



Airlock Microgateway Connaisseur

An Airlock Academy Training



Maximilian Dürr
Airlock Academy Teacher



Nicolas Röschli
Airlock Academy Teacher

Before We Start...

For **questions**, please use **voice chat**

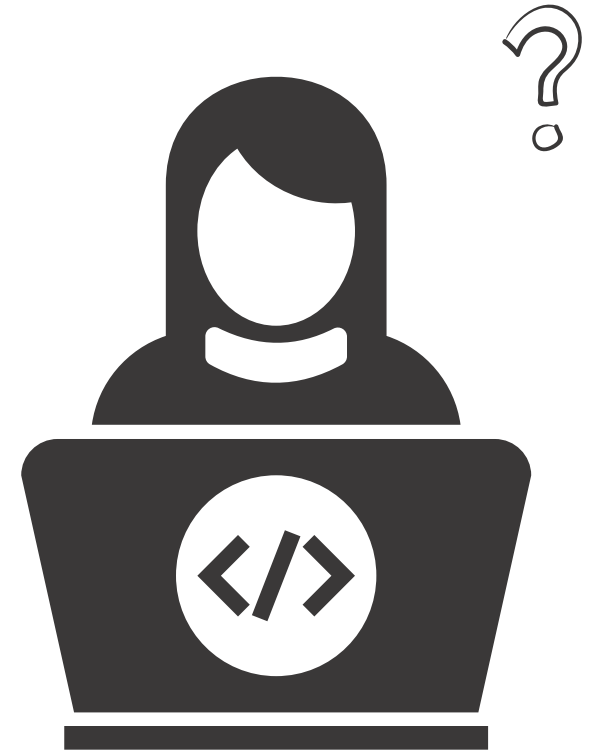
Text chat only for moderate discussion.



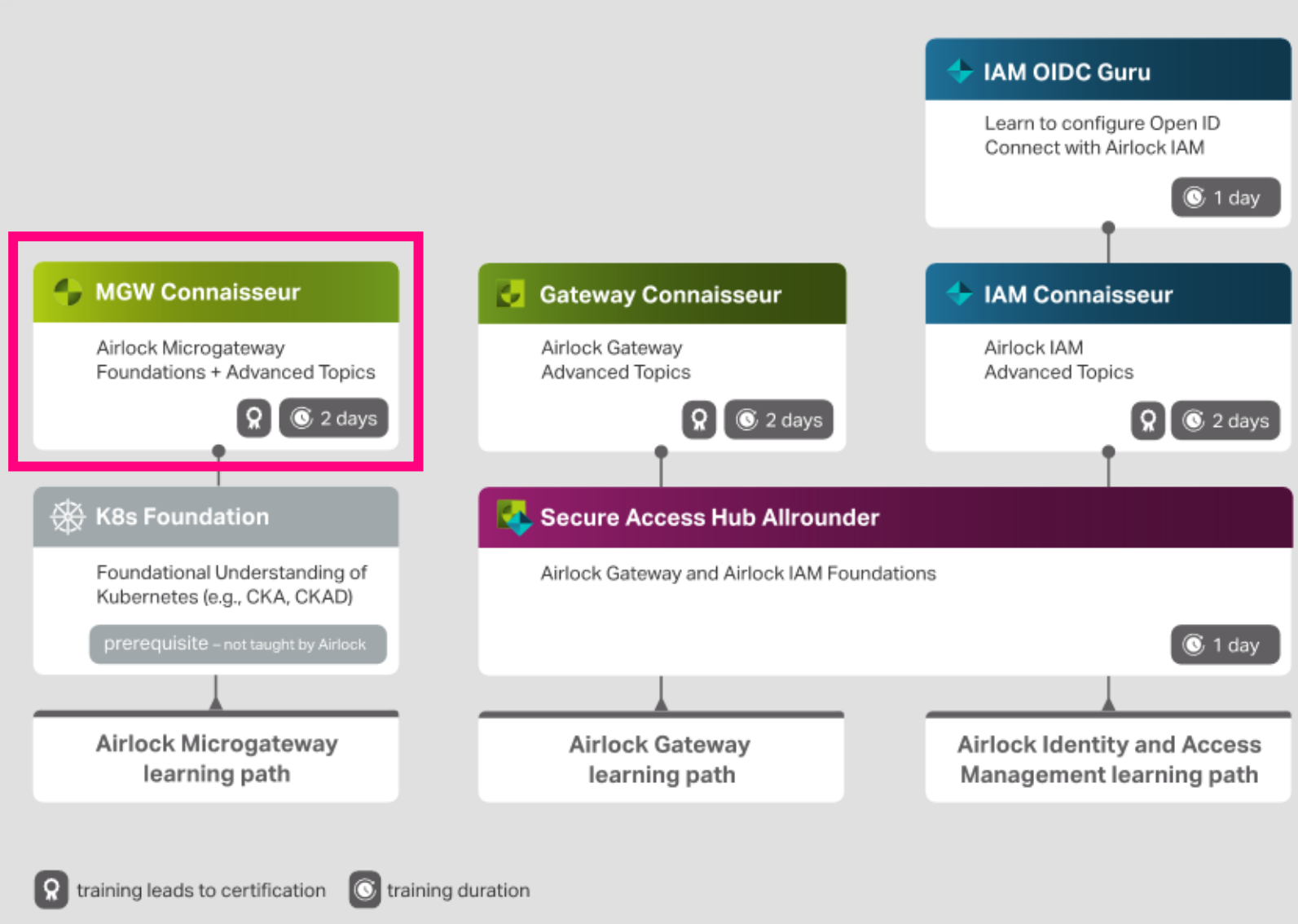
Participant Introductions

Please, **introduce yourself**, including:

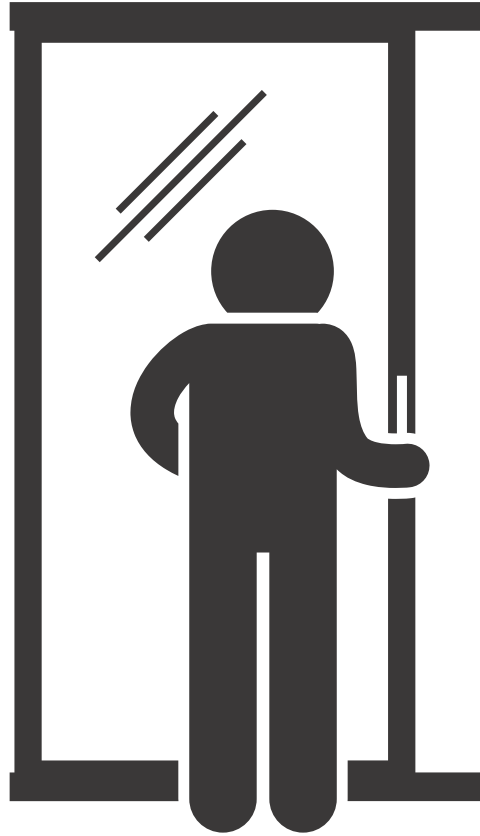
- Name
- Company, designation and responsibilities
- Expectations of this course and your goals
- The extent of your knowledge of application security

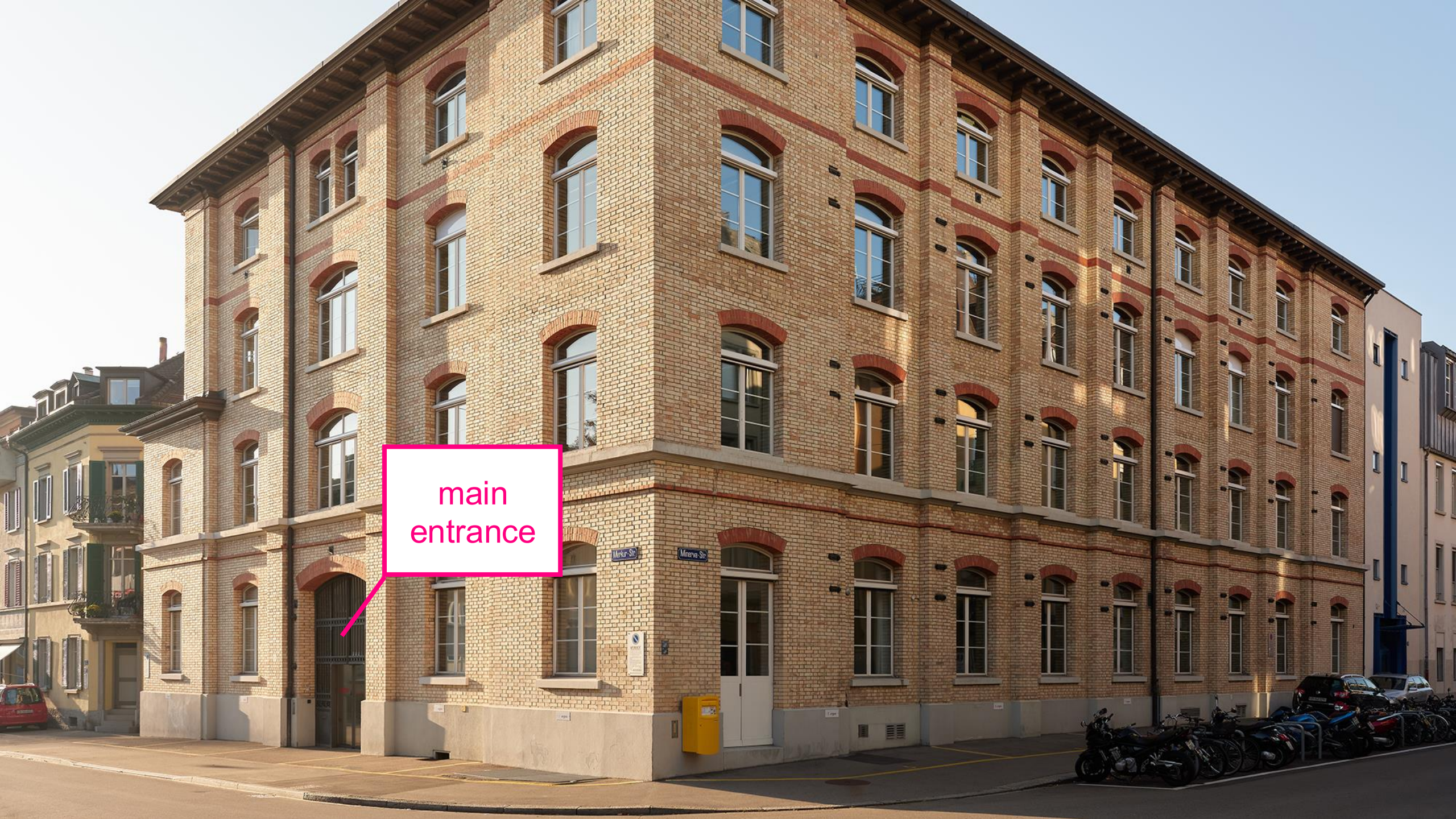


Training Overview



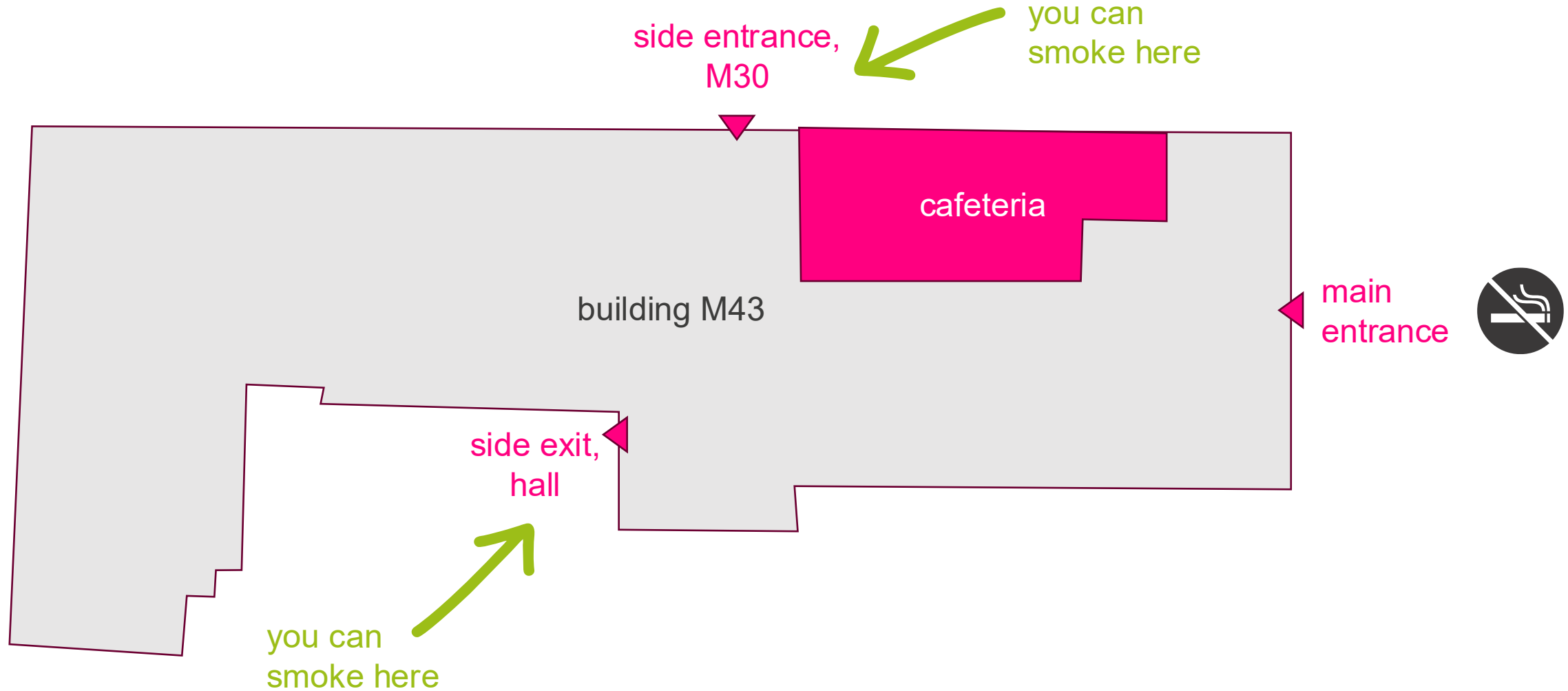
Please do not Smoke at the Building's Main Entrance





main
entrance

Smoking Areas: Merkurstrasse 43 (M43)



This Training



2 days



09:00 - 16:00
CET (winter)
CEST (summer)



Morning/
afternoon breaks



Lunch breaks

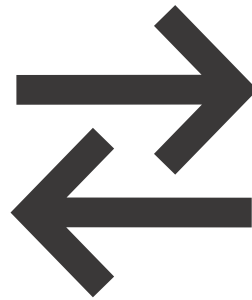


Certification

This Training



theory



hands-on LABs

Airlock Microgateway Version for This Training



AIRLOCK®

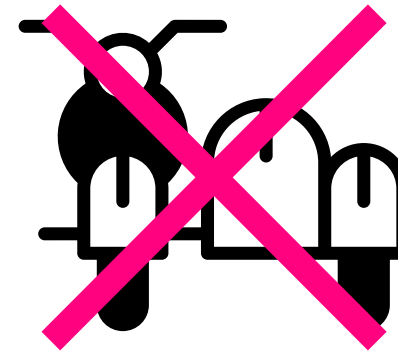
Microgateway **5.1**



Airlock Microgateway DataPlane Modes



Sidecarless,
based on K8s Gateway API



Sidecar

Deprecated with Airlock
Microgateway **5.0**

Learning Goal(s)



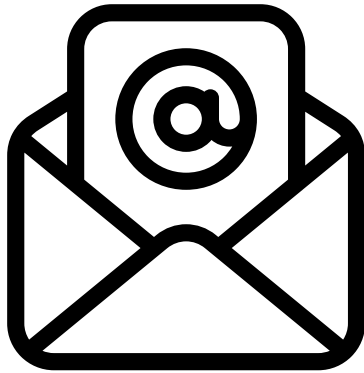
After this training you should be able to

install, configure, and operate Airlock Microgateway to protect web applications

As part of this, you should

- understand core concepts and features of Airlock Microgateway, based on hands-on experiences
- know where and how to find configuration related info (e.g., Documentation, CRD Reference)
- be able to tackle potential issues by yourself and know who to contact if you require help

We Would Appreciate Your Feedback!



link via e-mail



Airlock Microgateway Connaisseur Feedback

Hello!

We would appreciate it, if you could help us to improve the Airlock Microgateway training and product by filling our feedback questionnaire.

Please answer only half of the questionnaire at the begin of the training, and the other half at the end of the training (stopping-point indicated in questionnaire).

Your answers will help us to better address your needs.

[Click here to start →](#)

Please Fill Only Half of the Questionnaire Now

🙏 Thank you!

👋 Please stop here for now.

STOP once you see
this information



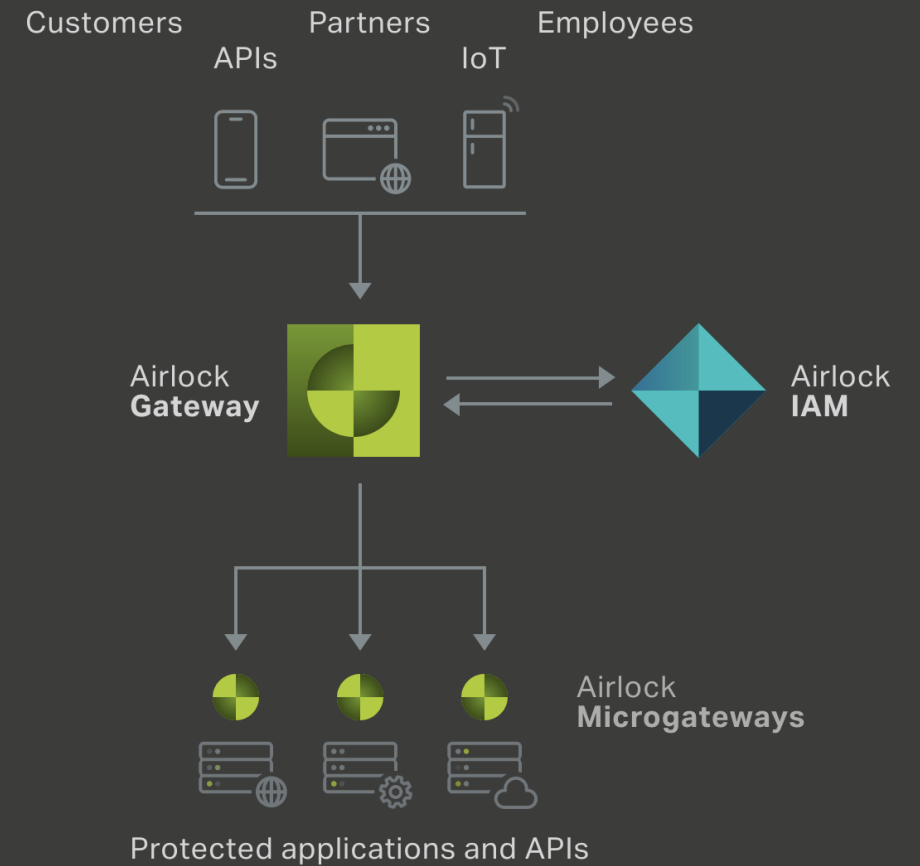
Re-open the link you received via e-mail when prompted at the end of the last day of the training and answer the remain questions.

You are at the end of the last day of the training and re-opened the link?

Please, click «**Next**» to continue!

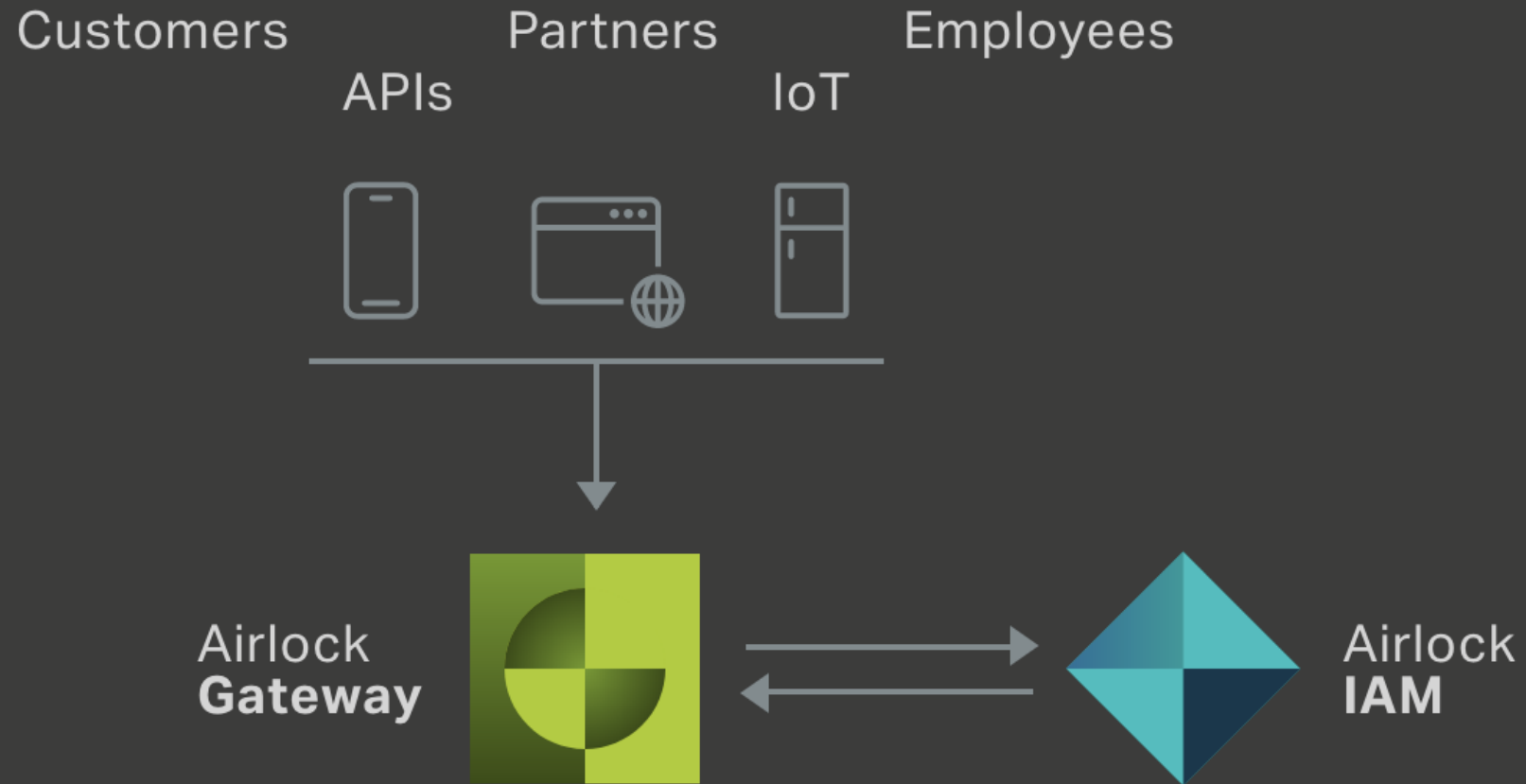
Friendly to users. Relentless to uninvited guests.

The Airlock components work together to provide maximum ease of use and effective protection without compromise.

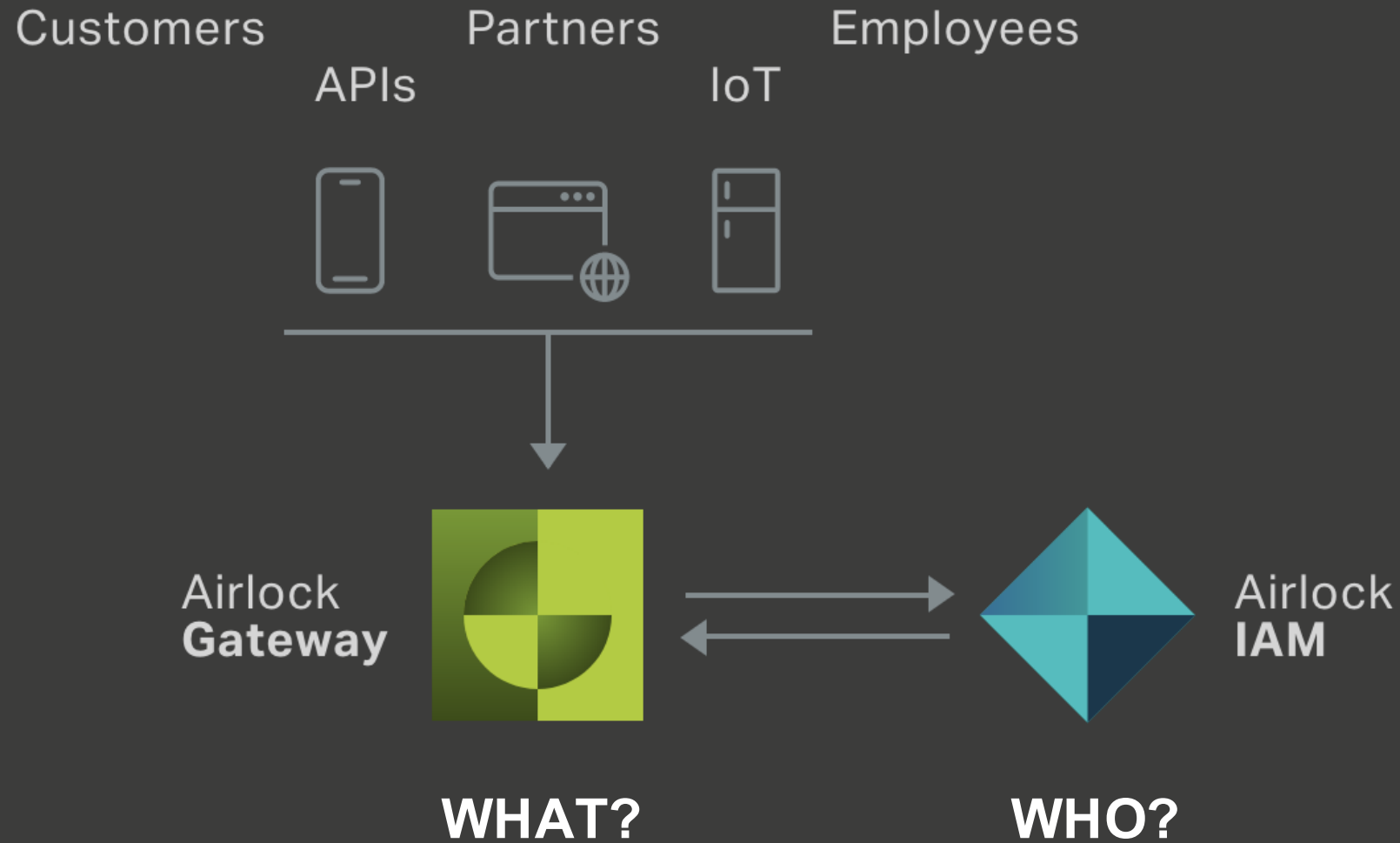


Secure Access Hub

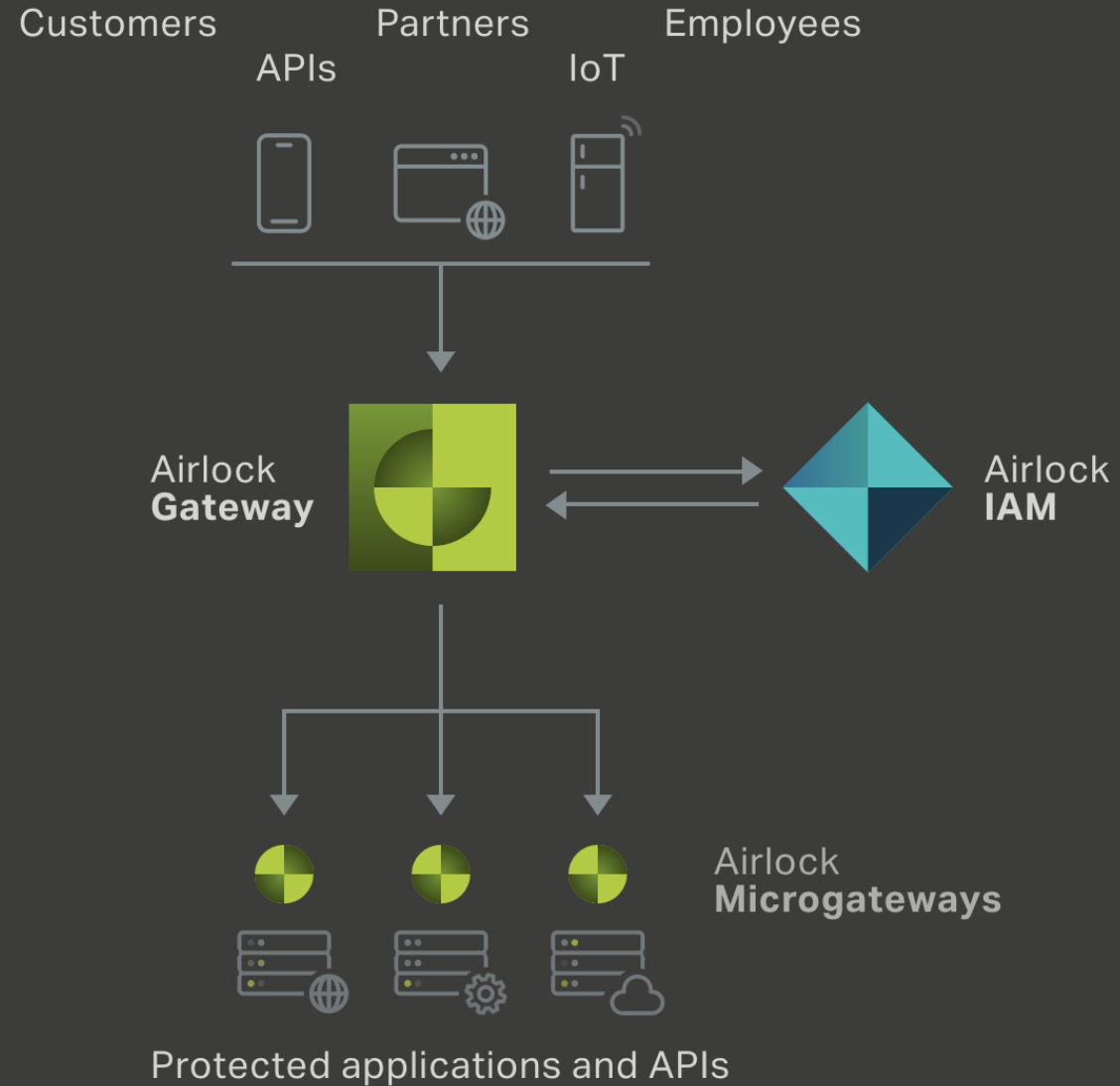
Secure Access Hub



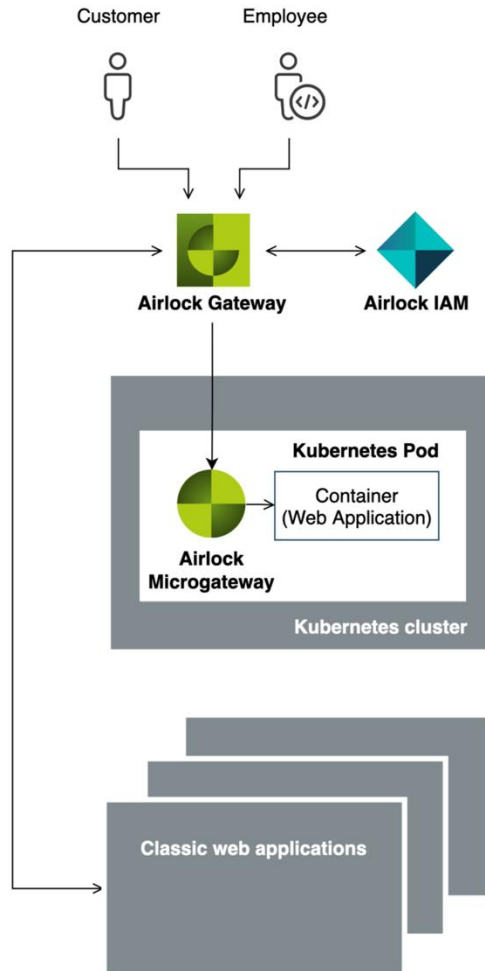
Secure Access Hub



Airlock Microgateway

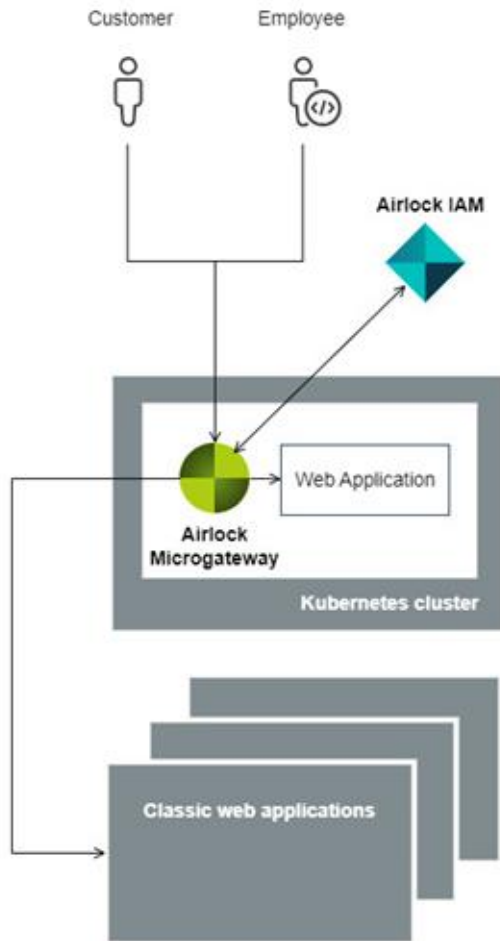


Microgateway Architecture **WITH** Classic Perimeter



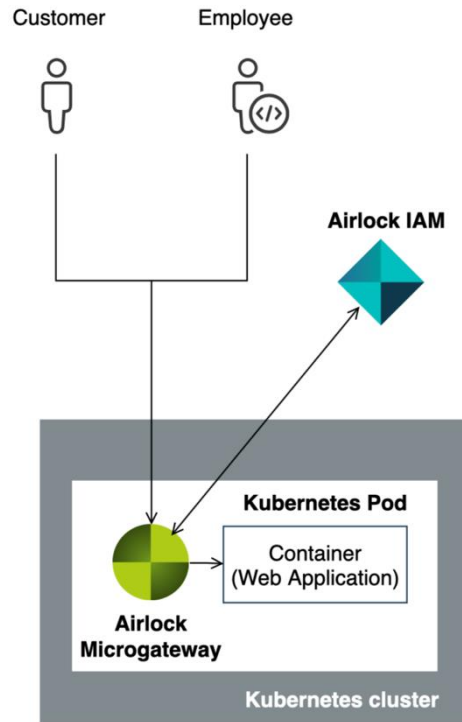
- **Perimeter to secure ANY enterprise workload**
- **Classic workloads must be protected**
- Kubernetes workloads must be protected:
 - Establish DevSecOps principles with shared responsibility
 - Zero Trust architecture
 - Short development lifecycle

Microgateway Architecture **WITH** Classic Workloads



- No need for a classic perimeter
- Single solution to protect classic and Kubernetes workloads
- Kubernetes workloads must be protected:
 - Establish DevSecOps principles with shared responsibility
 - Zero Trust architecture (requires dedicated Airlock Microgateway per web application)
 - Short development lifecycle

Microgateway Architecture **WITHOUT** Classic Workloads



- **NO** need for a perimeter
- **NO** classic workloads to protect
- Kubernetes workloads must be protected:
 - Establish DevSecOps principles with shared responsibility
 - Zero Trust architecture
 - Short development lifecycle

Key Features



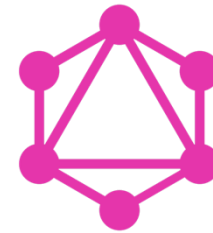
Filtering



API protections



OpenAPI



GraphQL

Key Features



Filtering



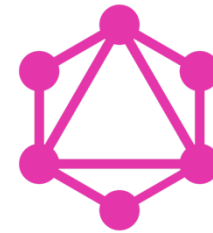
API protections



Access control with built-in
OIDC RP and JWT



OpenAPI



GraphQL



JWT



OIDC

Key Features



Filtering



API protections



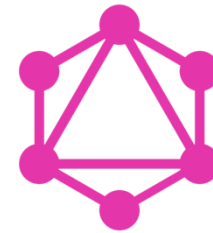
Access control with built-in
OIDC RP and JWT



Observability



OpenAPI



GraphQL



JWT



OIDC

Key Features



Filtering



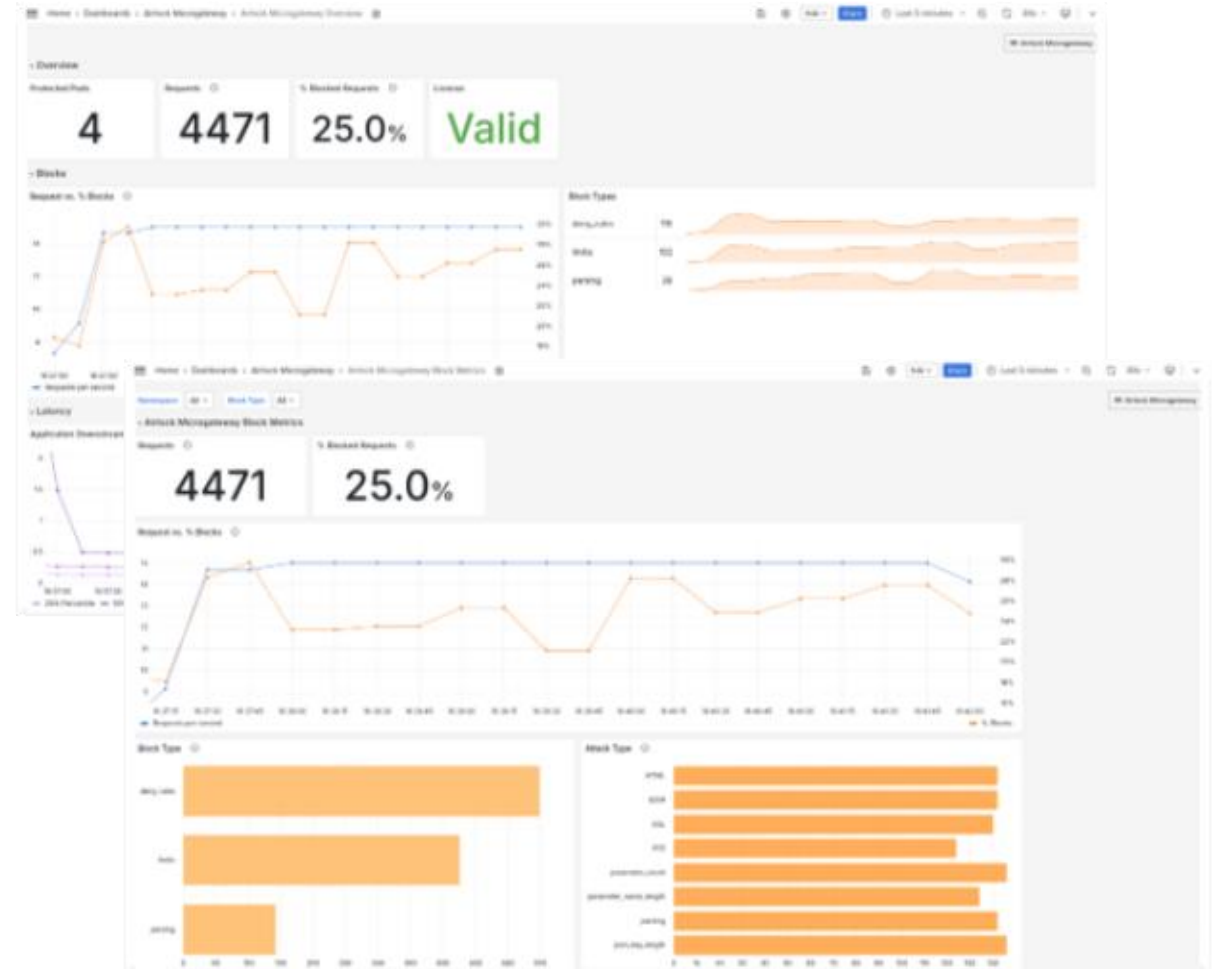
API protections



Access control with built-in
OIDC RP and JWT



Observability



What Sets us Apart



Kubernetes native
Runtime security
for pods



Comprehensive
API Protection



RedHat OpenShift
certified



Kubernetes Gateway API
support



Hyperscaler agnostic,
no **lock-in**



Identity aware proxy

Migration Path from Airlock Gateway

Airlock Microgateway is certified for OpenShift

OpenShift Virtualization support available with **Airlock Gateway ≥ 8.5**

Migration path

1. Move workload and Airlock Gateway in OpenShift
2. Migrate one service after another to Airlock Microgateway

Training Overview



Academy Labs and Certification Overview	Let's Explore Airlock Microgateway	Resources	Foundational Components	Setup Airlock Microgateway	K8s Gateway API
Basic Webapp Protection	Troubleshooting	Monitoring	DevSecOps Process (optional)	Deny Rule Overrides and Exceptions	Policy Enforcement (optional)
Header Rewrites	Request and Rate Limits	Request Body Parsing	OpenAPI and GraphQL	Session Handling	AuthN, AuthZ, and IdP
ICAP	Operations	Conclusion and Certification			

Academy Labs and Certification Overview



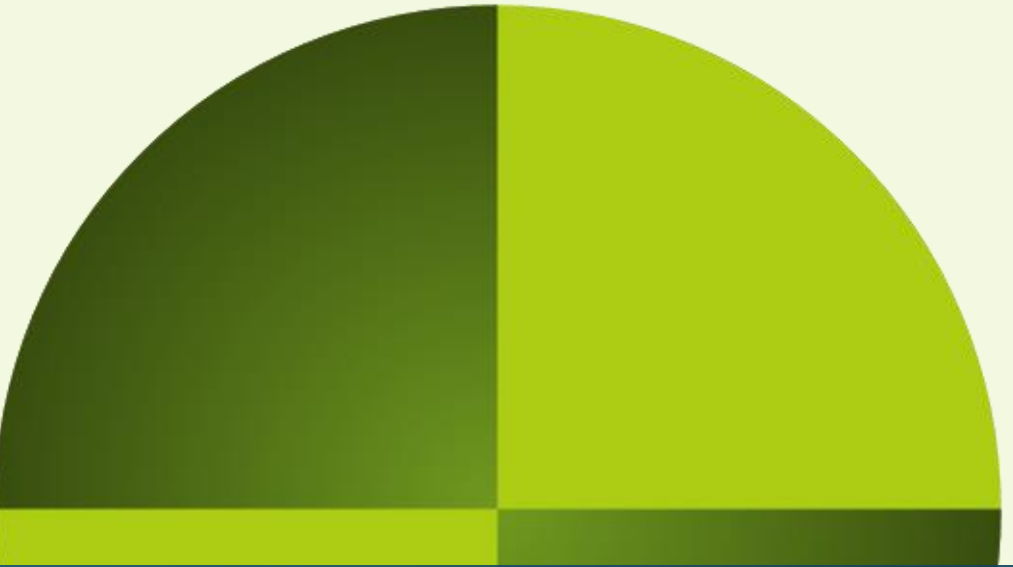
What **lab environment** and **use cases** will you work with *during the training*, and what will the *final* **certification** be like?

Academy Lab Environment
Certification



Academy Labs and Certification Overview

Academy Lab Environment
Certification



Academy Labs and Certification Overview

Academy Lab Infrastructure

Core



Kubernetes



Microgateway

Monitoring



Grafana



Prometheus



Loki

Back-ends



JuiceShop



EchoServer

Policy Enforcement



Kyverno

Session Handling and Authentication



Redis



Entra ID

Academy Labs in This Training



Explore MGW



Setup MGW



First Webapp



Monitoring



Deny Rule Overrides
and Exceptions



Policy
Enforcement



Session
Handling

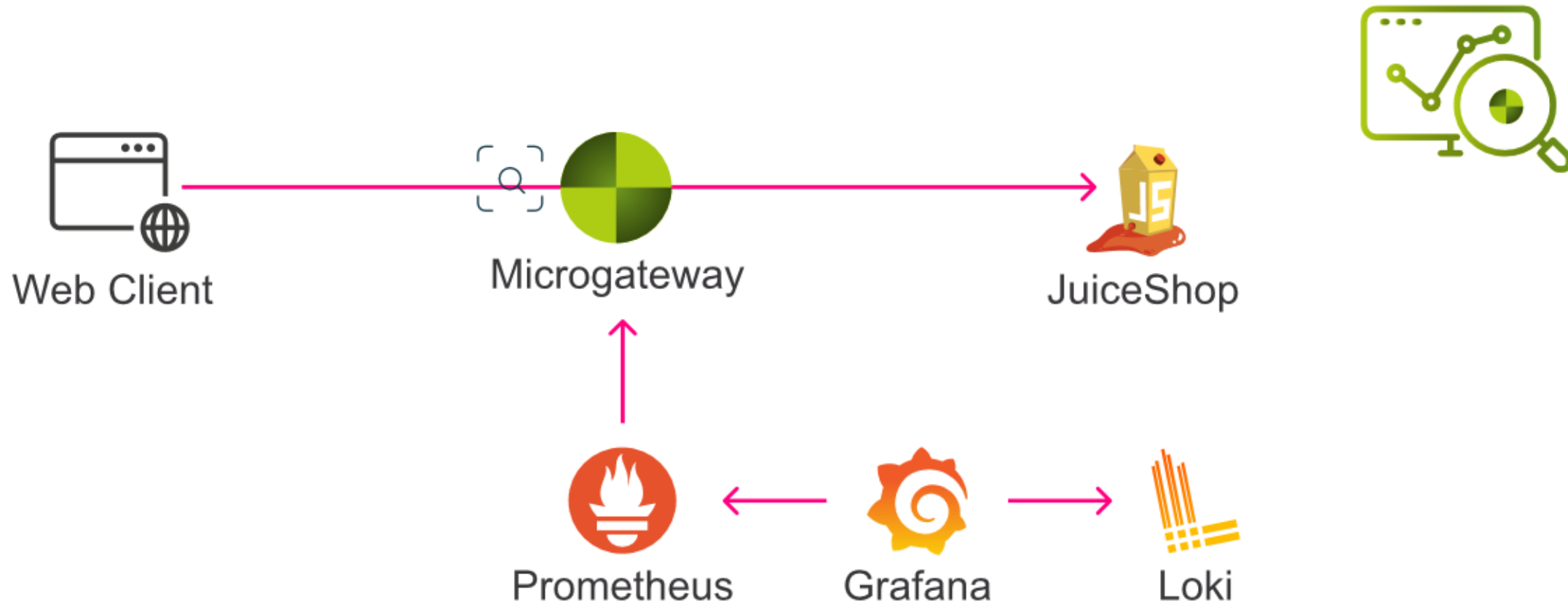


AuthN and AuthZ
with Entra ID

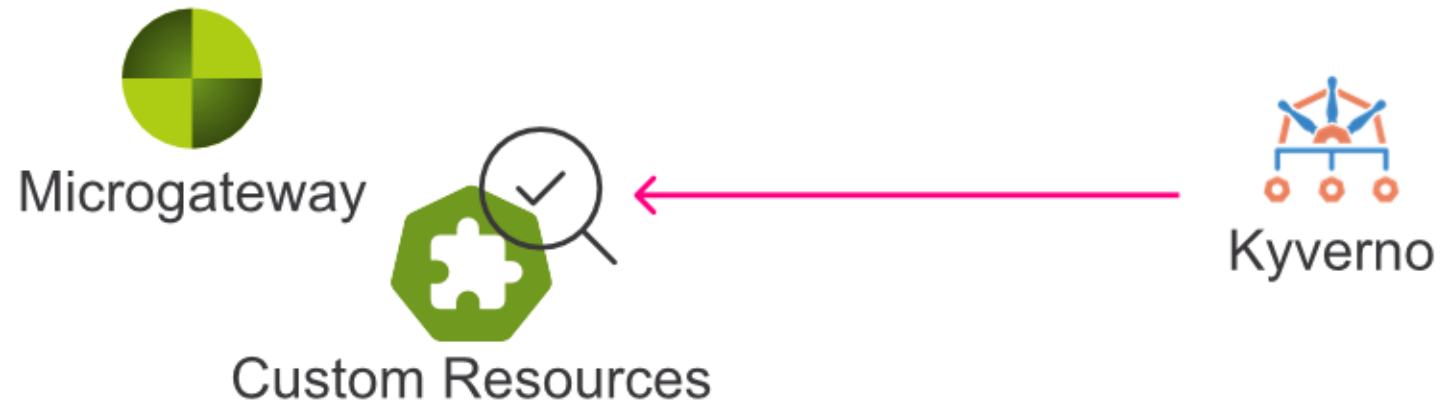
Usecase – Simple Webapp Protection



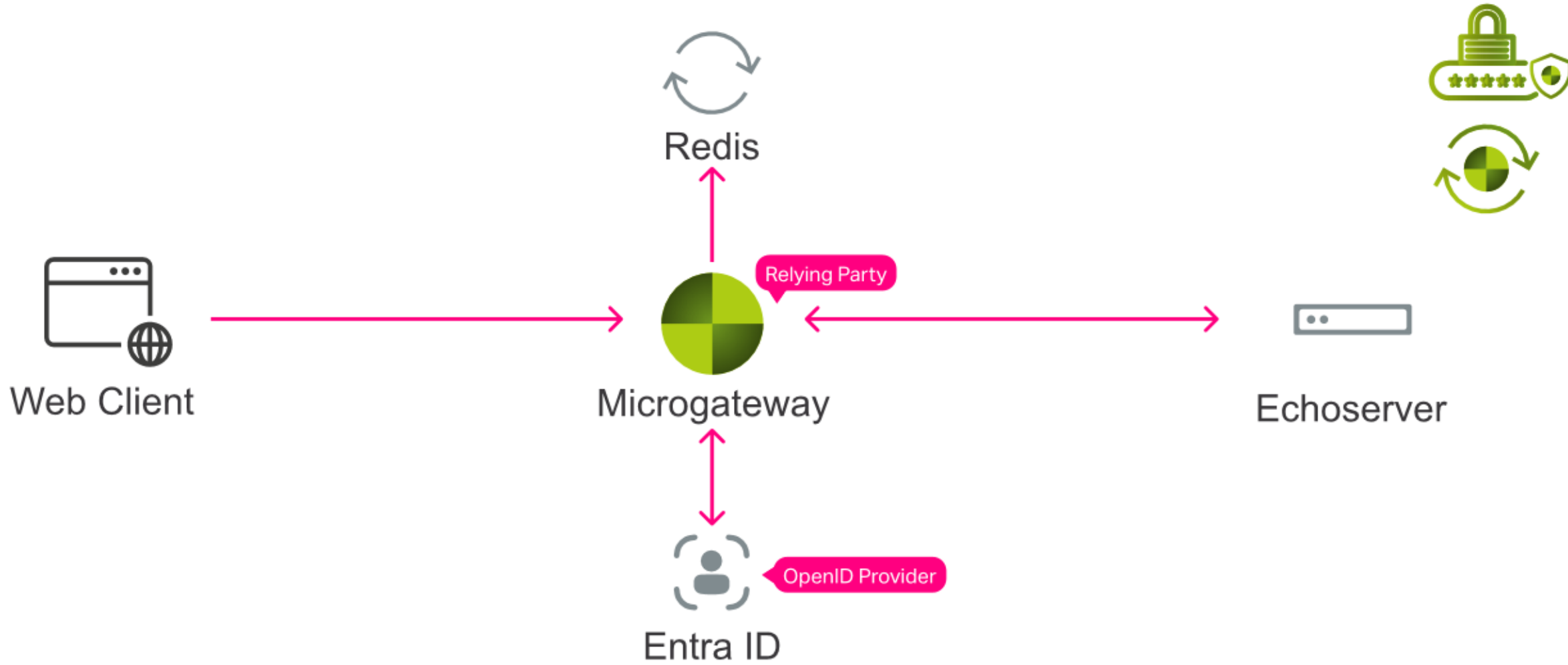
Usecase - Monitoring



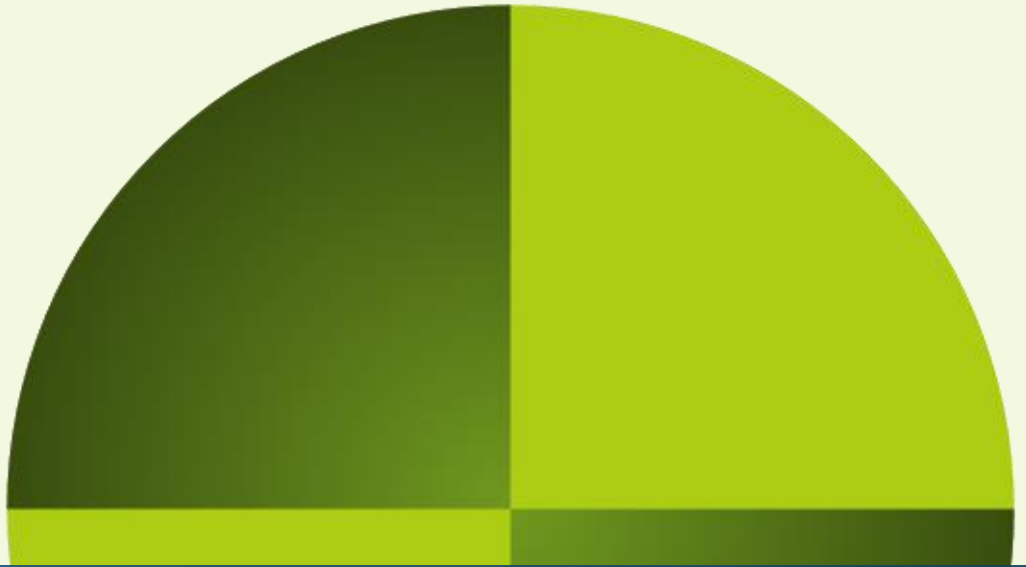
Usecase – Policy Enforcement



Usecase –Session Handling and Authentication



Academy Lab Environment
Certification



Academy Labs and Certification Overview

Certification Process



-  Online test via Testportal
 - An access code will be sent to your mail
 -  60 min
 -  Open book
 -  Single and multiple choice questions
 -  Answer 65% correctly to pass!
- **Further details at the end of the training!**

**Let's Explore
Airlock
Microgateway**



What is the **core architecture** of Airlock Microgateway
and what are the **building blocks** that allow its
configuration?

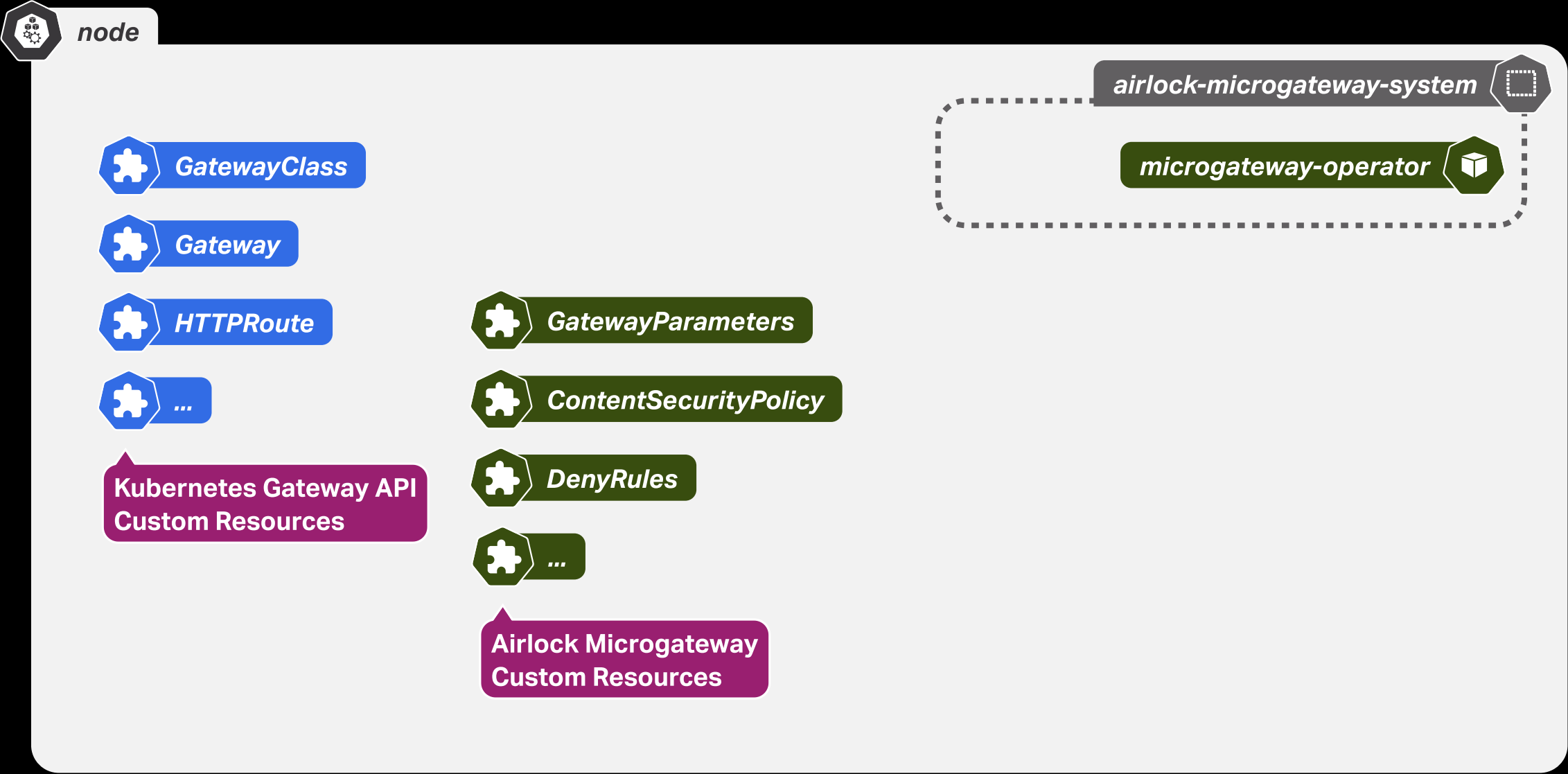
Airlock Microgateway Builds on Envoy Proxy



Airlock Microgateway is Based on the Operator Pattern



The Microgateway Can be Configured Based on Kubernetes Gateway API CRDs and Airlock Microgateway CRDs



You Can Find a Reference for Kubernetes Gateway API CRDs Online



node



GatewayClass



Gateway



HTTPRoute



...

Kubernetes Gateway API
Custom Resources

AIRLOCK®
Microgateway 5.x-dev

Search...

CRD Reference

Gateway API

BackendTLSPolicy

Gateway

GatewayClass

HTTPRoute

ListenerSet

ReferenceGrant

Microgateway

Gateway API

Gateway API

Reference documentation for Gateway API custom resource definitions

Gateway API Version v1.6.0-rc.1

Info

The Kubernetes Gateway API specification is an open standard for configuring and managing Kubernetes networking.

If Airlock Microgateway does not support an optional field or only supports specific values for a field, this is noted in the field's description within this documentation.

Note: The "Allowed Values" column is derived from the validation constraints defined in the Gateway API CRDs. It does not consider implementation support and may thus include values not supported by Airlock Microgateway.

BackendTLSPolicy

API Documentation for BackendTLSPolicy

Gateway

API Documentation for Gateway

GatewayClass

API Document

HTTP

API Document

Listener

API Document

Refer

API Document

<https://docs.airlock.com/microgateway/5.1/index/api/crds/gateway-api/index.html>

We also Provide a Reference for the Available Airlock Microgateway CRDs



node

 **GatewayParameters**

 **ContentSecurityPolicy**

 **DenyRules**

 ...

**Airlock Microgateway
Custom Resources**

Microgateway 5.x-dev

Search...

CRD Reference

- Gateway API
- Microgateway
 - AccessControlPolicy
 - APIProtection
 - ContentSecurityPolicy
 - CSRFProtection
 - CustomResponse
 - CustomResponsePolicy
 - DenyRules
 - EnvoyCluster
 - EnvoyExtensionPolicy
 - EnvoyHTTPFilter
 - GatewayParameters
 - GraphQL
 - HeaderRewrites
 - ICAPPolicy
 - ICAPProvider
 - Ider
 - JW
 - JW
 - Lim
 - Log
 - OID
 - OID
 - Ope
 - Par
 - Rat
 - Red
 - Ses

Microgateway

Reference documentation for Microgateway custom resource definitions

AccessControlPolicy

API Documentation for AccessControlPolicy

APIProtection

API Documentation for APIProtection

ContentSecurityPolicy

API Documentation for ContentSecurityPolicy

CSRFProtection

API Documentation for CSRFProtection

CustomResponse

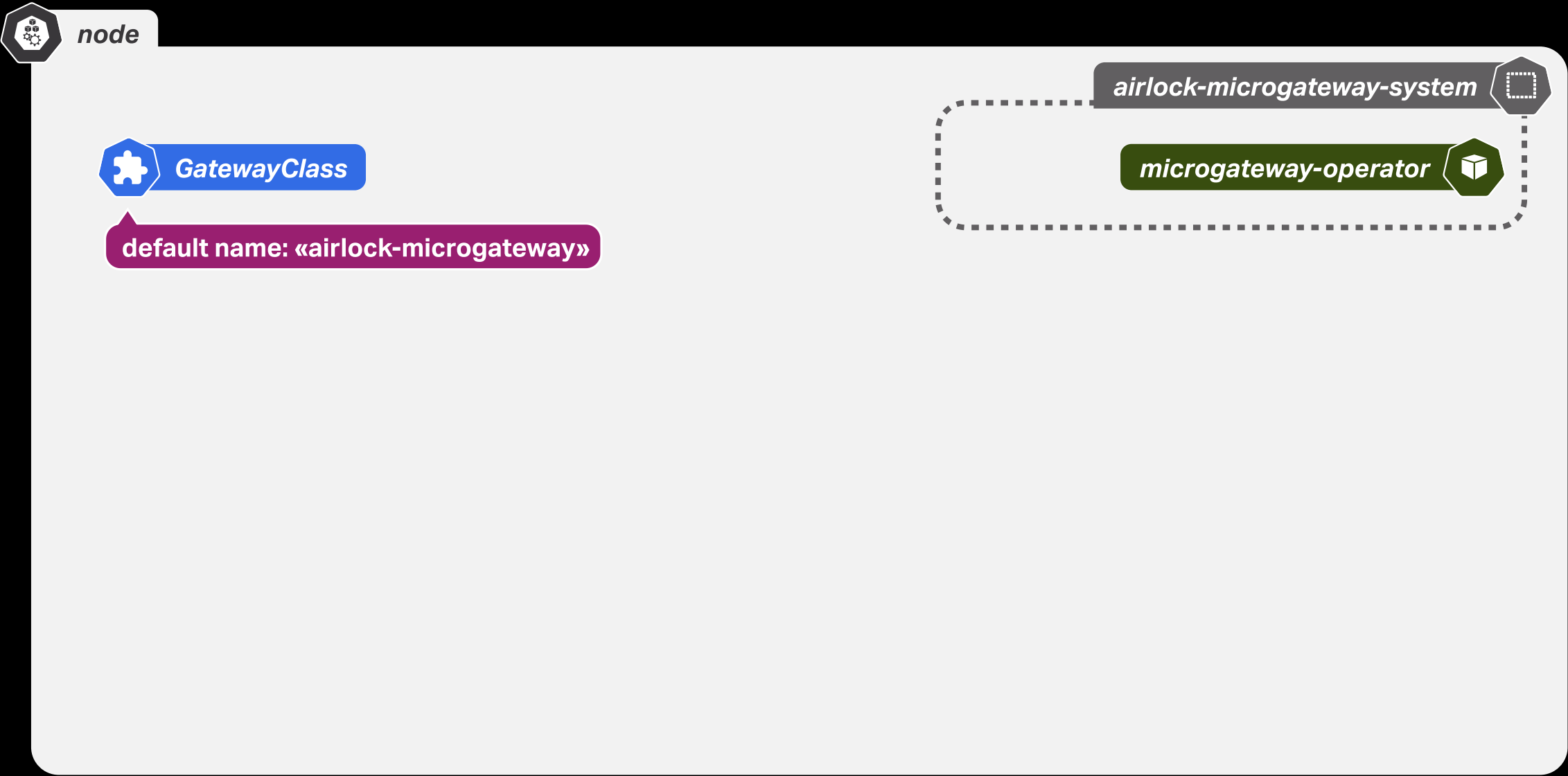
API Documentation for CustomResponse

CustomResponsePolicy

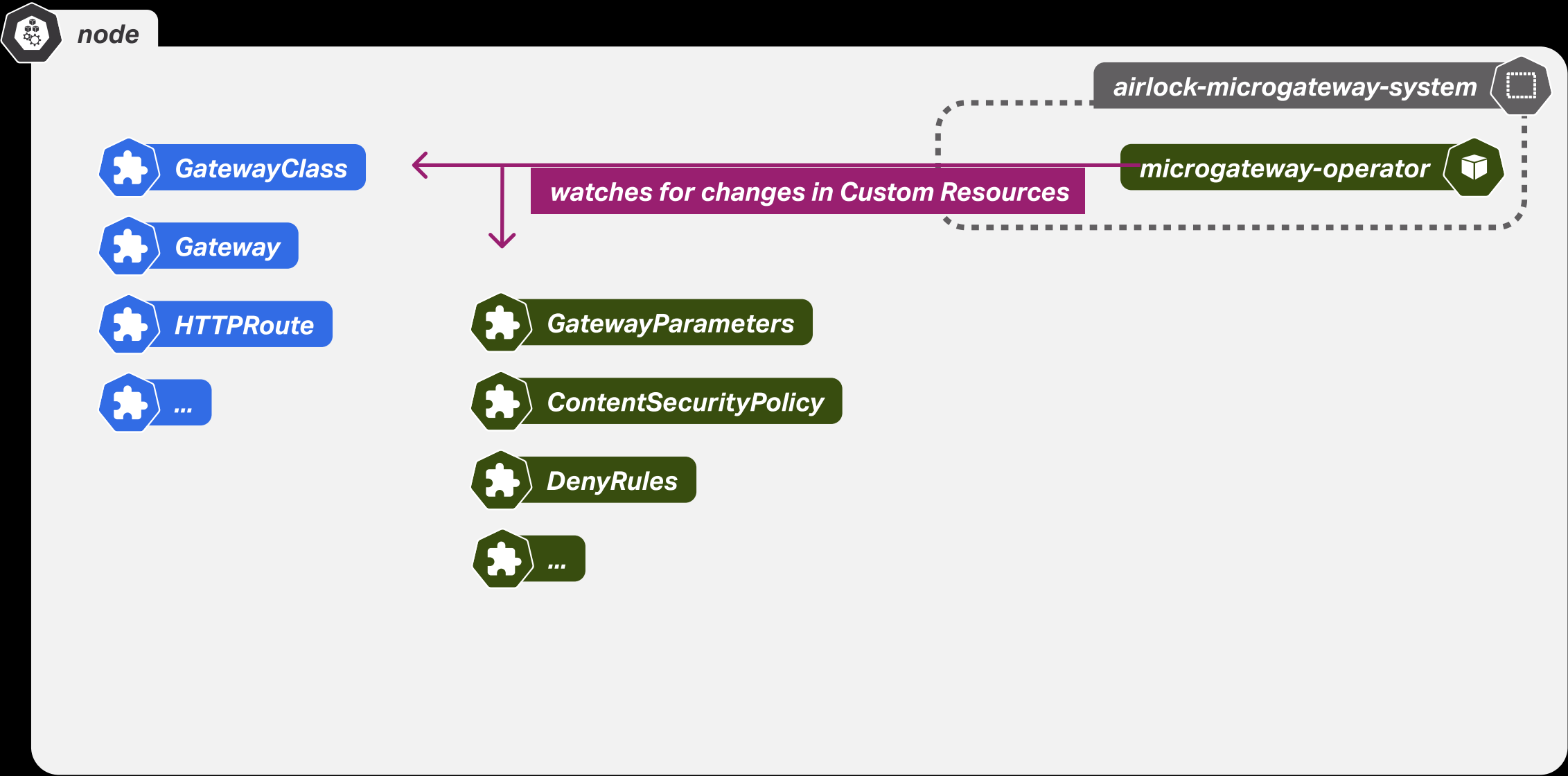
API Documentation for CustomResponsePolicy

<https://docs.airlock.com/microgateway/5.1/index/api/crds/microgateway/index.html>

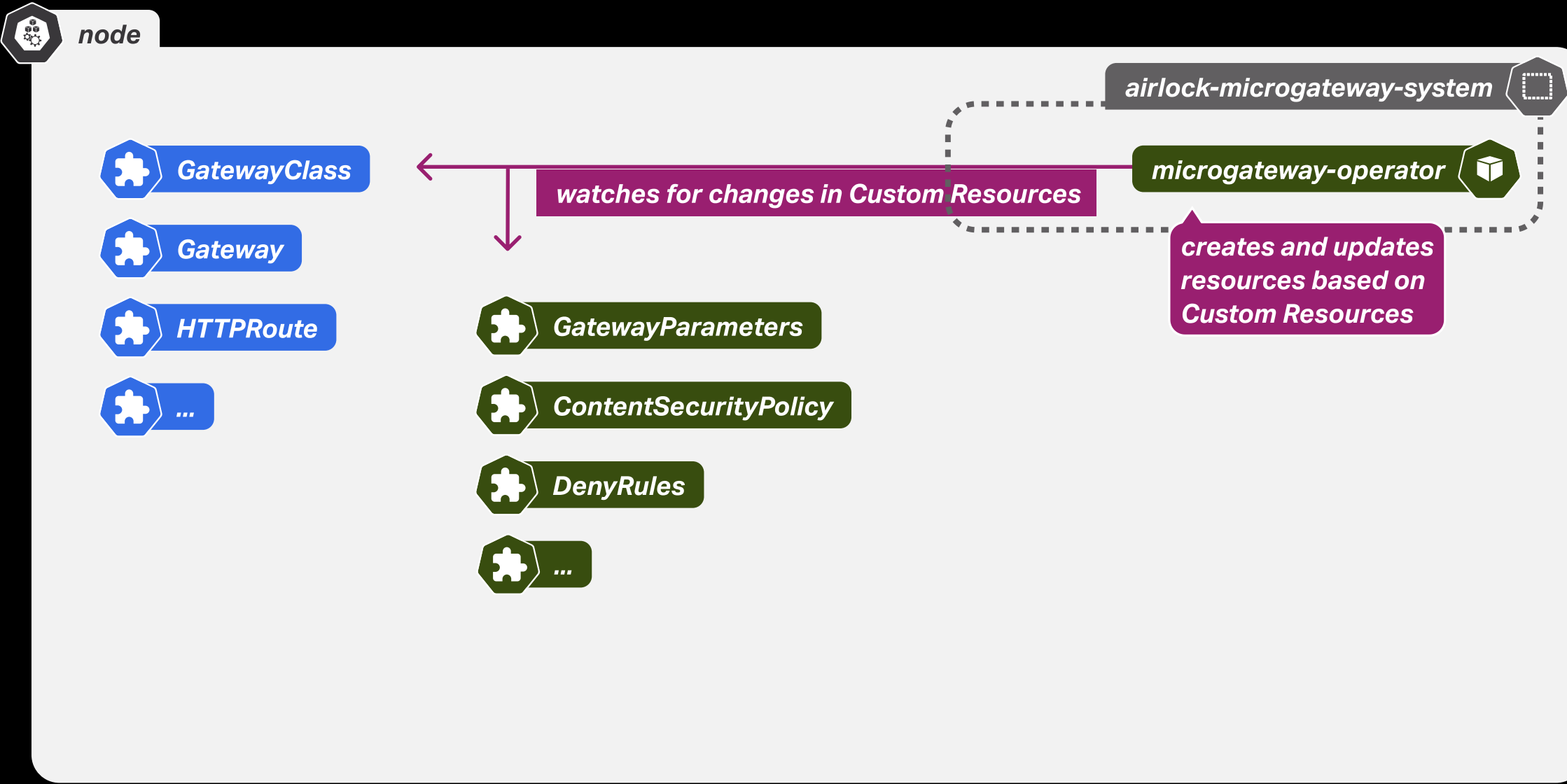
A Default GatewayClass CR is Deployed During the Microgateway Deployment in the Cluster (Cluster-scoped Resource)



The Microgateway Operator Watches for Changes in Kubernetes Gateway API CRs and Airlock Microgateway CRs



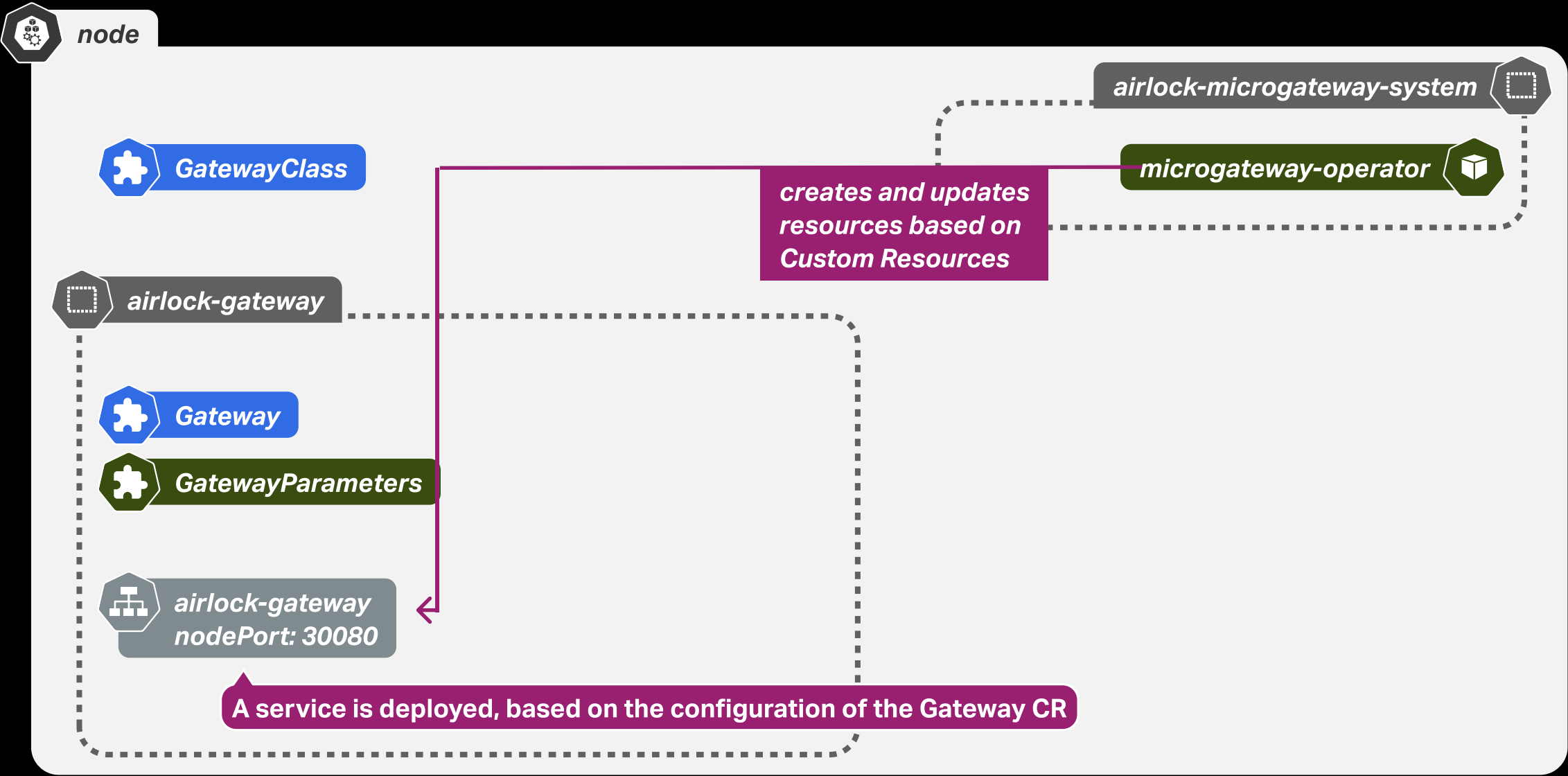
The Microgateway Operator Creates and Updates Resources based on the State of the Configured CRs



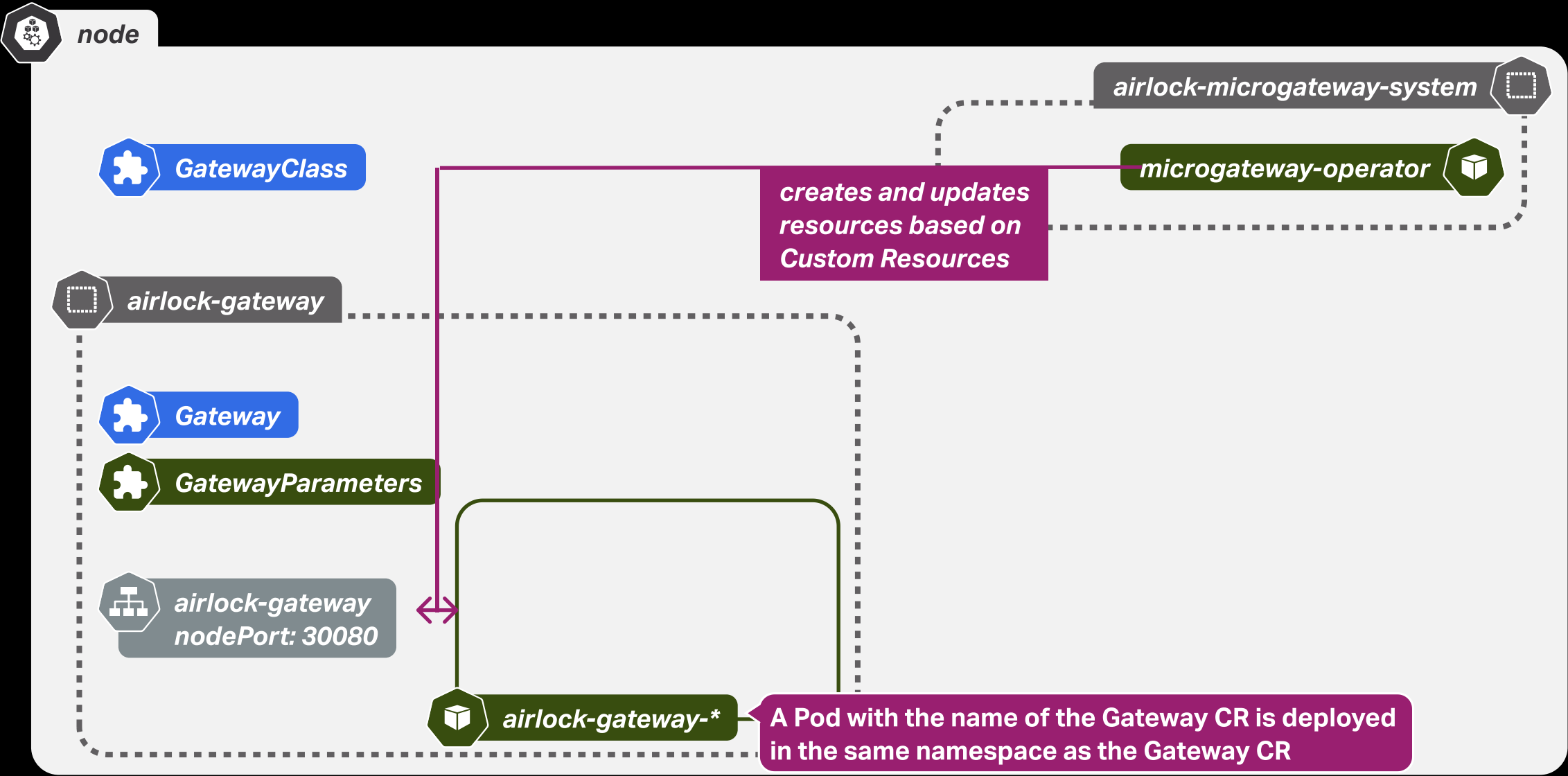
Let's Assume We Have a Gateway and a GatewayParameters CR Deployed in the Namespace «airlock-gateway»



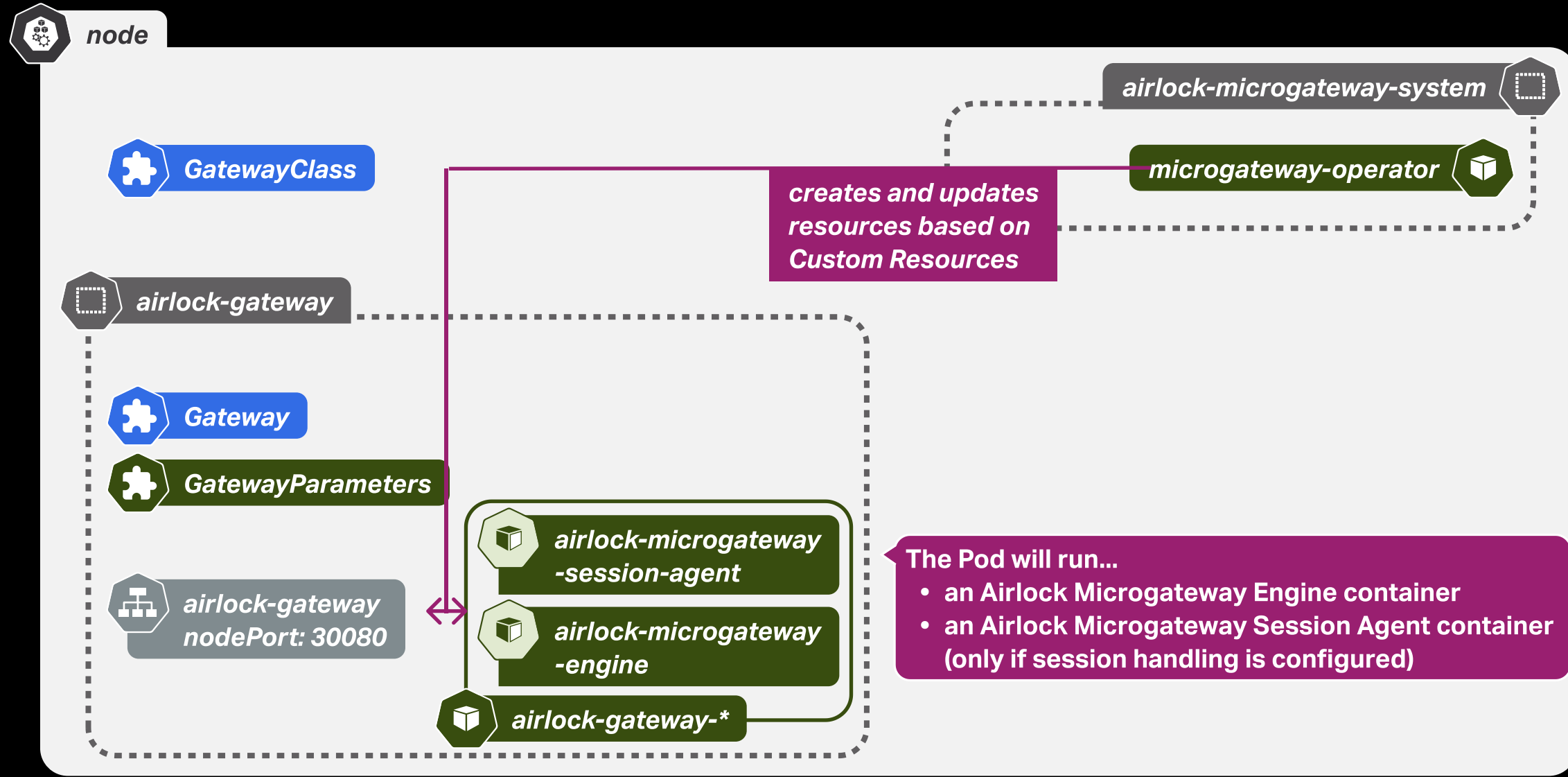
The Microgateway Operator Creates a Service Based on the Configuration of the Gateway CR



The Microgateway Operator also Creates a Pod with the Name of the Gateway CR



The Pod Will Run the Airlock Microgateway Engine, and Session Agent (only if Session Handling is Configured)

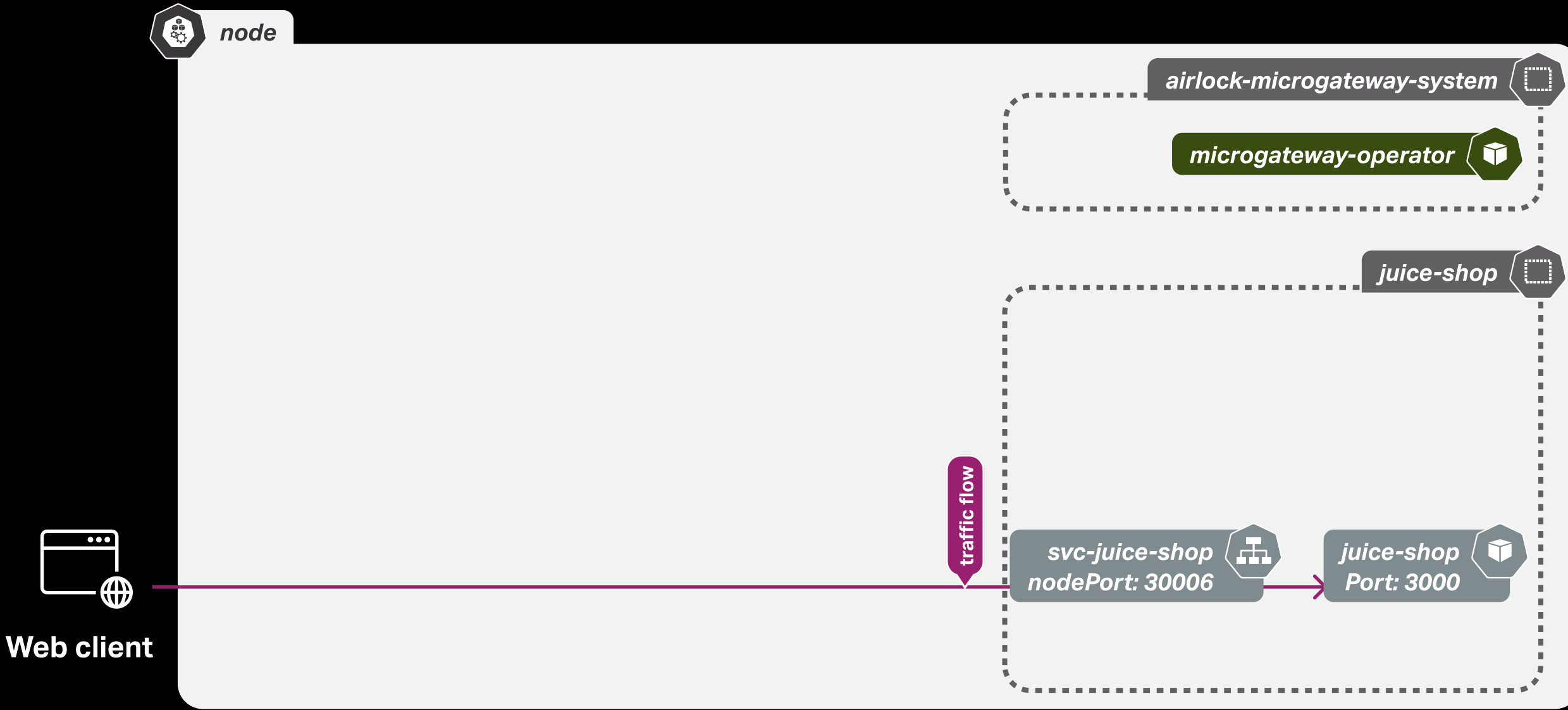


**Let's View A
Sample
Configuration...**

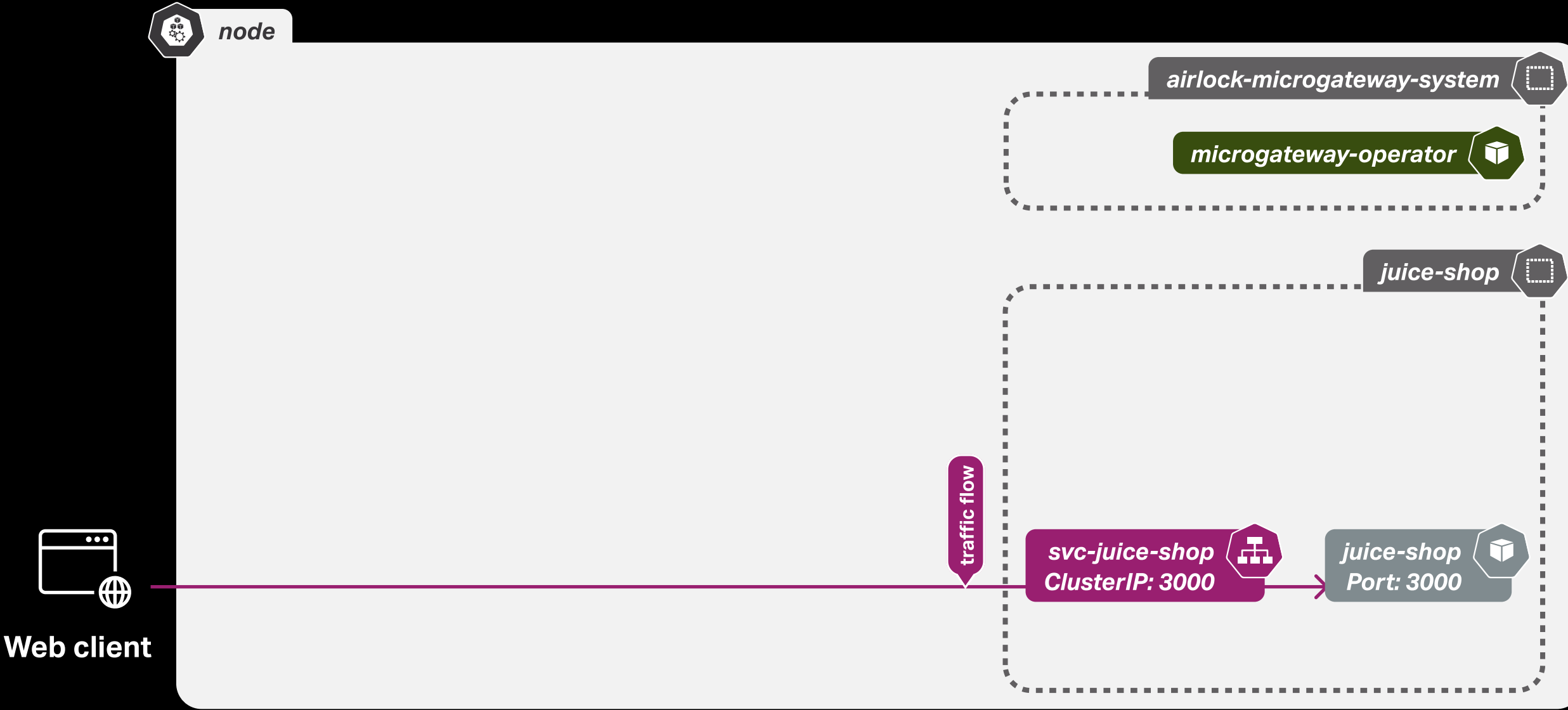
Usecase – Simple Webapp Protection



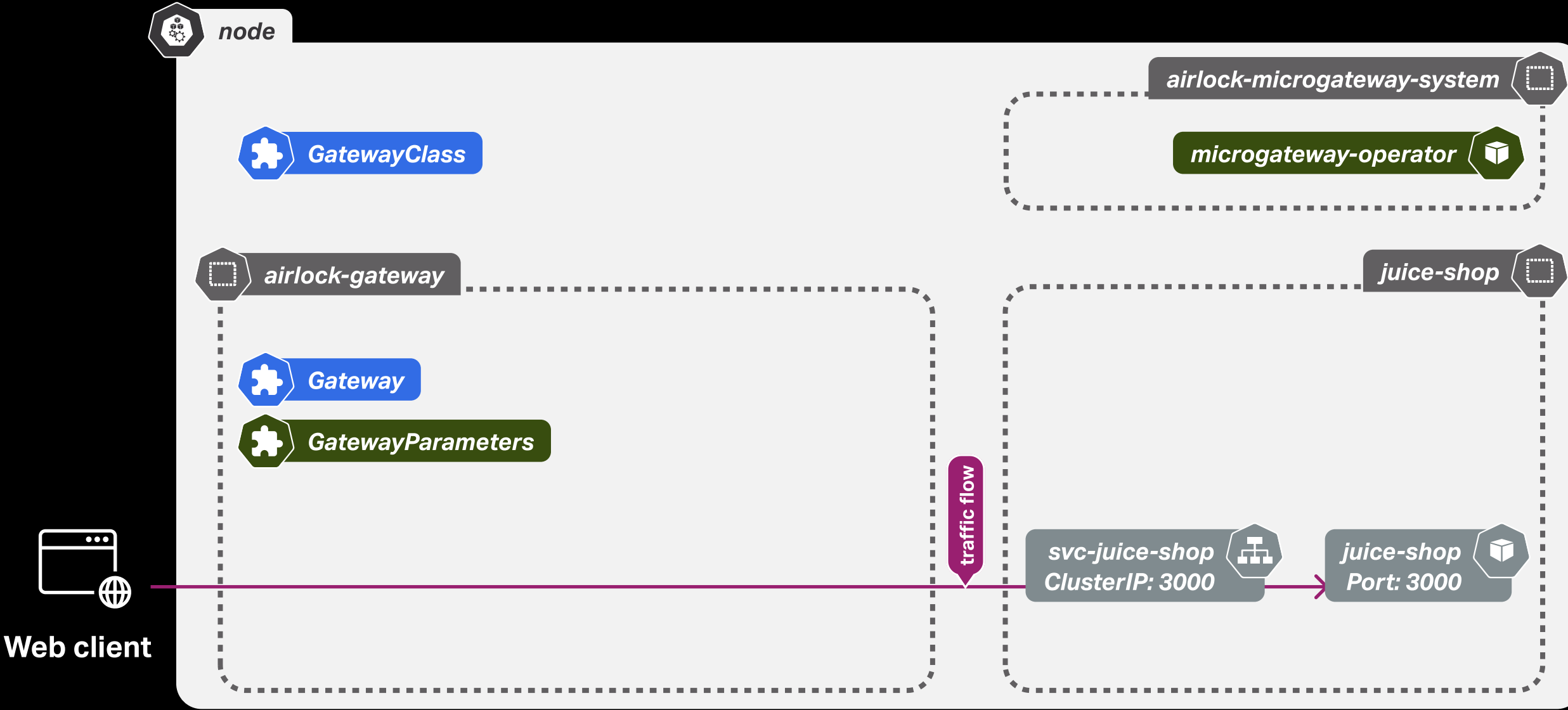
Access to Juice Shop, Traffic NOT Routed through the Airlock Microgateway Engine



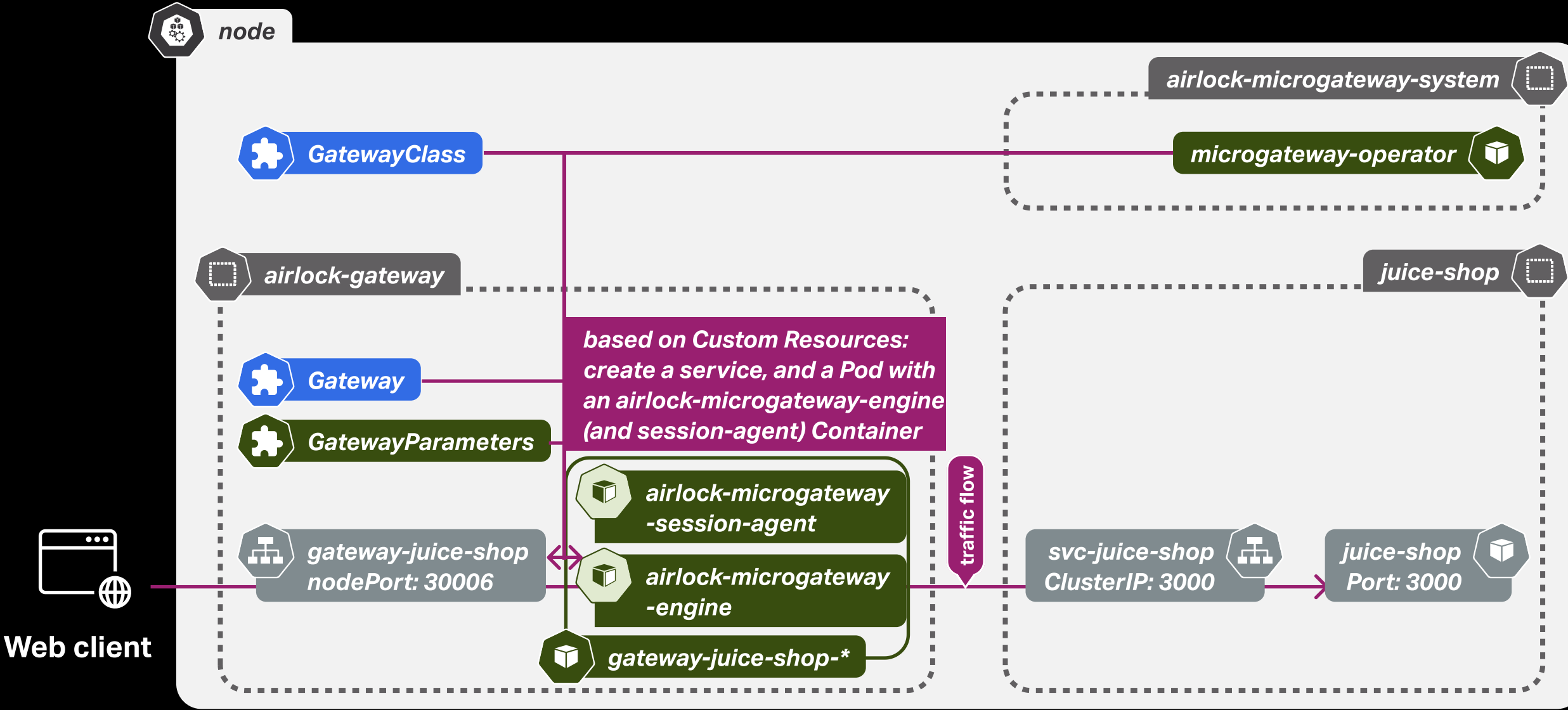
Change nodePort to ClusterIP for svc-juice-shop



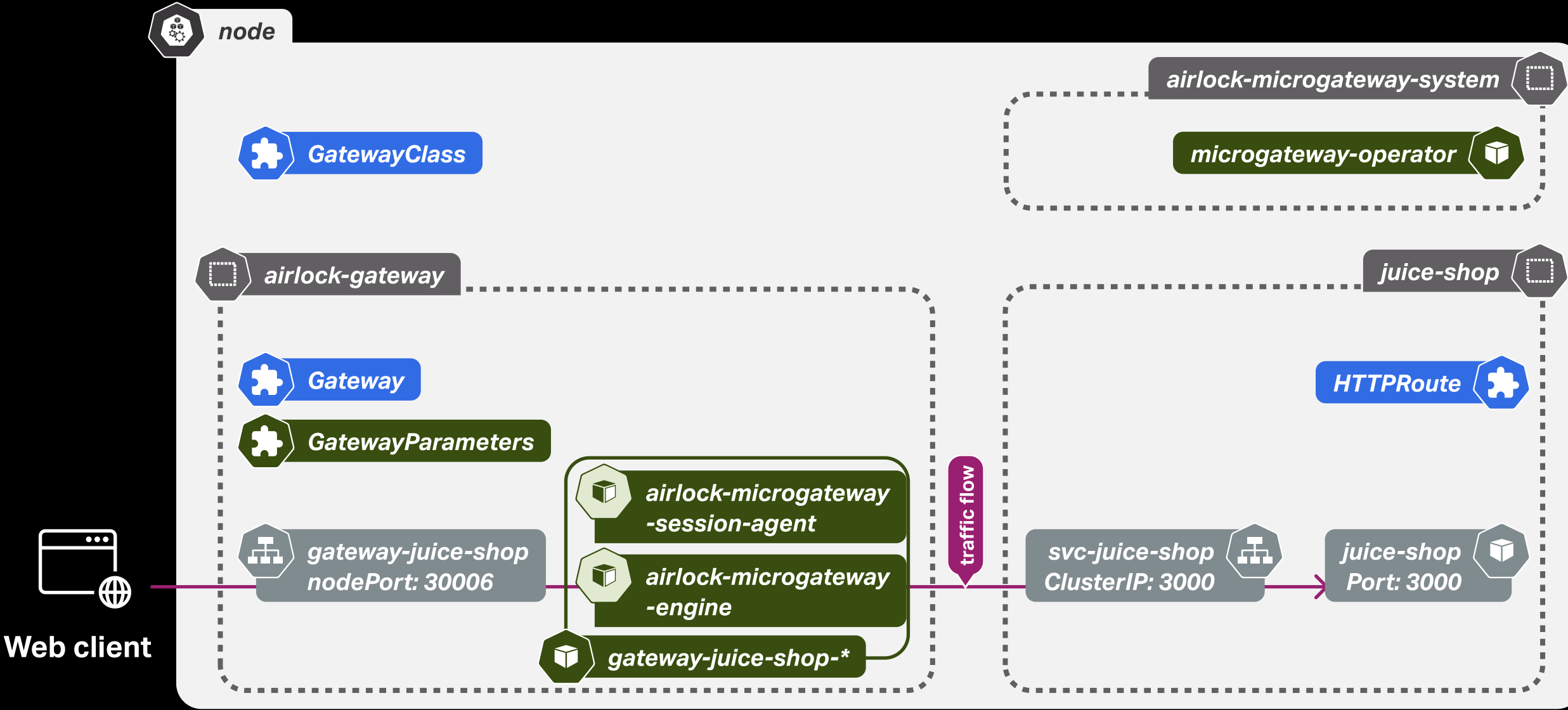
Deploy CRs to Create a Service and a Pod with an Airlock Microgateway Engine Container



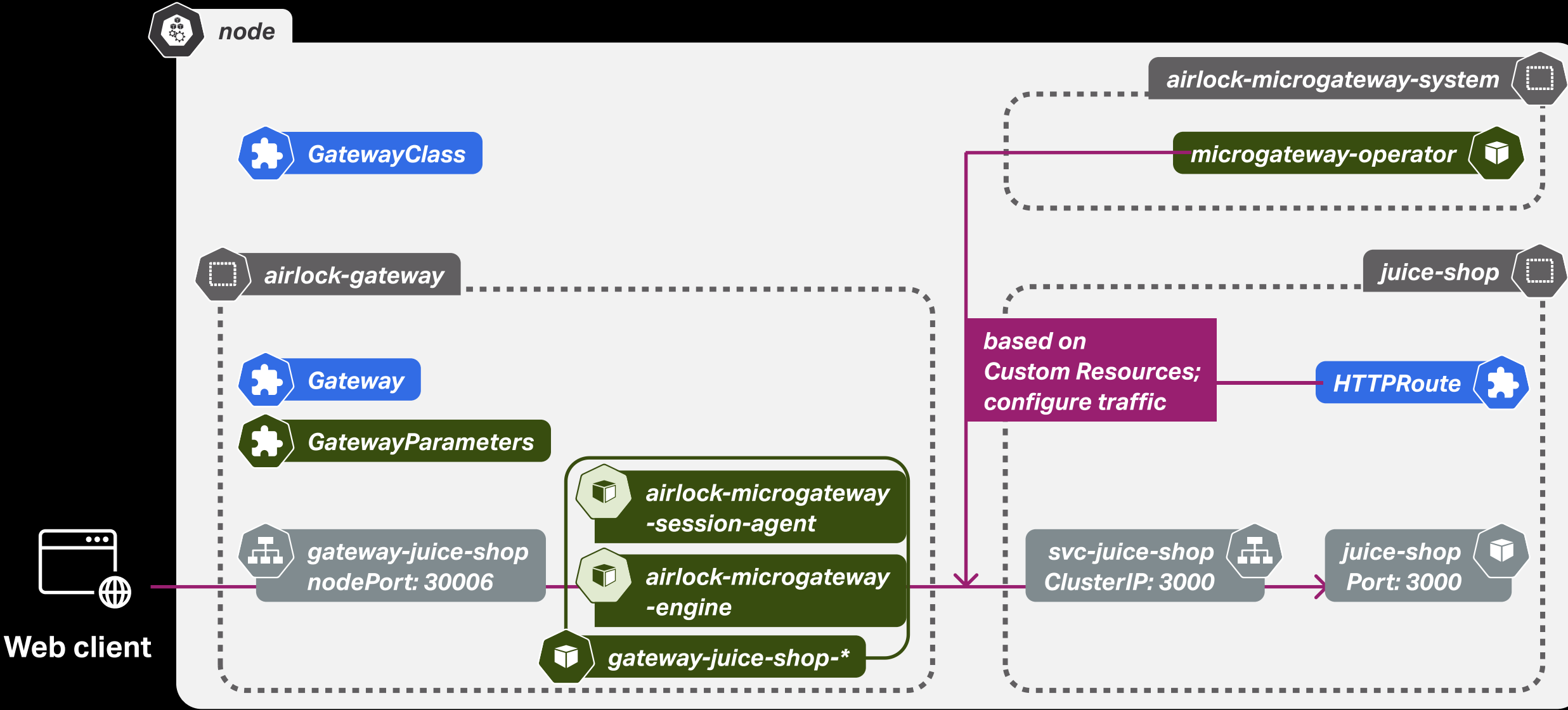
Deploy CRs to Create a Service and a Pod with an Airlock Microgateway Engine Container



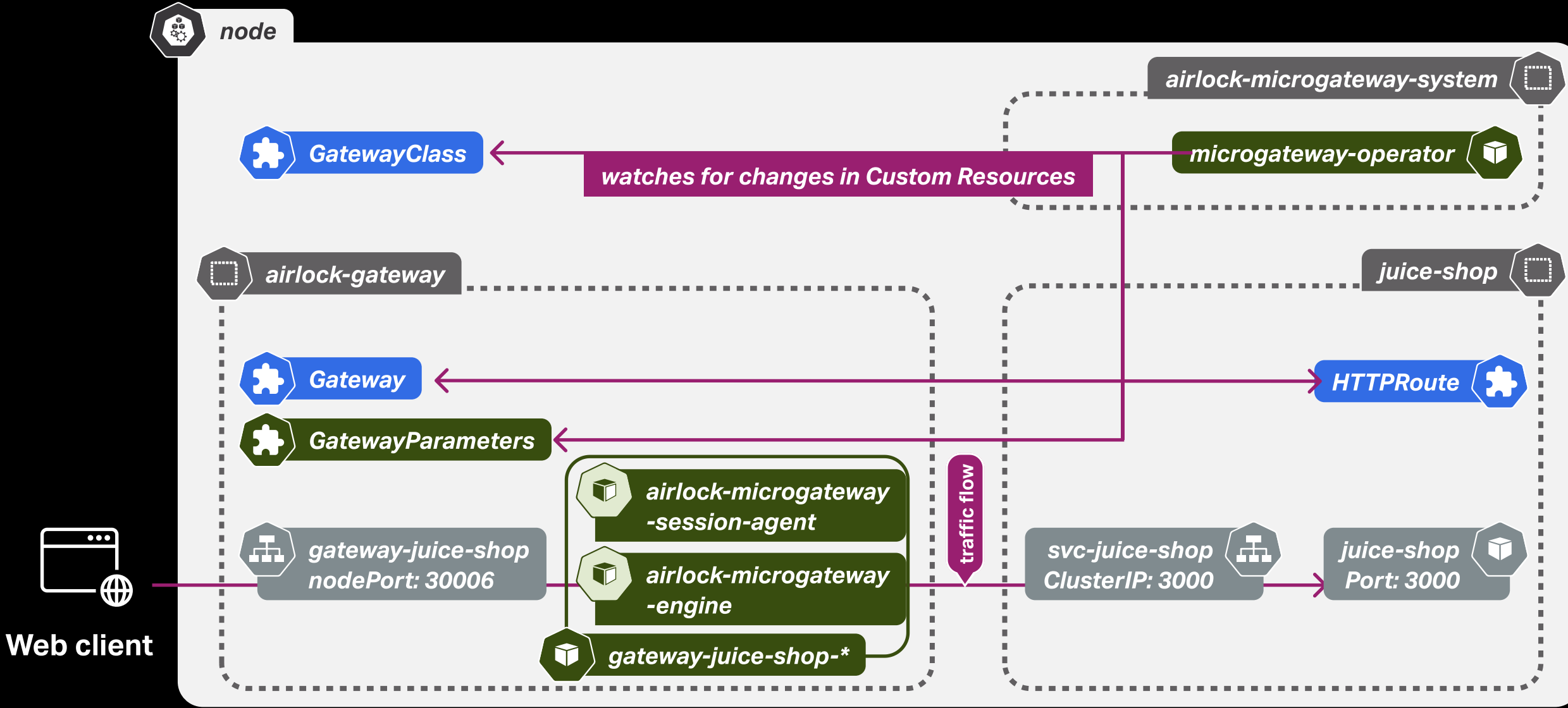
Deploy CRs to Configure Traffic between Airlock Microgateway and the Juice Shop Back-end



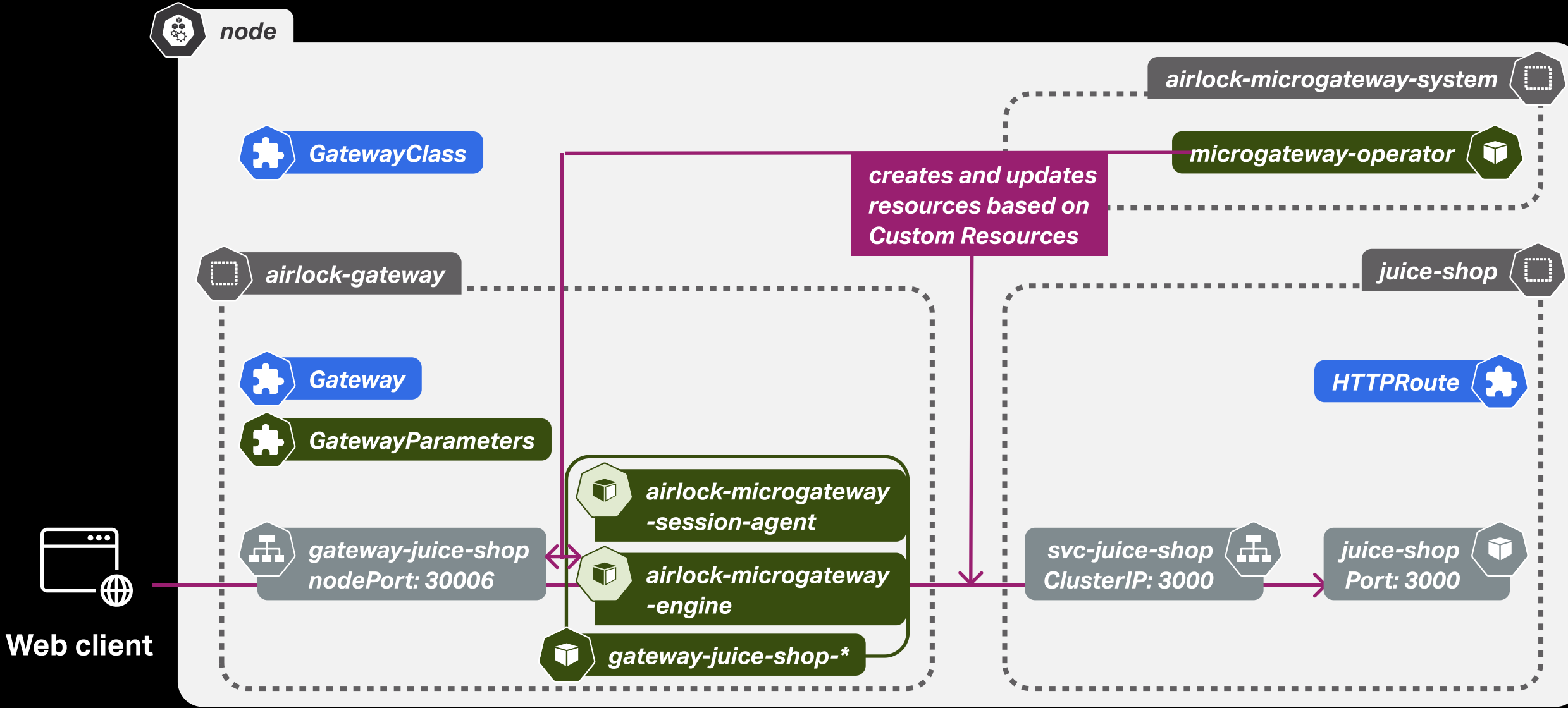
Deploy CRs to Configure Traffic between Airlock Microgateway and the Juice Shop Back-end



The Microgateway Operator Watches for Changes in CRs



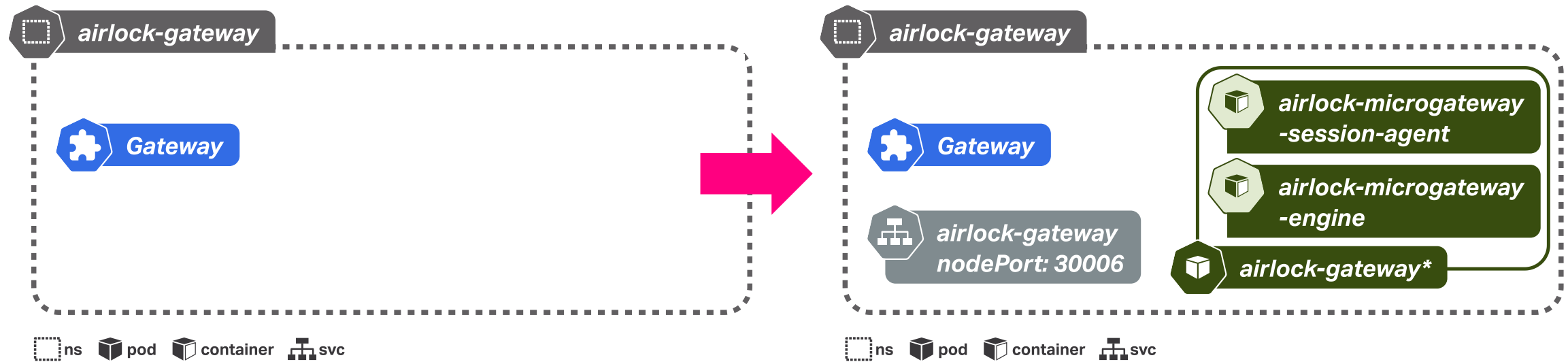
The Microgateway Operator Creates and Updates Resources based on the State of the Configured CRs



Recap – Airlock Microgateway Operator

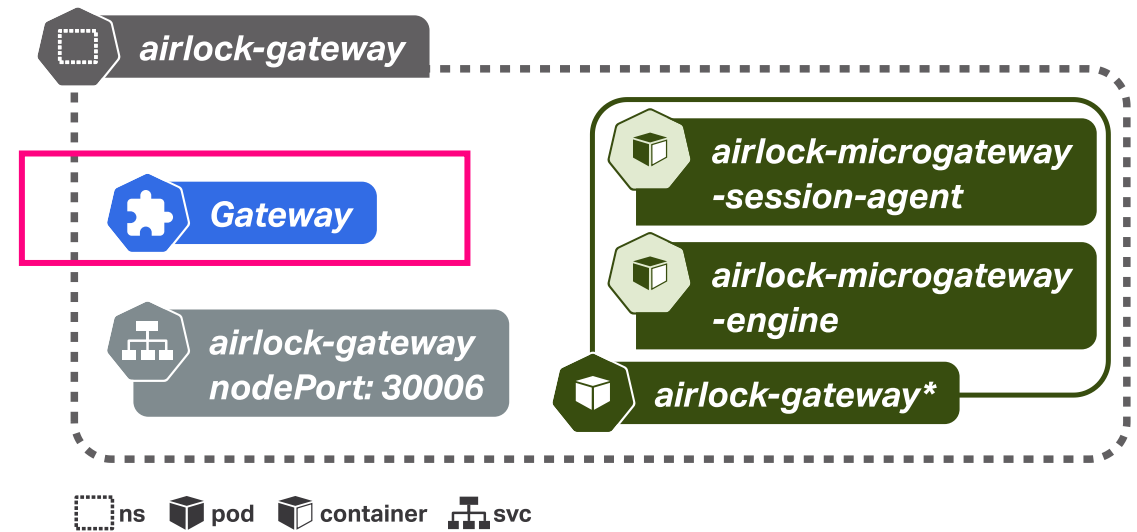
Watches for **changes** in K8s Gateway API CRs and Airlock Microgateway CRs

→ On change: **create / update resources**



Recap – Airlock Microgateway Operator

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: airlock-gateway
  namespace: airlock-gateway
spec:
  gatewayClassName: airlock-microgateway
  listeners:
    - name: http
      port: 30006
      protocol: HTTP
  allowedRoutes:
    namespaces:
      from: All
```

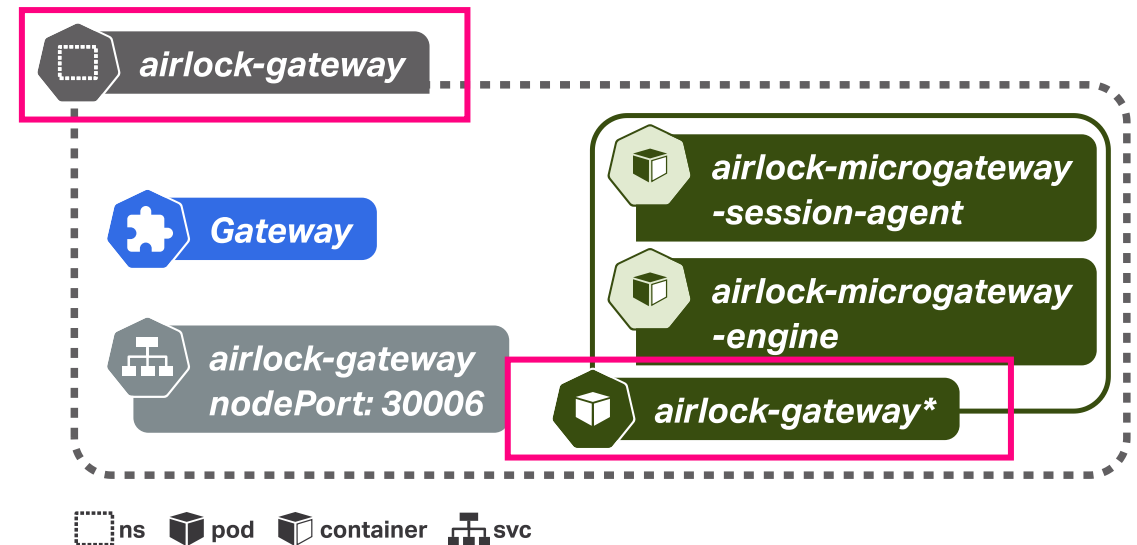


Recap – Airlock Microgateway Operator

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: airlock-gateway
  namespace: airlock-gateway
spec:
  gatewayClassName: airlock-microgateway
  listeners:
    - name: http
      port: 30006
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: All
```

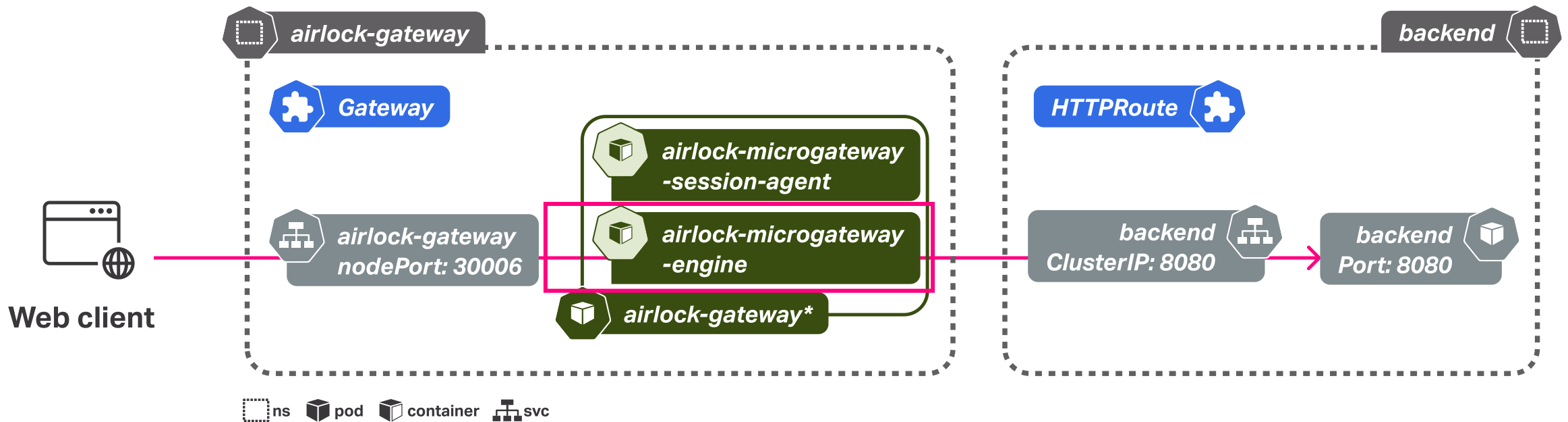
Pod name = based on Gateway CR name

Pod namespace = Gateway CR namespace



Recap – Airlock Microgateway Engine

Handles **filtering** and **AuthN, AuthZ, and Identity Propagation**

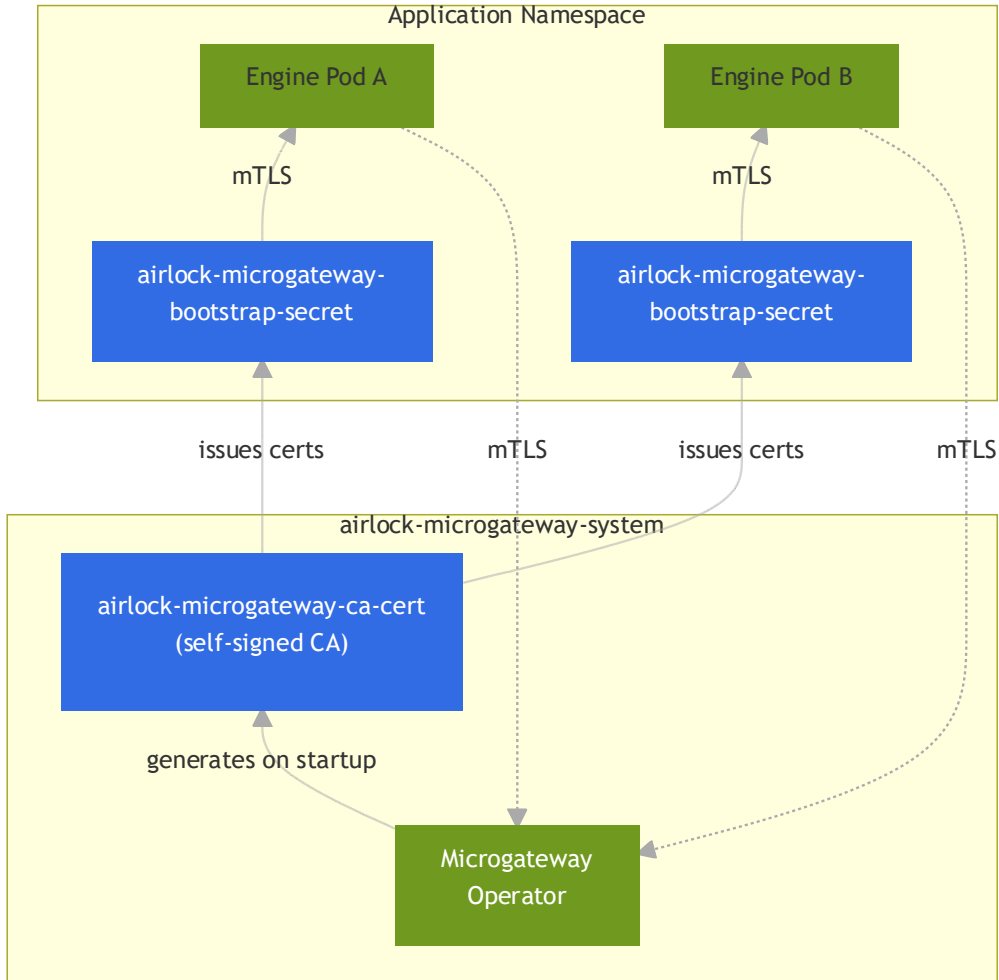


Airlock Microgateway Engine Communications



Any communication between Airlock Microgateway
Engine containers, is secured using **TLS/mTLS**

Internal mTLS Managed by the Microgateway Operator



self-signed CA

Secret: `airlock-microgateway-ca-cert`

Namespace: `airlock-microgateway-system`

bootstrap secret

Secret: `airlock-microgateway-bootstrap-secret`

Namespace... `...of engine deployment`

Contains... `...certificates + keys for mTLS`

Internal mTLS Managed by the Microgateway Operator

Certificates are **auto-renewed every 48 hours**

cert-manager only needed for
publicly trusted certificates on Ingress Gateways

Access to Hand-On Labs for the Training

Airlock Microgateway Training

Learn to setup and protect a web application with Airlock Microgateway 4.8!

Available content (7)

K8s

Setup Airlock Microgateway
Learn how to install Airlock Microgateway

View detailsStart

K8s

First Webapp
Learn how to secure a web application against security threats with Airlock Microgateway

K8s

Monitoring
Learn how to monitor blocked requests with Airlock Microgateway

K8s

Deny Rule Overrides and Exceptions
Learn how to configure Airlock Microgateway deny rule overrides and exceptions

K8s

Policy Enforcement
Learn how to enforce Airlock Microgateway settings with (Kyverno) policies

K8s

Session Handling
Learn how to Setup Session Handling with Airlock Microgateway

K8s

Authentication and Authorization with Entra ID
Learn how to Setup Authentication and Authorization with Airlock Microgateway and Microsoft Entra ID

View detailsStart

To access this content, please leave your details.
By submitting this form, you consent to Instruct sharing your details with Airlock.

First name*

Last name*

First name

Last name

Email address*

Email address

Company name

Company name

Job title

Job title

☐

Please agree to the following statement to access the Airlock Microgateway Labs for free: I acknowledge that any data or input I provide while using the Airlock Microgateway Labs may be processed in accordance with the [Airlock Microgateway Labs - Terms and Privacy Policy](#).

Submit and access

Please register, based on the **invite link** that you received in the calendar invitation for the training

Instruqt

instruqt

Deny Rule Overrides and Exceptions

Shell

Editor

Juice Shop

Grafana

MGW Docs

MGW API-Reference

Q-1

Q-2

airlock-mgw@kubernetes:~\$

Overview

Share

Invite details

Progress

Hide Instructions

30m

Instructions

Messages (1)

Introduction

This track is split into two *challenges* that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:

- Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
- Challenge 2: Create deny rule exceptions.

Use the figure below to get an **overview on the lab**.

Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass

Web client

Airlock Microgateway

Sidecarless

1 / 1

Base Setup

Skip

Check

Instruqt – Shell

instruqt

Deny Rule Overrides and Exceptions

Shell

Editor

Juice Shop

Grafana

MGW Docs

MGW API-Reference

Q-1

Q-2

```
airlock-mgw@kubernetes:~$
```

Overview

Share

Invite details

Progress

Hide Instructions

30m

Instructions

Messages (1)

Introduction

This track is split into two *challenges* that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:

- Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
- Challenge 2: Create deny rule exceptions.

Use the figure below to get an overview on the lab.

Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass

Web client

Airlock Microgateway

Sidecarless

Base Setup

Skip

Check

shell tab

Instruqt – Code Editor

The image shows the Instruqt interface, which is a platform for interactive learning. The left side features a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a file named `denyrule.yaml` under the `AIRLOCK-MGW` directory. The code editor displays the content of `denyrule.yaml`, which is a Kubernetes DenyRule configuration. The terminal shows the command `airlock-mgw@kubernetes:~$`.

The right side of the interface shows the lab overview. It includes a title bar with tabs for `Overview`, `Share`, `Invite details`, `Progress`, `Hide Instructions`, and a timer showing `26m`. Below the title bar, there are tabs for `Instructions` and `Messages (1)`. The `Instructions` tab is active, showing an `Introduction` section. The text in the introduction states: "This track is split into two challenges that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:".

- Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
- Challenge 2: Create deny rule exceptions.

Below the introduction, there is a section titled `Use the figure below to get an overview on the lab.` which contains a diagram. The diagram illustrates the architecture of the lab, showing a `Web client` connected to `Airlock Microgateway`, which is in turn connected to `Sidecarless Backend`. The diagram also shows a `DenyRule` configuration being applied to the `Airlock Microgateway`.

At the bottom of the interface, there is a `Base Setup` section with a `Skip` button and a `Check` button.

code editor tab

Instruqt – Back-ends, Microgateway Docs/CRD Reference

instruqt Deny Rule Overrides and Exceptions

Overview Share Invite details Progress Hide Instructions 26m

Instructions Messages (1)

Introduction

This track is split into two *challenges* that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:

- Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
- Challenge 2: Create deny rule exceptions.

Use the figure below to get an overview on the lab.

Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass

Diagram: A diagram showing a Web client connected to an Airlock Microgateway, which is connected to a Juice Shop Backend.

Base Setup

Skip Check

Back-ends, Microgateway Docs and CRD Reference

Instruqt – Reflective Questions

instruqt

Deny Rule Overrides and Exceptions

Shell

Editor

Juice Shop

Grafana

MGW Docs

MGW API-Reference

Q-1

Q-2

Question 1/2: Overrides in a DenyRules Custom Resource allow it to...

override the builtIn settings for Airlock Microgateway's default deny rules.

create and add new custom deny rule groups (ruleKeys) that do not exist in Airlock Microgateway.

change the system-wide evaluation order of deny rule ruleKeys and types.

reflective questions

Submit your Solution

Overview

Share

Invite details

Progress

Hide Instructions

23m

Instructions

Messages (1)

Introduction

This track is split into two challenges that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:

- Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
- Challenge 2: Create deny rule exceptions.

Use the figure below to get an overview on the lab.

Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass

```
graph LR; WC[Web client] --> AM[Airlock Microgateway]; AM --> SAM[Sidecarless Airlock Microgateway];
```

Base Setup

Skip

Check

Instruqt – Progress Overview

The screenshot displays the Instruqt interface for a lab titled "Deny Rule Overrides and Exceptions". The interface is divided into two main sections: a left sidebar and a right main content area.

Left Sidebar:

- Top bar: "instruqt Deny Rule Overrides and Exceptions"
- Navigation tabs: Shell, Editor, Juice Shop, Grafana, MGW Docs, MGW API-Reference, Q-1, Q-2
- Terminal output: `airlock-mgw@kubernetes:~$`

Right Main Content Area:

- Top bar: Overview, Share, Invite details, Progress (highlighted with a pink arrow), Hide Instructions, 30m
- Instructions tab: Messages (1)
- Introduction section:
 - Text: "This track is split into two challenges that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implementation."
 - Challenges:
 - Challenge 1: Create deny rule overrides for GraphQL, JSON, etc.
 - Challenge 2: Create deny rule exceptions for GraphQL, JSON, etc.
 - Text: "Use the figure below to get an overview of the architecture."
- Diagram section:
 - Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass
 - Diagram: A flow diagram showing a "Web client" sending traffic to an "Airlock Microgateway" (represented by a green circle with a plus sign), which then forwards traffic to a "Back End Service" (represented by a yellow house icon).
- Base Setup section: A dropdown menu with "Skip" and "Check" buttons.

access **Progress**
overview

Instruqt – Progress Overview

The screenshot displays the Instruqt interface for the 'Deny Rule Overrides and Exceptions' track. The top navigation bar includes links for Shell, Editor, Juice Shop, Grafana, MGW Docs, MGW API-Reference, Q-1, and Q-2. The main terminal window shows the command prompt 'airlock-mgw@kubernetes:~\$'. The right-hand 'Progress' panel lists two challenges: 'Reduce False Positives - Part I' (unlocked) and 'Reduce False Positives - Part II' (locked). A red arrow points to the locked challenge, and a callout box states 'Possible to Skip challenges'.

Progress Overview:

- Reduce False Positives - Part I** (Unlocked): Create Deny Rule Overrides related to Rule Groups and Attribute Types
- Reduce False Positives - Part II** (Locked): Create Deny Rule Exceptions

Instructions:

This track is split into rule overrides and exceptions (MGW)

- Challenge 1: Create attribute types (Gr)
- Challenge 2: Create

Use the figure below to

Base Setup

Restart **Stop & Exit**

Instruqt – Possibility to Skip

The screenshot displays the Instruqt interface for a track titled "Deny Rule Overrides and Exceptions". The interface includes a top navigation bar with tabs for Shell, Editor, Juice Shop, Grafana, MGW Docs, MGW API-Reference, Q-1, and Q-2. A terminal window shows the command prompt "airlock-mgw@kubernetes:~\$".

A dialog box titled "Skip to challenge?" is open, containing the text: "To skip to a challenge, we start the track and run scripts to solve every challenge leading up to the one you're skipping to." Below the text are "Cancel" and "Skip" buttons.

On the right side, a "Progress" panel shows a list of challenges. The first challenge, "Reduce False Positives - Part I", is highlighted in purple. The second challenge, "Reduce False Positives - Part II", is highlighted in red and has a lock icon, indicating it is the current challenge. A large red arrow points from the "Skip" button in the dialog box to the "Reduce False Positives - Part II" challenge.

A white box with a red border contains the text "Possible to Skip challenges".

Instruqt – Instructions

instruqt

Deny Rule Overrides and Exceptions

Shell

Editor

Juice Shop

Grafana

MGW Docs

MGW API-Reference

Q-1

Q-2

airlock-mgw@kubernetes:~\$

Overview

Share

Invite details

Progress

Hide Instructions

30m

Instructions

Messages (1)

Introduction

This track is split into two *challenges* that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:

- Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
- Challenge 2: Create deny rule exceptions.

Use the figure below to get an **overview on the lab**.

Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass

Web client

Airlock Microgateway

Sidecarless

1 / 1

instructions tab

Skip

Check

Airlock Microgateway Connaisseur – An Airlock Academy Training

AIRLOCK®

89

Instruqt – Interactive Figures

The screenshot displays the Instruqt interface for a track titled "Deny Rule Overrides and Exceptions". The top navigation bar includes links for Overview, Share, Invite details, Progress, Hide Instructions, and a 30-minute timer. Below the navigation bar, there are tabs for Shell, Editor, Juice Shop, Grafana, MGW Docs, MGW API-Reference, Q-1, and Q-2. The main terminal window shows the prompt `airlock-mgw@kubernetes:~$`. On the right side, there is a sidebar with an "Introduction" section. A pink callout box with the text "Possible to **view** interactive figures in full-screen" points to a diagram in the sidebar. The diagram, titled "Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass", shows a "Web client" connected to an "Airlock Microgateway" (represented by a green circle with a white 'X') and an "API-based sidecarless Airlock" (represented by a yellow house icon). The diagram is part of a sequence of 1/1 slides. At the bottom of the sidebar, there is a "Base Setup" section with "Skip" and "Check" buttons.

instruqt Deny Rule Overrides and Exceptions

Overview Share Invite details Progress Hide Instructions 30m

Shell Editor Juice Shop Grafana MGW Docs MGW API-Reference Q-1 Q-2

airlock-mgw@kubernetes:~\$

Instructions Messages (1)

Introduction

This track is split into two challenges that will show you how to configure deny

by API-based sidecarless Airlock

ns of deploying them:

the groups and limit them to

b.

Possible to **view** interactive figures in full-screen

Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass

Web client

Airlock Microgateway

API-based sidecarless Airlock

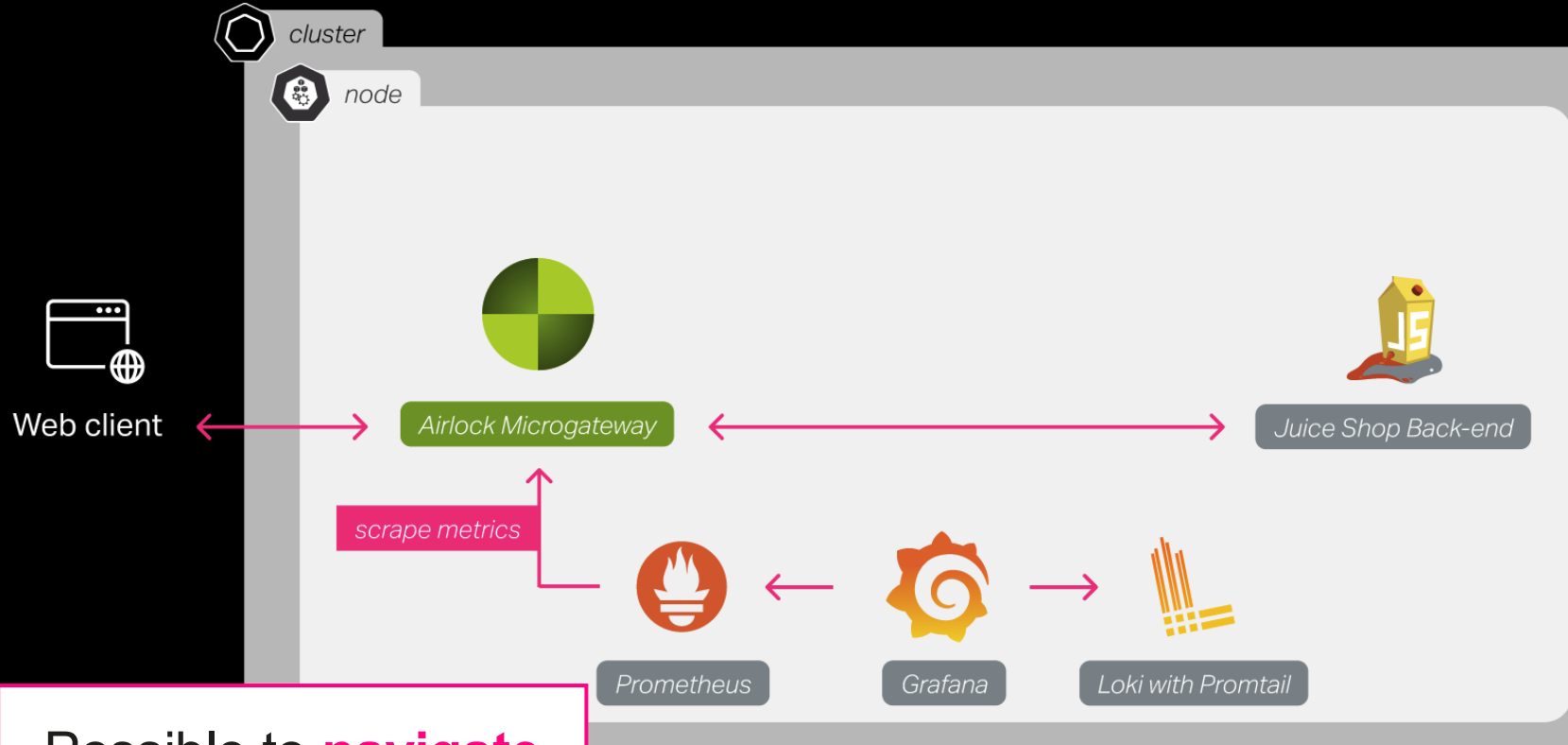
1/1

Base Setup

Skip Check

Instruqt – Interactive Figures Full-Screen

Assumed Foundation: You Should be Able to Setup Airlock Microgateway with a Backend (e.g., Juice Shop)



Possible to **navigate**
interactive figures

next >

1 of 5

Instruqt – Click Check to Check for Correctness and...

The screenshot shows the Instruqt interface for a lab titled "Deny Rule Overrides and Exceptions". The interface is divided into two main sections: a terminal on the left and a lab overview on the right.

Terminal Section:

- Top bar: Shell, Editor, Juice Shop, Grafana, MGW Docs, MGW API-Reference, Q-1, Q-2.
- Terminal output: `airlock-mgw@kubernetes:~$`

Lab Overview Section:

- Top bar: Overview, Share, Invite details, Progress, Hide Instructions, 30m.
- Instructions tab: Introduction.
- Introduction text: "This track is split into two challenges that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:"
- Challenges:
 - Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
 - Challenge 2: Create deny rule exceptions.
- Text: "Use the figure below to get an overview on the lab."
- Diagram: A diagram showing the lab setup. It includes a "Web client" connected to an "Airlock Microgateway" (MGW), which is connected to a "Backend Service". The MGW is also connected to a "Deny Rule" component.
- Bottom bar: Base Setup, Skip, Check.

A pink box highlights the "Check" button at the bottom right of the lab overview, with the text "Possible to **check** for correctness" overlaid on it.

Instruqt – ...receive Feedback

The screenshot displays the Instruqt interface for a lab titled "Deny Rule Overrides and Exceptions". The left pane shows a terminal with the prompt `airlock-mgw@kubernetes:~$`. The right pane contains the lab instructions, which are split into two challenges: "Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc)." and "Challenge 2: Create deny rule exceptions." Below the instructions is a diagram showing a "Web client" connected to "Airlock Microgateway", which is connected to "Juice Shop Backend". A "Focus of This Lab" box states: "Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass". At the bottom of the right pane, a feedback message is displayed: "Not quite right, try again HTML override at level 'Basic' not found. Did you add an override for ruleKeys: [HTML] and apply the file?". The message includes a warning icon and a close button. Below the message are "Skip" and "Check" buttons.

provides **feedback** if solution is not correct, yet

Instruqt – Instructor Messages / Replies

Instructor may provide additional LAB information for all participants

Overview Share Invite details Progress Hide Instructions 17m

Messages (1)

Introduction

This track is split into two *challenges* that will show you how to configure deny rule overrides and exceptions with a K8s Gateway API-based sidecarless Airlock Microgateway (MGW) and inspect the implications of deploying them:

- Challenge 1: Create deny rule overrides for rule groups and limit them to attribute types (GraphQL, JSON, etc).
- Challenge 2: Create deny rule exceptions.

Use the figure below to get an **overview on the lab**.

Focus of This Lab: Configure DenyRule Overrides and Exceptions to Allow Traffic that is Blocked but Should Pass

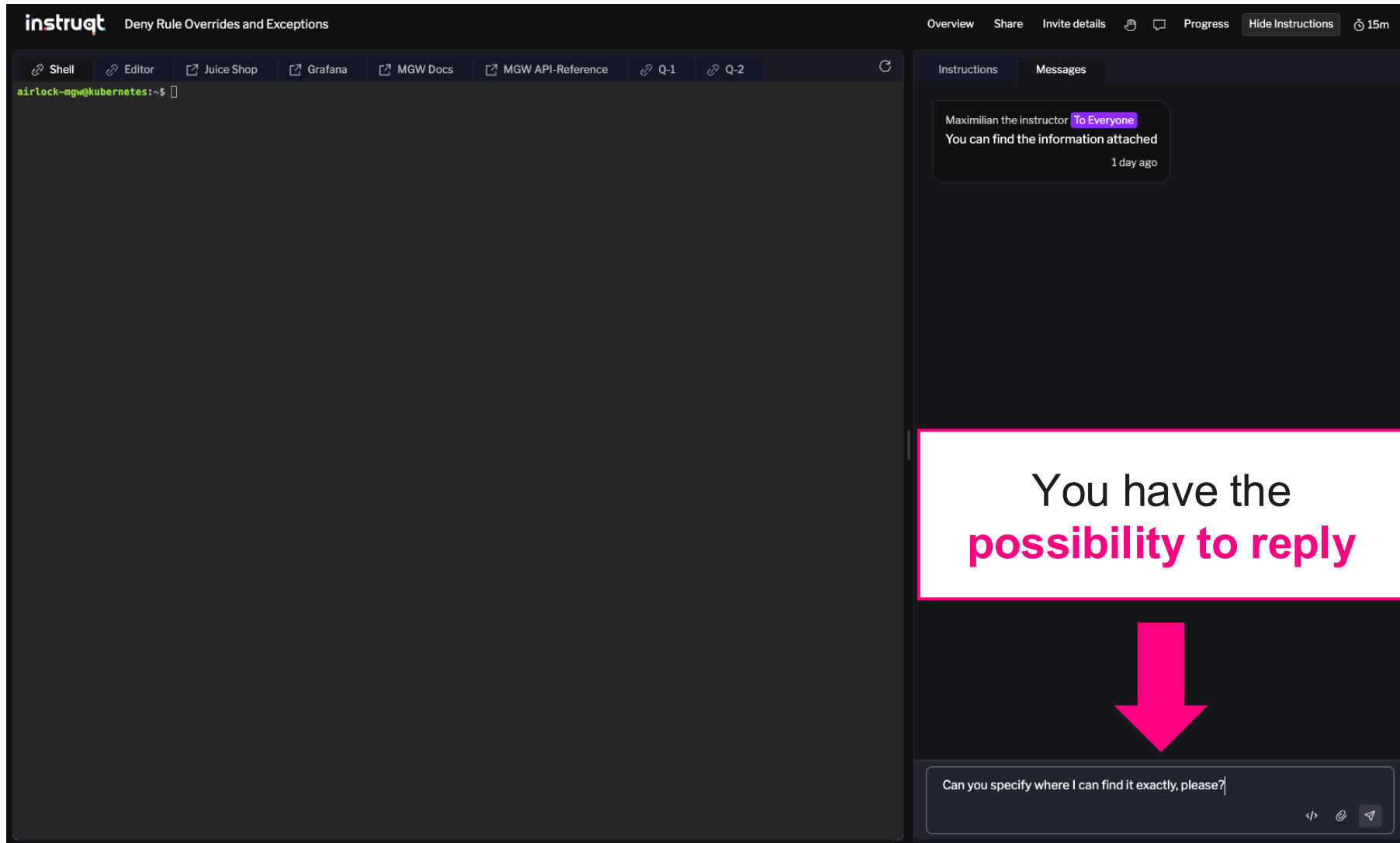
Diagram illustrating the lab architecture:

```
graph LR
    subgraph cluster
        direction TB
        node1(( )) --> node2(( ))
        node2 --> node3(( ))
    end
    WebClient[Web client] --> Airlock[Airlock Microgateway]
    Airlock --> JuiceShop[Juice Shop Backend]
```

Not quite right, try again
HTML override at level 'Basic' not found. Did you add an override for ruleKeys: [HTML] and apply the file?

Skip Check

Instruqt – Instructor Messages / Replies



The screenshot shows the Instruqt interface. On the left is a terminal window with the prompt `airlock-mgw@kubernetes:~$`. The top navigation bar includes links for Shell, Editor, Juice Shop, Grafana, MGW Docs, MGW API-Reference, Q-1, and Q-2. The right sidebar has tabs for Overview, Share, Invite details, Progress, and Hide Instructions. The Messages tab is active, displaying a message from Maximilian the instructor to everyone, stating that information is attached and dated 1 day ago. A large pink arrow points down to a reply box containing the text "Can you specify where I can find it exactly, please?".

instruqt Deny Rule Overrides and Exceptions

Overview Share Invite details Progress Hide Instructions 15m

Shell Editor Juice Shop Grafana MGW Docs MGW API-Reference Q-1 Q-2

airlock-mgw@kubernetes:~\$

Instructions Messages

Maximilian the instructor **To Everyone**
You can find the information attached
1 day ago

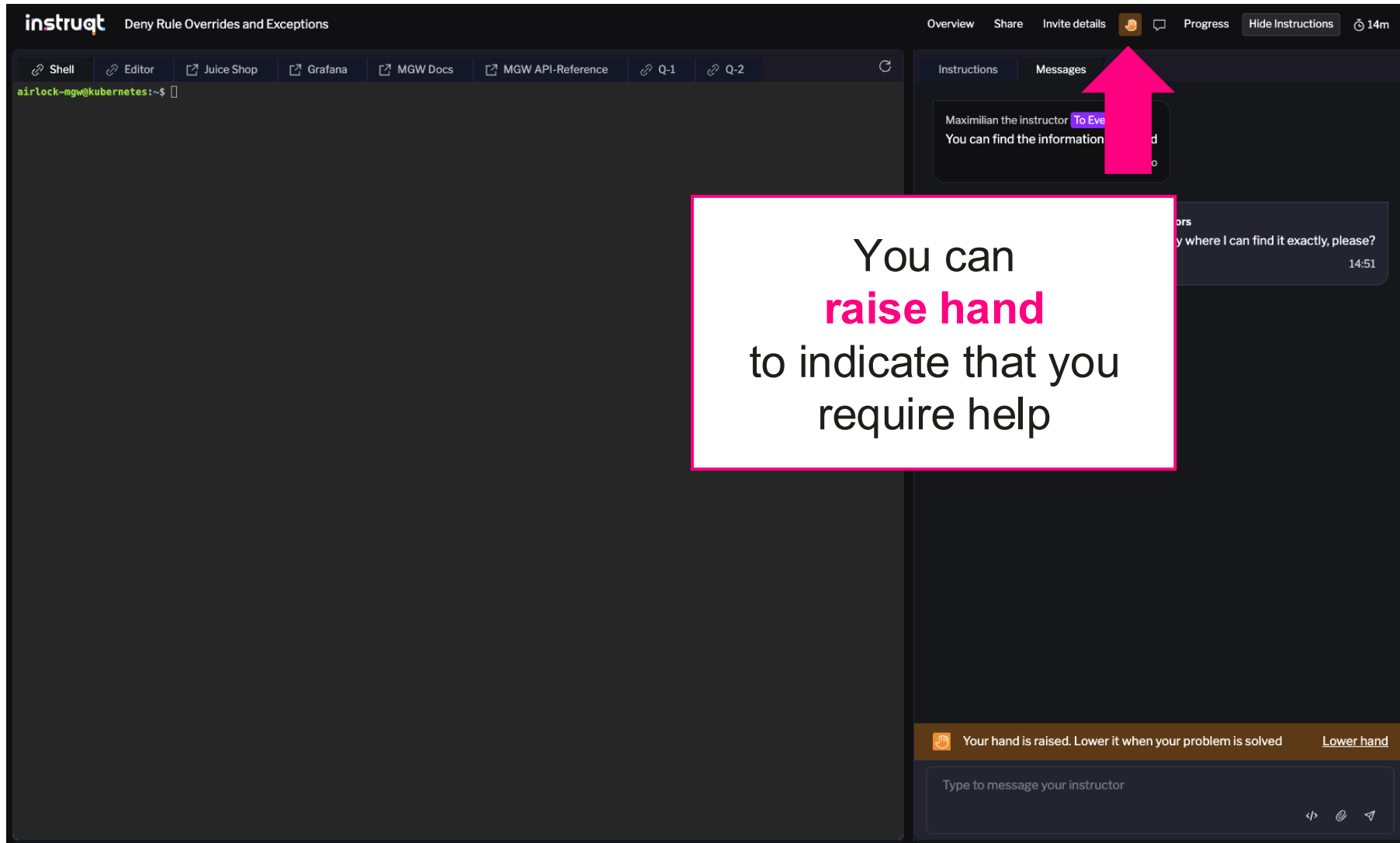
You have the
possibility to reply

Can you specify where I can find it exactly, please?

Instruqt – Instructor Messages / Replies

The screenshot displays the Instruqt web interface. The top navigation bar includes the 'instruqt' logo, the title 'Deny Rule Overrides and Exceptions', and several utility links: Overview, Share, Invite details, Progress, Hide Instructions, and a timer showing 14m. Below the navigation bar is a tabbed interface with tabs for Shell, Editor, Juice Shop, Grafana, MGW Docs, MGW API-Reference, Q-1, and Q-2. The 'Shell' tab is active, showing a terminal window with the prompt 'airlock-mgw@kubernetes:~\$'. To the right of the terminal is a sidebar with two tabs: Instructions and Messages. The Messages tab is selected, showing a list of messages. The first message is from 'Maximilian the instructor' with the subject 'To Everyone' and the text 'You can find the information attached', dated '1 day ago'. The second message is from 'You - To Instructors' with the text 'Can you specify where I can find it exactly, please?' and a timestamp of '14:51'. At the bottom of the sidebar is a text input field with the placeholder 'Type to message your instructor' and three icons for sending, attaching, and deleting.

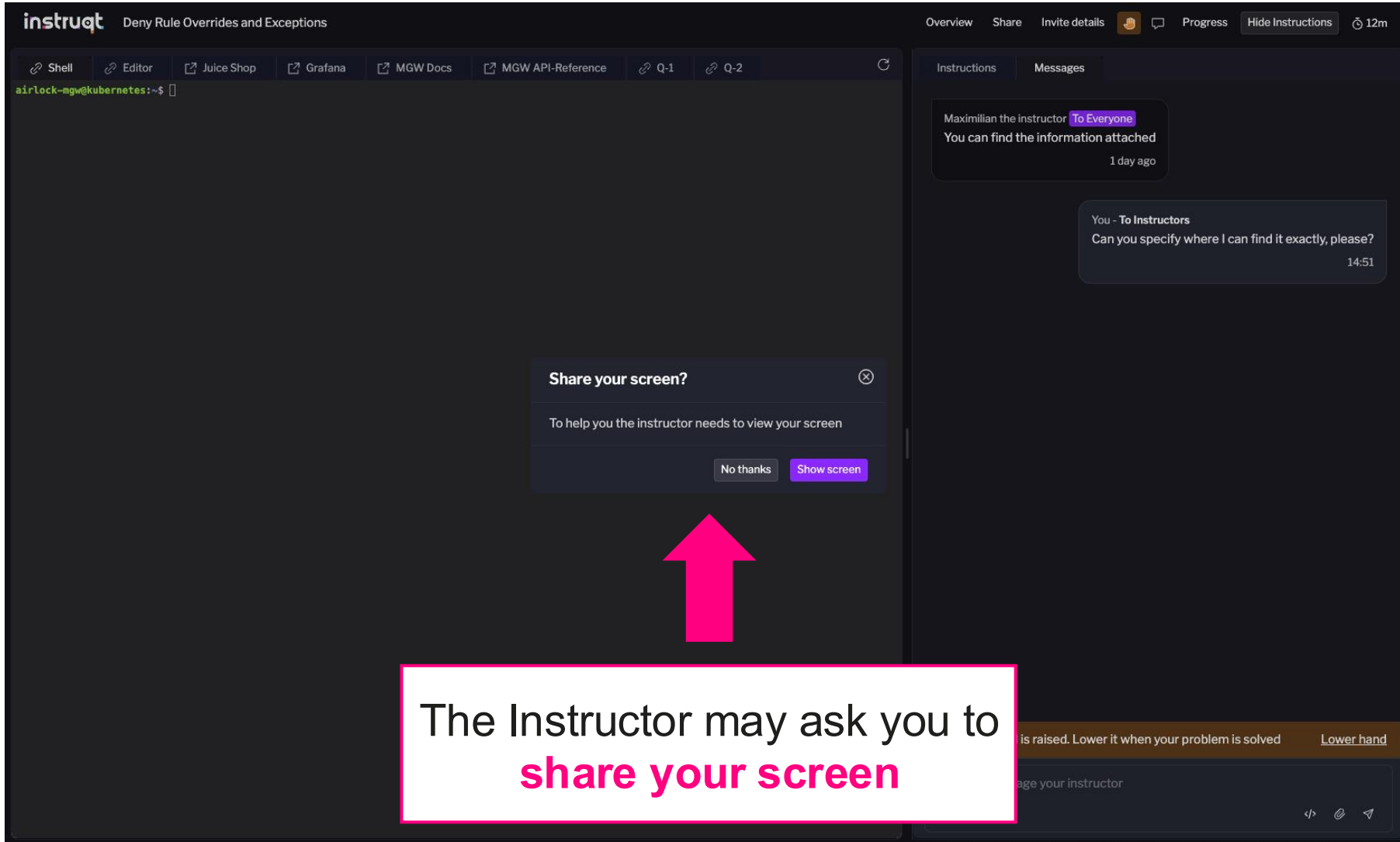
Instruqt – Request Help



The screenshot shows the Instruqt interface with a terminal window on the left and a chat area on the right. The terminal window displays the prompt `airlock-mgw@kubernetes:~$`. The chat area has tabs for 'Instructions' and 'Messages'. A pink arrow points to the 'raise hand' icon in the chat area. A text box in the center of the screen explains that this icon is used to request help.

You can **raise hand** to indicate that you require help

Instruqt – Possible to Allow Screen Share



The screenshot displays the Instruqt web application interface. At the top, the 'instruqt' logo is visible, followed by the title 'Deny Rule Overrides and Exceptions'. The navigation bar includes links for 'Overview', 'Share', 'Invite details', 'Progress', 'Hide Instructions', and a timer showing '12m'. Below the navigation bar, there are tabs for 'Shell', 'Editor', 'Juice Shop', 'Grafana', 'MGW Docs', 'MGW API-Reference', 'Q-1', and 'Q-2'. The 'Shell' tab is active, showing a terminal window with the prompt 'airlock-mgw@kubernetes:~\$'. A modal dialog box titled 'Share your screen?' is centered on the screen. The dialog contains the text 'To help you the instructor needs to view your screen' and two buttons: 'No thanks' and 'Show screen'. A large pink arrow points upwards from a text box at the bottom towards the 'Show screen' button. The text box contains the text 'The Instructor may ask you to share your screen'. On the right side of the interface, there is a 'Messages' panel with two messages. The first message is from 'Maximilian the instructor' to 'Everyone', stating 'You can find the information attached' and '1 day ago'. The second message is from 'You - To Instructors', asking 'Can you specify where I can find it exactly, please?' and '14:51'. At the bottom of the interface, there is a status bar with the text 'is raised. Lower it when your problem is solved' and a link 'Lower hand'. Below the status bar, there is a search bar with the placeholder text 'Search your instructor' and some icons.

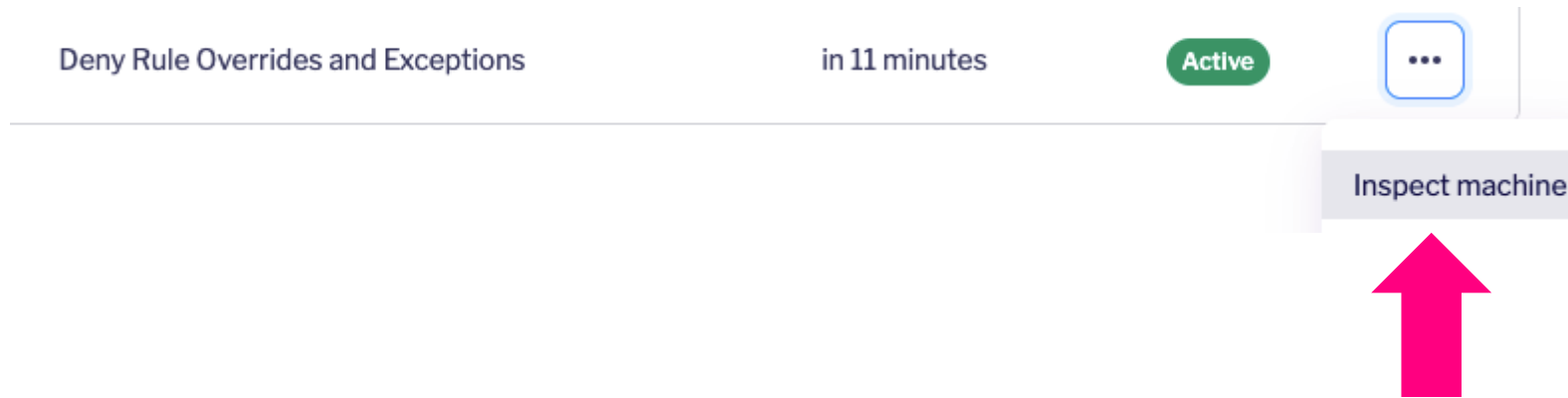
Share your screen?

To help you the instructor needs to view your screen

No thanks Show screen

The Instructor may ask you to
share your screen

Instruqt – Instructors can Inspect Your Machine to Help



The Instructor may also **inspect your machine** to help you with your configuration

Instruqt – Headlamp

ShellEditorHeadlampJuice ShopGrafanaMGW DocsMGW API-Reference

airlock-mgw@kubernetes:~\$

Headlamp tab

Headlamp auth token

Instructions

Goals of Challenge 1

Headlamp

You can access a graphical overview of the cluster resources via [Headlamp](#). Use the following token to login:

bash

eyJhbGciOiJIUzI1NiIsImtpZCI6IjVZNUY3dXFYTktZ1lRdXBNeHM1MkJudl

Pods
23 / 23
Requested

100.0 %

Nodes
1 / 1 Ready

100.0 %

☐	Name ↑↓	Namespace ↑↓	Class Name ↑↓	Conditions ↑↓
☐	gateway-juice-shop	airlock-gateway	airlock-microgateway	Accepted

Headlamp

Authentication

Please paste your authentication token.

ID token

.....

Check out how to generate a [service account token](#).

Cluster settings

Authenticate

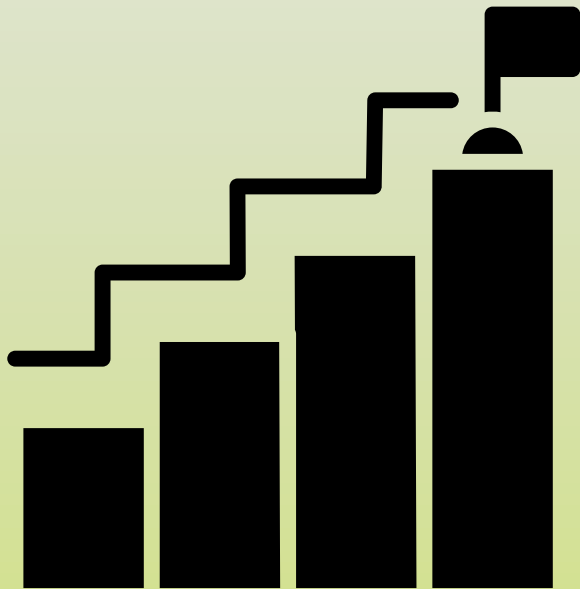
Lab 00

Explore Airlock Microgateway



Lab 00

Explore Airlock Microgateway



30 min

Steps

- Explore Webapp Protection
- Explore OIDC (optional)

Resources



Where can you find **documentation** and **release information** about Airlock Microgateway and its dependencies?

Microgateway Documentation
Microgateway CRD Reference
Release Notes
Incubating Features



Resources

Microgateway Documentation

Microgateway CRD Reference

Release Notes

Incubating Features



Resources

Microgateway Documentation

Airlock Microgateway

Version	Documentation	CRD Reference
5.0	Documentation	CRD Reference
4.8	Documentation	CRD Reference
4.7	Documentation	CRD Reference

See: <https://docs.airlock.com>

Microgateway Documentation

 Airlock Microgateway Documentation 5.0

CRD Reference [↗](#)

About 

License editions

Online labs

Running Example

Release and support 

System architectures

Getting started

Requirements and limitations 

Installation and upgrade 

Configuration 

Use cases 

Tooling 

Troubleshooting and support 

Reference documentation 

Airlock Microgateway

Modern application security lives inside your development workflow and runs natively in your Kubernetes clusters. Airlock Microgateway delivers exactly that: a powerful Web Application and API Protection that filters malicious requests, enforces authentication, and protects web applications and APIs without friction in your development lifecycle.

What is Airlock Microgateway?

Airlock Microgateway is a modern Web Application and API Protection (WAAP) solution - formerly known as Web Application Firewall (WAF) - and an identity-aware proxy. Out of the box, it provides:

- Secure single sign-on (SSO) experience for users, based on open standards.
- Filtering incoming requests to protect against OWASP Top 10 attacks.
- Ingress in Kubernetes with [Kubernetes Gateway API](#) support - the successor of the frozen [Kubernetes Ingress](#) project.
- Frictionless integration into enterprise environments and Dev(Sec)Ops processes, with native support for GitOps tools and standard deployment workflows.

It's enterprise-grade, secure by default, developer-friendly, flexible, and highly extensible. Rely on proven security and focus on your core business.

Who is Airlock Microgateway for?

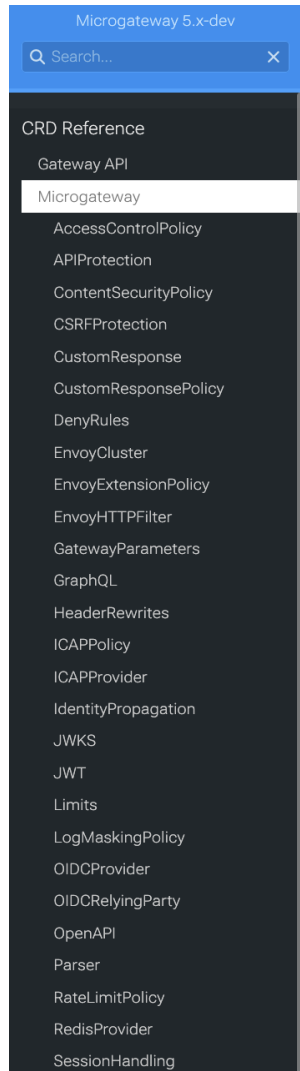
Airlock Microgateway is used by teams and organizations that need strong security for their web applications and APIs, including:

Microgateway Documentation
Microgateway CRD Reference
Release Notes
Incubating Features



Resources

Microgateway CRD Reference



Microgateway

Reference documentation for Microgateway custom resource definitions

AccessControlPolicy

API Documentation for AccessControlPolicy

APIProtection

API Documentation for APIProtection

ContentSecurityPolicy

API Documentation for ContentSecurityPolicy

CSRFProtection

API Documentation for CSRFProtection

CustomResponse

API Documentation for CustomResponse

CustomResponsePolicy

API Documentation for CustomResponsePolicy

DenyRules

API Documentation for DenyRules

EnvoyCluster

API Documentation for EnvoyCluster

EnvoyExtensionPolicy

API Documentation for EnvoyExtensionPolicy

EnvoyHTTPFilter

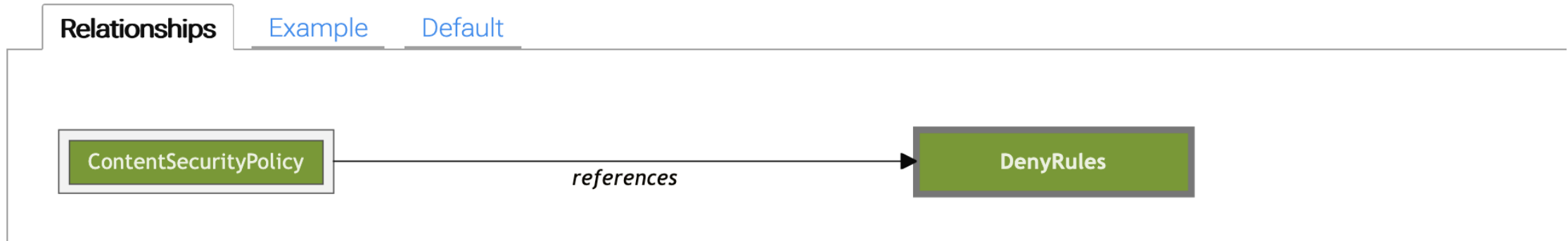
API Documentation for EnvoyHTTPFilter

[https://docs.airlock.com/microgateway/\[ver\]/index/api/crds](https://docs.airlock.com/microgateway/[ver]/index/api/crds)

i. e., **[ver]** could be **5.1**

CRD = Custom Resource Definition

Graphical Display of Relationships



Examples for Custom Configuration of Resources

Relationships

Example

Default

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: deny-rules-example
spec:
  request:
    builtIn:
      settings:
        # Use the deny rules in security level 'Strict'
        level: Strict
        # Explicitly set the 'threatHandlingMode' to 'Block'
        threatHandlingMode: Block
    overrides:
      # Set the deny rule security level to 'Standard' for
      # the deny rule 'XSS' if it is applied to request parameters.
      - conditions:
          ruleKeys:
            - XSS
          types:
            - Parameter
          settings:
            level: Standard
```

Default Configuration of Resources

Relationships

Example

Default

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: default
spec:
  request:
    builtIn:
      settings:
        level: Standard
        threatHandlingMode: Block
    custom: {}
```

Field Descriptions and Default Values

DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.exceptions[].requestConditions
DenyRules.spec.request.builtIn.overrides[]
DenyRules.spec.request.builtIn.overrides[].conditions
DenyRules.spec.request.builtIn.overrides[].settings
DenyRules.spec.request.builtIn.settings
DenyRules.spec.request.custom
DenyRules.spec.request.custom.rules[]
DenyRules.spec.request.custom.rules[].blockData

DenyRules.spec.request.builtIn.settings [🔗](#)

Field	Type	Description	Required	Default	Allowed Values
<code>level</code>	<i>enum</i>	Level represents a set of deny rules with different filter strengths.	no	<code>Standard</code>	<code>Basic</code> , <code>Standard</code> , <code>Strict</code> , <code>Unfiltered</code>
<code>threatHandlingMode</code>	<i>enum</i>	ThreatHandlingMode specifies how threats should be handled when a deny rule matches.	no	<code>Block</code>	<code>Block</code> , <code>LogOnly</code>

Microgateway Documentation
Microgateway CRD Reference
Release Notes
Incubating Features



Resources

Release Notes

Release description

Airlock Microgateway helps you to protect your services and APIs from unauthorized or malicious access with little effort. It is a lightweight and Kubernetes-native Web Application and API Protection (WAAP) solution designed to overcome the DevSecOps obstacles and to implement ZeroTrust.

Main new features

- Community Edition offers license free Gateway API features
- Flexible licenses for Authentication and Filtering modules
- Gateway API extensions
 - ListenerSet
 - Regex path matching
- Request frequency filter

[https://github.com/airlock/
microgateway/releases](https://github.com/airlock/microgateway/releases)

Microgateway Documentation
Microgateway CRD Reference
Release Notes
Incubating Features



Resources

Incubating Features are Version Dependent!



Check the Microgateway (MGW) documentation to determine **incubating features** for your MGW version

HTTP/3 Downstream Connections (Gateway-API based)



HTTP/3 connection between
the client and the Gateway

Limitations

No support for QUIC connection migration + upstream HTTP/3

HTTPRoute Retry

Configure to Retry Unsuccessful Requests to Backends

Limitation



requires Gateway API experimental channel (version **1.6**)

Foundational Components



What are the **foundational components** of Airlock
Microgateway and how are they **related**?

Editions and Licensing
Deployment Forms
Requirements and Limitations



**Foundational
Components**

Editions and Licensing

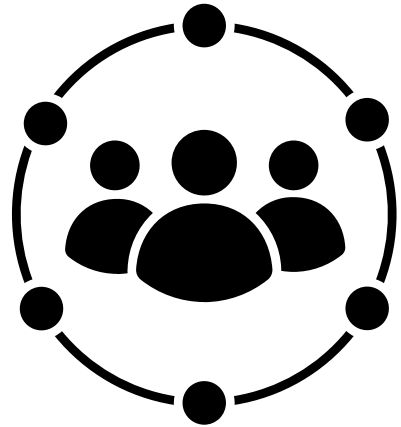
Deployment Forms

Requirements and Limitations



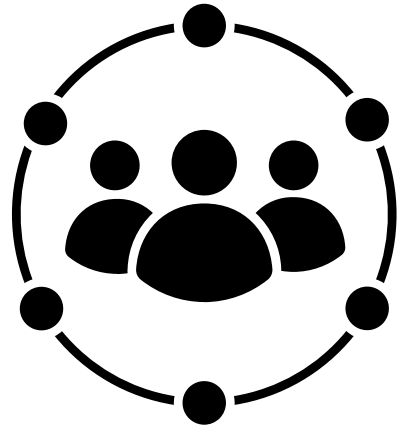
**Foundational
Components**

Editions

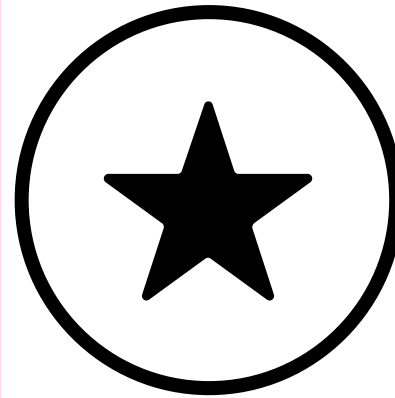


Community Edition

Editions



Community Edition



Premium Edition



with Airlock Microgateway 5.1

- Free
- Unlimited
- No license required
- Features
 - K8s Gateway API
 - Selected Airlock Features
- Use-Case: **Ingress NGINX replacement**

Base

Community Support

new

Evaluation License

- Expiry date
- Throughput is limited
- 2 replicas



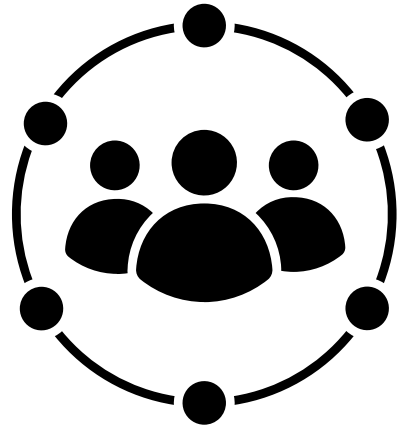
Filtering

Authentication

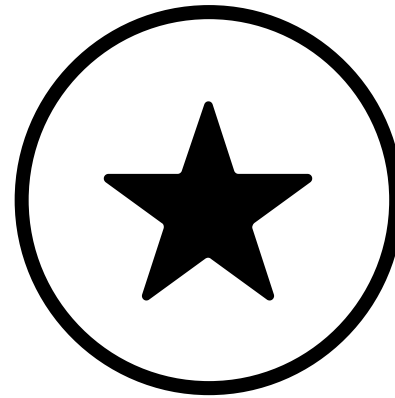
Base

Community Support

Editions



Community Edition



Premium Edition

<https://airlock.com/microgateway-premium>



with Airlock Microgateway 5.1

- Features
 - K8s Gateway API
 - Selected Airlock Features
- Use-Case: **Ingress NGINX replacement**
- **Airlock support**





+ For enterprise setups

+ Includes all licensed features

Available modules:

- *Microgateway Filtering*
- *Microgateway Authentication*

+ NO throughput enforcement

+ Up to unlimited requests per month
(licensing based on requests per calendar month)

+ Unlimited replica count per Gateway

+ Airlock support

Premium Edition – Please Monitor Your Throughput!



Licensing model: trust-based

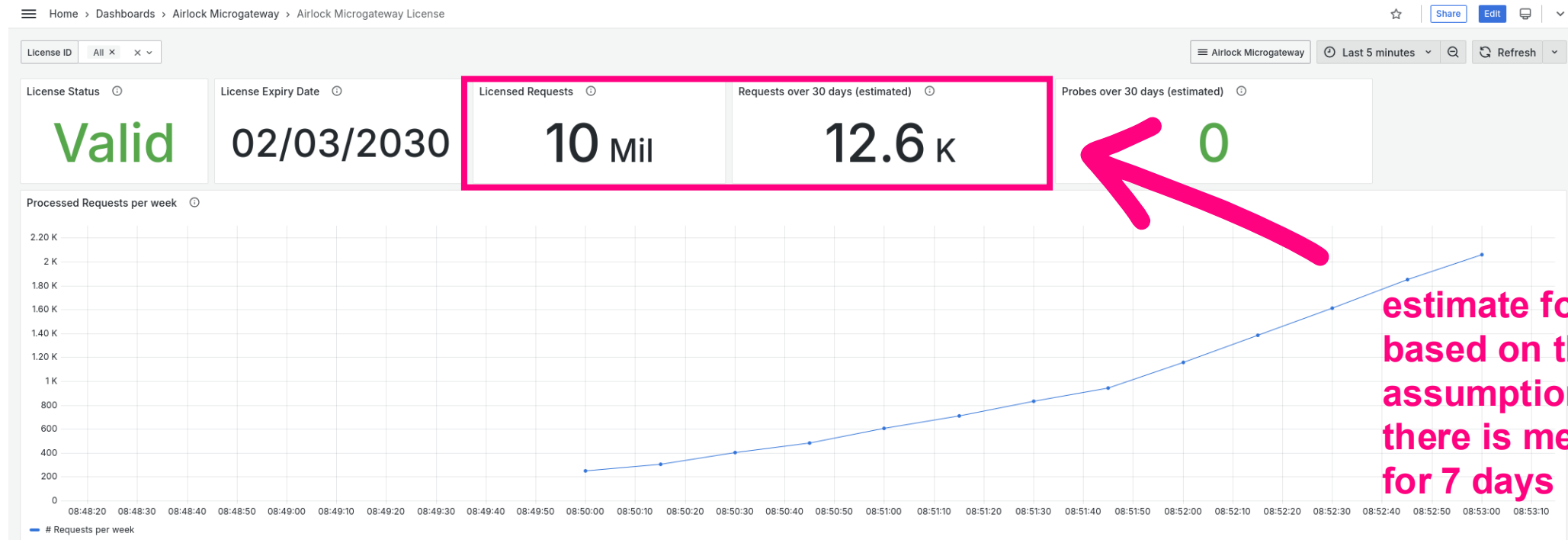
License type: calendar-based **monthly request rates**

Please upgrade license if: **number of requests > your licensed count**

Premium Edition – Please Monitor Your Throughput!



You can use Prometheus + Grafana, or Access Logs to monitor the requests per calendar month (explained later)



estimate for 30 days,
based on the
assumption that
there is metrics data
for 7 days

Editions and Licensing

Deployment Forms

Requirements and Limitations



**Foundational
Components**

Compatibility Requirements are Version Dependent!



Check the Microgateway (MGW) documentation to determine **compatibility** with your MGW version

Compatibility Overview (Microgateway 5.1)



Kubernetes (K8s)

1.34 – 1.36



OpenShift

4.18 – 4.22

Kubernetes Gateway API Support (Microgateway 5.1)



Standard Channel

1.2 – 1.6

Experimental Channel *(for some incubating features)*

1.6

Airlock Microgateway is Red Hat Certified

The screenshot shows the Red Hat Ecosystem Catalog interface. The top navigation bar includes the Red Hat logo, 'Red Hat Ecosystem Catalog', and links for Solutions, Products, Artifacts, and Partners. A search bar is present with the text 'Search Ecosystem Catalog'. On the right, there are links for 'Contact us', 'Resources', 'All Red Hat', and 'Log In'. The main content area features a breadcrumb trail: 'Home > Software > All software results > Containerized applications'. The product card for 'Airlock Microgateway' is highlighted, showing its logo and a 'Certified' badge. Below the product name, a description states: 'Airlock Microgateway is a Kubernetes native WAAP (Web Application and API Protection) solution to protect microservices.' A tabbed interface at the bottom of the card shows 'Overview' as the active tab, with other tabs for 'Resources', 'Certifications', and 'Deploy & use'. To the right of the product card, a 'Get started' section contains a prominent red 'Contact Red Hat' button and a 'Deploy & use' button. Below these buttons, it lists 'Provided by: Airlock' and 'Type: Containerized application'.

OpenShift supports
only K8s Gateway
API features in the
Standard Channel

Modern application security is embedded in the development workflow and follows DevSecOps paradigms. Airlock Microgateway is the perfect fit for these requirements. It is a lightweight WAAP solution (formerly known as WAF), optimized for Kubernetes environments. Airlock Microgateway protects your applications and microservices with the tried-and-tested Airlock security features against attacks, while also providing a high degree of scalability.

For a list of all features, view the [comparison of the community and premium editions](#).

→ Comprehensive WAAP

Airlock Microgateway includes deny rules that are extensively and continuously improved through bug bounty programs on HackerOne and BugBounty Switzerland. Features including OpenAPI specification enforcement, GraphQL schema validation, and more, increase security tremendously. The easy-to-use reporting dashboards, developed by Airlock, provide valuable insights, enabling users to monitor activity and investigate any potential issues.

<https://catalog.redhat.com/en/software/container-stacks/detail/67177f927cfedb209761e48f>

Editions and Licensing
Deployment Forms
Requirements and Limitations



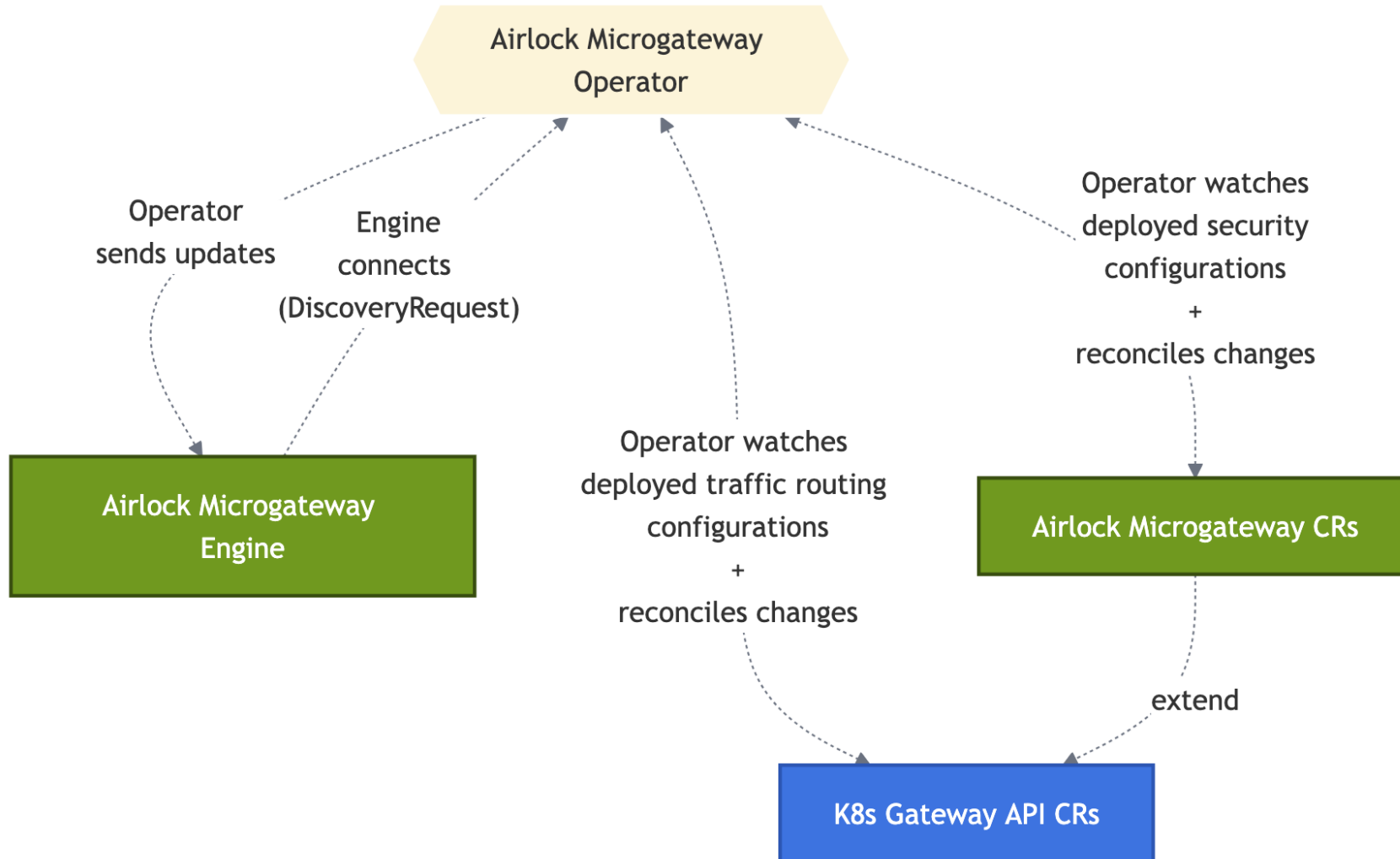
**Foundational
Components**

Requirements and Limitations are Version Dependent!

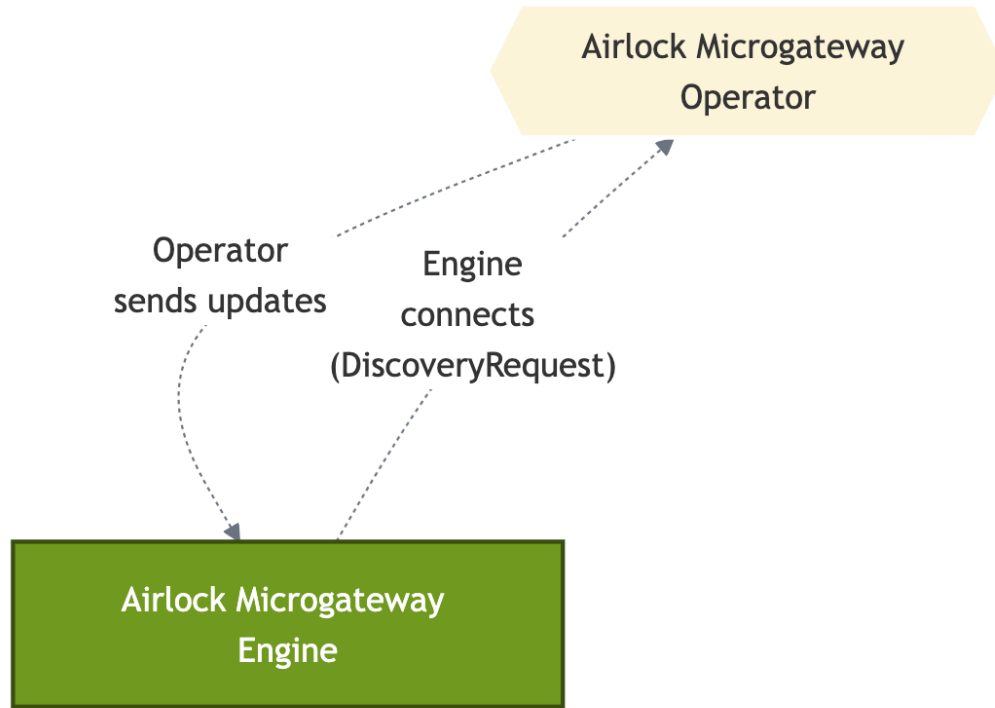


Check the Microgateway (MGW) documentation to determine **requirements** for your MGW version

Recap: Architecture



Required Microgateway Engine Network Communication



From Airlock Microgateway Engine

your Gateway's Namespace

gateway.networking.k8s.io/gateway-name=<Gateway Name>

To Microgateway Operator

airlock-microgateway-system (default namespace)

app.kubernetes.io/name=microgateway-operator (default label selector)

To Port

13377, 8082

To Protocol

TCP

Required for Session Handling (Microgateway 5.1)



Valkey

7.2, 8.0, 8.1, 9.0, 9.1

OR



Redis

7.2, 7.4, 8.4, 8.6, 8.8

Required Session Handling Network Communication



Valkey

7.2, 8.0, 8.1, 9.0, 9.1

OR



Redis

7.2, 7.4, 8.4, 8.6, 8.8

From Microgateway Session Agent

your Gateway's Namespace

gateway.networking.k8s.io/gateway-name=<Gateway Name>

To Valkey or Redis

check your deployment's namespace + label selector

To Port (defaults)

Standalone/Cluster: 6379

Sentinel: 26379

To Protocol

TCP

Limitations (Microgateway 5.1)

NO-Support for a **mix of different versions** of

- Microgateway **Operator** (e.g., 5.x)
- Microgateway **Engine** (e.g., 5.y)

WebSocket support is limited to **HTTP/1.1**

Setup Airlock Microgateway



What steps do you need to complete, to **install Airlock Microgateway** in its most simple form?

Images

Deployment

License Application



**Setup Airlock
Microgateway**

Images

Deployment

License Application



**Setup Airlock
Microgateway**

Container Image Repositories and Registries

Public images on  **QUAY**

- Microgateway Operator
- Microgateway Engine
- Microgateway Session Agent

→ Can be installed via **helm**

[https://docs.airlock.com/microgateway/
5.1/index/1652877995940.html](https://docs.airlock.com/microgateway/5.1/index/1652877995940.html)

Possible to Deploy via Red Hat Operator Hub

The screenshot shows the Red Hat Ecosystem Catalog interface. The top navigation bar includes the Red Hat logo, 'Red Hat Ecosystem Catalog', and links for Solutions, Products, Artifacts, and Partners. A search bar is present with the text 'Search Ecosystem Catalog'. On the right, there are links for 'Contact us', 'Resources', 'All Red Hat', and 'Log In'. The main content area displays the 'Airlock Microgateway' product, which is marked as 'Certified'. Below the product name, a description states: 'Airlock Microgateway is a Kubernetes native WAAP (Web Application and API Protection) solution to protect microservices.' A sidebar on the left contains tabs for 'Overview', 'Resources', 'Certifications', and 'Deploy & use'. A 'Get started' section on the right contains a red 'Contact Red Hat' button and a 'Deploy & use' button. Below this, it lists 'Provided by Airlock' and 'Type Containerized application'.

Red Hat Ecosystem Catalog Solutions Products Artifacts Partners All Search Ecosystem Catalog Contact us Resources All Red Hat Log In

Home > Software > All software results > Containerized applications

Airlock Microgateway Certified

Airlock Microgateway is a Kubernetes native WAAP (Web Application and API Protection) solution to protect microservices.

Overview Resources Certifications Deploy & use

Get started

[Contact Red Hat](#)

[Deploy & use](#)

Provided by
Airlock

Type
Containerized application

Modern application security is embedded in the development workflow and follows DevSecOps paradigms. Airlock Microgateway is the perfect fit for these requirements. It is a lightweight WAAP solution (formerly known as WAF), optimized for Kubernetes environments. Airlock Microgateway protects your applications and microservices with the tried-and-tested Airlock security features against attacks, while also providing a high degree of scalability.

For a list of all features, view the [comparison of the community and premium editions](#).

→ Comprehensive WAAP

Airlock Microgateway includes deny rules that are extensively and continuously improved through bug bounty programs on HackerOne and BugBounty Swi... features including OpenAPI specification enforcement, GraphQL schema va... increase security tremendously. The easy-to-use reporting dashboards, dev... Airlock, provide valuable insights, enabling users to monitor activity and investigate any potential issues.

<https://catalog.redhat.com/en/software/container-stacks/detail/67177f927cfedb209761e48f>

OpenShift or Helm-based Installation?

Recommendations

You already use RedHat OpenShift in your organization and project(s)?

→ **Install via OpenShift (Operator Lifecycle Manager)**

You do **NOT** use RedHat OpenShift?

→ **Install via helm chart**

Airlock Microgateway Is Also Available on Serval

Airlock Microgateway on Serval

Details about the service Airlock Microgateway on Serval



Airlock Microgateway

Security

 Get It

"The identity aware, Kubernetes native WAAP (Web Application and API Protection) solution."

Cloud Providers

-  Managed OpenShift
-  Enterprise Private Cloud

Licenses

[Show details](#)

-  Proprietary

External Links

-  [Microgateway Website](#)

Overview

Protect your cloud-native applications: Airlock Microgateway has been designed for use in Kubernetes environments. Security guidelines and compliance with them can be perfectly automated thanks to Security as Code, ensuring improved integration of security and governance throughout the entire DevSecOps lifecycle. Modern corporate architecture with micro-segmentation also benefits from distributed access controls based on the zero trust principle.

Features

Comprehensive App and API Protection

<https://serval.com/service/airlock-microgateway/>

Images

Deployment

License Application



**Setup Airlock
Microgateway**

Deploy the K8s Gateway API Resources

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/'<VERSION>'/standard-install.yaml
```

e.g., '**<VERSION>**' = **v1.5.1**



```
customresourcedefinition.apiextensions.k8s.io/backendtlspolicies.gateway.networking.k8s.io serverside-applied  
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io serverside-applied  
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io serverside-applied  
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io serverside-applied  
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io serverside-applied  
customresourcedefinition.apiextensions.k8s.io/listenersets.gateway.networking.k8s.io serverside-applied  
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io serverside-applied  
customresourcedefinition.apiextensions.k8s.io/tlsroutes.gateway.networking.k8s.io serverside-applied
```


Install the Airlock Microgateway Operator

Create the **airlock-microgateway-system** namespace

Get a **license** and store it in the Microgateway Operator namespace
(as Kubernetes secret)

install **airlock-microgateway** in the airlock-microgateway-system namespace

After the install, verify that the Airlock Microgateway Operator started successfully

Deployment of Multiple Airlock Microgateway Operators

If necessary it is possible to deploy **multiple** Airlock Microgateway Operators in the same cluster

High isolation on Operator-side, but some shared components:

- Custom Resource Definitions (CRDs)
- Kubernetes webhooks
- Kubernetes admission policies

General recommendation: Use **single** Operator instead

Images

Deployment

License Application



**Setup Airlock
Microgateway**

Install the Airlock Microgateway Operator

Create the **airlock-microgateway-system** namespace

Get a **license** and store it in the Microgateway Operator namespace
(as Kubernetes secret)

install **airlock-microgateway** in the airlock-microgateway-system namespace

After the install, verify that the Airlock Microgateway Operator started successfully

How to Apply Your License to Your Microgateway Setup?

Storage in a **Kubernetes secret** that has the key **microgateway-license.txt**

```
kubectl create secret generic airlock-microgateway-license \  
-n airlock-microgateway-system \  
--from-file=microgateway-license.txt=<path-to-your-local-microgateway-license.txt>
```



kubernetes secret

airlock-microgateway-license
n= airlock-microgateway-system

key= microgateway-license.txt
source= <path-to-your-local-
microgateway-license.txt>



How to Update Your Airlock Microgateway License?

To update your license, use

```
kubect1 apply
```

When upgrading from Community ($\leq 5.0.x$) or Evaluation ($>5.1.x$) to Premium

→ NO need to restart the Airlock Microgateway Operator

(Kubernetes secrets are updated at intervals of about one minute)

LAB 01

Setup Airlock Microgateway



LAB 01

Setup Airlock Microgateway

Steps

- Install Airlock Microgateway
- Explore the Airlock Microgateway Installation



LAB 01

Setup Airlock Microgateway



Any feedback?



K8s Gateway API



What is the **Kubernetes Gateway API** and where can you find **further information** about it?

Ingress NGINX Retirement
K8s Gateway API Overview
GatewayClass
Gateway
HTTPRoute
ListenerSet
Supported Features



K8s Gateway API

Ingress NGINX Retirement

K8s Gateway API Overview

GatewayClass

Gateway

HTTPRoute

ListenerSet

Supported Features

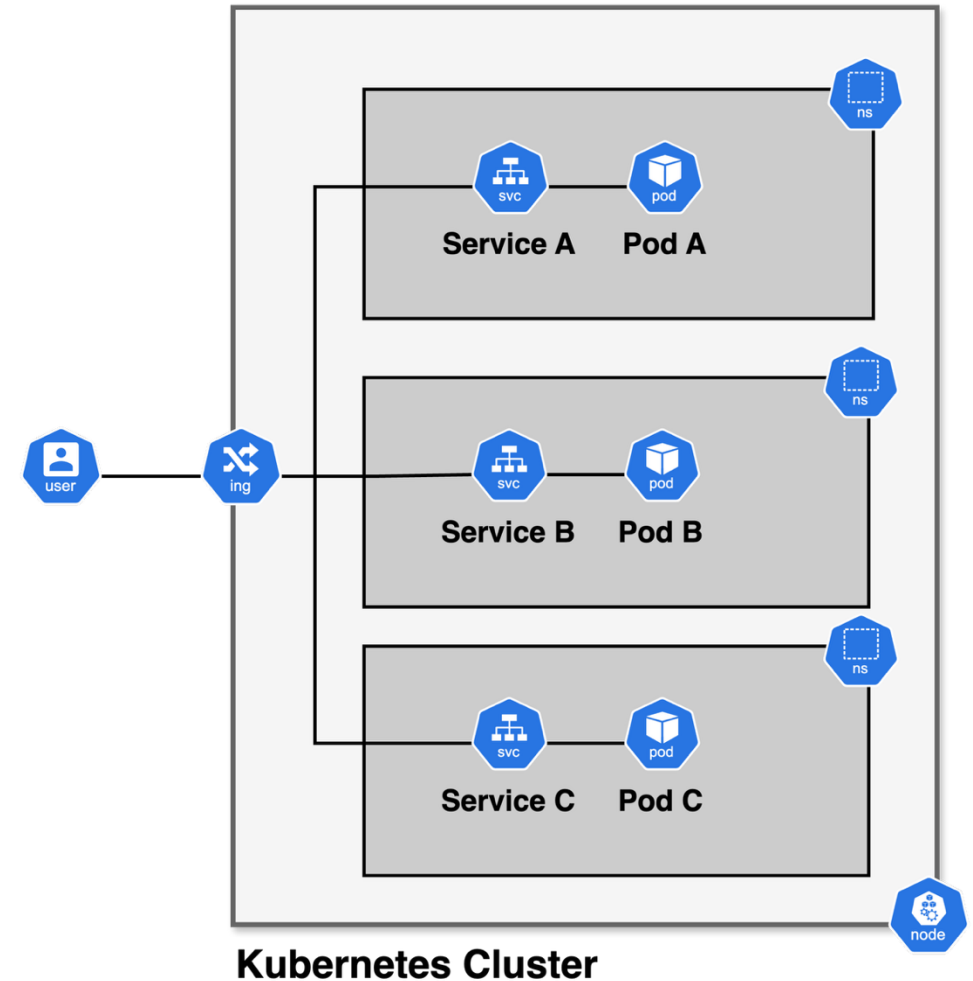


K8s Gateway API

Why an Ingress Controller?



- Centralized traffic management
 - Traffic routing (based on fqdn, path, ...)
 - Load balancing
 - Request mirroring
- TLS termination
- Security
 - Control which services are exposed
 - Control who can access exposed services



Understanding the Related Terminology



	Ingress	Kubernetes Ingress	Kubernetes Gateway API
Description	A component (reverse proxy, load balancer, API gateway) that accepts external traffic and forwards it to services.	A Kubernetes API resource (config) that defines HTTP/HTTPS routing rules to services.	Set of Kubernetes API resources (config) for managing L4-L7 traffic in a more flexible, structured way. Successor of Kubernetes Ingress
Form	Software	Kubernetes resource	Multiple Kubernetes resources
Examples	Ingress NGINX, Airlock Microgateway, ...	kind: Ingress	- kind: Gateway - kind: HTTPRoute - ...

Ingress NGINX Retirement



This repository was archived by the owner on Mar 24, 2026. It is now read-only.



→ Ingress NGINX got retired



<https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/>

<https://github.com/kubernetes/ingress-nginx>

K8s Gateway API – Advantages



- Mature, well-established and widely adopted configuration model
- Works for Kubernetes and classic workload
- Cloud agnostic solution
- Same configuration for North-South and East-West traffic
- Benefit of a growing ecosystem

K8s Gateway API – Growing Ecosystem



K8s Gateway API is a standard with a growing ecosystem

Name	Status	Kubernetes Version
demo-aks	Active	v1.23.12
docker-desktop	Bad Gateway	...
minikube	Active	v1.26.3

```
# List all GatewayClasses
gwctl get gatewayclasses

# List all Gateways in the default namespace
gwctl get gateways

# List all HTTPRoutes in the 'ns2' namespace in yaml format
gwctl get httproutes -n ns2 -o yaml

# List all Services in wide format in the default namespace
gwctl get svc -o wide
```

github.com/kubernetes-sigs/gwctl

github.com/kubernetes-sigs/headlamp

K8s Gateway API – Support in Microgateway

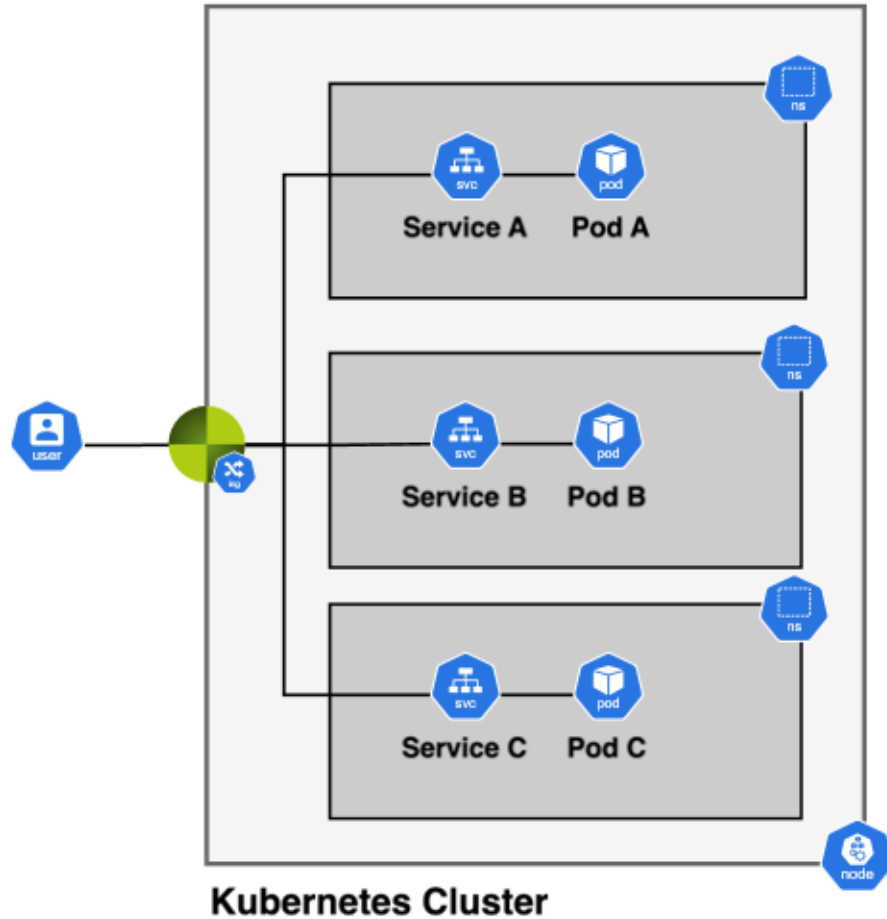


- Kubernetes Gateway API provides a standard for traffic routing
- Microgateway supports Gateway API and extends it with its CRDs:
 - Authentication enforcement (mTLS, OIDC, JWT, ...)
 - Filtering (deny rules, Open API Spec., GraphQL, ...)

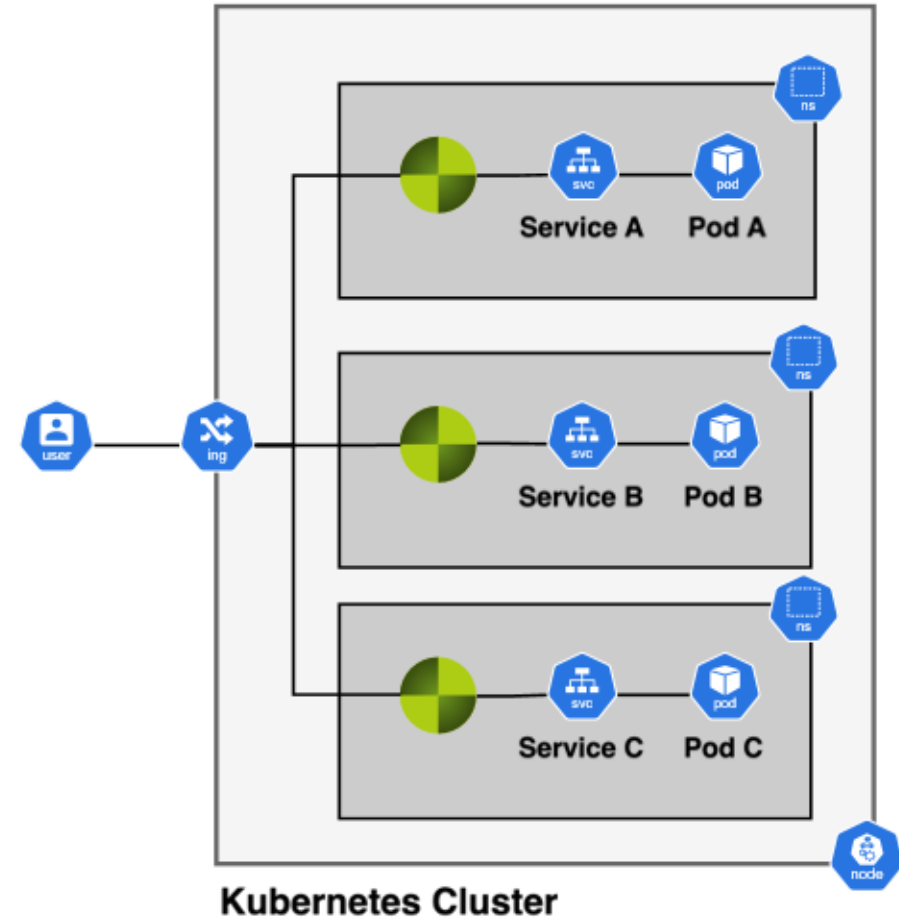
K8s Gateway API – Microgateway Deployment Options



Microgateway as Ingress (RECOMMENDED)



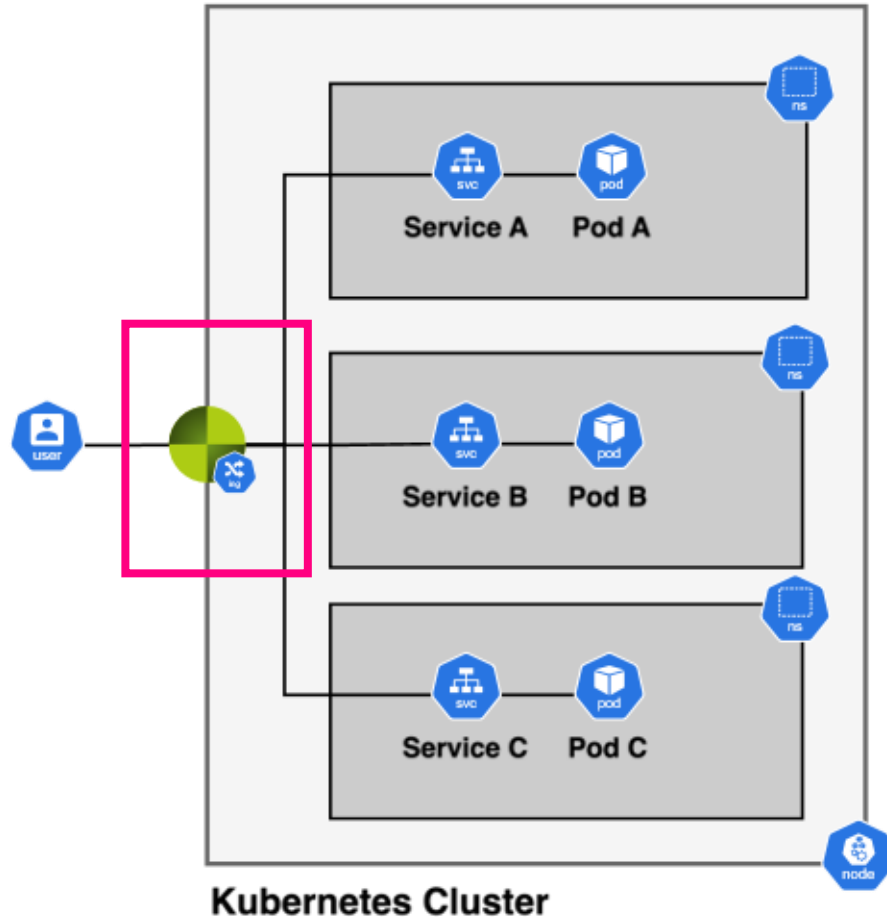
Microgateway as in-cluster Gateway



K8s Gateway API – Why «Microgateway as Ingress»?



Microgateway as Ingress (RECOMMENDED)



Incoming traffic (north-south)

automatically passes Microgateway

- NO bypass path
- Easily assure compliance
- Easily explainable in audits

"here is our Gateway, it is the ingress"

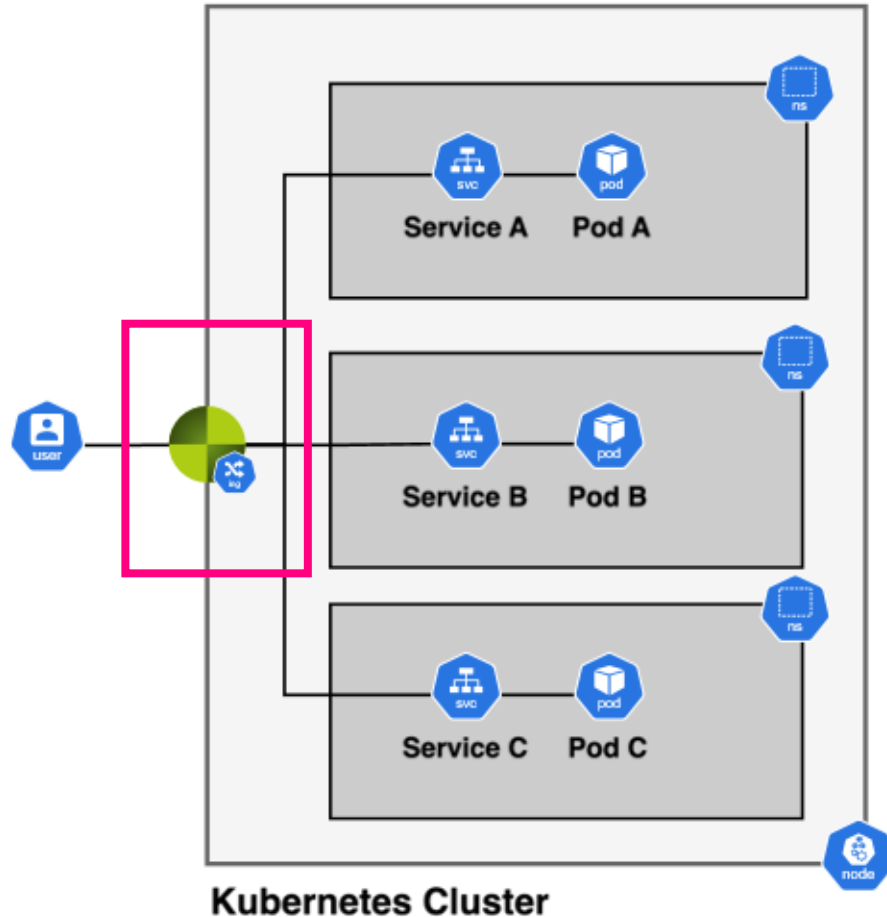
instead of...

"we have an ingress controller, and behind it, for some services, there is a Gateway."

K8s Gateway API – Why «Microgateway as Ingress»?



Microgateway as Ingress (RECOMMENDED)

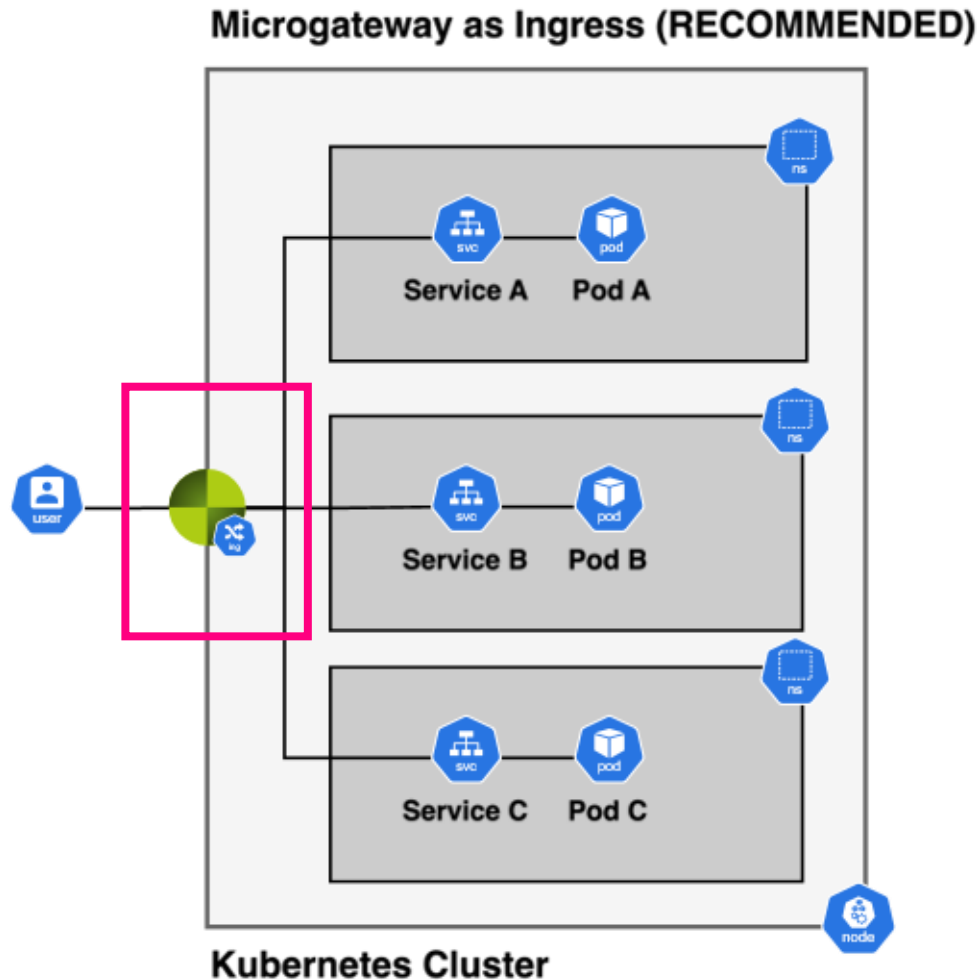


NO additional hop

- reduced processing cost / latency
- less parts → easier failure analysis

Lower complexity (one or few Gateways)

K8s Gateway API – Why «Microgateway as Ingress»?



Kubernetes Gateway API is a
standard protocol

Allows **simple + quick migration**
from Ingress NGINX
to **Airlock Microgateway as Ingress**

<https://docs.airlock.com/microgateway/5.1/index/1776358271342.html>

Ingress NGINX Retirement

K8s Gateway API Overview

GatewayClass

Gateway

HTTPRoute

ListenerSet

Supported Features



K8s Gateway API

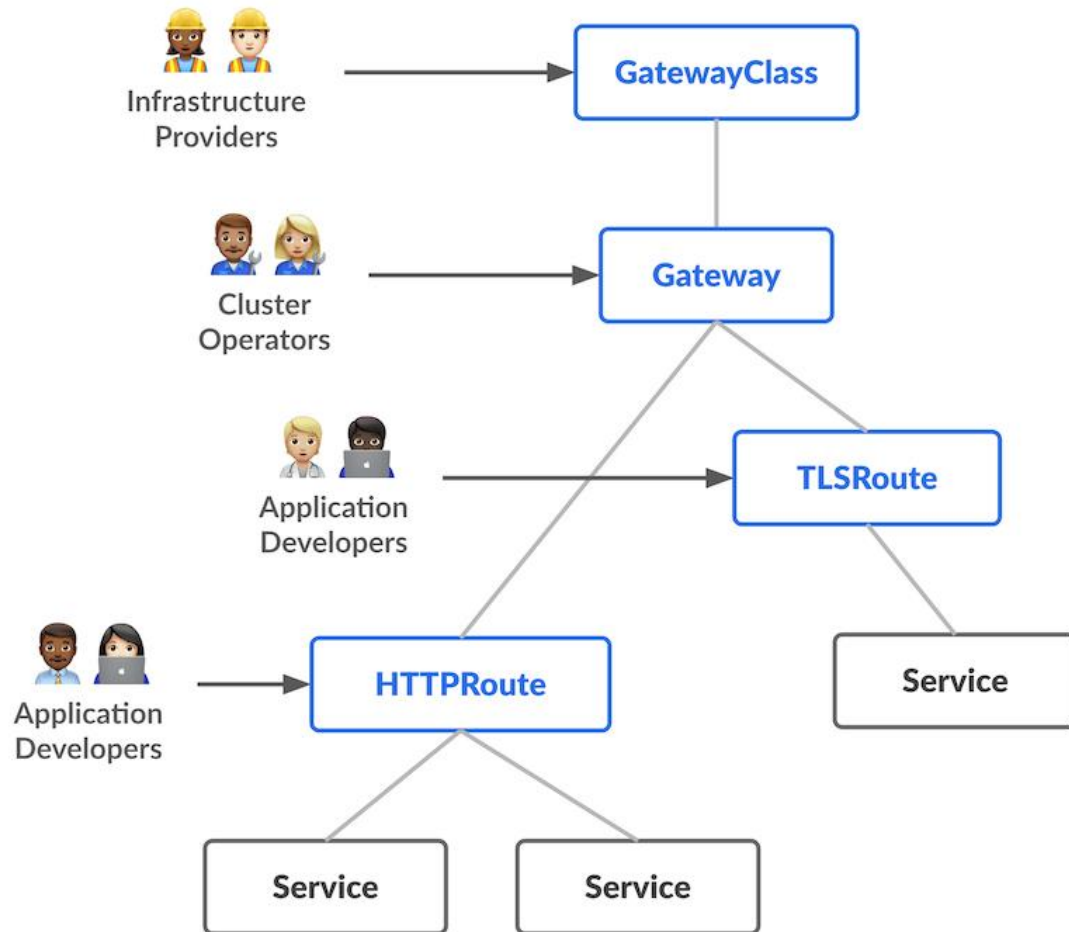
K8s Gateway API – Airlock's Role



We **implement** the K8s Gateway API standard.

We **discuss** and **contribute** together with major vendors (Google, Microsoft, Red Hat, Isovalent, ...) to the K8s Gateway API project.

K8s Gateway API – Key Roles

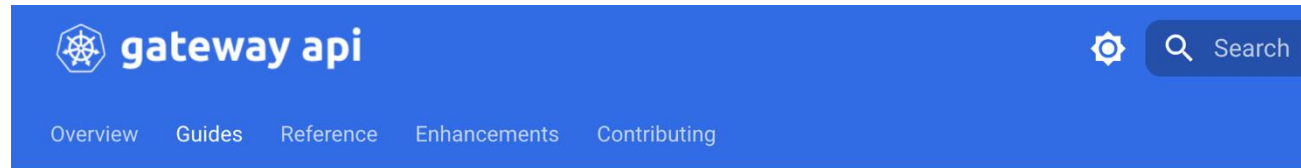


Three key roles with corresponding resources

Multiple roles may map to the same person

Image: <https://gateway-api.sigs.k8s.io/images/resource-model.png>

K8s Gateway API – User Guides



Guides

User Guides

[Getting started](#)

[Simple Gateway](#)

[HTTP routing](#)

[HTTP redirects and rewrites](#)

[HTTP header modifier](#)

[HTTP traffic splitting](#)

[HTTP request mirroring](#)

[Cross-Namespace routing](#)

[TLS](#)

[TCP routing](#)

[gRPC Routing](#)

[Migrating from Ingress](#)

[Backend Protocol Selection](#)

[Infrastructure attributes](#)

[API Design](#)

[CRD Management](#)

[Implementer's Guide](#)

Getting started with Gateway API

1. [Install a Gateway controller](#) OR [install the Gateway API CRDs manually](#)

THEN

2. Try out one of the available guides:

- [Simple Gateway](#) (a good one to start out with)
- [HTTP routing](#)
- [HTTP redirects and rewrites](#)
- [HTTP traffic splitting](#)
- [Routing across Namespaces](#)
- [Configuring TLS](#)
- [TCP routing](#)
- [gRPC routing](#)
- [Migrating from Ingress](#)

<https://gateway-api.sigs.k8s.io/guides/>

K8s Gateway API – Custom Resource Definitions



Reference

API Types

Gateway

GatewayClass

GRPCRoute

HTTPRoute

Policy

BackendTLSPolicy

BackendTrafficPolicy

ReferenceGrant

API specification

Standard

Experimental

Policy Attachment

API Reference

Packages

- gateway.networking.k8s.io/v1
- gateway.networking.k8s.io/v1alpha2
- gateway.networking.k8s.io/v1alpha3
- gateway.networking.k8s.io/v1beta1

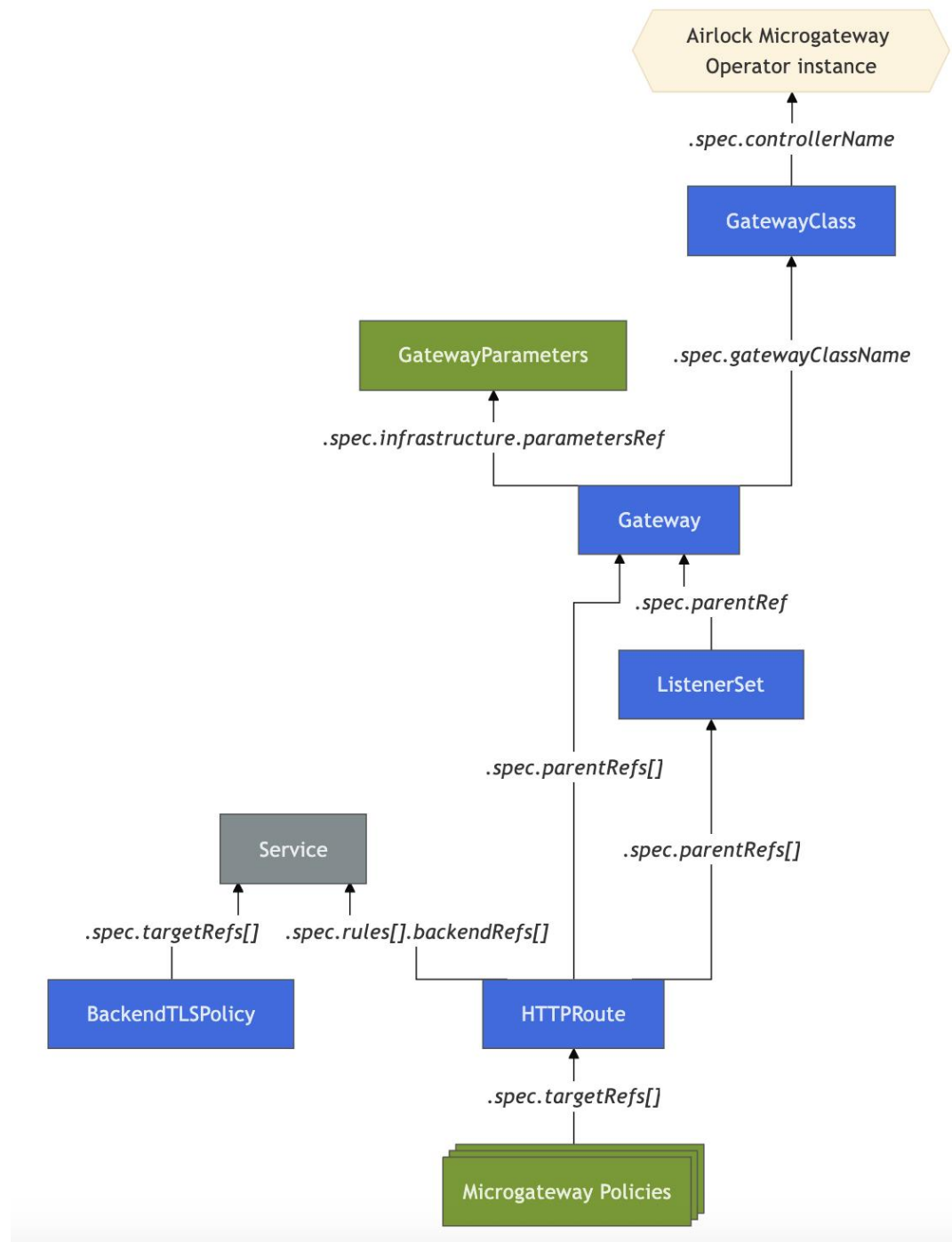
gateway.networking.k8s.io/v1

Package v1 contains API Schema definitions for the gateway.networking.k8s.io API group.

Resource Types

- [BackendTLSPolicy](#)
- [GRPCRoute](#)
- [Gateway](#)
- [GatewayClass](#)
- [HTTPRoute](#)

<https://gateway-api.sigs.k8s.io/reference/spec/>
<https://github.com/kubernetes-sigs/gateway-api>

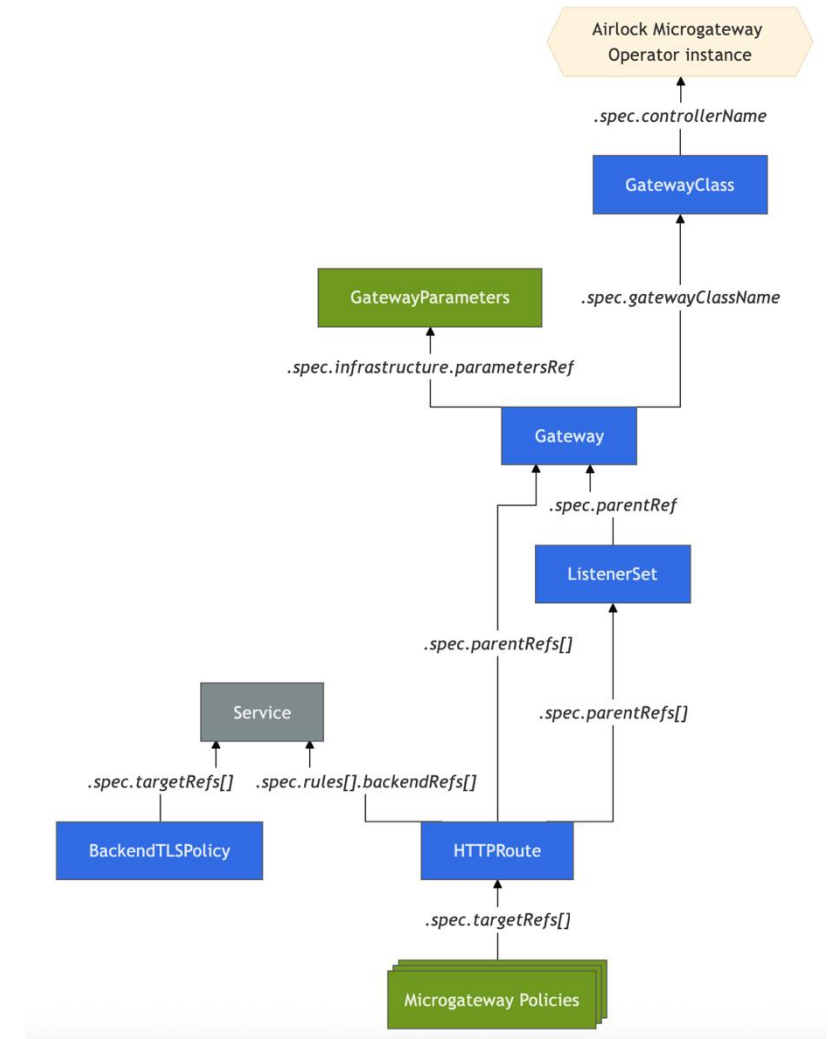


K8s Gateway API – Custom Resource Definitions



API Types:

- GatewayClass
- Gateway
- GRPCRoute
- HTTPRoute
- Policy
- ReferenceGrant
- TLS Route
- ListenerSet

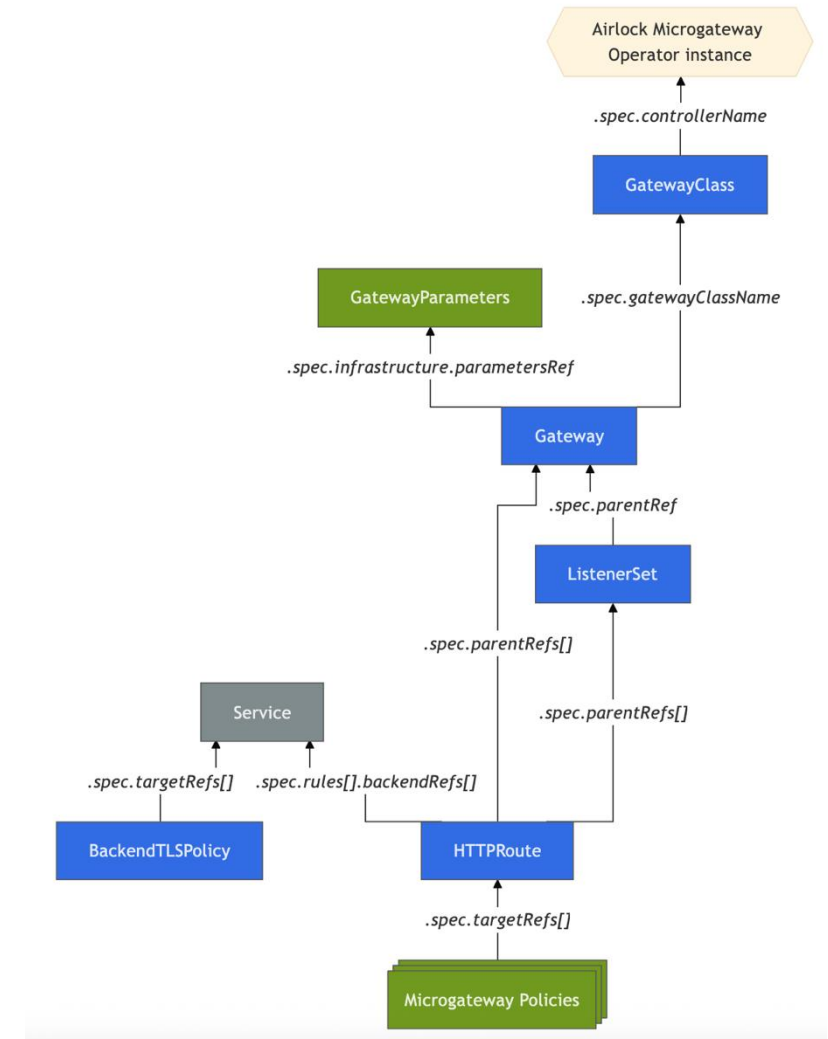


K8s Gateway API – Custom Resource Definitions



API Types:

- **GatewayClass**
- **Gateway**
- GRPCRoute
- **HTTPRoute**
- Policy
- ReferenceGrant
- TLS Route
- **ListenerSet**



Ingress NGINX Retirement
K8s Gateway API Overview

GatewayClass

Gateway

HTTPRoute

ListenerSet

Supported Features



K8s Gateway API

GatewayClass



A **cluster-scoped** resource

- Defined by the infrastructure provider



- Represents a class of Gateways that can be instantiated
- Serves the same function as **networking.IngressClass**

GatewayClass – Decoupling Implementation from User



Allows decoupling of which mechanism (e.g. controller) implements the Gateways from the user

```
kind: GatewayClass
metadata:
  name: internet
  ...
---
kind: GatewayClass
metadata:
  name: private
  ...
```

GatewayClass – Controller Selection



`GatewayClass.spec.controllerName` → controller to manage GatewayClass

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: airlock-microgateway
spec:
  controllerName: microgateway.airlock.com/gatewayclass-controller
```

Must match the **Microgateway controllerName** configured
in the **Airlock Microgateway Operator Helm chart** !

GatewayClass – Default GatewayClass



A **default GatewayClass** with the name **airlock-microgateway** is created when **deploying the Microgateway Operator**.

GatewayClass – Airlock Microgateway CRDs



NOT recommended!
→ reference via **Gateway** instead

Ingress NGINX Retirement
K8s Gateway API Overview
GatewayClass
Gateway
HTTPRoute
ListenerSet
Supported Features

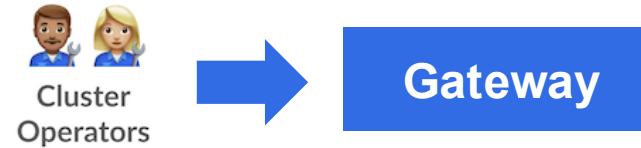


K8s Gateway API



Resource that **triggers actions** in the API

- Defined by the cluster operator



- Other resources in the API are config snippets until a Gateway has been created to link them

Gateway – GatewayClass Reference



```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: airlock-gateway
  namespace: airlock-gateway
spec:
  gatewayClassName: airlock-microgateway
```

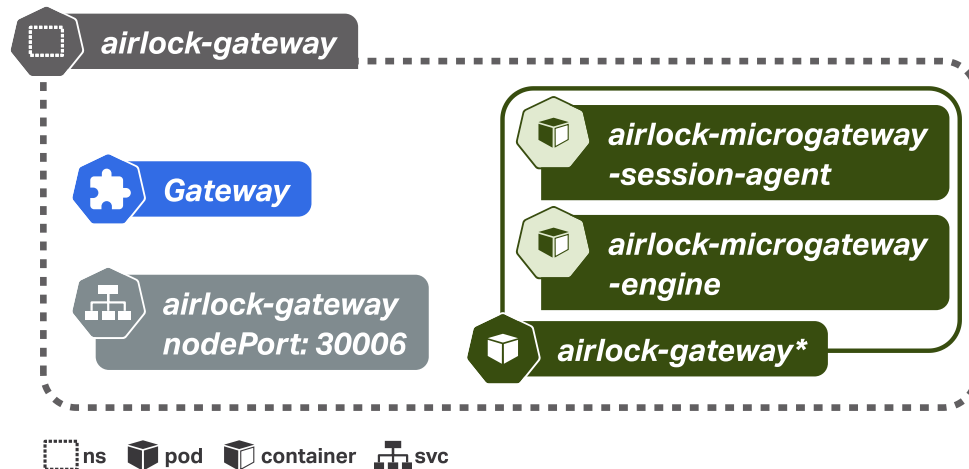
Defined based on a **GatewayClass** object, which it references via the **gatewayClassName** spec

Gateway – Microgateway Engine Deployment

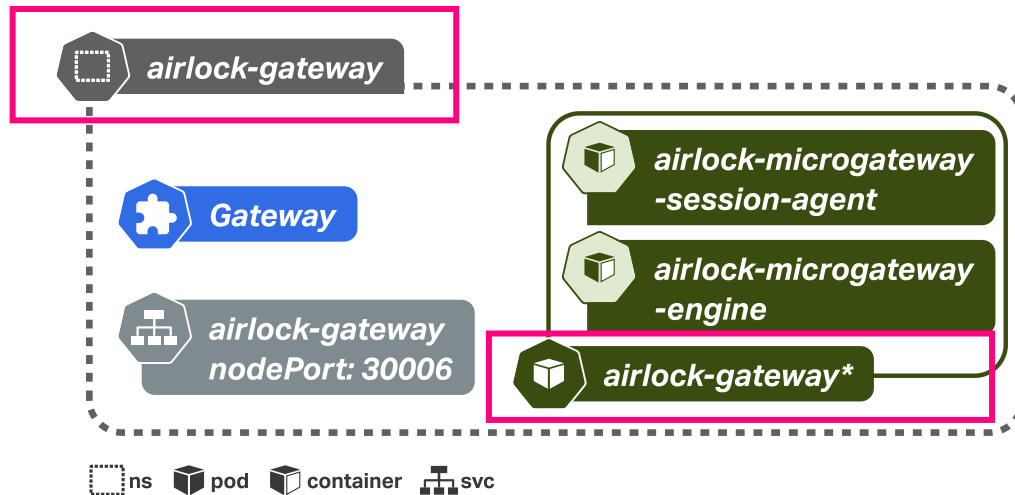


If the Airlock Microgateway Controller supports the referenced GatewayClass:

- A Pod with the **name** of the Gateway CR is deployed in the same **namespace** as the Gateway CR
- The Pod will run an **Airlock Microgateway Engine container**

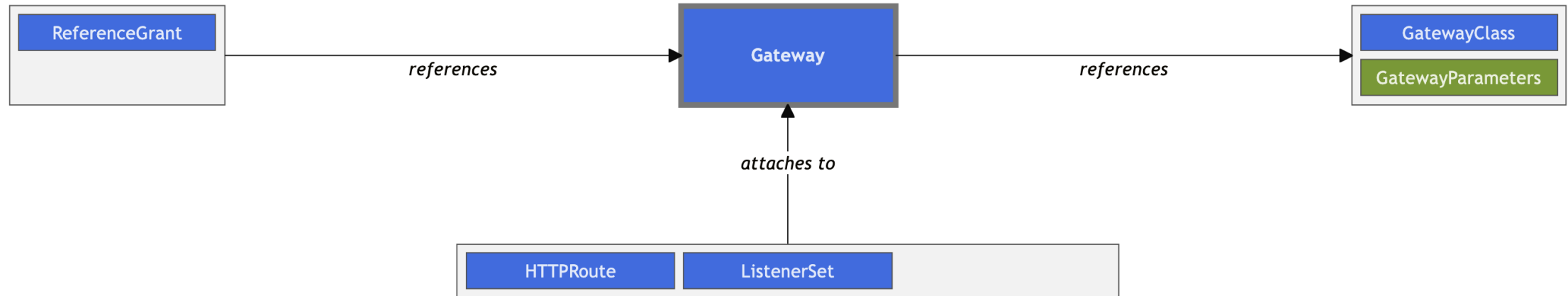


Gateway – Microgateway Engine Deployment



```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: airlock-gateway
  namespace: airlock-gateway
spec:
  gatewayClassName: airlock-microgateway
  listeners:
    - name: http
      port: 30006
      protocol: HTTP
  allowedRoutes:
    namespaces:
      from: All
```

Gateway – Airlock Microgateway CRDs



Ingress NGINX Retirement
K8s Gateway API Overview
GatewayClass
Gateway
HTTPRoute
ListenerSet
Supported Features

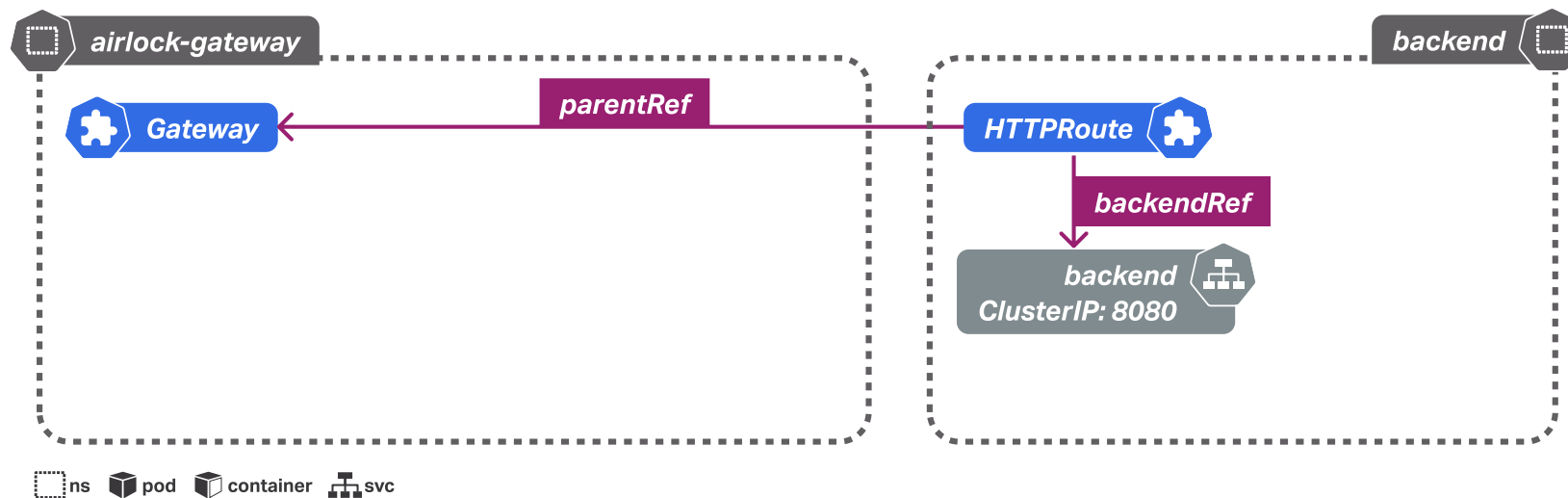


K8s Gateway API

HTTPRoute



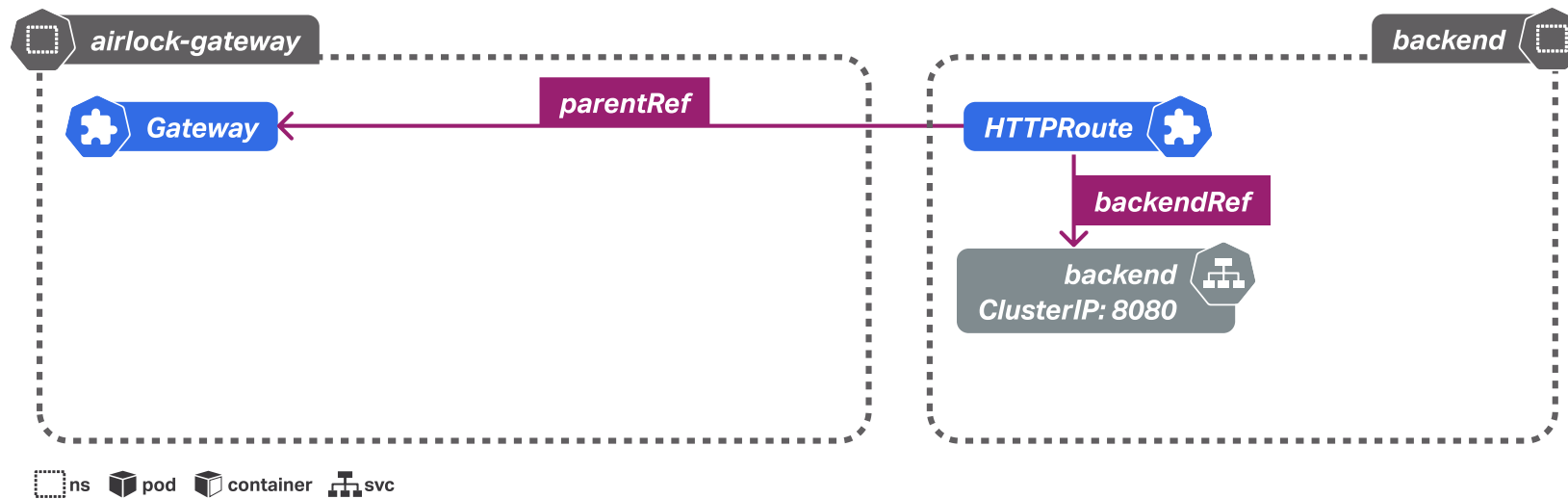
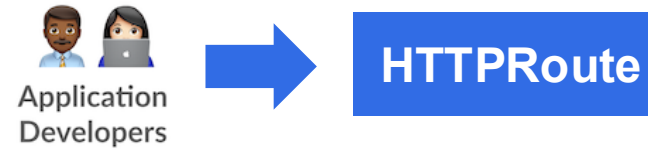
Allows to specify routing behavior of HTTP requests from a **Gateway listener** to an **API object, i.e., Service**



HTTPRoute



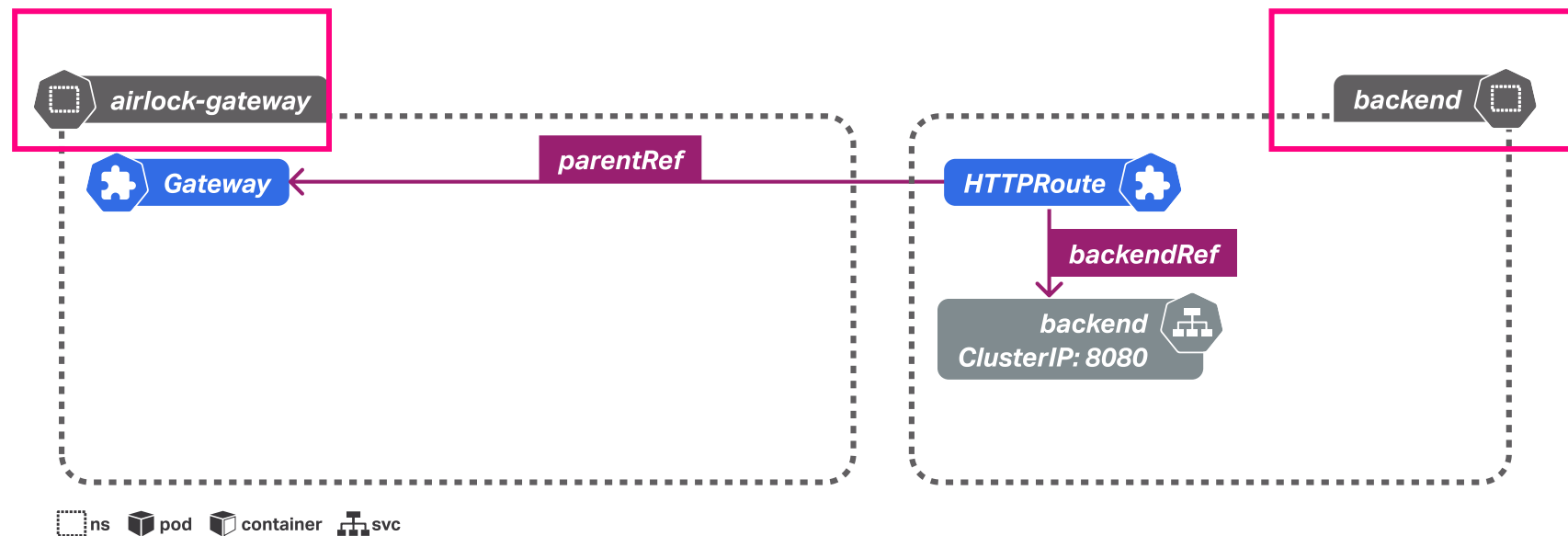
Defined by the application developer



HTTPRoute – Cross-namespace Routing



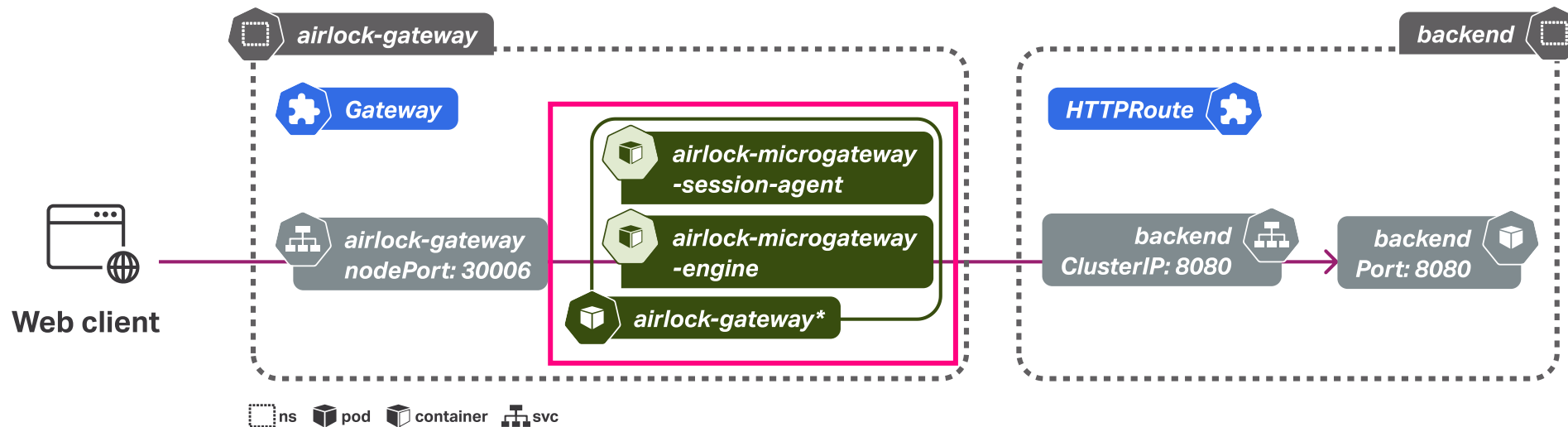
Gateways and HTTPRoutes can be deployed into **different Namespaces** and HTTPRoutes can attach to Gateways **across Namespace boundaries**



HTTPRoute – Route Through Microgateway Engine!



Use network policies to assure that the **incoming traffic** is routed **through the Microgateway Engine!**



HTTPRoute – parentRefs



Define which Gateways this
Route shall be attached to

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: backend
  namespace: backend
spec:
  parentRefs:
  - group: gateway.networking.k8s.io
    kind: Gateway
    name: airlock-gateway
    namespace: airlock-gateway
  rules:
  - backendRefs:
    - group: ""
      kind: Service
      name: backend
      namespace: backend
      port: 8080
```

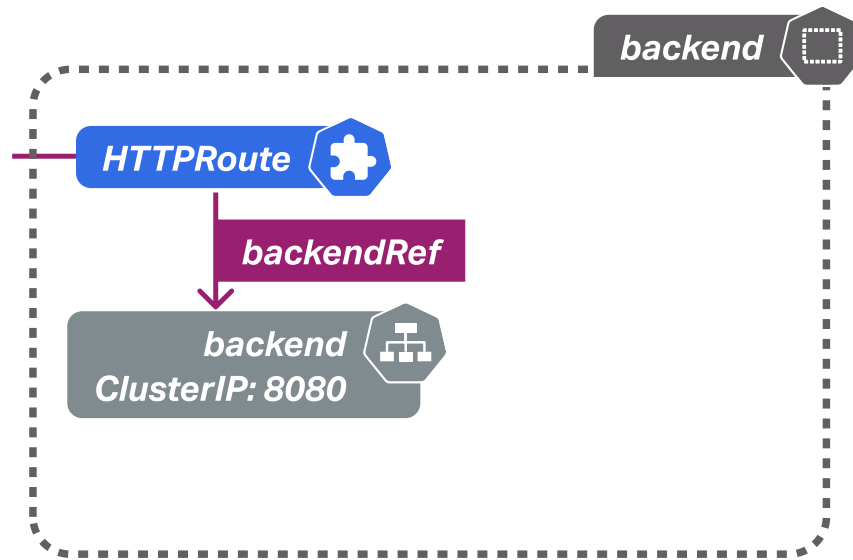
HTTPRoute – rules



Define semantics for matching a HTTP request based on conditions

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: backend
  namespace: backend
spec:
  parentRefs:
  - group: gateway.networking.k8s.io
    kind: Gateway
    name: airlock-gateway
    namespace: airlock-gateway
  rules:
  - backendRefs:
    - group: ""
      kind: Service
      name: backend
      namespace: backend
      port: 8080
```

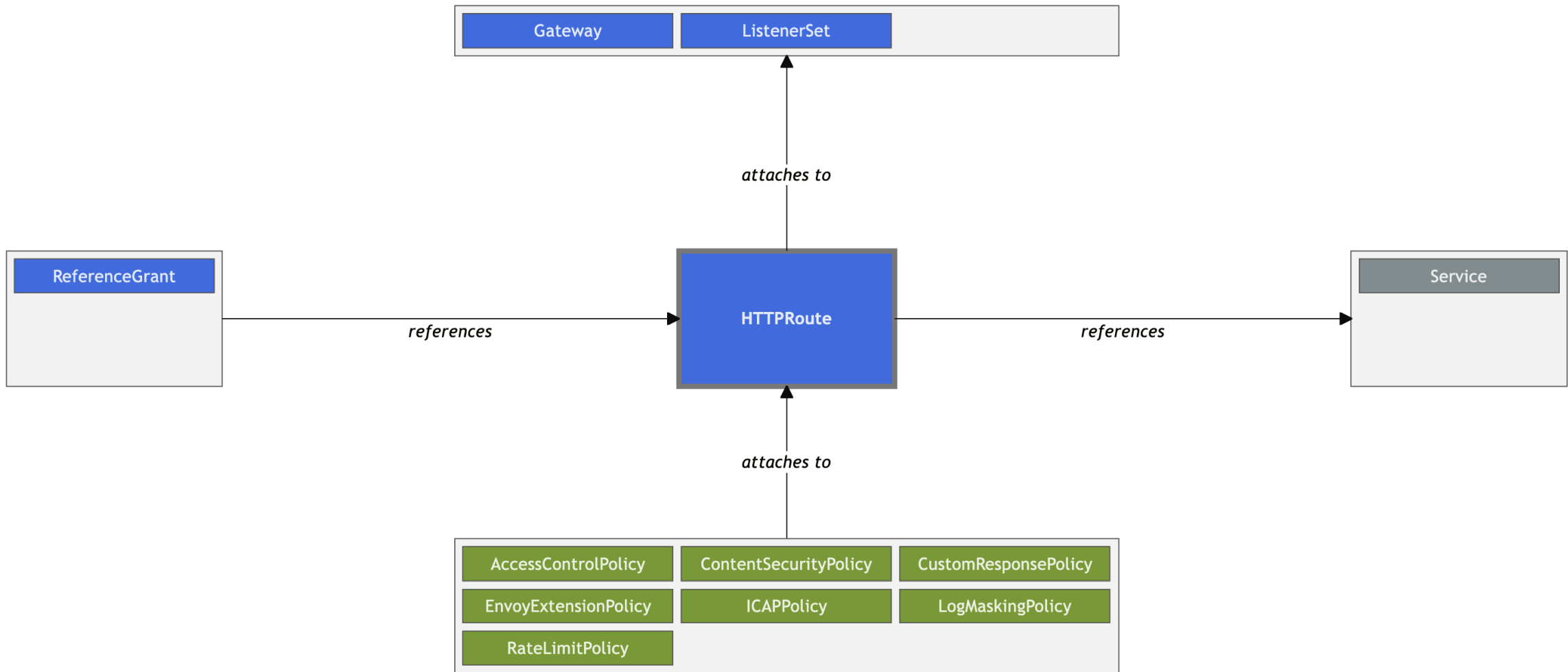
HTTPRoute – rules



Defines API objects where matching requests should be sent

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: backend
  namespace: backend
spec:
  parentRefs:
  - group: gateway.networking.k8s.io
    kind: Gateway
    name: airlock-gateway
    namespace: airlock-gateway
  rules:
  - backendRefs:
    - group: ""
      kind: Service
      name: backend
      namespace: backend
      port: 8080
```

HTTPRoute – Airlock Microgateway CRDs



Ingress NGINX Retirement
K8s Gateway API Overview
GatewayClass
Gateway
HTTPRoute
ListenerSet
Supported Features

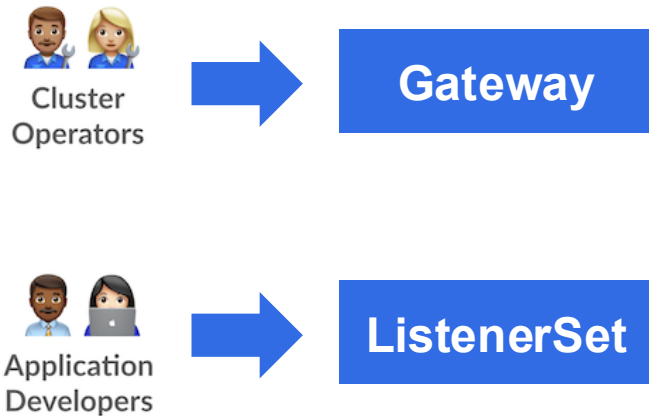


K8s Gateway API

ListenerSet



Allows to specify **additional listeners** for a Gateway
and to **delegate listener management**





Create ListenerSets as team / developer
(and use shared Gateway + load balancing infrastructure)

Go beyond 64-listeners limit of Gateway resource

Create service-level certificates

Ingress NGINX Retirement
K8s Gateway API Overview
GatewayClass
Gateway
HTTPRoute
ListenerSet
Supported Features



K8s Gateway API

Supported Features – Feature Comparison View



gateway api

Overview Guides Reference Enhancements Contributing

Overview

Introduction

Concepts

API Overview

Troubleshooting and Status

Conformance

Roles and Personas

Security

Tools

Use Cases

Versioning

Traffic Matching

Hostnames

Service Mesh

Overview

GAMMA Initiative

Service Facets

Implementations

List

Matching Wizard

Comparisons

v1.5

v1.4

v1.3

v1.2

FAQ

Glossary

v1.5

The following tables are populated from the conformance reports [uploaded by project implementations](#). They are separated into the extended features that each project supports listed in their reports. Implementations only appear in this page if they pass Core conformance for the resource type, and the features listed should be Extended features. Implementations that submit conformance reports with skipped tests won't appear in the tables.

Warning

This page is under active development and is not in its final form, especially for the project name and the names of the features. However, as it is based on submitted conformance reports, the information is correct.

Gateway Profile

HTTPRoute

Organization	Project	Version	Mode	Standard Features	Experimental Features	Core
agentgateway	agentgateway	v1.0.0	default	34/34	1/1	✓
airlock	microgateway	5.0.0	default	27/34	1/1	✓

Indicates which **extended (optional)**
K8s Gateway API features are
supported by **Airlock Microgateway**

<https://gateway-api.sigs.k8s.io/implementations/v1.5/>

Supported Features – Enhancements



gateway api

Overview Guides Reference Enhancements Contributing

Enhancements

Overview

Implementable

Experimental

GEP-1016: GRPCRoute

GEP-1713: ListenerSets - Standard Mechanism to Merge Multiple Gateways

GEP-1731: HTTPRoute Retries

GEP-1748: Gateway API Interaction with Multi-Cluster Services

GEP-1767: CORS Filter

GEP-2162: Supported features in GatewayClass Status

GEP-3155: Complete Backend mutual TLS Configuration

Standard

GEP-709: Cross Namespace References from Routes

GEP-718: Rework forwardTo segment in routes

GEP-724: Refresh Route-Gateway Binding

GEP-726: Add Path Redirects and Rewrites

GEP-746: Replace Cert Refs on HTTPRoute with Cross

Gateway Enhancement Proposal (GEP)

Gateway Enhancement Proposals (GEPs) serve a similar purpose to the [KEP](#) process for the main Kubernetes project:

1. Ensure that changes to the API follow a known process and discussion in the OSS community.
2. Make changes and proposals discoverable (current and future).
3. Document design ideas, tradeoffs, decisions that were made for historical reference.
4. Record the results of larger community discussions.
5. Record changes to the GEP process itself.

Process

This diagram shows the state diagram of the GEP process at a high level, but the details are below.

```
graph TD; A([Discuss with the community]) --> B([Issue Created]);
```

Indicates which K8s Gateway API Enhancements

- are available in **Standard**
- are still **Experimental**

<https://gateway-api.sigs.k8s.io/geps/overview/>

Basic Webapp Protection



What steps do you need to **protect a web-application** with Airlock Microgateway in its **most simple form**?

Recap: Traffic Routing
Default Content Security
Custom Microgateway Resources



**Basic Web-app
Protection**

Recap: Traffic Routing

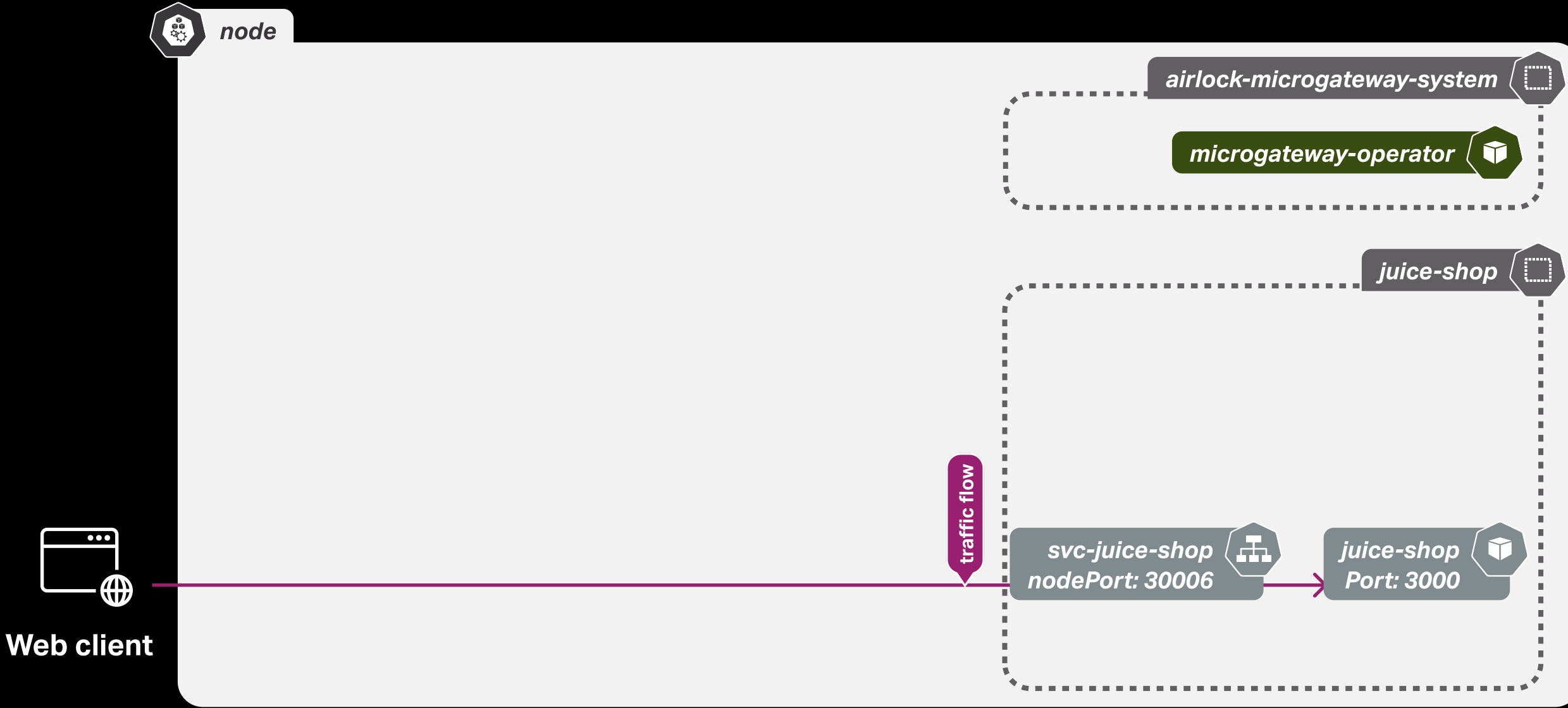
Default Content Security

Custom Microgateway Resources

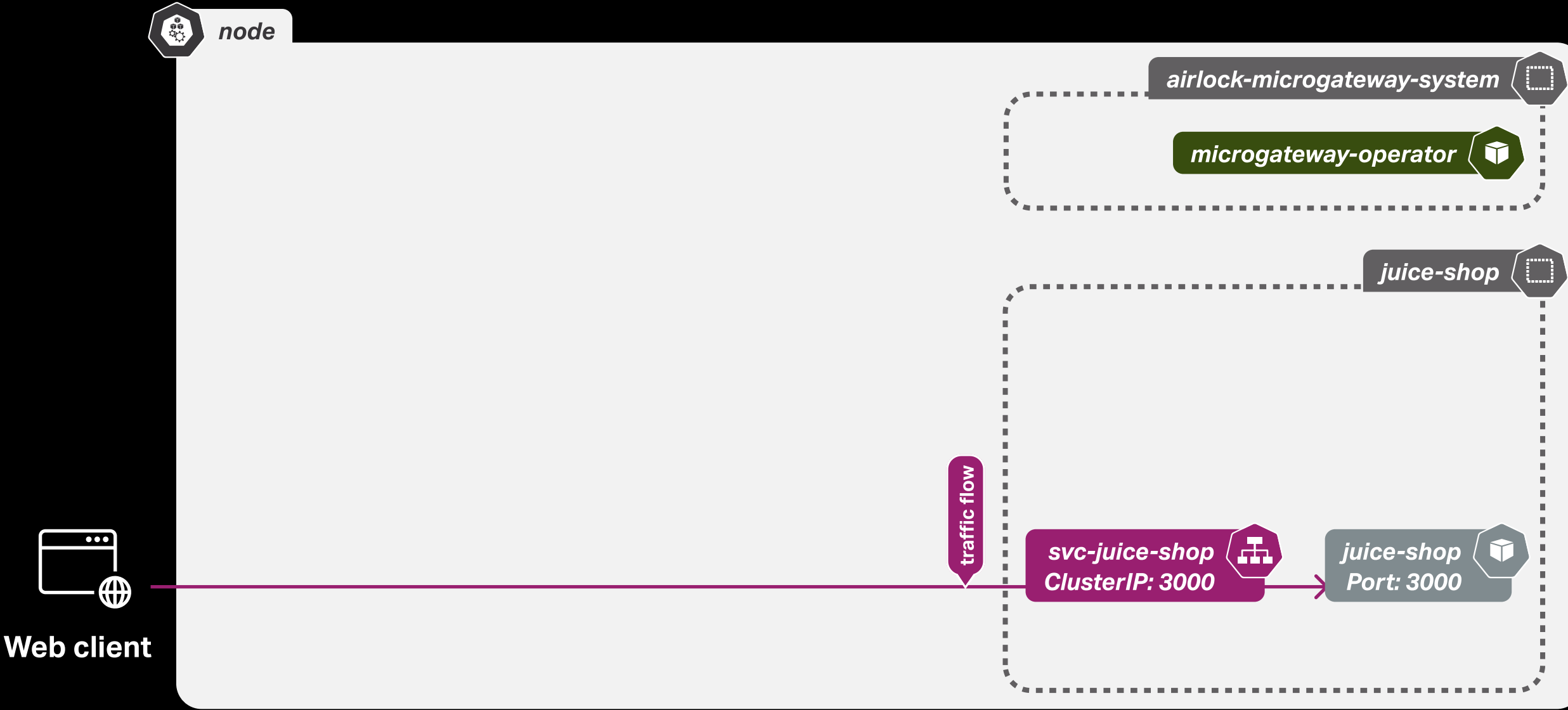


**Basic Web-app
Protection**

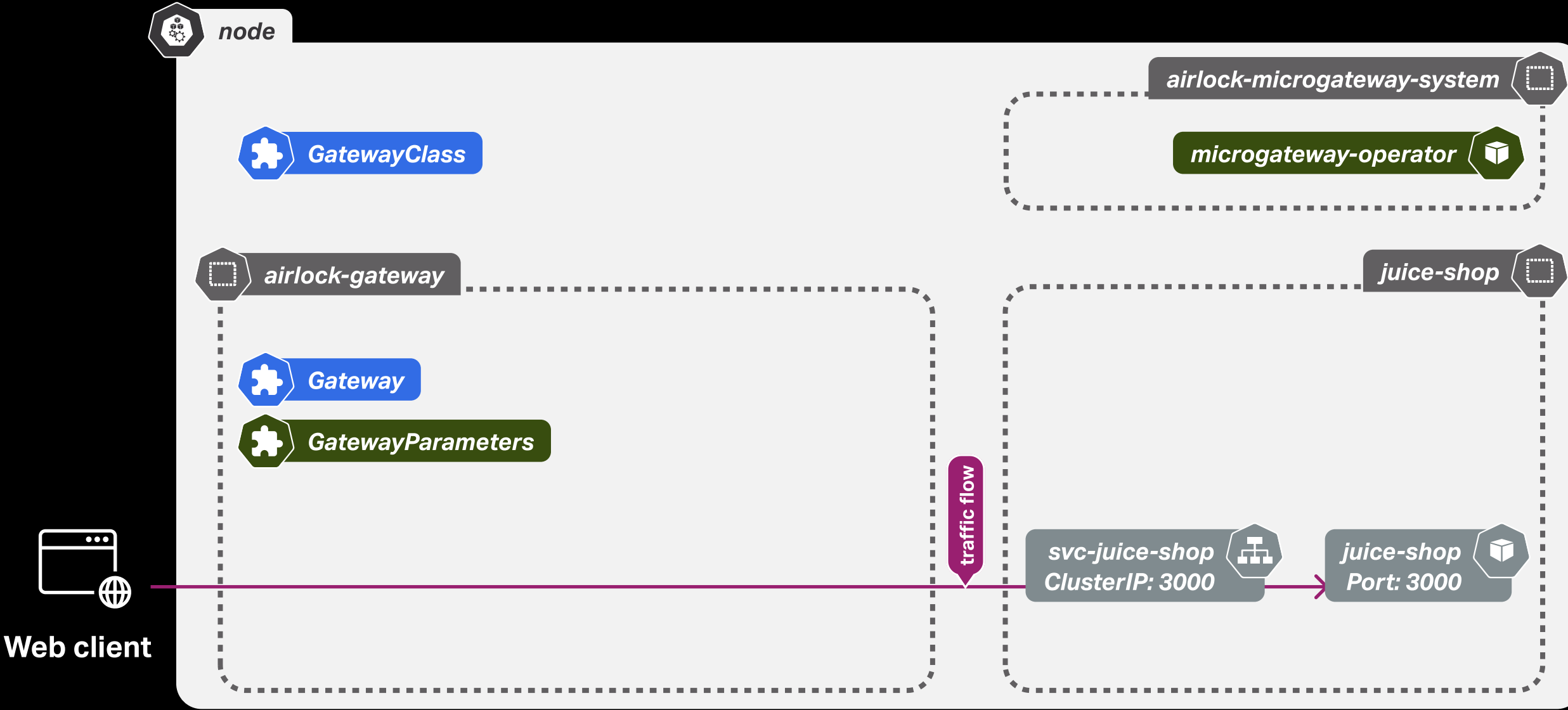
Access to Juice Shop, Traffic NOT Routed through the Airlock Microgateway Engine



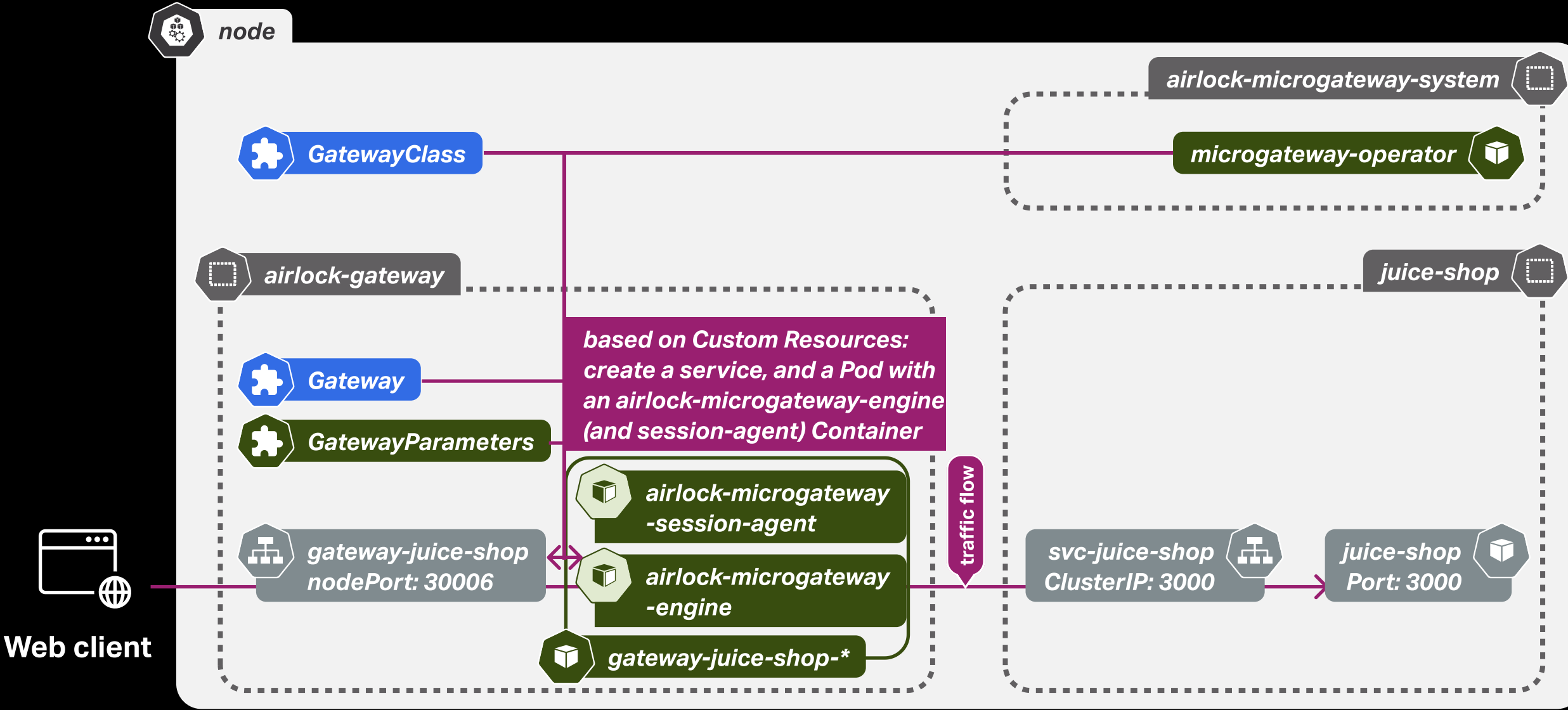
Change nodePort to ClusterIP for svc-juice-shop



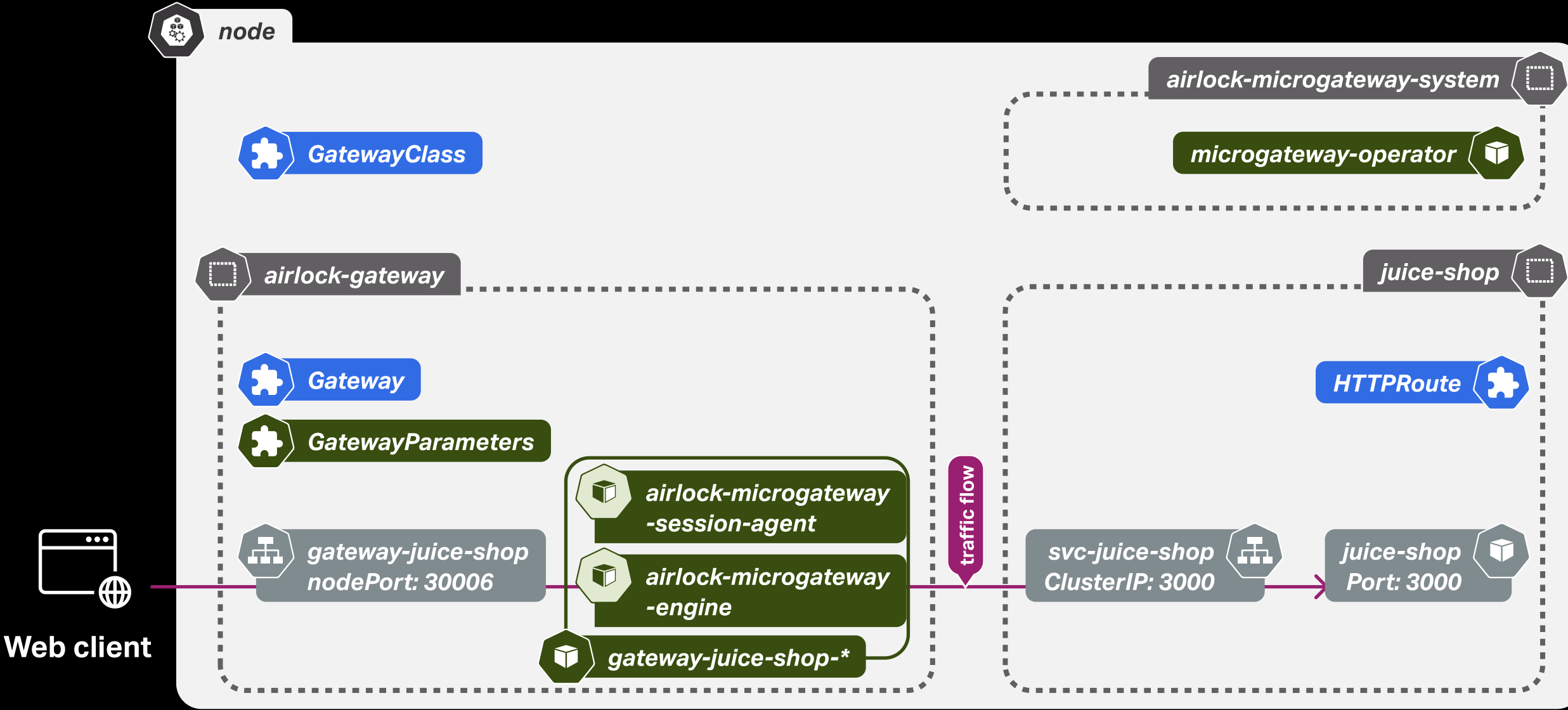
Deploy CRs to Create a Service and a Pod with an Airlock Microgateway Engine Container



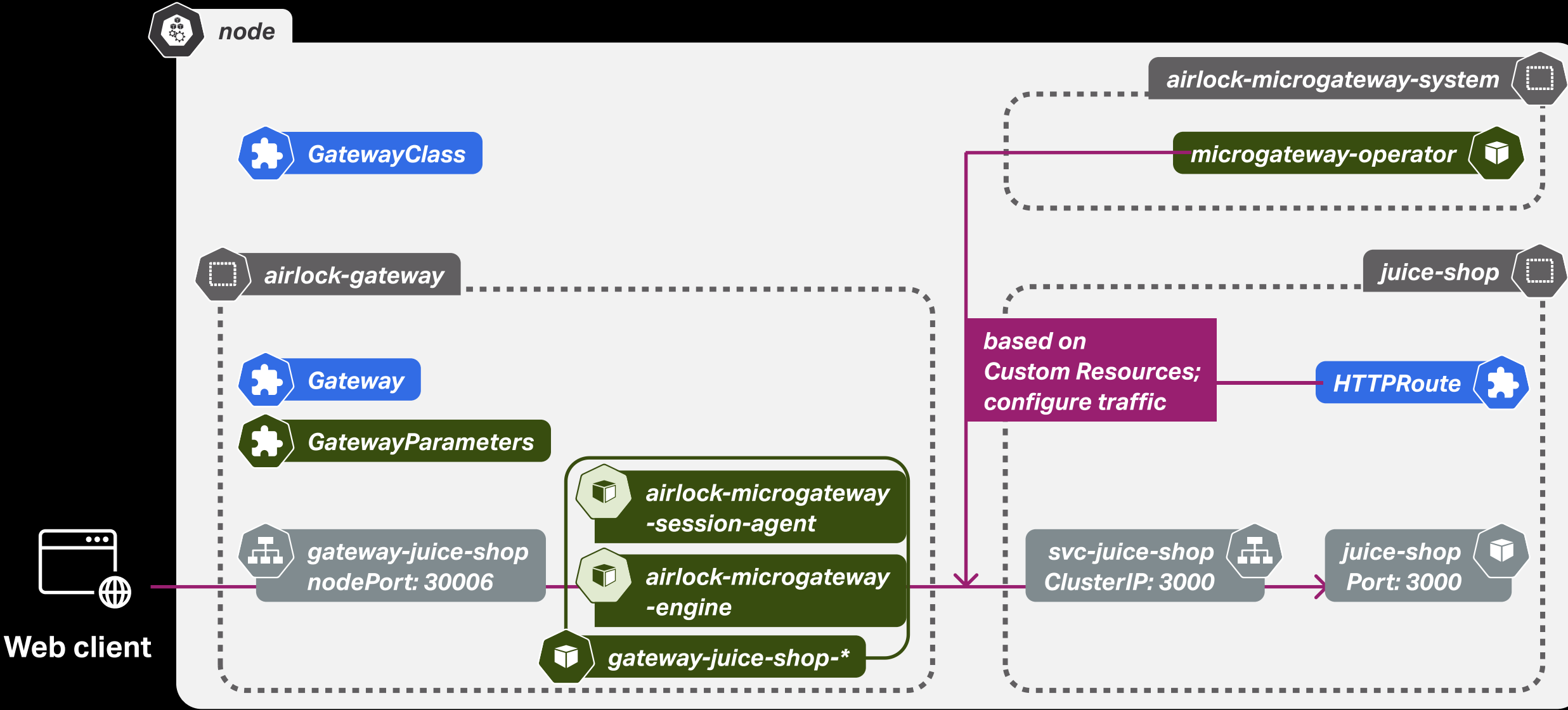
Deploy CRs to Create a Service and a Pod with an Airlock Microgateway Engine Container



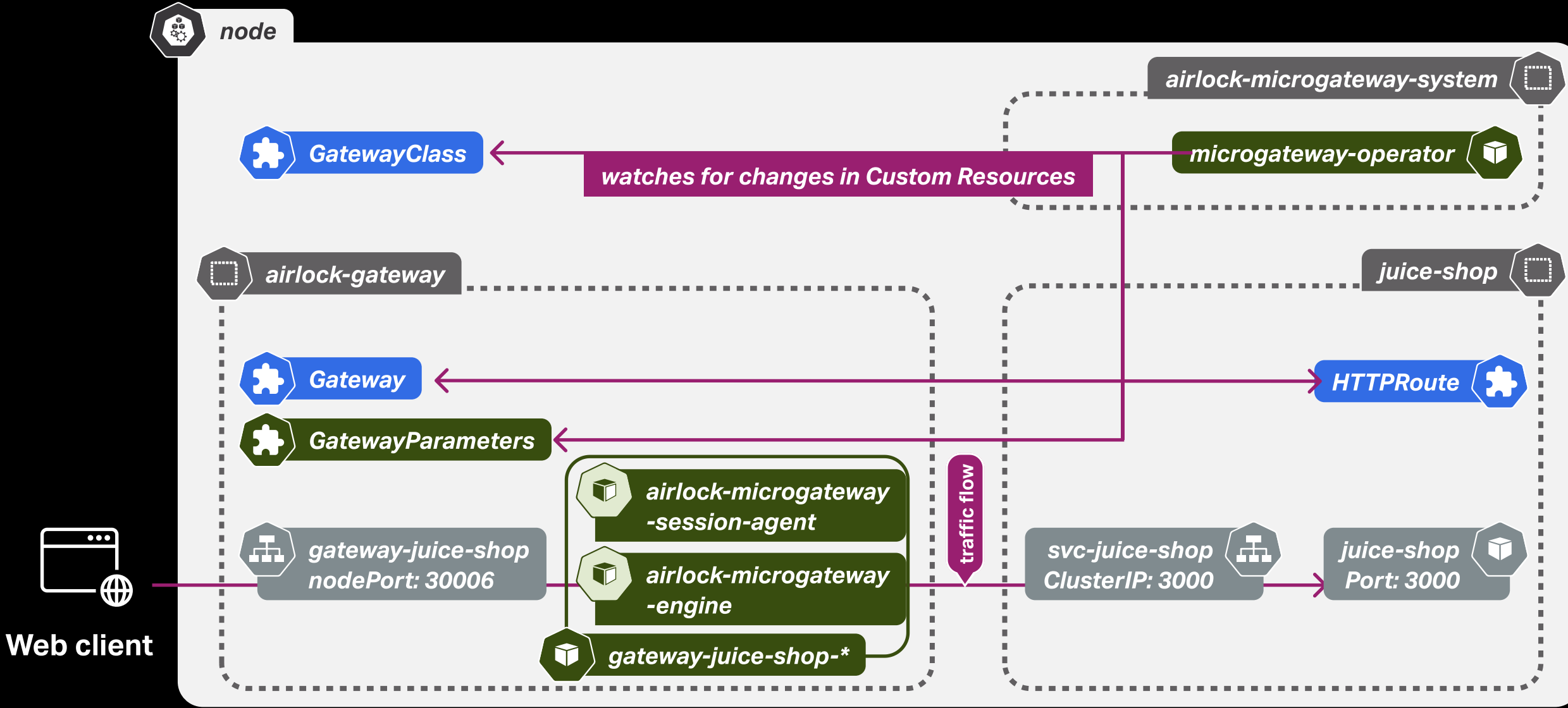
Deploy CRs to Configure Traffic between Airlock Microgateway and the Juice Shop Back-end



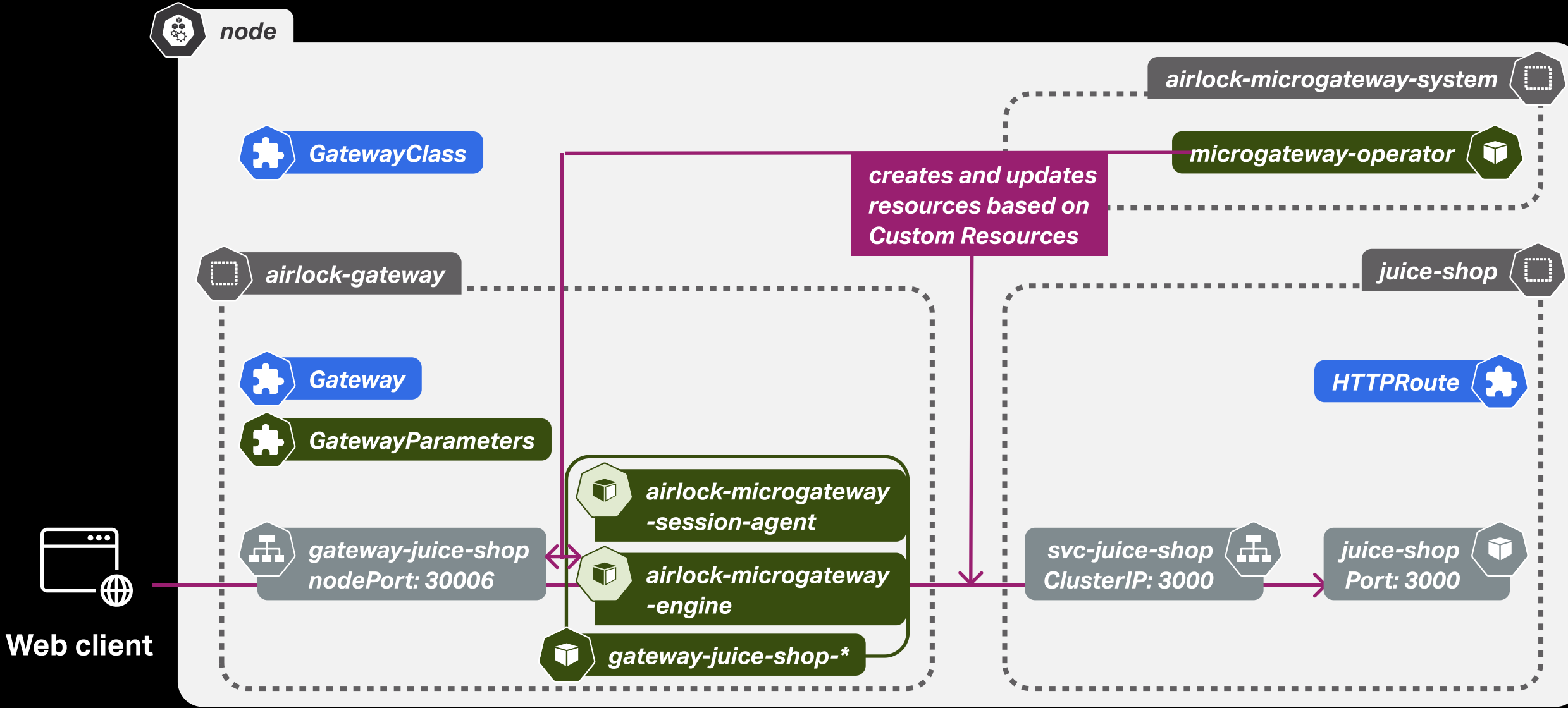
Deploy CRs to Configure Traffic between Airlock Microgateway and the Juice Shop Back-end



The Microgateway Operator Watches for Changes in CRs



The Microgateway Operator Creates and Updates Resources based on the State of the Configured CRs



Recap: Traffic Routing

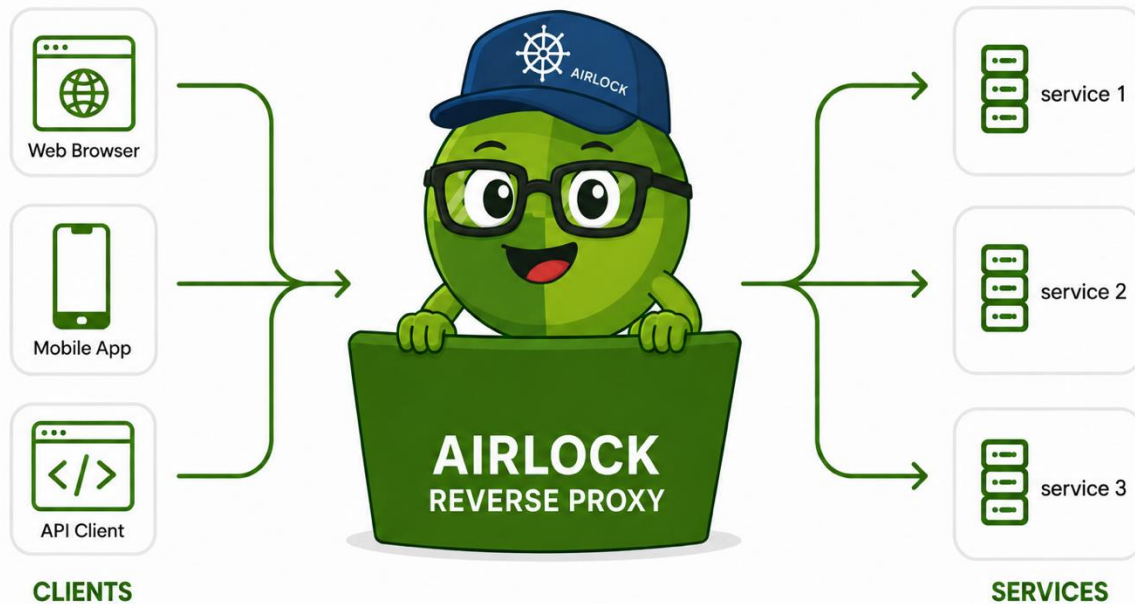
Default Content Security

Custom Microgateway Resources



**Basic Web-app
Protection**

«Reverse Proxy Only» as Default (New with **MGW 5.1**)



After Deployment

Content security defaults = **inactive**

Enable Content Security Defaults?

Only, if **module «filter»** is licensed

Enable Content Security Defaults (GatewayParameters)

Configure

`GatewayParameters.spec.defaults.contentSecurity = secured`

Implications

HTTPRoute without ContentSecurityPolicy

→ default content security processing

HTTPRoute with ContentSecurityPolicy

→ default content security processing + configured ContentSecurityPolicy

Content Security Defaults

Built-in content security defaults are applied, **if enabled**, to secure traffic routed via the Airlock Microgateway Engine

- built-in deny rules
- header sanitization
- limits
- etc.



**default settings for
Microgateway 5.1**
(see Microgateway
CRD Reference)

DenyRules.spec.request.builtIn.settings

Field	Description	Type	Required	Default	Allowed Values
level	represents a set of deny rules with different filter strengths.	enum	no	Standard	Basic, Standard, Strict, Unfiltered
threatHandlingMode	specifies how threats should be handled when a deny rule matches.	enum	no	Block	Block, LogOnly

HeaderRewrites.spec.request.allow.matchingHeaders.builtIn

Field	Description	Type	Required	Default	Allowed Values
standardHeaders	defines whether the request headers which are forwarded to the upstream will be restricted to a set of common request headers. ► Show details	bool	no	true	true, false

Limits.spec

Field	Description	Type	Required	Default	Allowed Values
request	defines the limits for requests.	object	no	limited{...}	limited{}, unlimited{}

Content Security Defaults

Built-in content security defaults are applied, **if enabled**, to secure traffic routed via the Airlock Microgateway Engine

- built-in deny rules
- header sanitization
- limits
- etc.

summary in tab
«**Default**» of
CRD Reference



Example

Default

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: default
spec:
  request:
    builtIn:
      settings:
        level: Standard
        threatHandlingMode: Block
    custom: {}
```

Recap: Traffic Routing
Default Content Security
Custom Microgateway Resources



**Basic Web-app
Protection**

Custom Resources (CRs)

K8s-style objects

Allow **customizing Microgateway behavior**

Declarative configuration of routing and security rules

- Kubernetes Gateway API CRs (e.g., HTTPRoute)
- Airlock Microgateway CRs (e.g., DenyRules)

→ **Airlock Microgateway Operator** watches for and reconciles changes to CRs

Custom Resources – Example «DenyRules»



Examples:

- Virus/malware scanner
- Airport security x-ray
- etc.

**Deny rules are necessary,
but on their own not
sufficient against today's
targeted attacks!**



Custom Resources – Example «DenyRules»



A **DenyRules CR** allows to configure

- Settings for built-in deny rules
- Overrides and exceptions for built-in deny rules
- Custom additional deny rules

Custom Resources – Example «DenyRules»



Example:

«Strict» mode + «Block» all threats

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: my-app
  namespace: my-app
spec:
  request:
    builtIn:
      settings:
        level: Strict
        threatHandlingMode: Block
```


Custom Resources – ContentSecurityPolicy

The DenyRules CR needs to be referenced by a ContentSecurityPolicy CR

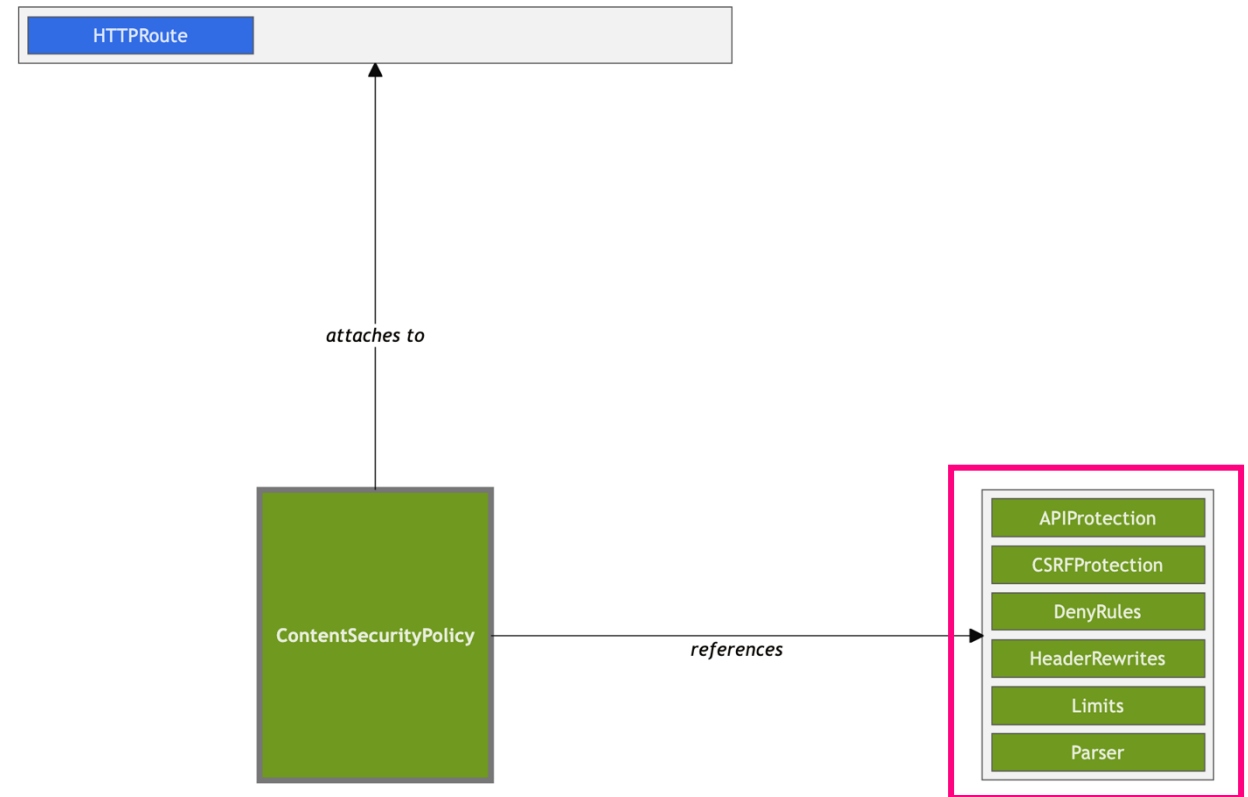
```
apiVersion:
microgateway.airlock.com/v1alpha1
kind: ContentSecurityPolicy
metadata:
  name: example-policy
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: backend
  denyRulesRef:
    name: example-denyrules
  headerRewritesRef:
    name: example-headerrewrites
```

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: example-denyrules
spec:
  request:
    builtIn:
      overrides:
        - conditions:
            ruleKeys: ["HTML"]
          settings:
            level: Basic
```

Custom Resources – ContentSecurityPolicy References

[https://docs.airlock.com/microgateway/\[ver\]/index/api/crds/microgateway/content-security-policy/v1alpha1/](https://docs.airlock.com/microgateway/[ver]/index/api/crds/microgateway/content-security-policy/v1alpha1/)

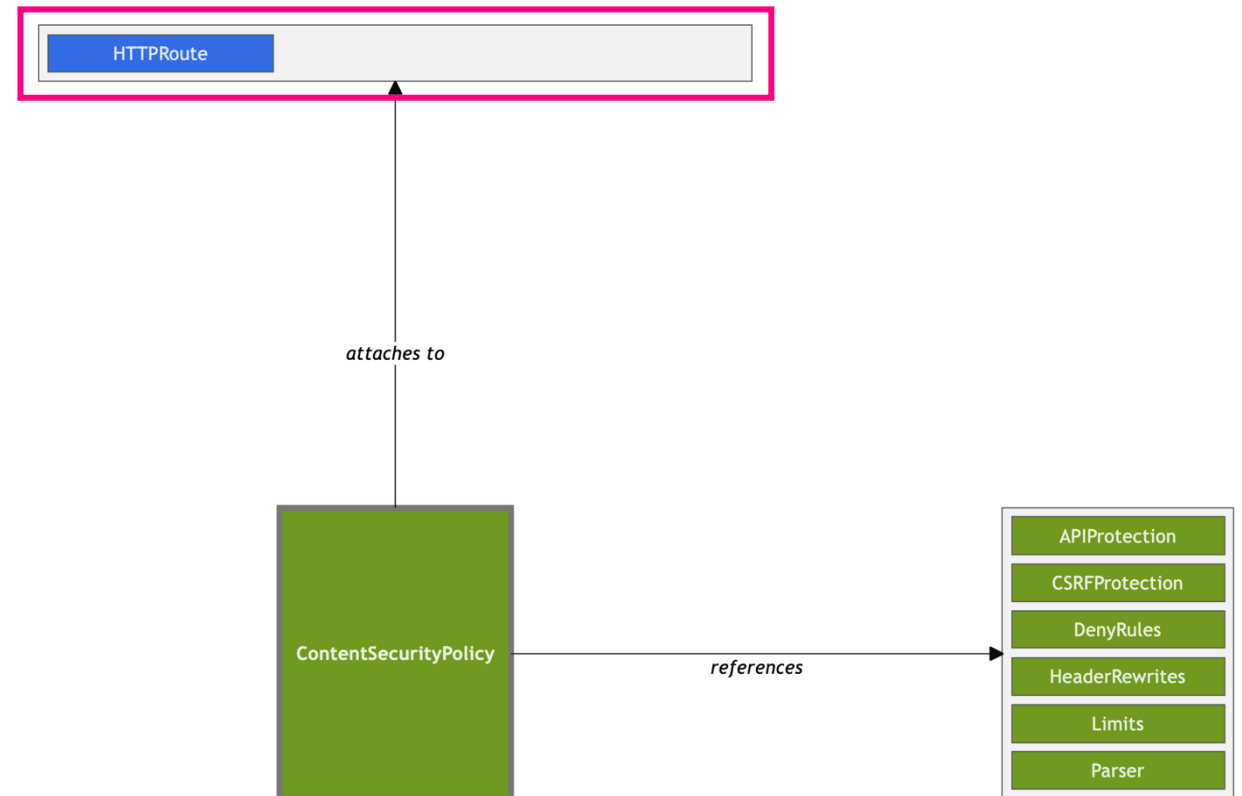
i. e., **[ver]** could be **5.1**



Custom Resources – ContentSecurityPolicy Attachement

A ContentSecurityPolicy CR attaches security configurations to K8s Gateway API

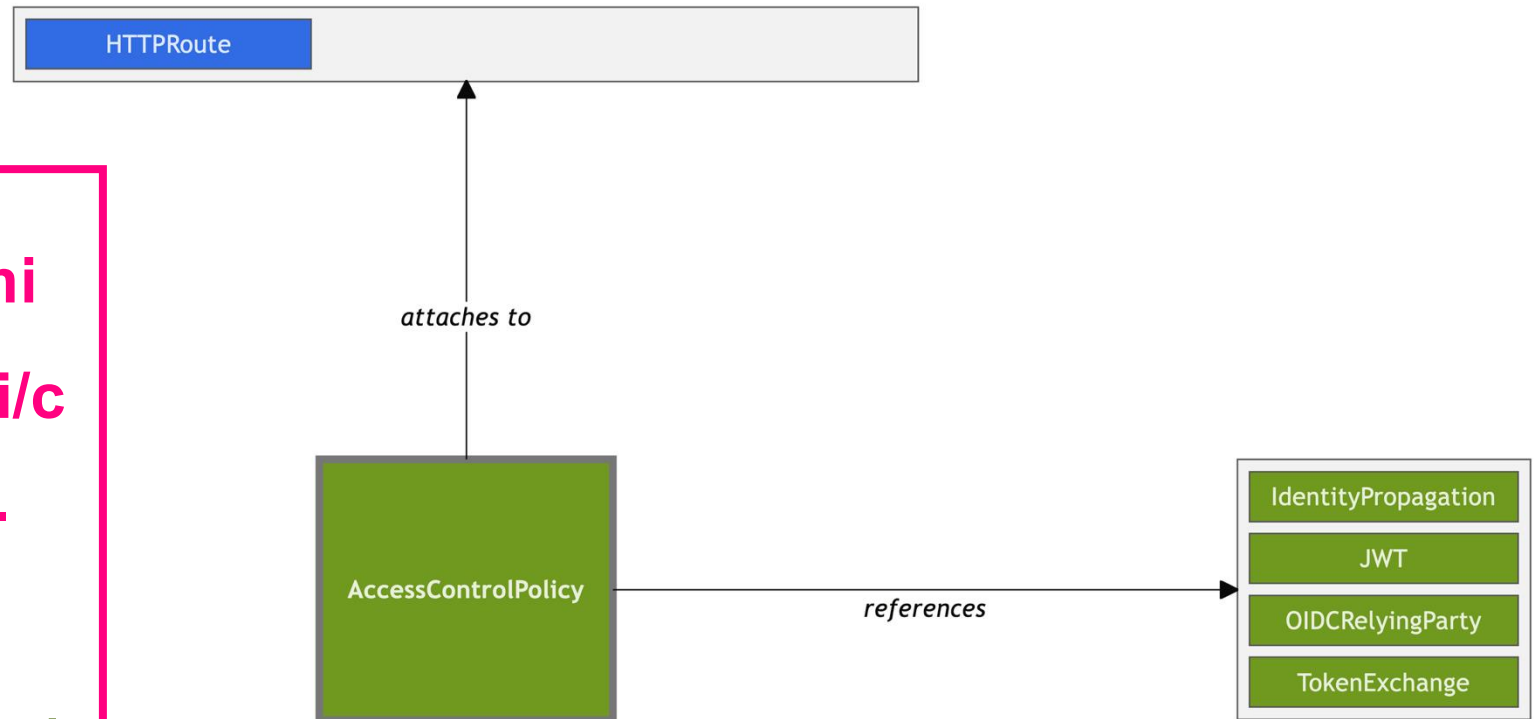
```
apiVersion:
microgateway.airlock.com/v1alpha1
kind: ContentSecurityPolicy
metadata:
  name: example-policy
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: backend
  denyRulesRef:
    name: example-denyrules
  headerRewritesRef:
    name: example-headerrewrites
```



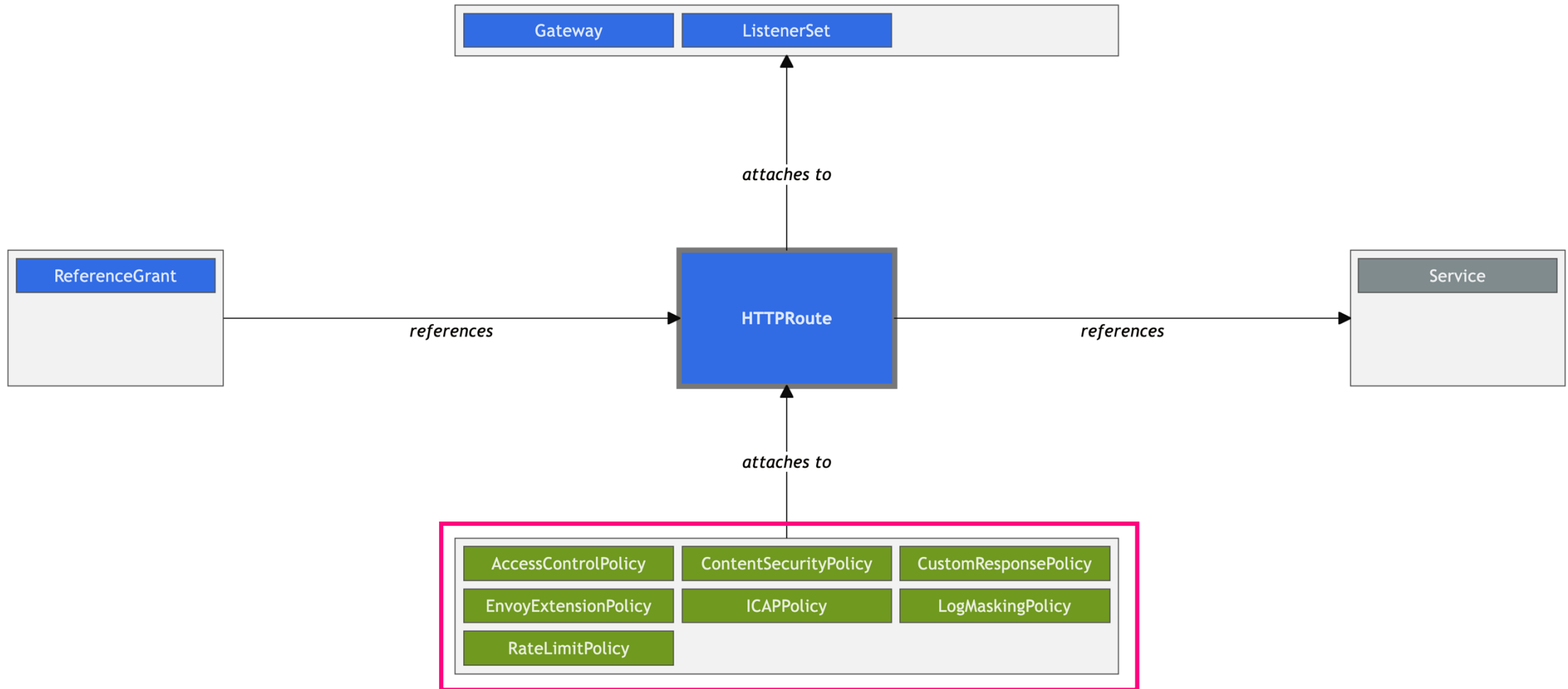
Custom Resources – AccessControlPolicy

[https://docs.airlock.com/microgateway/\[ver\]/index/api/crds/microgateway/access-control-policy/v1alpha1/](https://docs.airlock.com/microgateway/[ver]/index/api/crds/microgateway/access-control-policy/v1alpha1/)

i. e., **[ver]** could be **5.1**



Custom Resources – Other Policies



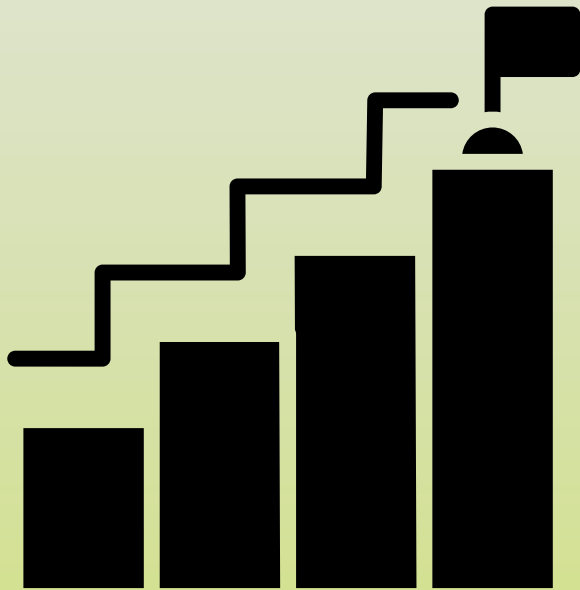
LAB 02

First Webapp



LAB 02

First Webapp



 50-60 min

Steps

- Check if the Juice-Shop back-end is protected
- Configure traffic routing with Custom Resources (CRs)
- Fix a broken Custom Resources configuration
- Deploy additional Custom Resources

LAB 02

First Webapp



Any feedback?



Troubleshooting



How can you **troubleshoot** your Airlock Microgateway configuration if something goes wrong?

Troubleshooting – General Approach

1. Check resource conditions for potential problems
 - Start with **Gateway** CRs (skip, if you lack the permissions)
 - Continue with **HTTPRoute** CRs
 - Finish with **policy** CRs
 2. Review K8s events
 3. Inspect logs of the involved components
- We will mainly use **gwctl** instead of kubectl in the training!

Troubleshooting – Have a Look at the Doc's Article!

General approach

Troubleshooting Microgateway resources follows the same problem-solving approach as troubleshooting other Kubernetes workloads:

1. Check resource conditions for potential problems.
 - a. Start with inspecting the **Gateway** resource.
Note: If the Gateway resource is not accessible due to missing permissions, skip this step.
 - b. Continue with inspecting **HTTPRoute** resources.
 - c. Finish with inspecting policy resources.
2. Review events that may point to pending or unresolved issues.
3. Inspect logs of the involved components for traces or details of faulty behavior.

While **kubectl** can be used for all of these tasks, this guide uses **gwctl**, a tool specifically designed for Kubernetes Gateway API resources.

Check Gateway resource conditions

- Getting a status overview
- Verifying the GatewayClass
- Verifying the license validity
- Verifying the GatewayParameters
- Verifying all listeners
- Checking for invalid routes

Check HTTPRoute resource conditions

- Getting a status overview
- Verifying route attachment
- Checking faulty references
- Identifying invalid policies

Check policy resource conditions

- Getting a status overview
- Verifying policy attachment
- Checking faulty references

Check events

Check logs

- Microgateway Operator logs
- Microgateway Engine logs

[https://docs.airlock.com/
microgateway/5.1/index/1
762466259858.html](https://docs.airlock.com/microgateway/5.1/index/1762466259858.html)

Monitoring



How can you configure Airlock Microgateway for **log**
and metrics monitoring with **Grafana**?

What to Monitor and Analyze?

Log Access from Shell

How to Setup Monitoring?

Grafana Dashboards



Monitoring

What to Monitor and Analyze?

Log Access from Shell

How to Setup Monitoring?

Grafana Dashboards



Monitoring

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent

follows the
Elastic Common
Schema (ECS)



Tracing

for end-to-end
path of a
request

What to Investigate with the Monitoring Information?

e.g.,

- Track **blocked vs. allowed requests** over time
- Watch for **spikes** in attack types (SQLi, XSS, BOT, IDOR)
- Correlate **deny-rule activity** with app behavior
- Use **Grafana dashboards + Loki queries** for detail
- Investigate **false positives** and **attack origins**
- Identify **latency hotspots** and **failures** in distributed systems (tracing)

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Prometheus Metrics – Airlock Microgateway Engine

microgateway_license_http_rq_total

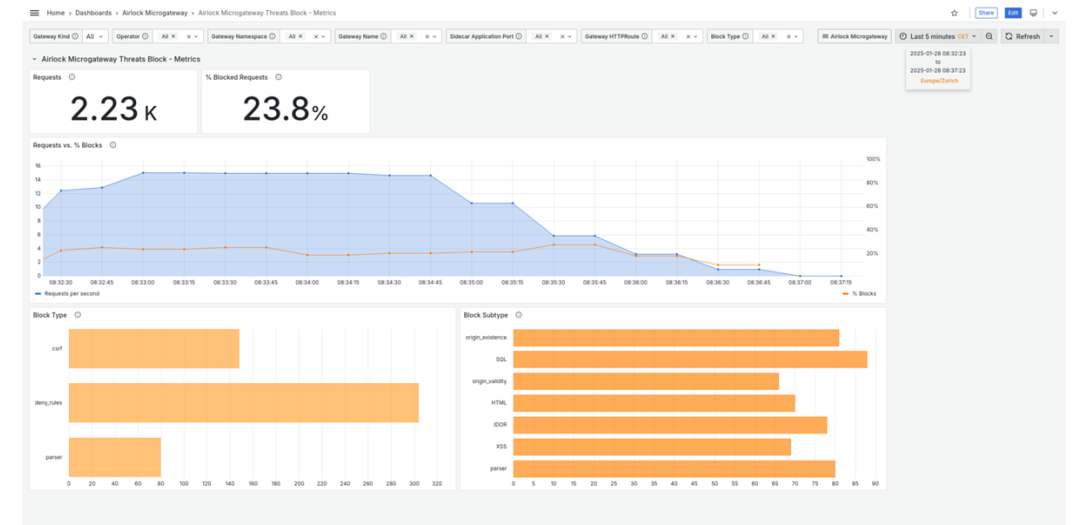
Number of total requests

microgateway_http_downstream_rq_threats_blocked_total

*Number of requests (*rq*) blocked with *threatHandlingMode: Block* by an Engine filter*

etc.

e.g., see the
«Airlock Microgateway
Thread Block Metrics»
dashboard



Prometheus Metrics – Airlock Microgateway Engine

microgateway_license_http_rq_total

Number of total requests

microgateway_http_downstream_rq_threats_blocked_total

*Number of requests (**rq**) blocked with **threatHandlingMode: Block** by an Engine filter*

etc.

<https://docs.airlock.com/microgateway/5.1/index/1683824575325.html>

Prometheus Metrics – Airlock Microgateway Operator

microgateway_license_valid

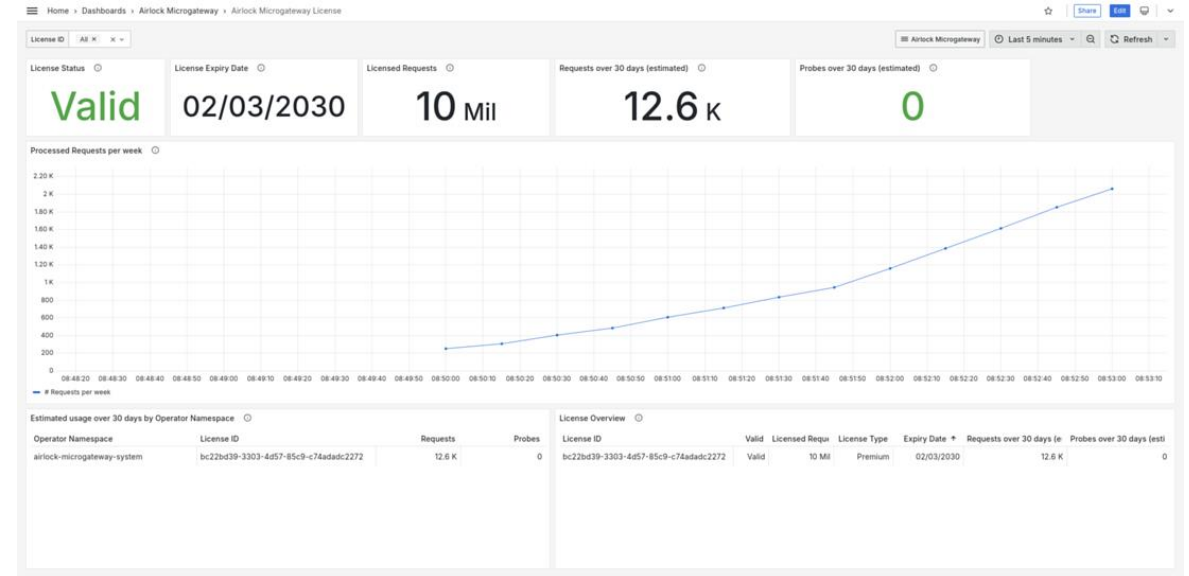
Whether the Airlock Microgateway Operator has a valid license 1 or not 0

microgateway_license_max_rq_count_per_month

The licensed request count per month

etc.

e.g., see the
«**Airlock Microgateway
License**» dashboard



Prometheus Metrics – Airlock Microgateway Operator

`microgateway_license_valid`

Whether the Airlock Microgateway Operator has a valid license 1 or not 0

`microgateway_license_max_rq_count_per_month`

The licensed request count per month

etc.

<https://docs.airlock.com/microgateway/5.1/index/1683824575325.html>

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Access Log – Airlock Microgateway Engine

Access log fields reflect logging output based on *Microgateway* command operators or *standard Envoy* command operators

Access Log – Airlock Microgateway Engine

airlock.access_control

Access control-related logs with authorization and authentication information

Microgateway command operator: %ACCESS_CONTROL%

event.start

Time the Microgateway Engine received the request

Standard Envoy command operator: %START_TIME%

etc.

example from
an access log



```
...  
"access_control": {  
  "authenticated": true,  
  "type": "oidc",  
  "user_id":  
    "rEI5tGPdafd9BUdSj8S38504GjQ62En07XDddXkB  
    zp8;https://login.microsoftonline.com/009  
    63d21-d627-4229-9193-8b9516bf9782/v2.0"  
}  
...
```

Access Log – Airlock Microgateway Engine

airlock.access_control

Access control-related logs with authorization and authentication information

Microgateway command operator: `%ACCESS_CONTROL%`

event.start

Time the Microgateway Engine received the request

Standard Envoy command operator: `%START_TIME%`

etc.

access log fields are reflected in dashboards;
e.g., see
«Airlock Microgateway Access Control»



Request Size ▾	HTTP Status ▾	Access Control Policy ▾	User ID ▾	Authenticated ▾	Authorized ▾	Status ▾
162 B	302	default[0]		false	false	pending
168 B	302	default[0]		false	false	pending
156 B	200	default[0]	1https://authlib.org	true	true	completed
156 B	200	default[0]	1https://authlib.org	true	true	completed
267 B	302	default[0]	1https://authlib.org	true	true	completed
162 B	302	default[0]		false	false	pending
162 B	401			false	false	pending

Access Log – Airlock Microgateway Engine

airlock.access_control

Access control-related logs with authorization and authentication information

Microgateway command operator: %ACCESS_CONTROL%

event.start

Time the Microgateway Engine received the request

Standard Envoy command operator: %START_TIME%

etc.

a **Telemetry CR**
can be used to
customize the
access log



```
...
event:
  ...
  start: "%START_TIME(%Y-%m-%dT%T.%3f%z)%"
  ...
airlock:
  access_control: "%ACCESS_CONTROL%"
...
```

Access Log – Airlock Microgateway Engine

airlock.access_control

Access control-related logs with authorization and authentication information

Microgateway command operator: %ACCESS_CONTROL%

event.start

Time the Microgateway Engine received the request

Standard Envoy command operator: %START_TIME%

etc.

<https://docs.airlock.com/microgateway/5.1/index/1683824575325.html>

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Application Log – Airlock Microgateway Engine

Surfaces **Envoy logs** with related log categories:

- upstream
- admin
- config
- runtime
- etc.

```
...
  "log": {
    "logger": "upstream",
    "level": "info",
    "origin": {
      "file": {
        "name": "external/envoy/source/common/listener_manager/lds_api.cc",
        "line": 109
      },
      "function": "onConfigUpdate"
    }
  },
  "event": {
    "module": "envoy",
    "dataset": "envoy.application"
  },
  "message": "lds: add/update listener 'tcp-30080'"
...

```

Application Log – Airlock Microgateway Operator

Shows information related to **control-plane operations**



Application Log – Airlock Microgateway SessionAgent

Recommendations

- Check the **Access logs** for session-related error handling instead
- SessionAgent log only as last resort
(if you have deep background knowledge)

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Tracing – What can You do with It?

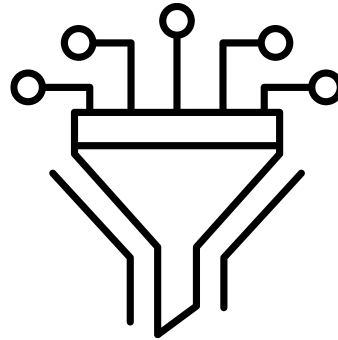
Identify **latency hotspots** and **failures** in distributed systems

Tracing support is based on W3C Trace Context specification

Microgateway can export traces to collectors / back-end services

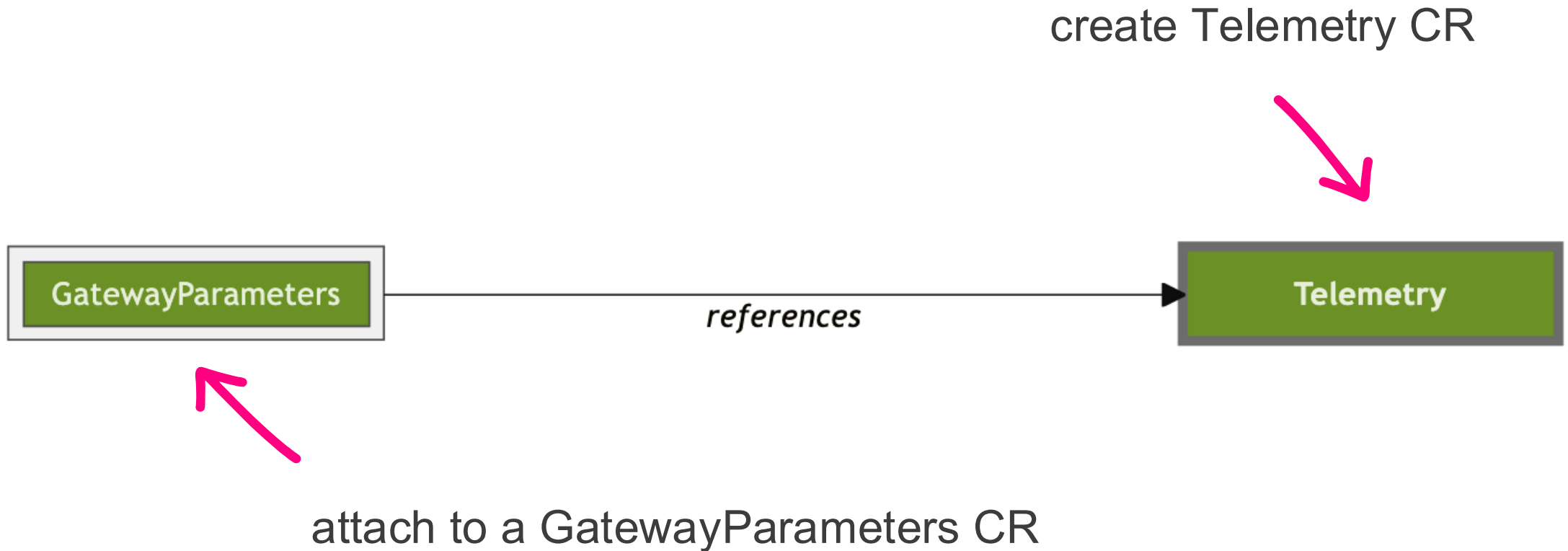
Tracing – Prerequisite

OpenTelemetry-compatible collector / back-end



e.g., Grafana Tempo, Jaeger, ...

Tracing – Configuration



Tracing – Sampling Strategies



Tail-based Sampling

Collector decides



Head-based Sampling



Microgateway decides

Tracing – Sampling Strategies: Tail-based Sampling

Telemetry.spec.tracing.provider.openTelemetry.sampling.strategy



Tail-based Sampling

Collector decides



always

sample on
every request

Tracing – Sampling Strategies: Head Sampling

Telemetry.spec.tracing.provider.openTelemetry.sampling.strategy

random

sample random based on a defined probability

inheritParentDecision

inherit sampling decision from parent span

→ *If no parents span: fallback strategy*



Head-based Sampling



Microgateway decides

Tracing – Example

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: Telemetry
metadata:
  name: telemetry-example
  namespace: example-backend-gw
spec:
  tracing:
    provider:
      openTelemetry:
        serviceName: "telemetry-example"
        sampling:
          strategy:
            always: {}
        exporter:
          grpcEndpoint:
            uri: "https://opentelemetry-collector.observability:4317"
```

What to Monitor and Analyze?

Log Access from Shell

How to Setup Monitoring?

Grafana Dashboards



Monitoring

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Log Access from Shell – Access Log

Use **kubectl logs** and filter for
.log.logger:"access" to inspect the
raw Airlock Microgateway **Engine**
Access logs

```
kubectl -n <airlock-gateway-namespace> logs -l  
gateway.networking.k8s.io/gateway-name= <my-  
gateway-name> \  
-c airlock-microgateway-engine -f \  
| jq 'select(.log.logger=="access" and  
.airlock.summary.details=="airlock_request_blocked_  
_deny_rule")'
```

```
...  
"airlock": {  
  "summary": {  
    "action": "denied",  
    "details": "airlock_request_blocked_deny_rule",  
    "flags": "-"  
  },  
  "actions": {  
    "block": {  
      "details": {  
        "deny_rules": {  
          "level": "standard",  
          "rule_id": 2,  
          "blocked_data": {  
            "json": {  
              "json_path": "$['email']",  
              "value": "admin' or 1=1--"  
            }  
          },  
          "rule_key": "SQL"  
        }  
      },  
      "block_subtype": "SQL",  
      "block_type": "deny_rules"  
    }  
  },  
  ...  
}
```

Log Access from Shell – Access Log

Use **jq** to filter for

- response code details, e.g.,
airlock_request_blocked_deny_rule
- other fields, e.g., **rule_key**, **path**,
blocked_data

Different response code details can appear in the **airlock.summary.details** log field

```
...
"airlock": {
  "summary": {
    "action": "denied",
    "details": "airlock_request_blocked_deny_rule",
    "tags": "-"
  },
  "actions": {
    "block": {
      "details": {
        "deny_rules": {
          "level": "standard",
          "rule_id": 2,
          "blocked_data": {
            "json": {
              "json_path": "$['email']",
              "value": "admin' or 1=1--"
            }
          },
          "rule_key": "SQL"
        }
      },
      "block_subtype": "SQL",
      "block_type": "deny_rules"
    }
  },
  ...
}
```

Log Access from Shell – Response Code Details

airlock_request_blocked_deny_rule

Request blocked; a deny rule detected malicious data

airlock_request_blocked_encoding

Request blocked; contains data with invalid encoding (UTF-8 required)

etc.

<https://docs.airlock.com/microgateway/5.1/index/1683824575325.html>

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Log Access from Shell – Engine Application Log

Use **kubectl logs** and filter for logs **without** **"logger": "access"** to inspect the raw Airlock Microgateway **Engine Application logs**

```
kubectl -n <airlock-gateway-namespace> logs \
  -l gateway.networking.k8s.io/gateway-name=<my-gateway-name> \
  -c airlock-microgateway-engine -f \
  | jq 'select(.log.logger!="access")'
```



```
...
  "log": {
    "logger": "upstream",
    "level": "info",
    "origin": {
      "file": {
        "name": "external/envoy/source/common/
listener_manager/lds_api.cc",
        "line": 109
      },
      "function": "onConfigUpdate"
    },
  },
  "event": {
    "module": "envoy",
    "dataset": "envoy.application"
  },
  "message": "lds: add/update listener 'tcp-30080'"
...

```


What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Log Access from Shell – Operator Application Log

Use **kubectl logs** to inspect the Airlock
Microgateway **Operator Application logs**

```
kubectl -n airlock-microgateway-system logs -l  
app.kubernetes.io/name=microgateway-operator -f \  
| jq
```



```
...  
"message": "Successfully reconciled GatewayClass",  
"controller": "gatewayclass",  
"controller_group": "gateway.networking.k8s.io",  
"controller_kind": "GatewayClass",  
"gateway_class": {  
  "name": "airlock-microgateway"  
},  
"namespace": "",  
"name": "airlock-microgateway",  
"reconcile_id": "8e64279b-f4bf-466b-8832-ec8c0988227e",  
"log": {  
  "level": "info",  
  "logger": "manager"  
}  
...
```

What Monitoring Information is Available?



Prometheus Metrics

for Airlock
Microgateway
Engine and
Operator



Access Log

for Airlock
Microgateway
Engine



Application Log

for Airlock Microgateway
Engine, **Operator**, and
SessionAgent



Tracing

for end-to-end
path of a
request

Log Access from Shell – Session Agent Application Log

Use **kubectl logs** to inspect the Airlock Microgateway **Session Agent Application logs** (only if session handling is enabled)

```
kubectl -n <airlock-gateway-namespace> logs \
-l gateway.networking.k8s.io/gateway-name=<my-gateway-name> \
-c airlock-microgateway-session-agent -f \
| jq
```

What to Monitor and Analyze?

Log Access from Shell

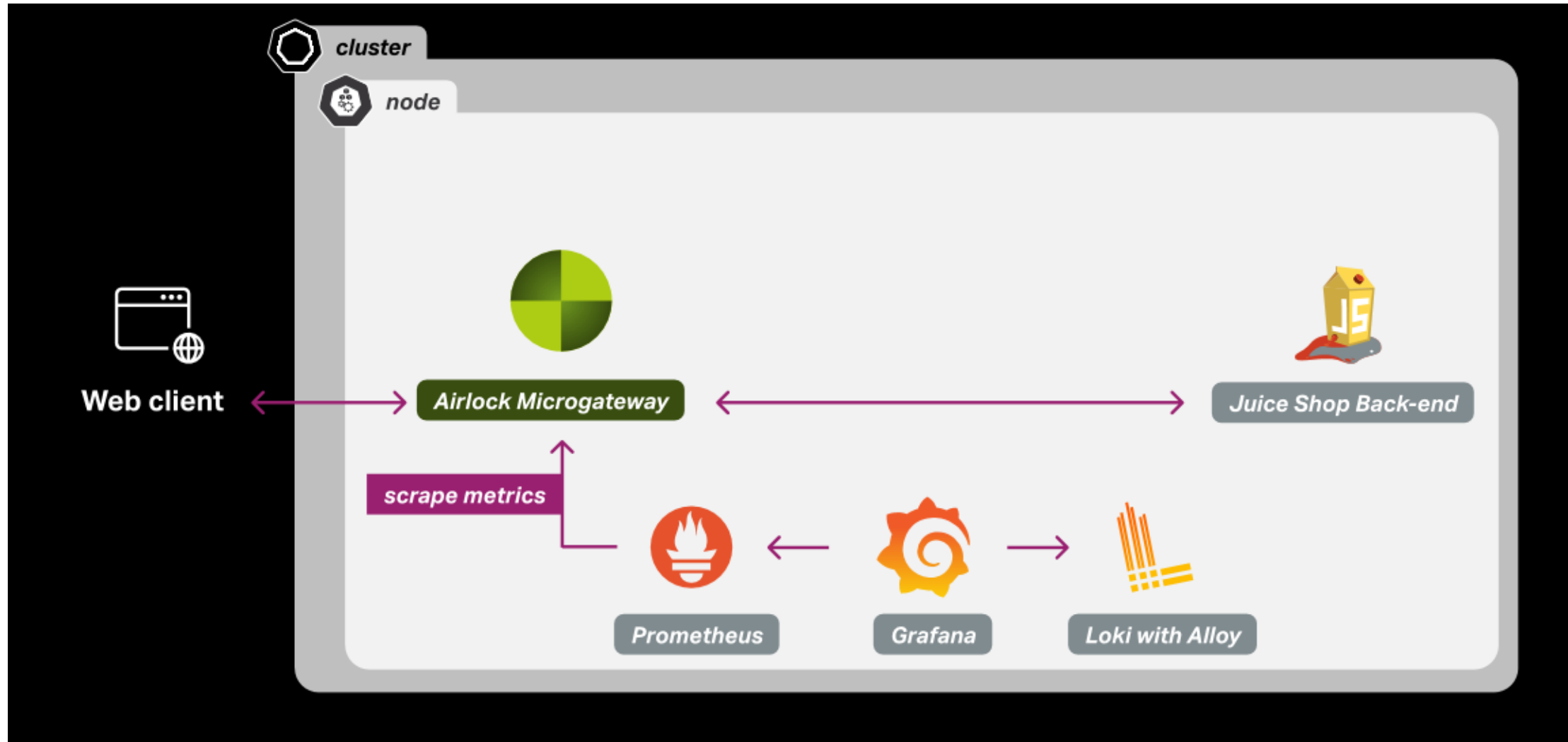
How to Setup Monitoring?

Grafana Dashboards



Monitoring

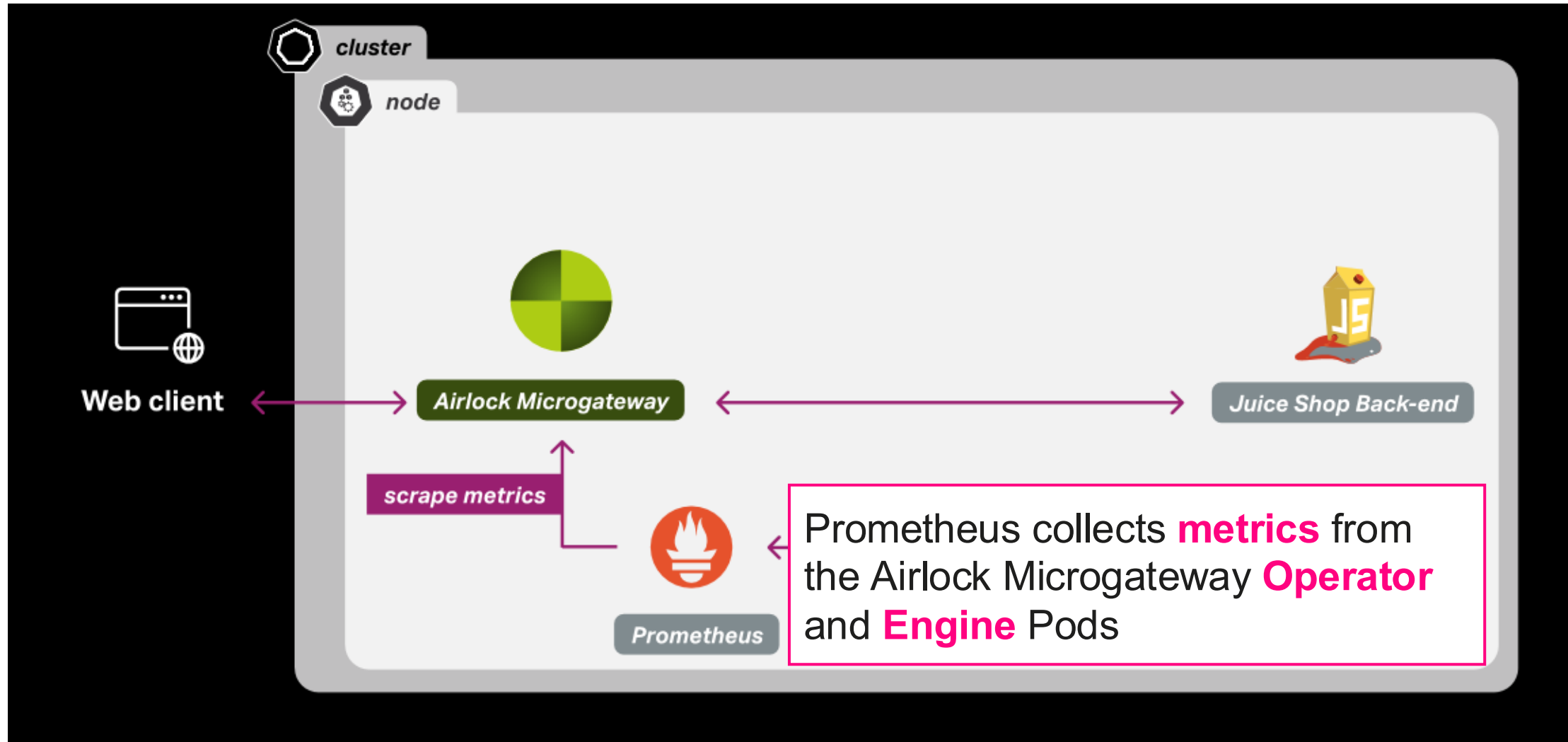
Monitoring Setup



Monitoring Setup



Monitoring Setup – Prometheus



Monitoring Setup – Prometheus Default Ports

Two default Airlock Microgateway ports are available

Type	Port
Engine metric	19002
Operator metric	8080



Assure that metrics are **NOT accessible outside your network!**

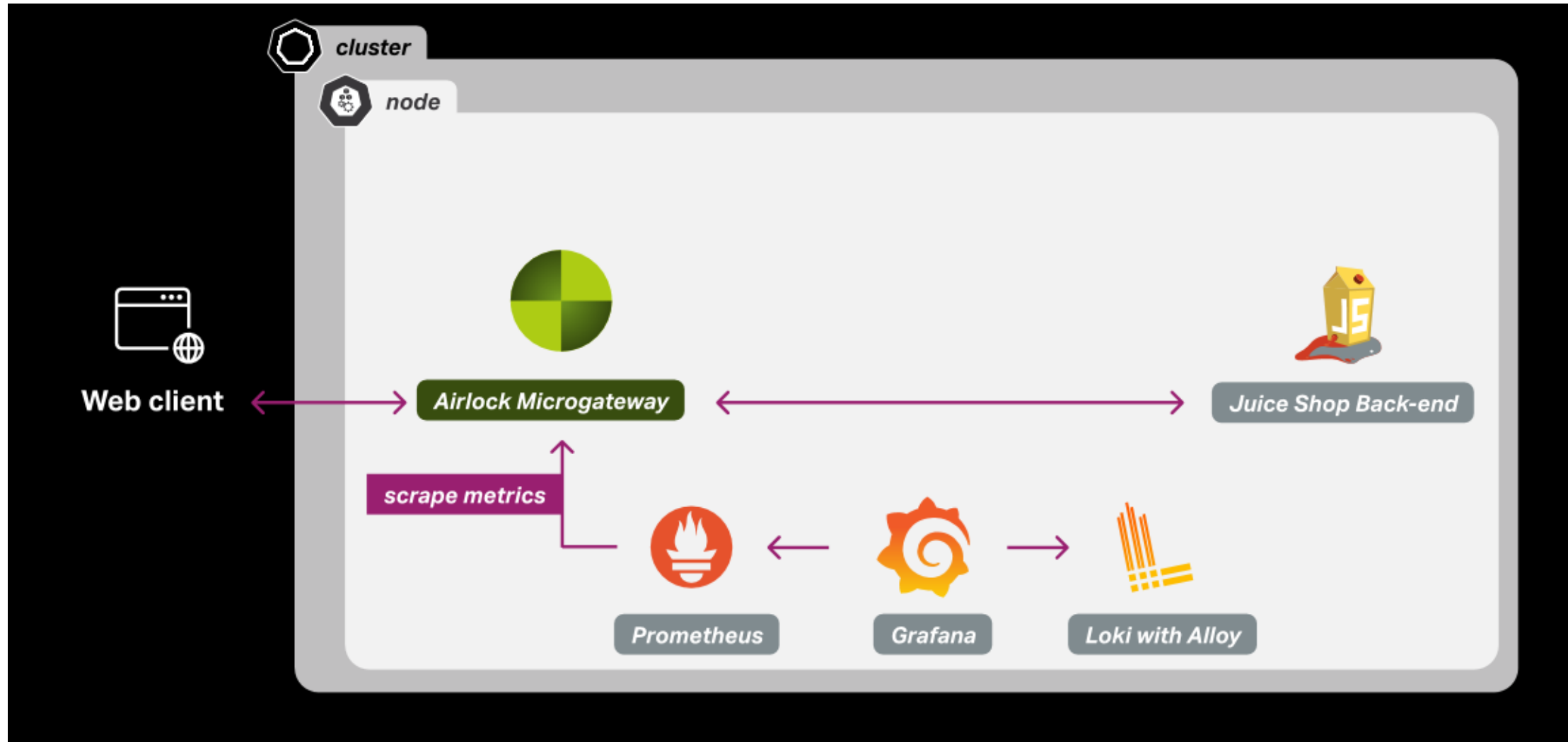
Monitoring Setup – Prometheus Monitors

A **serviceMonitor** and **podMonitor** configuration is required to scrape the Microgateway metrics

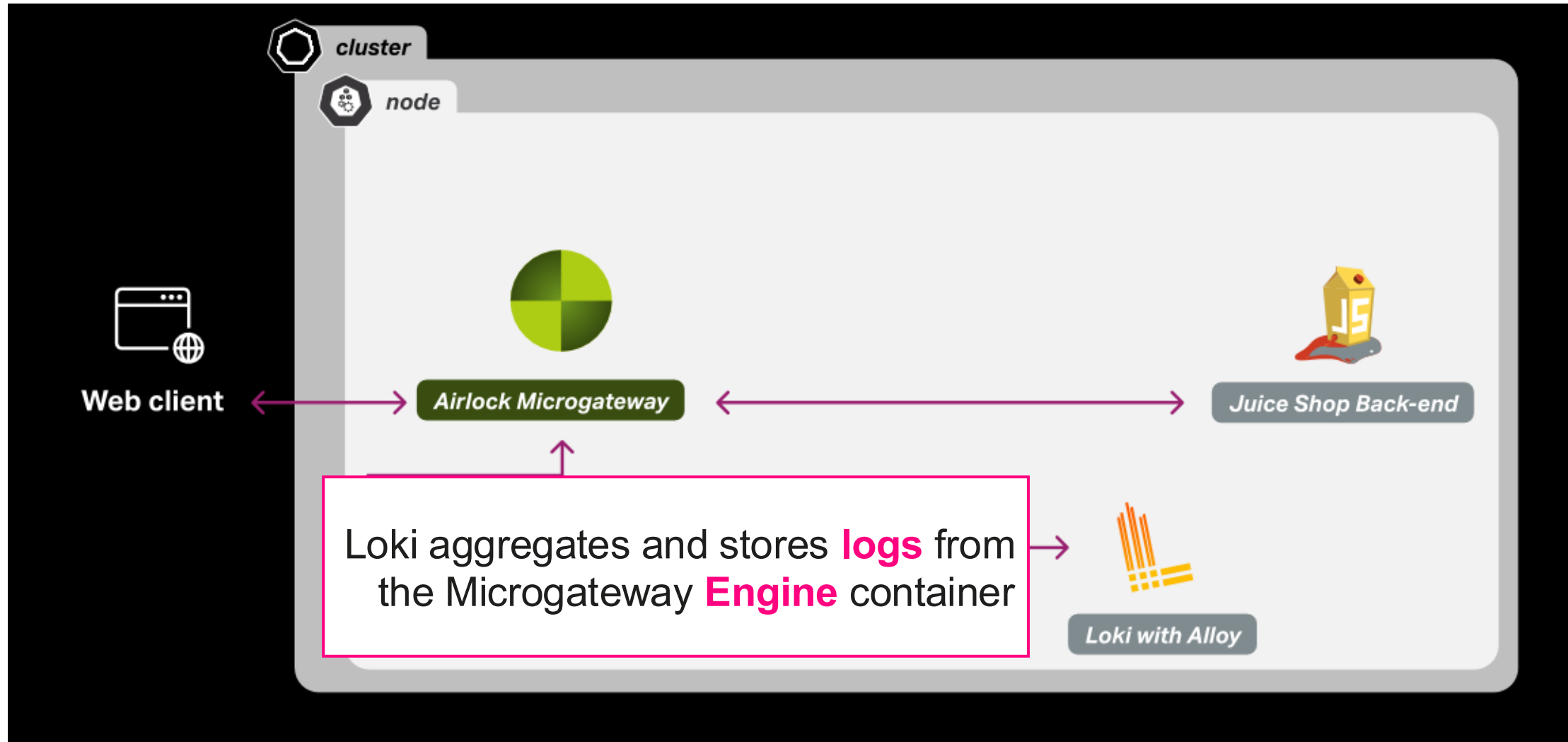
Airlock Microgateway needs to be deployed with:

```
--set config.gatewayPodMonitor.create=true \  
--set config.gatewayPodMonitor.labels.release=<PROMETHEUS-OPERATOR-RELEASE> \  
--set serviceMonitor.create=true \  
--set serviceMonitor.labels.release=<PROMETHEUS-OPERATOR-RELEASE>
```

Monitoring Setup



Monitoring Setup – Loki + Log Agent

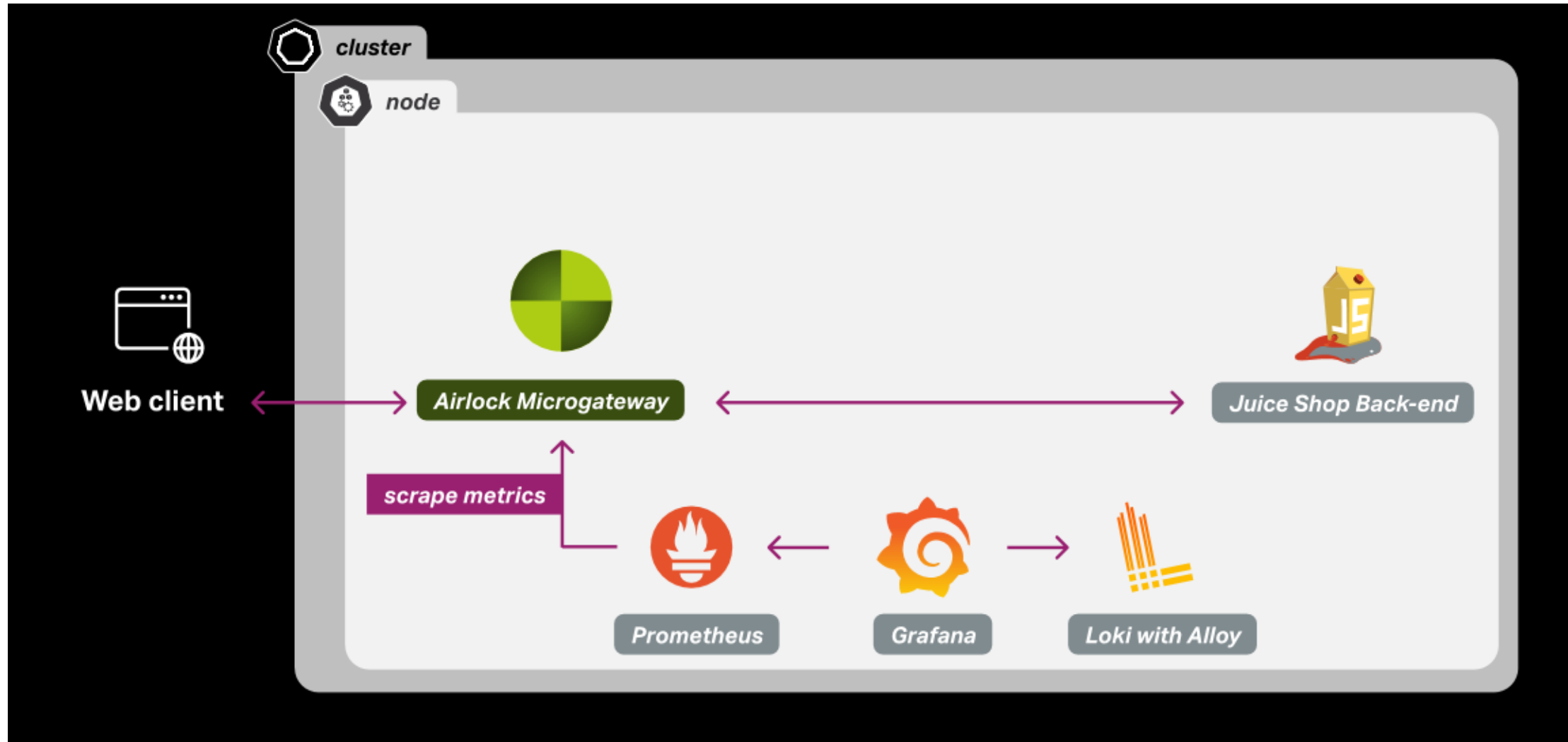


Monitoring Setup – Loki + Log Agent

Grafana Loki needs to be installed in combination with a suitable log agent

- Alloy
- ~~Promtail~~ (EOL March 2026)

Monitoring Setup



Monitoring Setup – Grafana



Monitoring Setup – Grafana Installation

Grafana needs to be deployed with

```
sidecar.dashboards.enabled=true
```

so Grafana can automatically add, update, and remove Airlock Microgateway dashboards

Monitoring Setup – Grafana Dashboard Creation

Airlock Microgateway needs to be deployed with

```
--set dashboards.create=true
```

to enable the cluster-wide search for the dashboard templates



Only **one** Microgateway Operator **per cluster** may provide the Grafana dashboard templates (multiple not supported)

Monitoring Setup – Grafana Dashboard Creation

Default: **dashboards.create= true**

→ Creates all Microgateway dashboards

Do not require all dashboards?

→ Create only some!

→ **To do so:** overwrite `values.yaml`

```
instances:
  # Available dashboard instances that can be individually created/deployed.
overview:
  # -- Whether to create the overview dashboard.
  create: true
license:
  # -- Whether to create the license dashboard.
  create: true
blockMetrics:
  # -- Whether to create the block metrics dashboard.
  create: true false
blockLogs:
  # -- Whether to create the block logs dashboard.
  create: true
headerLogs:
  # -- Whether to create the header rewrite logs dashboard.
  create: true
logOnlyMetrics:
  # -- Whether to create the log only metrics dashboard
  create: true
logOnlyLogs:
  # -- Whether to create the log only logs dashboard.
  create: true
...
```

Monitoring Setup – Deployment In One Command

```
helm upgrade -i airlock-microgateway \
  oci://quay.io/airlockcharts/microgateway \
  --version '<VERSION>' \
  -n airlock-microgateway-system \
  --wait \
  --set serviceMonitor.create=true \
  --set serviceMonitor.labels.release=<PROMETHEUS-OPERATOR-RELEASE> \
  --set config.gatewayPodMonitor.create=true \
  --set config.gatewayPodMonitor.labels.release=<PROMETHEUS-OPERATOR-RELEASE> \
  --set dashboards.create=true \
  --reuse-values
```

e.g., '**<VERSION>**' = **5.1.0**

Monitoring Setup – Grafana Dashboard Artifacts

4.7.3 Latest

Version 4.7.3

Release description

Airlock Microgateway helps you to protect your services and APIs from unauthorized or malicious access with little effort. It is a lightweight and Kubernetes-native Web Application and API Protection (WAAP) solution designed to overcome the DevSecOps obstacles and to implement ZeroTrust.

This Airlock Microgateway release contains security and regular updates of dependencies and improvements.

Helpful links:

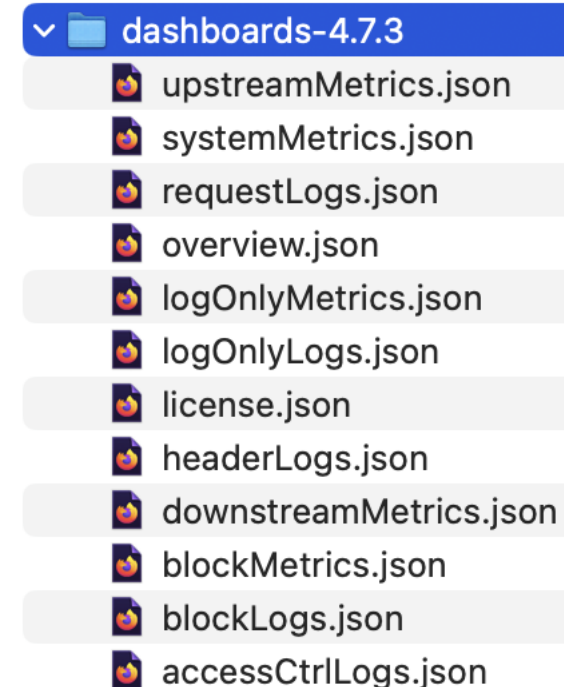
- [Documentation](#)
- [Airlock Microgateway Labs](#)
- [Airlock Community Forum](#)

Changelog

- SEC: AM-6277 Updated Go to v1.25.3, updated Envoy to v1.35.6, updated base images

Assets 3

 dashboards-4.7.3.zip	sha256:1e2fd3576d2ff7...	42.9 KB	last month
 Source code (zip)			last month
 Source code (tar.gz)			last month



<https://github.com/airlock/microgateway/releases>

What to Monitor and Analyze?

Log Access from Shell

How to Setup Monitoring?

Grafana Dashboards



Monitoring

Grafana Dashboard Overview

General Dashboards

Overview

License

Metrics Dashboards

Threats Block Metrics

Threats LogOnly Metrics

Upstream Metrics

System Metrics

Downstream Metrics

Access Control Metrics

Log Dashboards

Request Logs

Threats Block Logs

Threats LogOnly Logs

Access Control Logs

Rewrites Logs

Grafana Dashboard Overview

General Dashboards

Overview

License

Metrics Dashboards

Threats Block Metrics

Threats LogOnly Metrics

Upstream Metrics

System Metrics

Downstream Metrics

Access Control Metrics

Log Dashboards

Request Logs

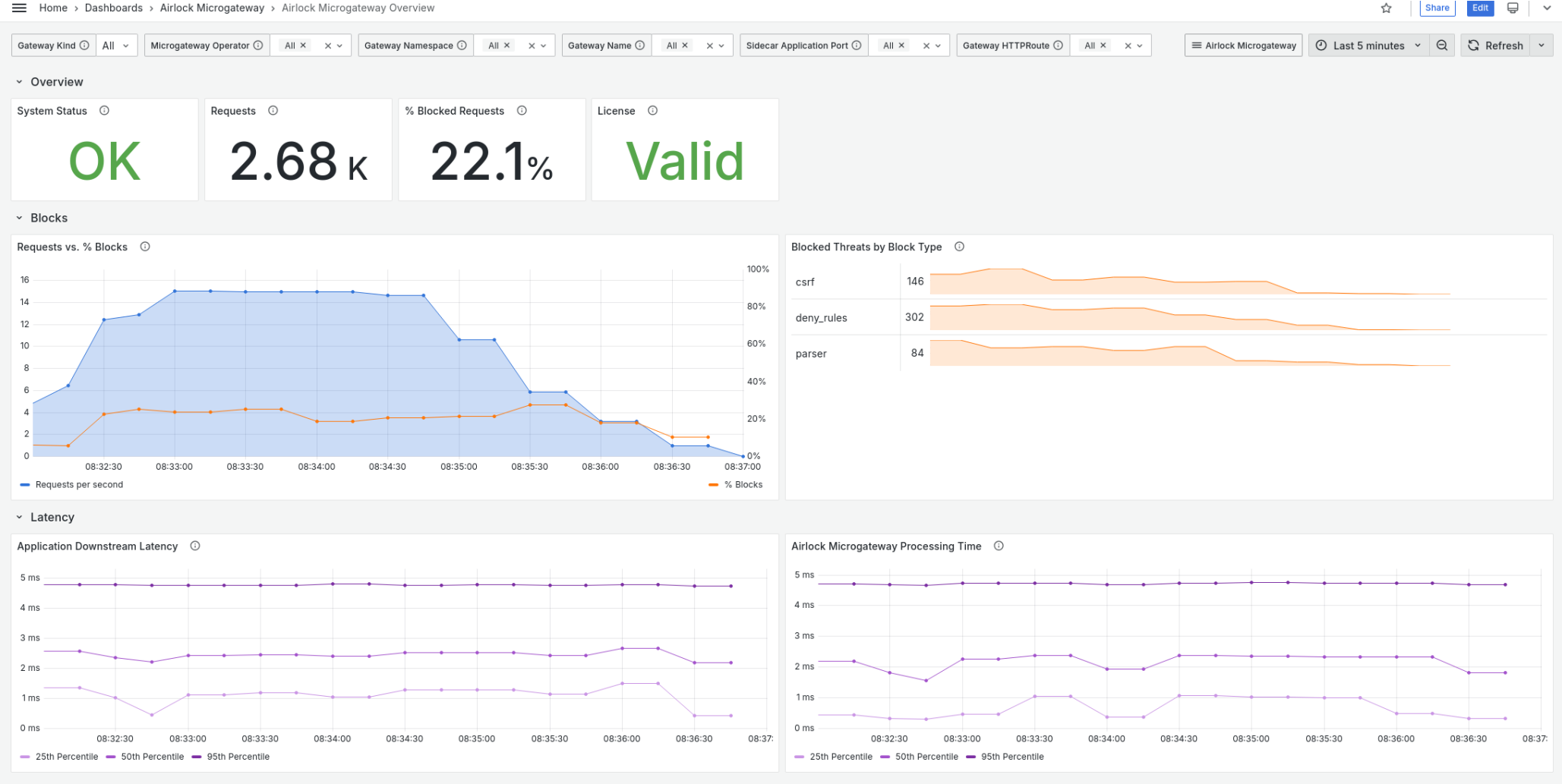
Threats Block Logs

Threats LogOnly Logs

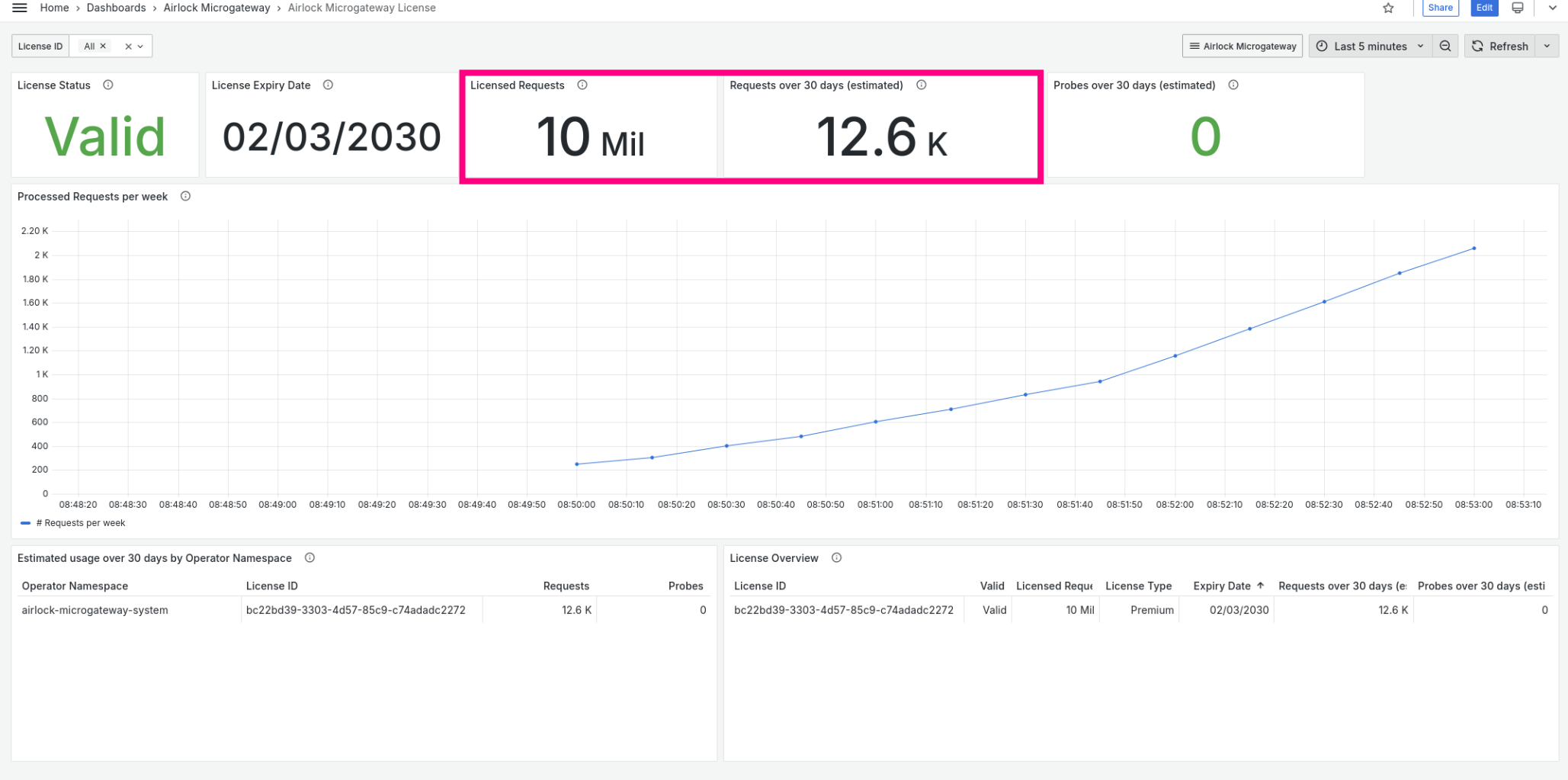
Access Control Logs

Rewrites Logs

General – Airlock Microgateway Overview



General – Airlock Microgateway License



Grafana Dashboard Overview

General Dashboards

Overview

License

Metrics Dashboards

Threats Block Metrics

Threats LogOnly Metrics

Upstream Metrics

System Metrics

Downstream Metrics

Access Control Metrics

Log Dashboards

Request Logs

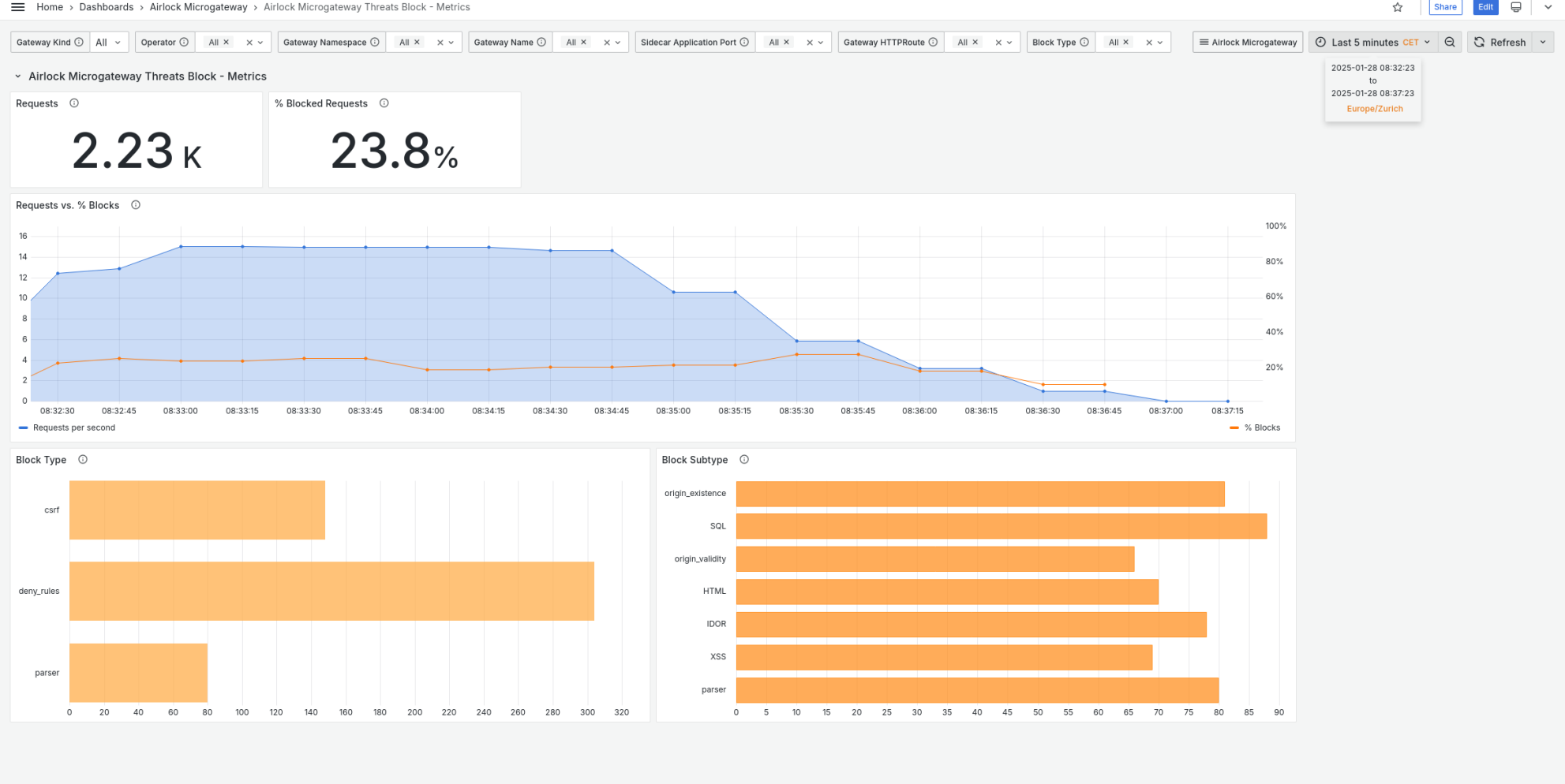
Threats Block Logs

Threats LogOnly Logs

Access Control Logs

Rewrites Logs

Metrics – Airlock Microgateway Threats Block



Grafana Dashboard Overview

General Dashboards

Overview

License

Metrics Dashboards

Threats Block Metrics

Threats LogOnly Metrics

Upstream Metrics

System Metrics

Downstream Metrics

Access Control Metrics

Log Dashboards

Request Logs

Threats Block Logs

Threats LogOnly Logs

Access Control Logs

Rewrites Logs

Logs – Airlock Microgateway Request

Grafana

Home

Starred

Dashboards

Explore

Drilldown

Alerting

Connections

Administration

Home > Dashboards > Airlock Microgateway > Airlock Microgateway Requests - Logs

Q Search...

+ v

🕒

Sign in

Edit

Export v

Share

Gateway Kind All

Microgateway Operator All x x v

Gateway Namespace All x x v

< 🕒 2025-09-12 17:31:15 to 2025-09-12 17:32:58 CEST v

> 🔍

Refresh v

Gateway Name All x x v

Airlock Microgateway

Request Logs ⓘ

Timestamp	Namespace	URL Domain	HTTP Method	URL Path	HTTP Status	Request Size	Response Size
2025-09-12 17:31:45.828	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.828	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B
2025-09-12 17:31:45.699	airlock-gateway	kubernetes	POST	/rest/user/login	400	295 B	607 B

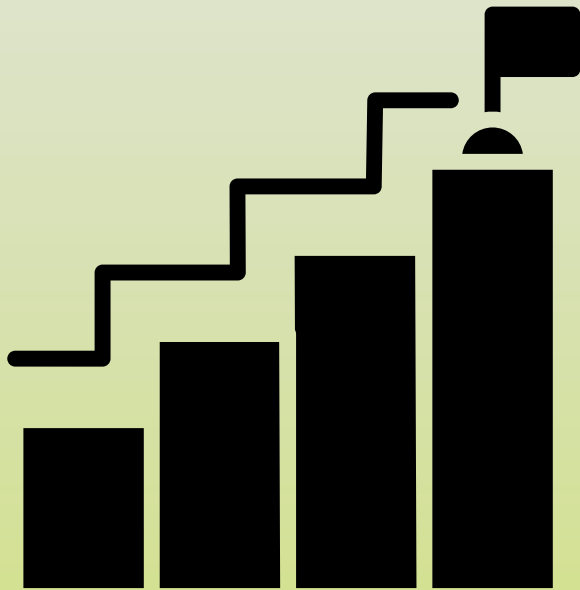
LAB 03

Monitoring



LAB 03

Monitoring



 30–40 min

Steps

- Analyze logs from the shell
- Configure and explore the Grafana Dashboards
- Analyze data for a simulated attack with Grafana
- *Optional: Explore Loki as additional investigation tool*

LAB 03

Monitoring



Any feedback?



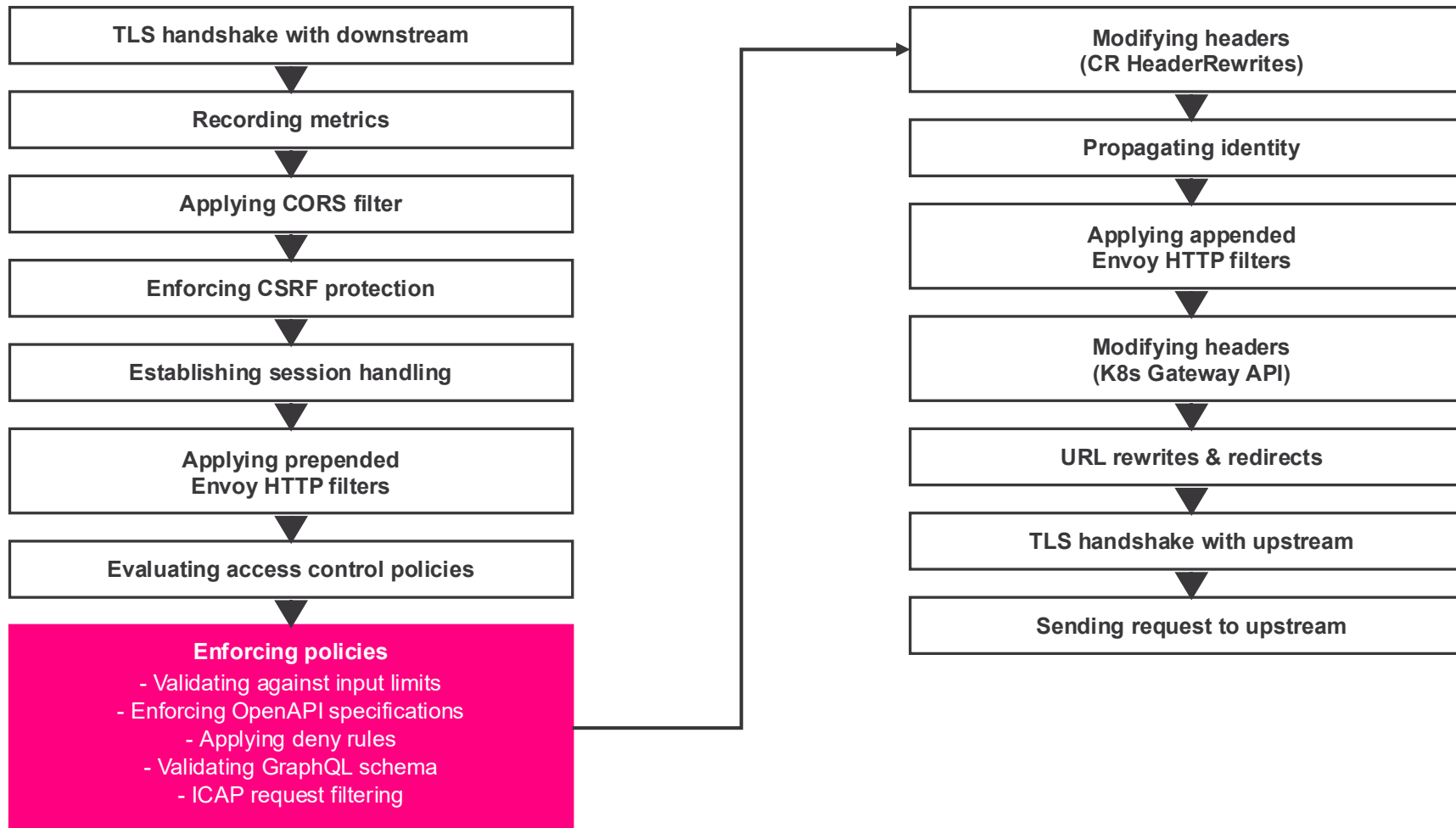
DevSecOps Process (optional)

Deny Rule Overrides and Exceptions

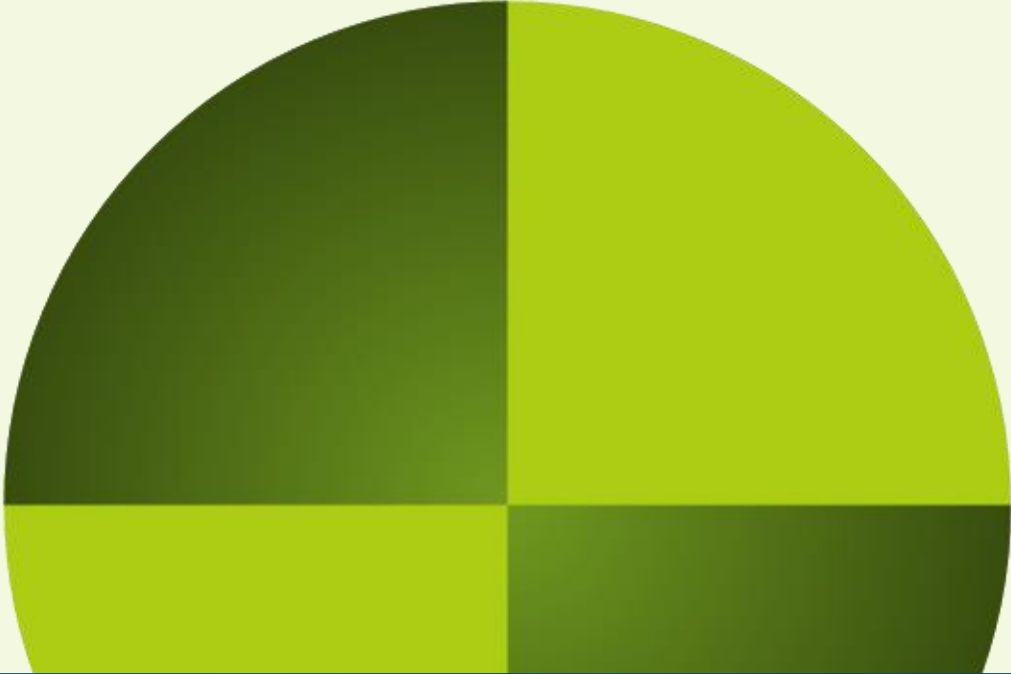


How can you configure **deny rule overrides** and **exceptions** to reduce **false-positive blocks** by Airlock Microgateway?

Processing Sequence – Request



Built-in Deny Rules
Deny Rule Overrides
Deny Rule Exceptions

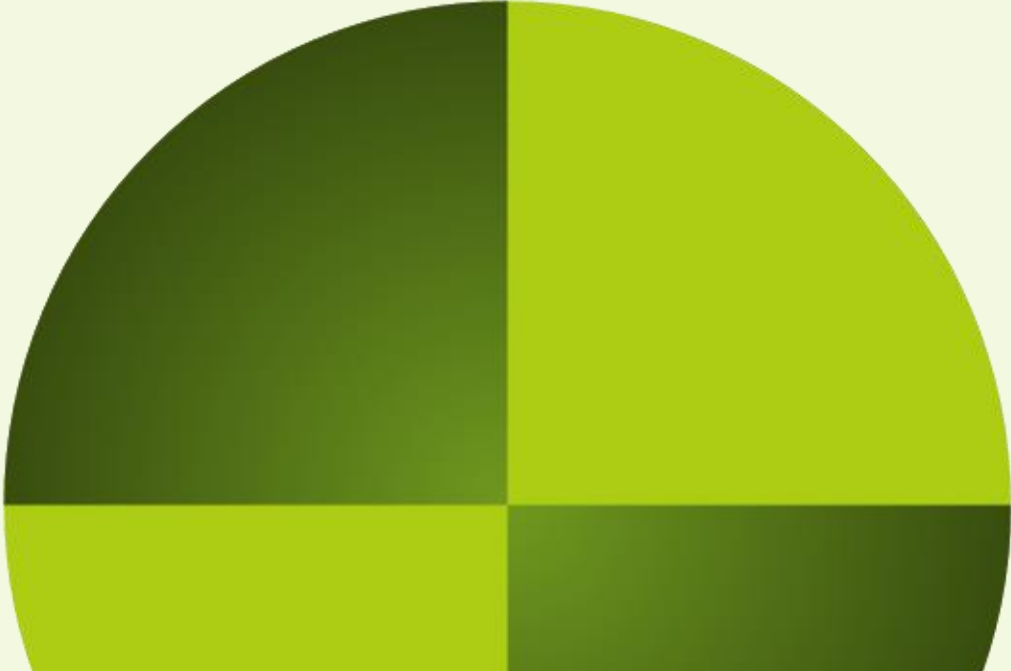


Deny Rule Overrides and Exceptions

Built-in Deny Rules

Deny Rule Overrides

Deny Rule Exceptions



Deny Rule Overrides and Exceptions

Built-in Deny Rules – Deny Rule Keys

Key	Name	Description
BOT	Bot detection in Header Value	Prevents access from bots, web-crawlers and web-scraper. The security levels Basic and Standard blocks the User-Agent known bots. The security level Strict blocks additional suspicious User-Agent.
ENCODING	Encoding and Conversion Exploits in Header and Parameter Value	Prevents injection of special encoded characters, such as %00, %0A, %0D, %0B, %0C, %0E, %0F, %1B, %1E, %1F, %2D, %2F, %3A, %3B, %3C, %3E, %3F, %5C, %5D, %5E, %5F, %60, %63, %64, %65, %66, %67, %68, %69, %6A, %6B, %6C, %6D, %6E, %6F, %70, %71, %72, %73, %74, %75, %76, %77, %78, %79, %7A, %7B, %7C, %7D, %7E, %7F, %80, %81, %82, %83, %84, %85, %86, %87, %88, %89, %8A, %8B, %8C, %8D, %8E, %8F, %90, %91, %92, %93, %94, %95, %96, %97, %98, %99, %9A, %9B, %9C, %9D, %9E, %9F, %A0, %A1, %A2, %A3, %A4, %A5, %A6, %A7, %A8, %A9, %AA, %AB, %AC, %AD, %AE, %AF, %B0, %B1, %B2, %B3, %B4, %B5, %B6, %B7, %B8, %B9, %BA, %BB, %BC, %BD, %BE, %BF, %C0, %C1, %C2, %C3, %C4, %C5, %C6, %C7, %C8, %C9, %CA, %CB, %CC, %CD, %CE, %CF, %D0, %D1, %D2, %D3, %D4, %D5, %D6, %D7, %D8, %D9, %DA, %DB, %DC, %DD, %DE, %DF, %E0, %E1, %E2, %E3, %E4, %E5, %E6, %E7, %E8, %E9, %EA, %EB, %EC, %ED, %EE, %EF, %F0, %F1, %F2, %F3, %F4, %F5, %F6, %F7, %F8, %F9, %FA, %FB, %FC, %FD, %FE, %FF, %20, %21, %22, %23, %24, %25, %26, %27, %28, %29, %2A, %2B, %2C, %2D, %2E, %2F, %30, %31, %32, %33, %34, %35, %36, %37, %38, %39, %3A, %3B, %3C, %3D, %3E, %3F, %40, %41, %42, %43, %44, %45, %46, %47, %48, %49, %4A, %4B, %4C, %4D, %4E, %4F, %50, %51, %52, %53, %54, %55, %56, %57, %58, %59, %5A, %5B, %5C, %5D, %5E, %5F, %60, %61, %62, %63, %64, %65, %66, %67, %68, %69, %6A, %6B, %6C, %6D, %6E, %6F, %70, %71, %72, %73, %74, %75, %76, %77, %78, %79, %7A, %7B, %7C, %7D, %7E, %7F, %80, %81, %82, %83, %84, %85, %86, %87, %88, %89, %8A, %8B, %8C, %8D, %8E, %8F, %90, %91, %92, %93, %94, %95, %96, %97, %98, %99, %9A, %9B, %9C, %9D, %9E, %9F, %A0, %A1, %A2, %A3, %A4, %A5, %A6, %A7, %A8, %A9, %AA, %AB, %AC, %AD, %AE, %AF, %B0, %B1, %B2, %B3, %B4, %B5, %B6, %B7, %B8, %B9, %BA, %BB, %BC, %BD, %BE, %BF, %C0, %C1, %C2, %C3, %C4, %C5, %C6, %C7, %C8, %C9, %CA, %CB, %CC, %CD, %CE, %CF, %D0, %D1, %D2, %D3, %D4, %D5, %D6, %D7, %D8, %D9, %DA, %DB, %DC, %DD, %DE, %DF, %E0, %E1, %E2, %E3, %E4, %E5, %E6, %E7, %E8, %E9, %EA, %EB, %EC, %ED, %EE, %EF, %F0, %F1, %F2, %F3, %F4, %F5, %F6, %F7, %F8, %F9, %FA, %FB, %FC, %FD, %FE, %FF, %20, %21, %22, %23, %24, %25, %26, %27, %28, %29, %2A, %2B, %2C, %2D, %2E, %2F, %30, %31, %32, %33, %34, %35, %36, %37, %38, %39, %3A, %3B, %3C, %3D, %3E, %3F, %40, %41, %42, %43, %44, %45, %46, %47, %48, %49, %4A, %4B, %4C, %4D, %4E, %4F, %50, %51, %52, %53, %54, %55, %56, %57, %58, %59, %5A, %5B, %5C, %5D, %5E, %5F, %60, %61, %62, %63, %64, %65, %66, %67, %68, %69, %6A, %6B, %6C, %6D, %6E, %6F, %70, %71, %72, %73, %74, %75, %76, %77, %78, %79, %7A, %7B, %7C, %7D, %7E, %7F, %80, %81, %82, %83, %84, %85, %86, %87, %88, %89, %8A, %8B, %8C, %8D, %8E, %8F, %90, %91, %92, %93, %94, %95, %96, %97, %98, %99, %9A, %9B, %9C, %9D, %9E, %9F, %A0, %A1, %A2, %A3, %A4, %A5, %A6, %A7, %A8, %A9, %AA, %AB, %AC, %AD, %AE, %AF, %B0, %B1, %B2, %B3, %B4, %B5, %B6, %B7, %B8, %B9, %BA, %BB, %BC, %BD, %BE, %BF, %C0, %C1, %C2, %C3, %C4, %C5, %C6, %C7, %C8, %C9, %CA, %CB, %CC, %CD, %CE, %CF, %D0, %D1, %D2, %D3, %D4, %D5, %D6, %D7, %D8, %D9, %DA, %DB, %DC, %DD, %DE, %DF, %E0, %E1, %E2, %E3, %E4, %E5, %E6, %E7, %E8, %E9, %EA, %EB, %EC, %ED, %EE, %EF, %F0, %F1, %F2, %F3, %F4, %F5, %F6, %F7, %F8, %F9, %FA, %FB, %FC, %FD, %FE, %FF, %20, %21, %22, %23, %24, %25, %26, %27, %28, %29, %2A, %2B, %2C, %2D, %2E, %2F, %30, %31, %32, %33, %34, %35, %36, %37, %38, %39, %3A, %3B, %3C, %3D, %3E, %3F, %40, %41, %42, %43, %44, %45, %46, %47, %48, %49, %4A, %4B, %4C, %4D, %4E, %4F, %50, %51, %52, %53, %54, %55, %56, %57, %58, %59, %5A, %5B, %5C, %5D, %5E, %5F, %60, %61, %62, %63, %64, %65, %66, %67, %68, %69, %6A, %6B, %6C, %6D, %6E, %6F, %70, %71, %72, %73, %74, %75, %76, %77, %78, %79, %7A, %7B, %7C, %7D, %7E, %7F, %80, %81, %82, %83, %84, %85, %86, %87, %88, %89, %8A, %8B, %8C, %8D, %8E, %8F, %90, %91, %92, %93, %94, %95, %96, %97, %98, %99, %9A, %9B, %9C, %9D, %9E, %9F, %A0, %A1, %A2, %A3, %A4, %A5, %A6, %A7, %A8, %A9, %AA, %AB, %AC, %AD, %AE, %AF, %B0, %B1, %B2, %B3, %B4, %B5, %B6, %B7, %B8, %B9, %BA, %BB, %BC, %BD, %BE, %BF, %C0, %C1, %C2, %C3, %C4, %C5, %C6, %C7, %C8, %C9, %CA, %CB, %CC, %CD, %CE, %CF, %D0, %D1, %D2, %D3, %D4, %D5, %D6, %D7, %D8, %D9, %DA, %DB, %DC, %DD, %DE, %DF, %E0, %E1, %E2, %E3, %E4, %E5, %E6, %E7, %E8, %E9, %EA, %EB, %EC, %ED, %EE, %EF, %F0, %F1, %F2, %F3, %F4, %F5, %F6, %F7, %F8, %F9, %FA, %FB, %FC, %FD, %FE, %FF, %20, %21, %22, %23, %24, %25, %26, %27, %28, %29, %2A, %2B, %2C, %2D, %2E, %2F, %30, %31, %32, %33, %34, %35, %36, %37, %38, %39, %3A, %3B, %3C, %3D, %3E, %3F, %40, %41, %42, %43, %44, %45, %46, %47, %48, %49, %4A, %4B, %4C, %4D, %4E, %4F, %50, %51, %52, %53, %54, %55, %56, %57, %58, %59, %5A, %5B, %5C, %5D, %5E, %5F, %60, %61, %62, %63, %64, %65, %66, %67, %68, %69, %6A, %6B, %6C, %6D, %6E, %6F, %70, %71, %72, %73, %74, %75, %76, %77, %78, %79, %7A, %7B, %7C, %7D, %7E, %7F, %80, %81, %82, %83, %84, %85, %86, %87, %88, %89, %8A, %8B, %8C, %8D, %8E, %8F, %90, %91, %92, %93, %94, %95, %96, %97, %98, %99, %9A, %9B, %9C, %9D, %9E, %9F, %A0, %A1, %A2, %A3, %A4, %A5, %A6, %A7, %A8, %A9, %AA, %AB, %AC, %AD, %AE, %AF, %B0, %B1, %B2, %B3, %B4, %B5, %B6, %B7, %B8, %B9, %BA, %BB, %BC, %BD, %BE, %BF, %C0, %C1, %C2, %C3, %C4, %C5, %C6, %C7, %C8, %C9

[https://docs.airlock.com/microgateway/\[ver\]/index/api/crds/deny-rules/v1alpha1/#default-deny-rule-keys](https://docs.airlock.com/microgateway/[ver]/index/api/crds/deny-rules/v1alpha1/#default-deny-rule-keys)

i. e., **[ver]** could be **5.1**

Built-in Deny Rules – Defaults

Example

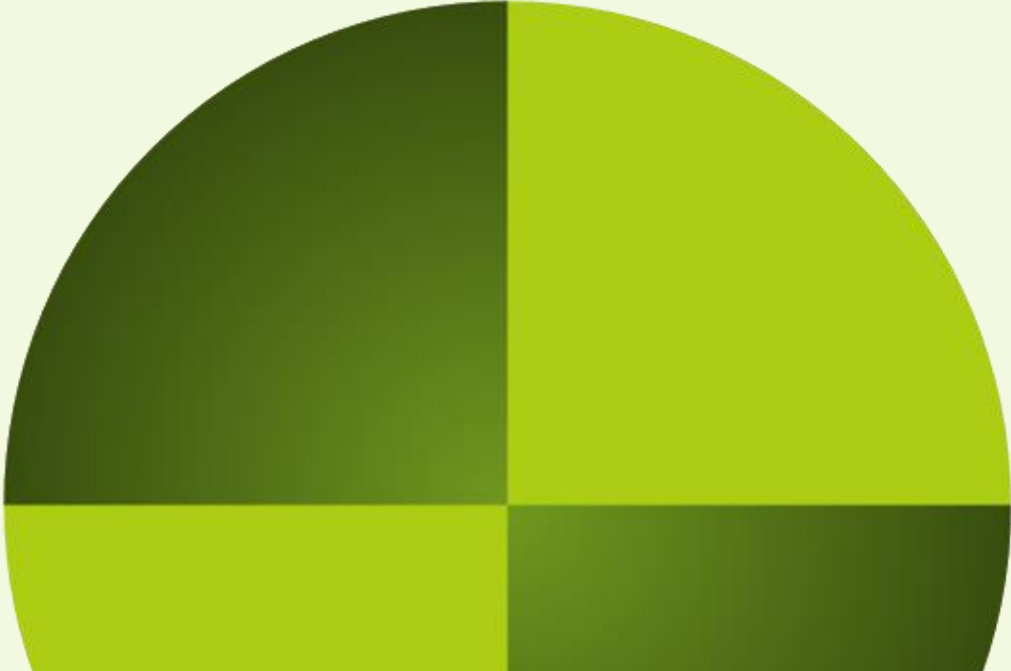
Default

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: default
spec:
  request:
    builtIn:
      settings:
        level: Standard
        threatHandlingMode: Block
  custom: {}
```


Built-in Deny Rules

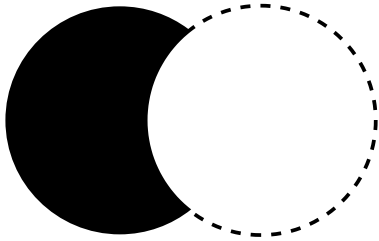
Deny Rule Overrides

Deny Rule Exceptions



Deny Rule Overrides and Exceptions

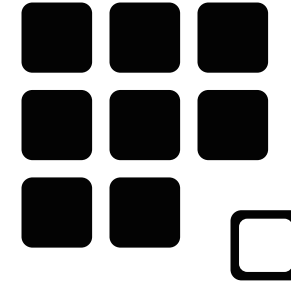
Deny Rule Overrides vs Deny Rule Exceptions



Overrides

to override settings of built-in
deny rule(s) and supported
attribute types

*override deny rule level, threat handling
mode*

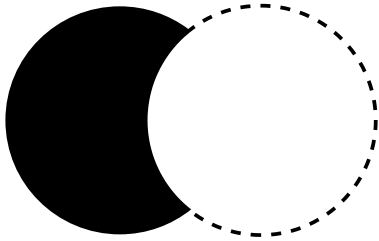


Exceptions

if requests with certain blocked data,
request conditions, and/or deny rule
keys should not be blocked

*e.g., an exception based on a JSON path, URL
path, media type, HTTP method, and rule key*

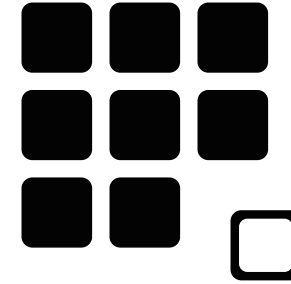
Deny Rule Overrides vs Deny Rule Exceptions



Overrides

to override settings of built-in
deny rule(s) and supported
attribute types

*override deny rule level, threat handling
mode*



Exceptions

if requests with certain blocked data,
request conditions, and/or deny rule
keys should not be blocked

*e.g., an exception based on a JSON path, URL
path, media type, HTTP method, and rule key*

Deny Rule Overrides – Conditions

conditions allow to select which built-in deny rule(s) settings will be adjusted

- **ruleKeys** = built-in deny rule(s)
e.g., HTML, XSS
- **types** = type of attributes the override should be applied on
e.g., JSON, GraphQL

Deny Rule Overrides – Settings

settings specify the properties to override for the selected rules

- **level** = filter strength

Basic, Standard, Strict, Unfiltered

- **threatHandlingMode** = how threats should be handled

Block, LogOnly

Deny Rule Overrides – Example

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: example-override
  namespace: override
spec:
  request:
    builtIn:
      settings:
        level: Strict
        threatHandlingMode: Block
    overrides:
      - conditions:
          ruleKeys:
            - XSS
          settings:
            level: Standard
```

Key	Name
XSS	Cross-Site Scripting (XSS) in Path, Header and Parameter Value

The security level Strict prevents injection of JavaScript code in unquoted context.

Deny Rule Overrides – Example

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: DenyRules
metadata:
  name: example-override
  namespace: override
spec:
  request:
    builtIn:
      settings:
        level: Strict
        threatHandlingMode: Block
    overrides:
      - conditions:
          ruleKeys:
            - XSS
          settings:
            level: Standard
```

Key	Name
XSS	Cross-Site Scripting (XSS) in Path, Header and Parameter Value

The security level Standard prevents injection of JavaScript code in quoted context.

Deny Rule Overrides

Adjust **specific rule groups** without lowering (or increasing) global posture

→ Can be used for **recurring false-positives** from one rule key

Deny Rule Overrides by ruleKey and type

Potentially combine...

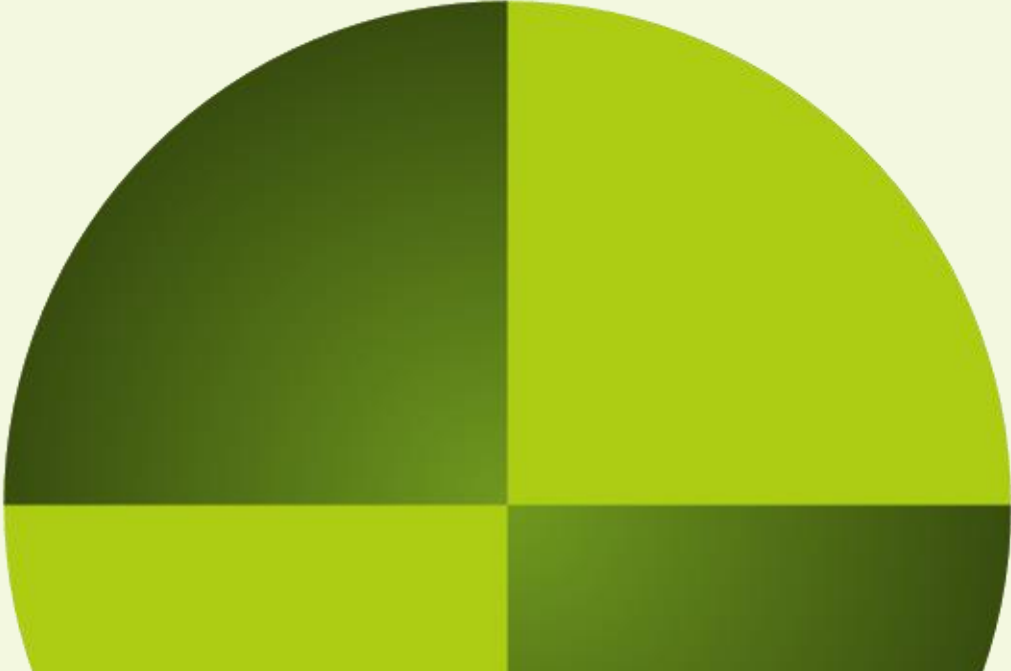
ruleKey (e.g., HTML) + **limitation by attribute type** (e.g., JSON)

= narrow override (no unnecessary relaxation)

→ Validate via Engine access logs

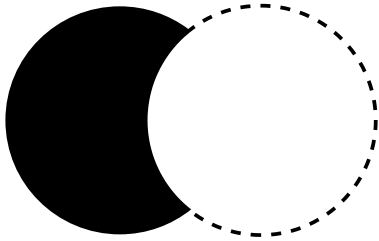
e.g., check for rule_key, blocked_data, request context

Built-in Deny Rules
Deny Rule Overrides
Deny Rule Exceptions



Deny Rule Overrides and Exceptions

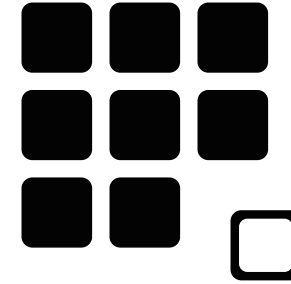
Deny Rule Overrides vs Deny Rule Exceptions



Overrides

to override settings of built-in
deny rule(s) and supported
attribute types

*override deny rule level, threat handling
mode*



Exceptions

if requests with certain blocked data,
request conditions, and/or deny rule
keys should not be blocked

*e.g., an exception based on a JSON path, URL
path, media type, HTTP method, and rule key*

Deny Rule Exceptions

To allow **specific requests** that would otherwise be blocked

Combine...

- **blockedData** = exception based on request data causing the block
e.g., jsonPath, graphql.argument
 - **requestConditions** = exception based on request properties (all must match)
e.g., path, method, mediaType
 - **ruleKeys** = restrict exception to deny rule(s)
e.g., HTML, XSS
- ← Possible to define one exception for **multiple deny ruleKeys**

Deny Rule Exceptions – Application

- Be as specific as possible
- Audit with **threatHandlingMode: LogOnly** before enforcing

Reflection – Deny Rule Overrides vs Exceptions

Overrides:

Adjust deny-rule *settings* (e.g., `level`, `mode`) for a **ruleKey** (and **type**)

Exceptions:

Explicitly *allow* narrow cases that still match a rule

Reflection – Deny Rule Overrides vs Exceptions

- Use Overrides for **many similar** false positives
- Use Exceptions for **precise, isolated** cases
- Prefer **Exceptions** to keep overall security strong

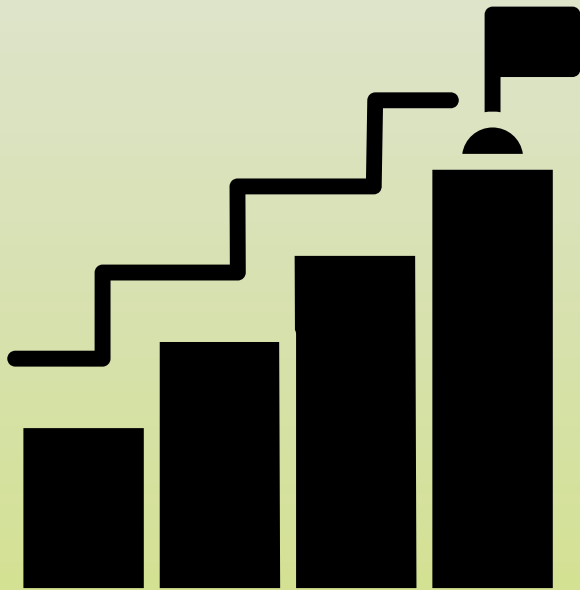
LAB 04

Deny Rule Overrides and Exceptions



LAB 04

Deny Rule Overrides and Exceptions



 40–50 min

Steps

- Create a deny rule override related to a rule group
- Restrict the override to an attribute type
- Configure a deny rule exception (optional)

LAB 04

Deny Rule Overrides and Exceptions



Any feedback?



Use Case: JuiceShop Customer Feedback Form

Customer Feedback

Comment*

I would like to suggest that you add the following `<code></code>` to your documentation to explain example 3.2 in more detail.

Max. 160 characters

125/160

Rating

CAPTCHA: What is ?

Result*

2

> Submit

Header Rewrites



How can you configure header filtering and rewriting with
Airlock Microgateway?

K8s Gateway API vs MGW CRD
Request and Response Actions
Envoy Command Operators
Logging



K8s Gateway API vs MGW CRD

Request and Response Actions

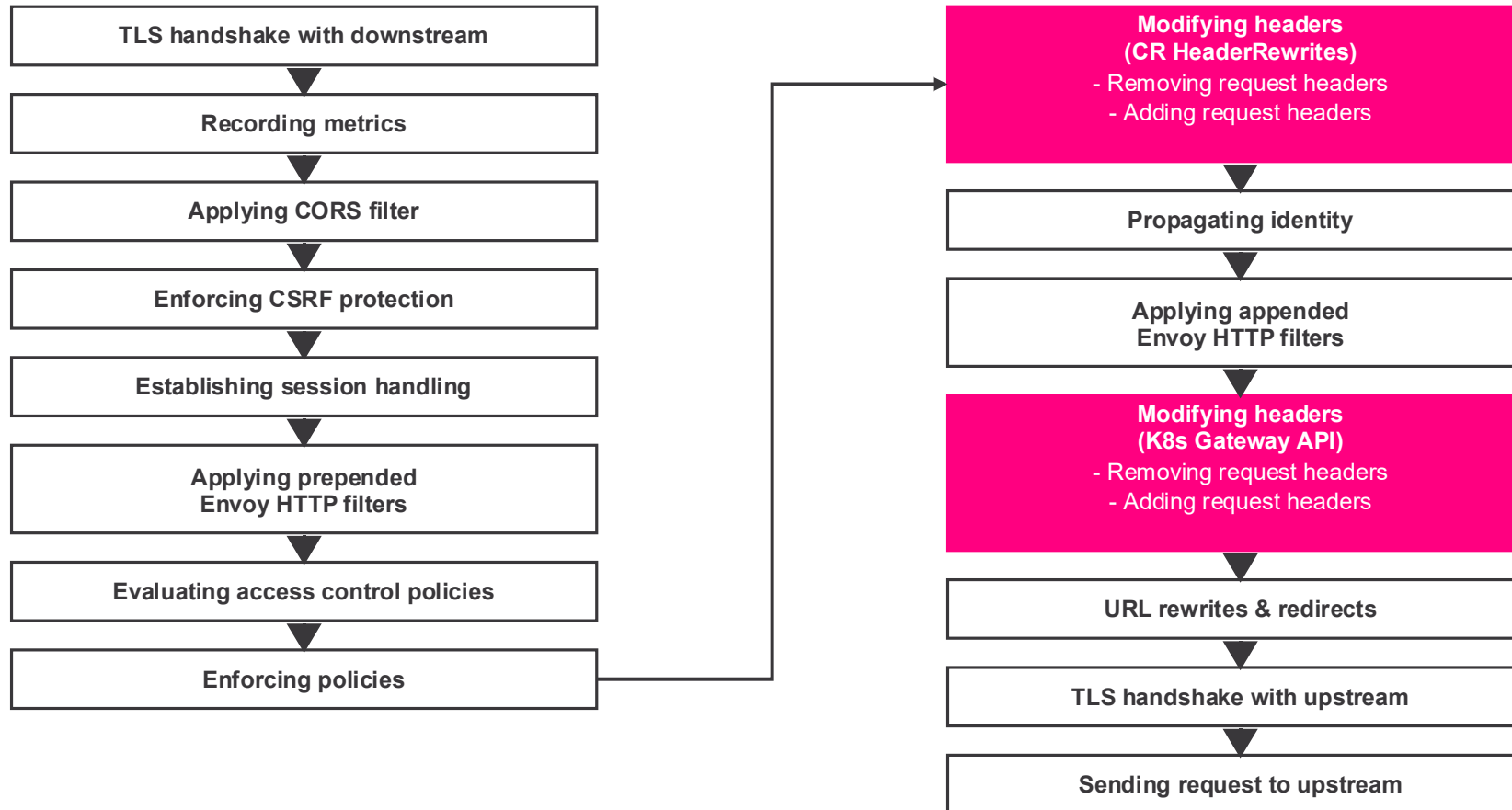
Envoy Command Operators

Logging

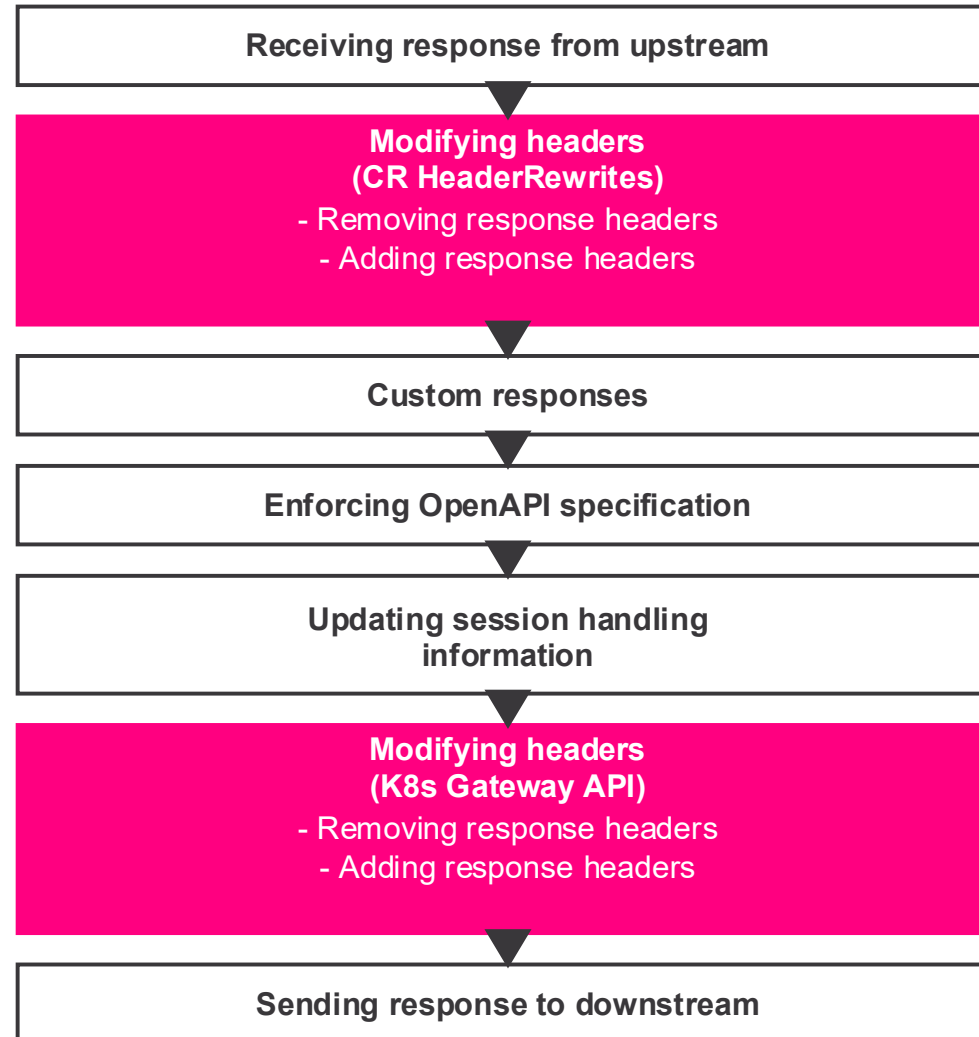


**Header
Rewrites**

Processing Sequence – Request



Processing Sequence – Response



Modifying Headers via K8s Gateway API



```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: header-http-echo
spec:
  parentRefs:
  - name: acme-gw
  rules:
  - filters:
    - type: RequestHeaderModifier
      requestHeaderModifier:
        add:
        - name: my-header-name
          value: my-header-value
  ...
```

Remove, **set** (edit) and **add** headers
in requests and responses

<https://gateway-api.sigs.k8s.io/guides/http-header-modifier/>

Modifying Headers via «HeaderRewrites» CR



```
apiVersion: microgateway.airlock.com/v1alpha1
kind: HeaderRewrites
metadata:
  name: example-header-rewrites
spec:
  response:
    add:
      builtIn:
        permissionsPolicy: true
  ...
```

Allow, **remove** and **add** headers in requests and responses

Field	Description	Type	Required	Default	Allowed Values
permissionsPolicy	sets a permissions policy which prevents cross-origin use of several browser features if the 'Permissions-Policy' header is not set by the upstream. ▼ Hide details Adds missing headers: • Permissions-Policy: accelerometer=(self), ambient-light-sensor=(self), autoplay=(self), camera=(self), display-capture=(self), document-domain=(self), encrypted-media=(self), fullscreen=(self), geolocation=(self), gyroscope=(self), magnetometer=(self), microphone=(self), midi=(self), payment=(self), usb=(self), xr-spatial-tracking=(self)	bool	no	true	true, false

K8s Gateway API vs. Microgateway Header Modification



K8s Gateway API

vendor-agnostic

focus of
upcoming slides



HeaderRewrites» CR

built-in header lists

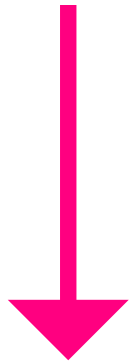
operationalMode: Integration

→ we recommend using
«HeaderRewrites» CR

K8s Gateway API vs MGW CRD
Request and Response Actions
Envoy Command Operators
Logging



Header Rewrites – Request and Response Processing



1. Remove headers NOT explicitly listed to be «allowed»
2. Remove headers explicitly listed to «remove»
3. Add headers explicitly listed to «add»



Not possible to add headers and remove them accidentally afterward
(by design)

«HeaderRewrites» CR

Supports integration:

- remove sensitive headers
- insert security headers (HSTS, CSP, etc.)



A HeaderRewrites CR needs to be referenced in a
ContentSecurityPolicy CR

Request and Response Actions

Envoy Command Operators

Logging



**Header
Rewrites**

Envoy Command Operators in Header Rewrites CR

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: HeaderRewrites
metadata:
  name: example-header-rewrites
spec:
  requests:
    add:
      custom:
        - name: Add headers with TLS info of
            downstream connection to
            member requests
          headers:
            - name: X-TLS-DOWNSTREAM-PEER-CERT
              value: "%DOWNSTREAM PEER CERT%"
          mode: OverwriteOrAdd
          requestConditions:
            path:
              matcher:
                regex: ^/member/
  ...
```

Envoy Command Operators can
be used within HeaderRewrites
CRs

[https://www.envoyproxy.io/docs/
envoy/latest/configuration/obser
vability/access_log/usage.html](https://www.envoyproxy.io/docs/envoy/latest/configuration/observability/access_log/usage.html)

Request and Response Actions
Envoy Command Operators
Logging



**Header
Rewrites**

Header Rewrites Logging



Default: header rewrite actions are **NOT reflected in access logs**

→ Use **operationalMode: Integration** to enrich logs with header rewrite details

Logs will include

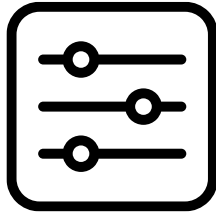
- rule names
- action types
- headers affected

Request and Rate Limits



How can you configure and customize **request content limits** and **rate limits** with Airlock Microgateway?

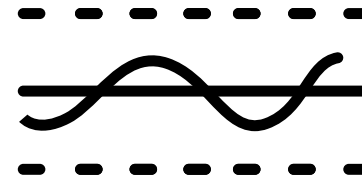
Request Content Limits vs Rate Limits



Request Content Limits

to limit content
based on thresholds

*e.g., request body size,
GraphQL query size*



Rate Limits

to limit number of
requests over time

for a route, selected requests based on request
conditions, or per client IP

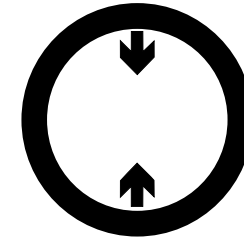
Request Content Limits

evaluated **per request**

Request Content Limits

Can help to

- reduce load
- prevent resource exhaustion and DoS
 - undesired resource intensive requests
 - oversized payloads
 - memory exhaustion
 - parsing overhead



validation against
limits



deny rule
evaluation

Request Content Limits

Default limits are pre-configured

Supported **threatHandlingModes**:

- LogOnly
- Block **(default)**

Example

Default

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: Limits
metadata:
  name: default
spec:
  settings:
    threatHandlingMode: Block
  request:
    limited:
      general:
        bodySize: "100Mi"
        pathLength: "1Ki"
      parameter:
        bodySize: "100Ki"
        count: 128
        nameLength: "128"
        valueLength: "8Ki"
      json:
        bodySize: "100Ki"
        nestingDepth: 100
        keyCount: 250
        elementCount: 10000
```

Request Content Limits

Limits can be set for

- **general** (e.g., maximum pathLength for all requests)
- **graphql** (e.g., querySize)
- **json** (e.g., valueLength)
- **multipart** (e.g., bodySize)
- **parameter** (e.g., count)



It is also possible to
define limit **exceptions**

Request Content Limits – Example

```
apiVersion:
microgateway.airlock.com/v1alpha1
kind: Limits
metadata:
  name: limits-example
spec:
  settings:
    threatHandlingMode: Block
  request:
    limited:
      general:
        bodySize: 10Mi
        pathLength: 1Ki
      json:
        bodySize: 10Ki
        nestingDepth: 20
        keyCount: 250
```



```
graphql:
  querySize: 512
  nestingDepth: 5
  valueLength: 128
  exceptions:
    - length:
        json:
          jsonPath: "$.logo"
        requestConditions:
          path:
            matcher:
              prefix: /project/
```

A Limits CR allows fine grained control;
e.g., different **bodySize limits** for **general**
(unparsed requests) and **json**

Request Content Limits – Logs in thMode «Block»

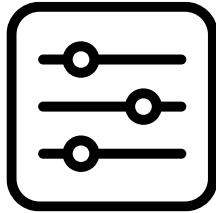


Logging is limited with **threatHandlingMode: Block**
(to mitigate the risk of being vulnerable to DoS attacks)

Want detailed information on limit violations during integration?

- Set **threatHandlingMode: LogOnly**
- Check the Microgateway Engine access log

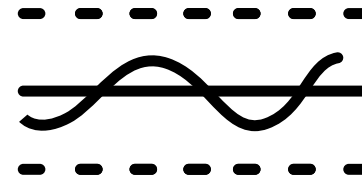
Request Content Limits vs Rate Limits



Request Content Limits

to limit content
based on thresholds

*e.g., request body size,
GraphQL query size*



Rate Limits

to limit number of
requests over time

for a route, selected requests based on request
conditions, or per client IP

Rate Limits

evaluated **based on request history**

Rate Limits

Can help to

- protect against excessive request frequency
- avoid individual customer monopolizing API capacity
- enforce limits affixed in an SLA
- reduce infrastructure cost (lower back-end load)
- prevent DoS

Rate Limits

Supported **threatHandlingModes**:

- LogOnly
- Block **(default)**

Field	Description	Type	Required	Default	Allowed Values
threatHandlingMode	specifies how a threat is handled when the rate limit is exceeded.	enum	no	Block	Block, LogOnly

Rate Limits – Example: Limit All Requests on A Route

Limit requests

on route **juice-shop**

to **1000 requests / s**



Field	Description	Type	Required	Default
per	is the time window of the requests/ time rate.	string (duration)	no	1s

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: RateLimitPolicy
metadata:
  name: rate-limit-example
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: juice-shop
  policies:
    - local:
        limit:
          requests: 1000
```

Rate Limits – Example: Limit Selected Requests Only

Limit requests

on route **juice-shop**

to **1000 requests / s**

if **path prefix matches /api1/**

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: RateLimitPolicy
metadata:
  name: rate-limit-example
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: juice-shop
  policies:
    - local:
        limit:
          requests: 1000
        requestConditions:
          path:
            matcher:
              prefix: /api1/
```

Rate Limits – Example: Count by Client IP

Limit requests

on route **juice-shop**

to **1000 requests / s**

if **path prefix matches /api1/**

Apply limit per client IP

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: RateLimitPolicy
metadata:
  name: rate-limit-example
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: juice-shop
  policies:
    - local:
        limit:
          requests: 1000
        requestConditions:
          path:
            matcher:
              prefix: /api1/
          countBy:
            ip: {}
```

Rate Limits – Memory Considerations

Counting per client IP can increase
memory usage

Use

countBy.ip.numberOfCounters

to restrict IP counters

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: RateLimitPolicy
metadata:
  name: rate-limit-example
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: juice-shop
  policies:
    - local:
        limit:
          requests: 1000
        requestConditions:
          path:
            matcher:
              prefix: /api1/
        countBy:
          ip: {}
```

Request Body Parsing



How can you customize **request body parsing**
with Airlock Microgateway?

Request Body Parsing

Airlock Microgateway Engine parses the request body

BEFORE applying filters to a request's content

Request Body Parsing – CR «Parser»

The CR **Parser** contains the configuration for content parsers
(default and custom)

Request Body Parsing – Configure with Caution!



The parser configuration **influences the behavior of other filters**

POTENTIAL: Hardening / customization

RISK: Accidental creation of «holes» in security setup
(e.g., by undesired skipping of filters)

Request Body Parsing – Example

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: Parser
metadata:
  name: parser-example
spec:
  request:
    defaultContentType: application/json
  parsers:
    json: { enable: true, mediaTypePattern: ".*json.*" }
    form: { enable: true, mediaTypePattern: ".*urlencoded.*" }
    multipart: { enable: true, mediaTypePattern: ".*multipart.*" }
  custom:
    rules:
      - requestConditions:
          method: [POST]
          path: { matcher: { exact: /company/contact.php } }
          action: { parse: { json: {} } }
      - requestConditions:
          path: { matcher: { exact: /support/upload.php } }
          action: { skip: {} }
```

OpenAPI and GraphQL

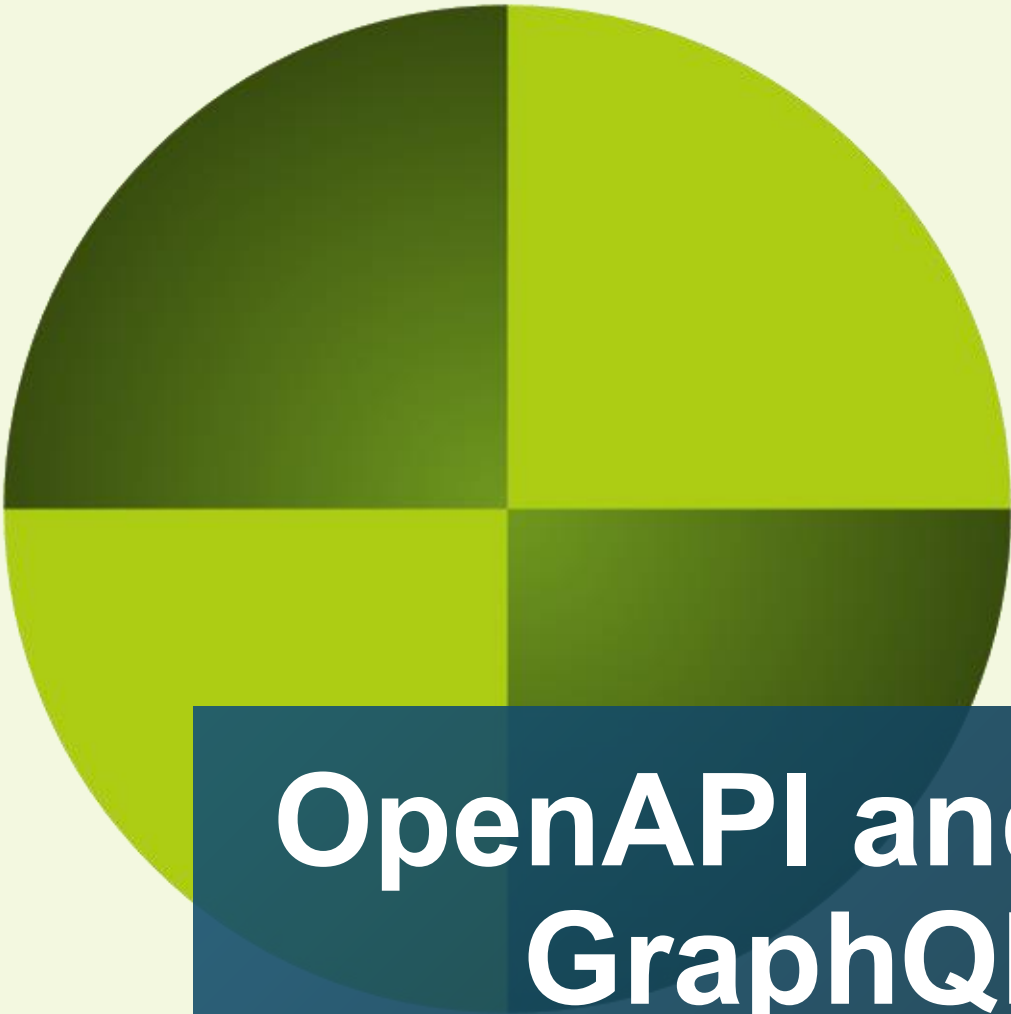


How can you use a **specification/schema** with Airlock
Microgateway to reduce the attack surface of your setup?

APIProtection Custom Resource

OpenAPI

GraphQL

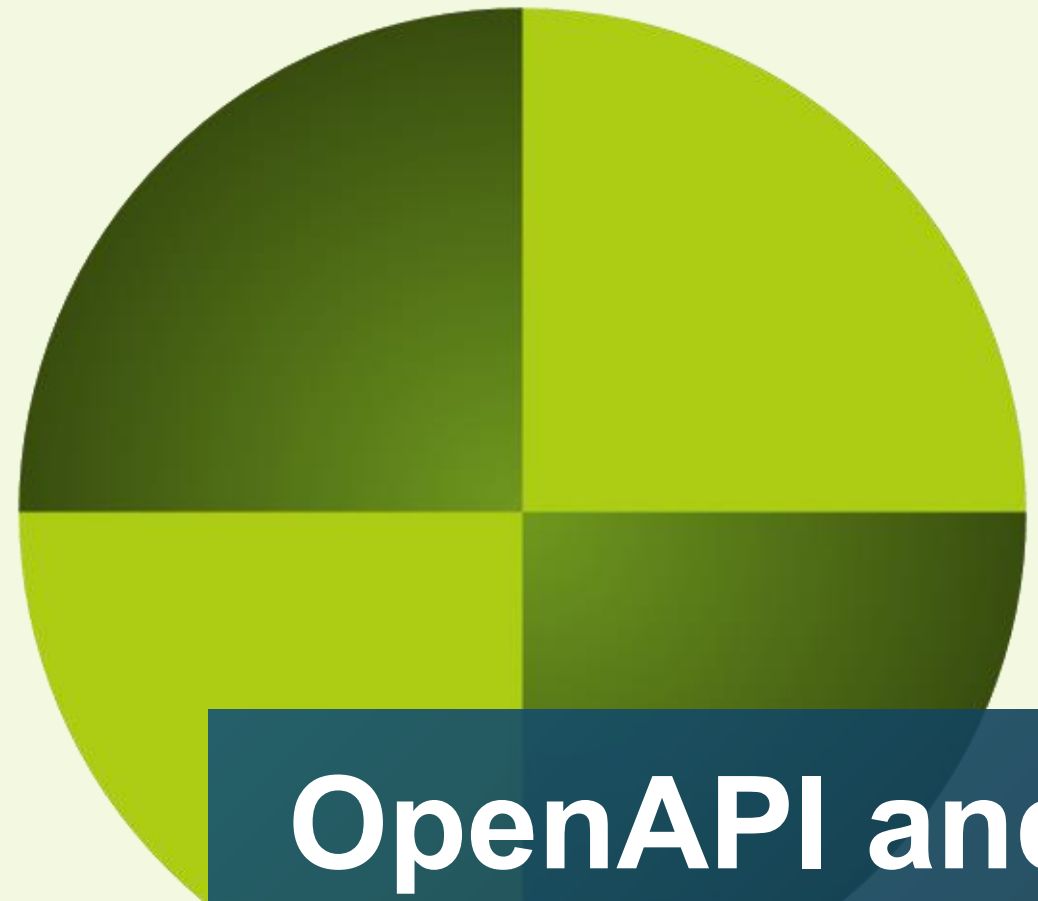


**OpenAPI and
GraphQL**

APIProtection Custom Resource

OpenAPI

GraphQL



**OpenAPI and
GraphQL**

CR «APIProtection»

Contains the **configuration for API security**

CR «APIProtection» – requestConditions

You can configure **request properties** that must be matched to apply...

- ... GraphQL validation

- ... OpenAPI validation

- ... noValidation

CR «APIProtection» – Example

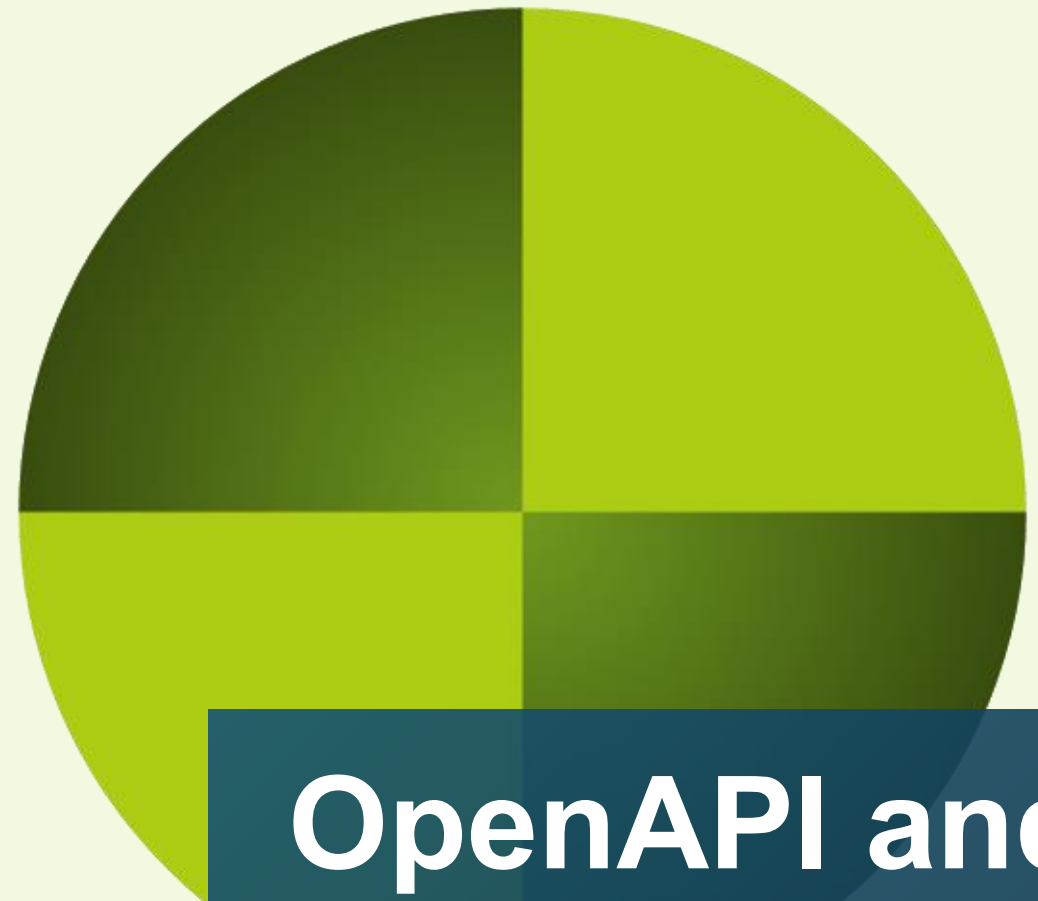
```
apiVersion:
microgateway.airlock.com/v1alpha1
kind: APIProtection
metadata:
  name: api-protection-example
spec:
  policies:
    - name: My REST API exception
      requestConditions:
        path:
          matcher:
            prefix: /rest/no-validation
          noValidation: {}
```

```
- name: My REST API
  requestConditions:
    path:
      matcher:
        prefix: /rest
  openAPIRef:
    name: open-api-example
- name: My GraphQL API
  requestConditions:
    path:
      matcher:
        prefix: /graphql
  graphqlRef:
    name: graphql-example
```

APIProtection Custom Resource

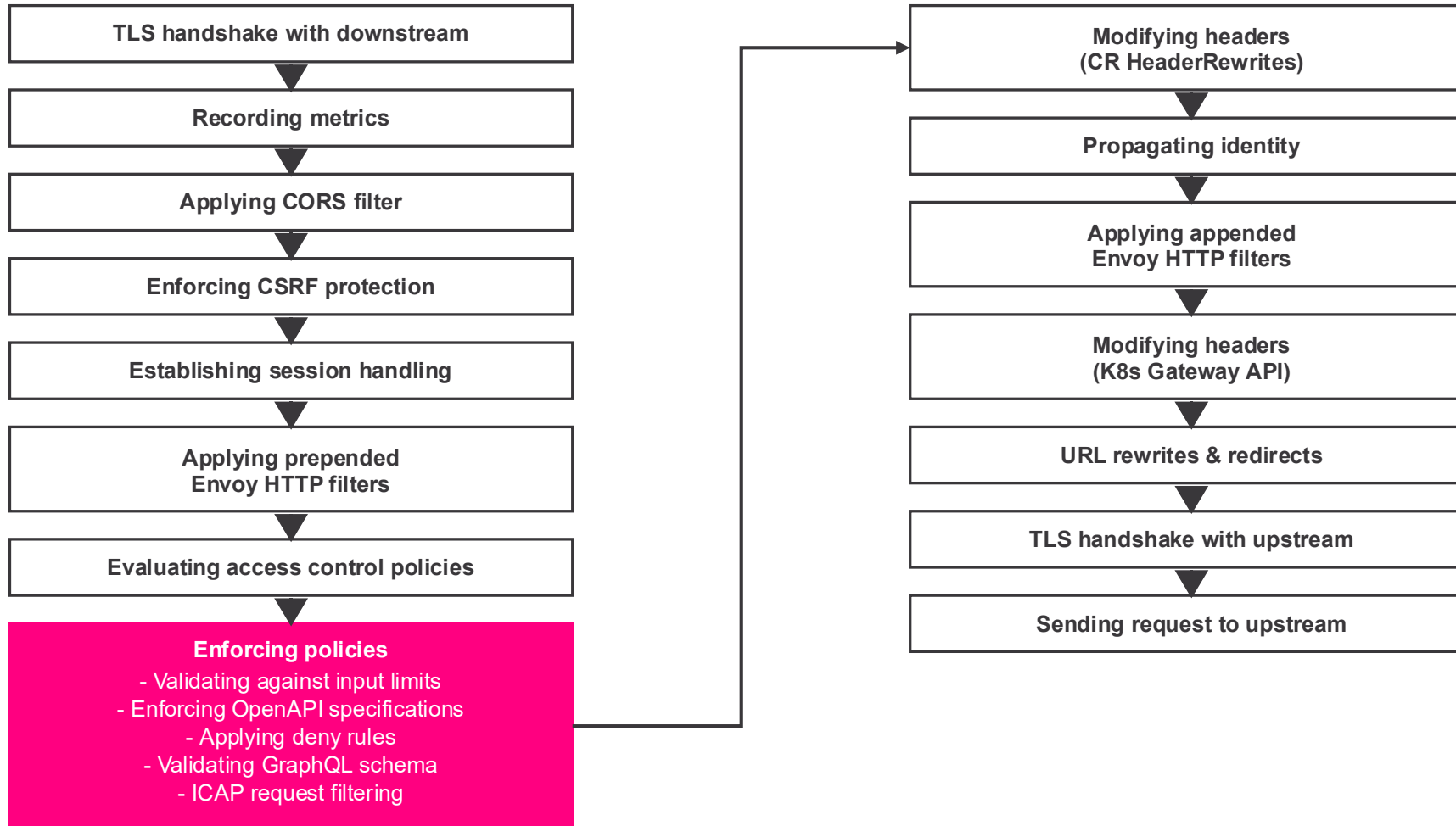
OpenAPI

GraphQL

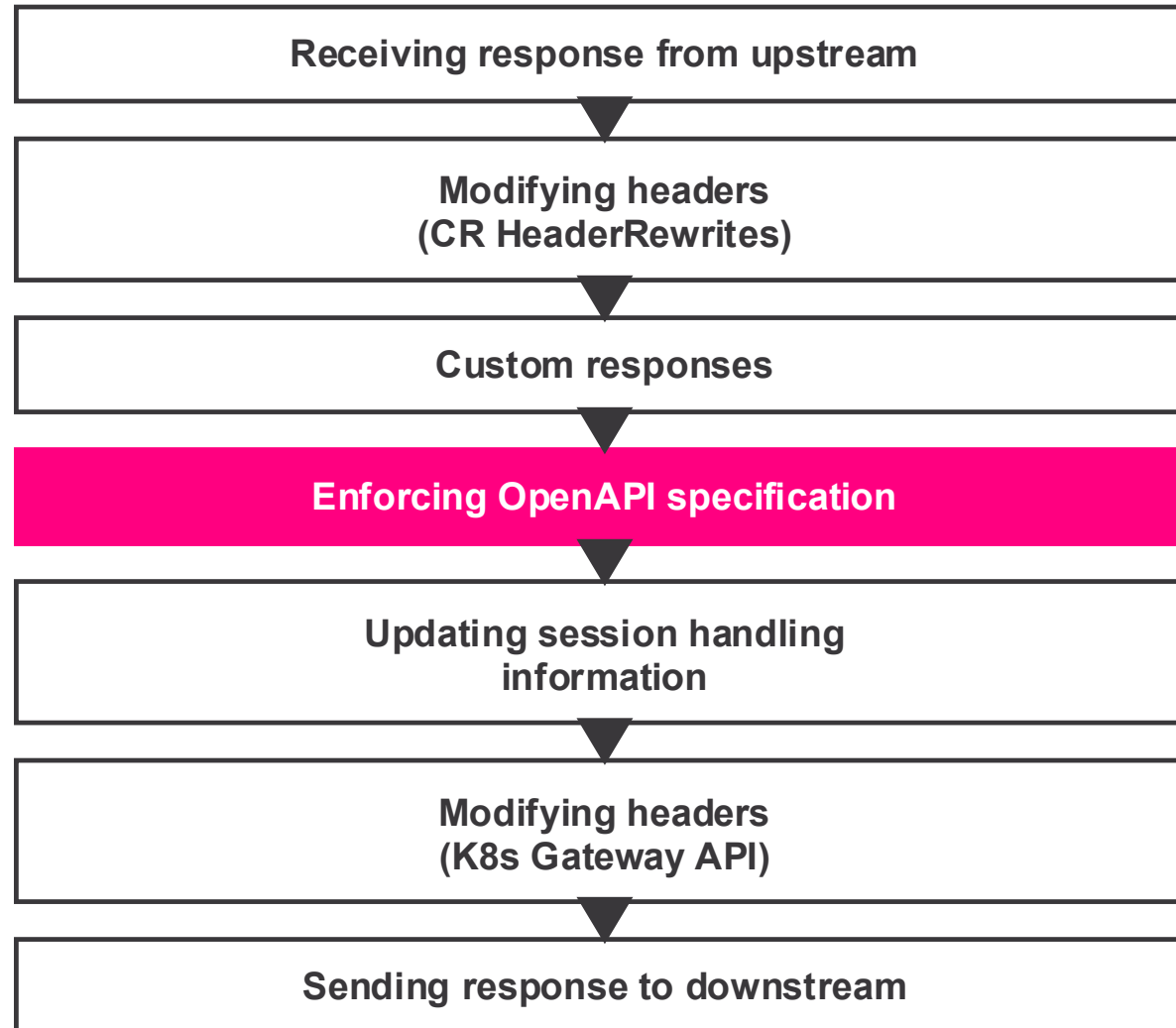


**OpenAPI and
GraphQL**

Processing Sequence – Request



Processing Sequence – Response



CR «OpenAPI»

To validate **requests** and **responses** against an API specification

Detect malformed or unexpected inputs before deeper processing

Required specification format: **OpenAPI version 3.0** as JSON

CR «OpenAPI»

Supported **threatHandlingModes** (if validation fails):

- LogOnly
- Block **(default)**

Response validation **optional**

→ Default = **unsecured{...}**

Example OpenAPI Schema → In ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: openapi3-spec
  namespace: app-space
data:
  openapi.json: |
    {
      "openapi": "3.0.3",
      "info": {
        "title": "Juice Shop API",
        "version": "1.0.0"
      },
      "paths": {
        "/api/Feedbacks/": {
          "post": {
            "summary": "Submit customer feedback",
            "requestBody": {
              "required": true,
              "content": {
                "application/json": {
                  "schema": {
                    "type": "object",
```

```
                    "properties": {
                      "captchaId": { "type": "string" },
                      "captcha": { "type": "string" },
                      "comment": { "type": "string" },
                      "rating": { "type": "integer" }
                    },
                    "required": ["comment", "rating"]
                  }
                }
              },
            "responses": {
              "200": { "description": "Feedback submitted successfully" },
              "400": { "description": "Invalid input" }
            }
          }
        }
      }
    }
```

→ in a **K8s ConfigMap**



CR «OpenAPI» – Example

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: OpenAPI
metadata:
  name: open-api-example
  namespace: app-space
spec:
  settings:
    schema:
      source:
        configMapRef:
          name: openapi3-spec
  threatHandlingMode: Block
  logging:
    maxFailedSubvalidations: 10
  validation:
    authentication:
      oAuth2:
        allowedParameters:
          builtIn:
            standardParameters: true
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: openapi3-spec
  namespace: app-space
data:
  openapi.json: |
    {
      "openapi": "3.0.3",
      "info": {
        "title": "Juice Shop API",
        "version": "1.0.0"
      },
      "paths": {
        "/api/Feedbacks/": {
          "post": {
            "summary": "Submit customer feedback",
            "requestBody": {
              "required": true,
              "content": {
                "application/json": {
                  "schema": {
                    "type": "object",

```

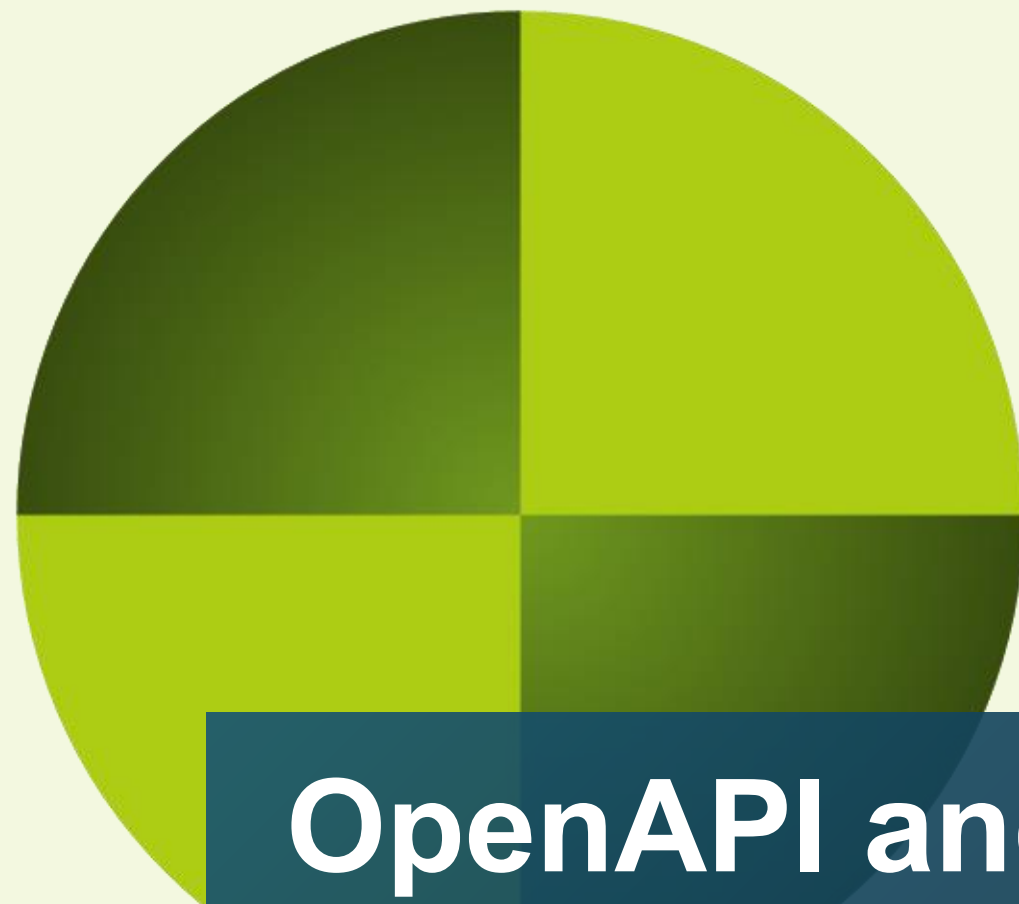
...



APIProtection Custom Resource

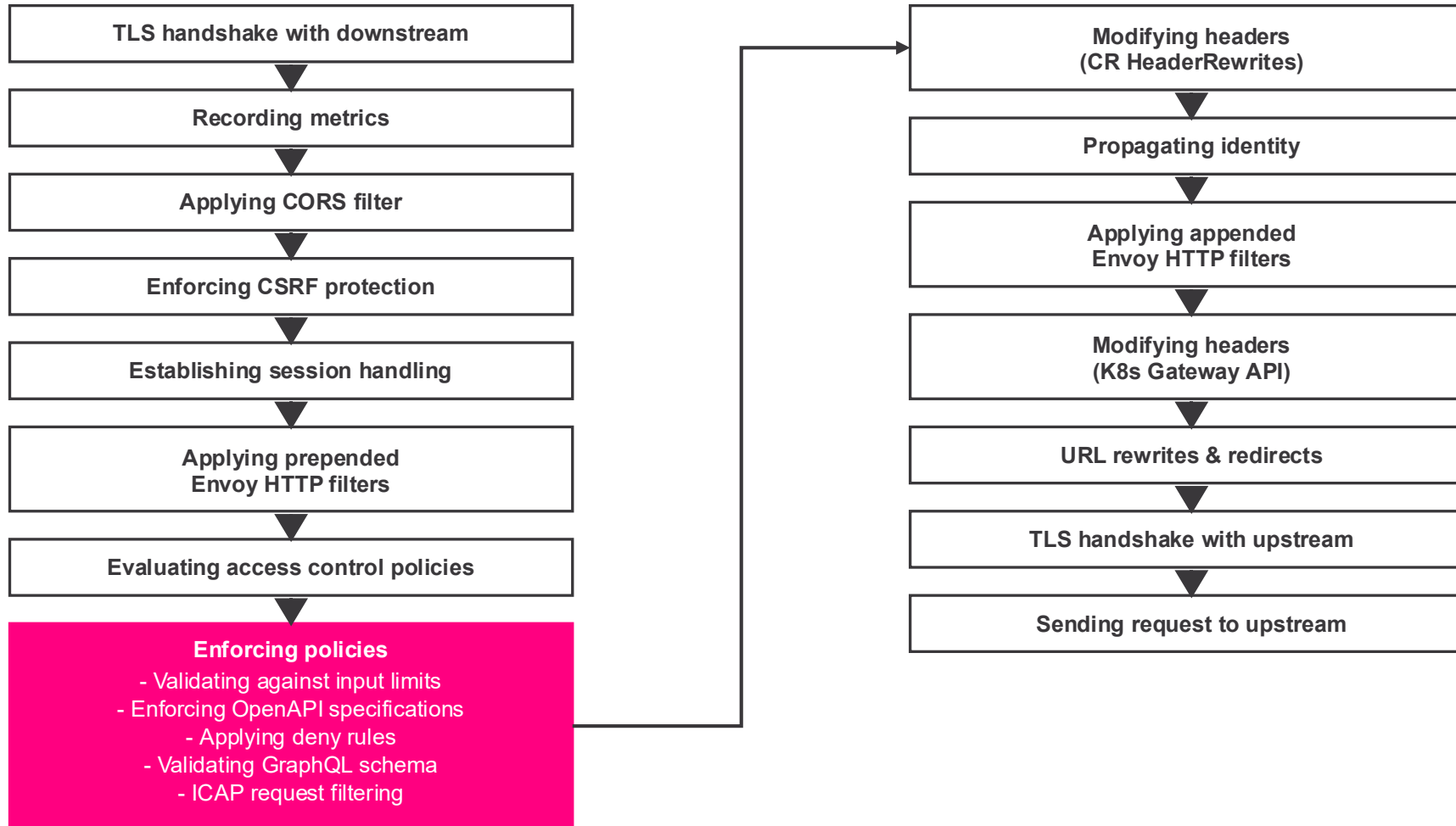
OpenAPI

GraphQL



**OpenAPI and
GraphQL**

Processing Sequence – Request



CR «GraphQL»

To validate request against a GraphQL schema resource

Required Schema Compatibility: **GraphQL schema definition (Oct. 2021)**

Supports to allow or prohibit **introspection** and **mutation** queries

CR «GraphQL»

Supported **threatHandlingModes** (if validation fails):

- LogOnly
- Block **(default)**

Example GraphQL Schema → In ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: graphql-schema
  namespace: app-space
data:
  schema.graphql: |-
    type Feedback {
      id: ID!
      comment: String!
      rating: Int!
    }

    type Query {
      feedbacks: [Feedback!]!
      feedback(id: ID!): Feedback
    }

    type Mutation {
      createFeedback(comment: String!, rating: Int!): Feedback!
    }
```



CR «GraphQL» – Example

```
apiVersion:
microgateway.airlock.com/v1alpha1
kind: GraphQL
metadata:
  name: graphql-example
  namespace: app-space
spec:
  settings:
    threatHandlingMode: Block
    allowMutations: true
    allowIntrospection: false
    schema:
      source:
        configMapRef:
          name: graphql-schema
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: graphql-schema
  namespace: app-space
data:
  schema.graphql: |-
    type Feedback {
      id: ID!
      comment: String!
      rating: Int!
    }

    type Query {
      feedbacks: [Feedback!]!
      feedback(id: ID!): Feedback
    }

    type Mutation {
      createFeedback(comment: String!, rating: Int!): Feedback!
    }
```



OpenAPI and GraphQL – ConfigMap Compression

ConfigMaps have a size limit of **1MiB (Mebibyte)**

If ConfigMap size > 1MiB

Reference a **.zstd**, **.gzip** or **.zip** compressed file in a Base64 string as binaryData



Compression creates **overhead**. Use only if necessary!

Session Handling



How can you configure **Session Handling** with Redis and Airlock Microgateway, and protect traffic with **mutual TLS**?

Overview

Prerequisite: Redis

Connect Microgateway to Redis

The Microgateway Session Agent

Estimate Session Storage Memory

Protect Traffic with mutual TLS



Session Handling

Overview

Prerequisite: Redis

Connect Microgateway to Redis

The Microgateway Session Agent

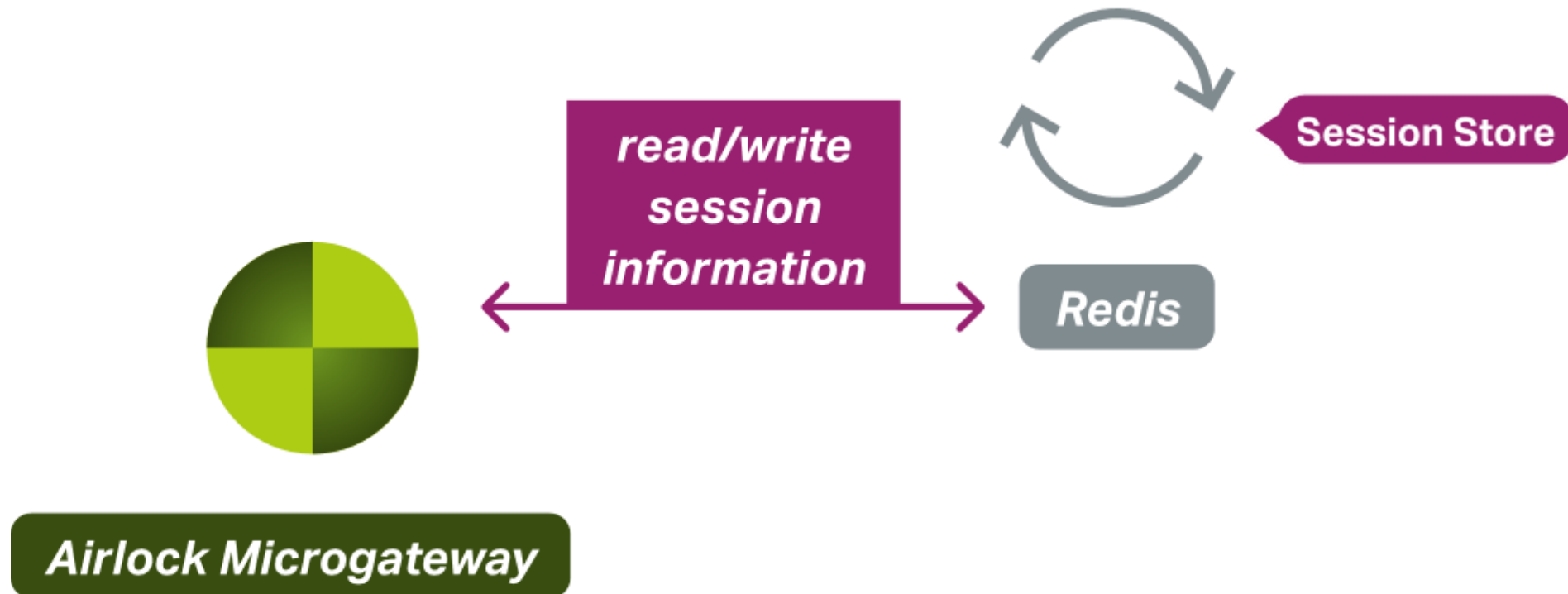
Estimate Session Storage Memory

Protect Traffic with mutual TLS

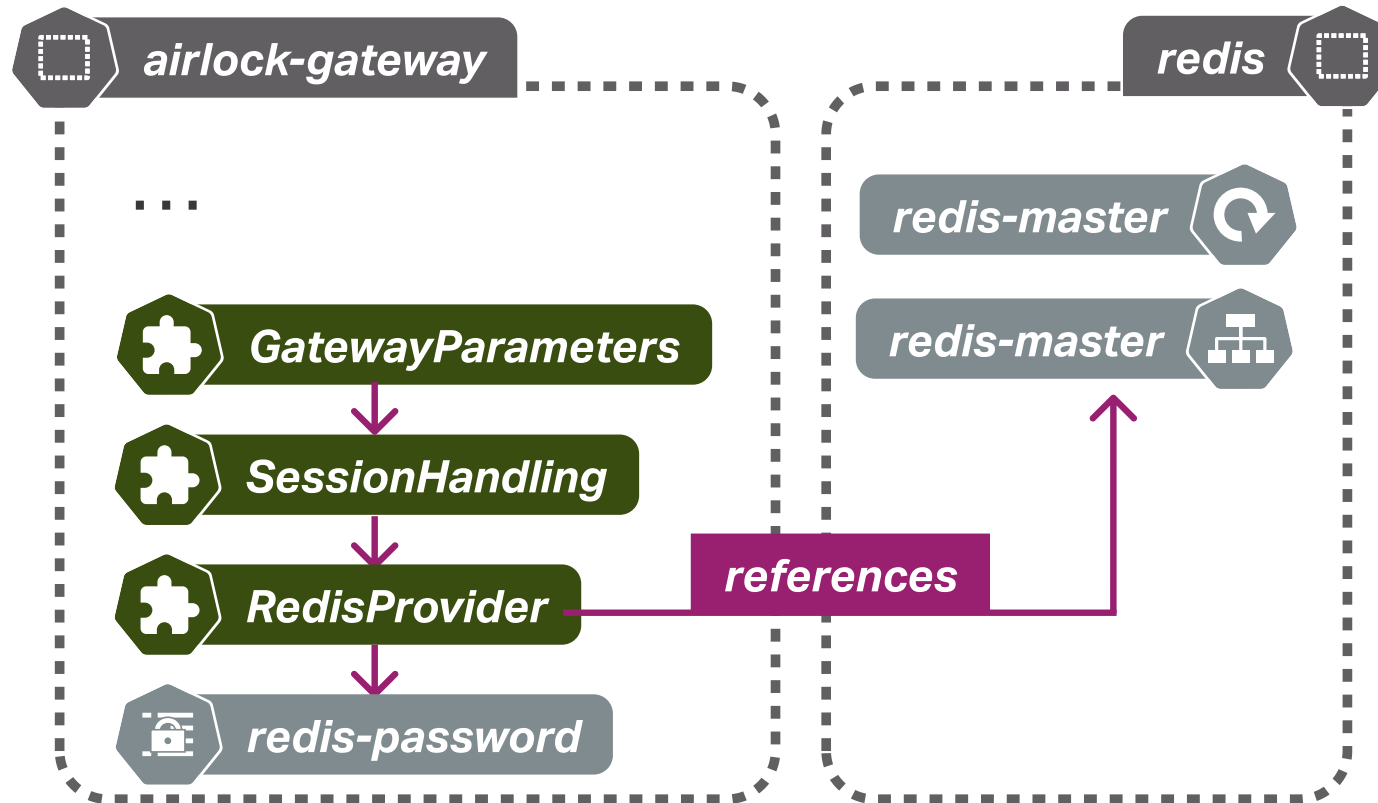


**Session
Handling**

Overview – Session Handling

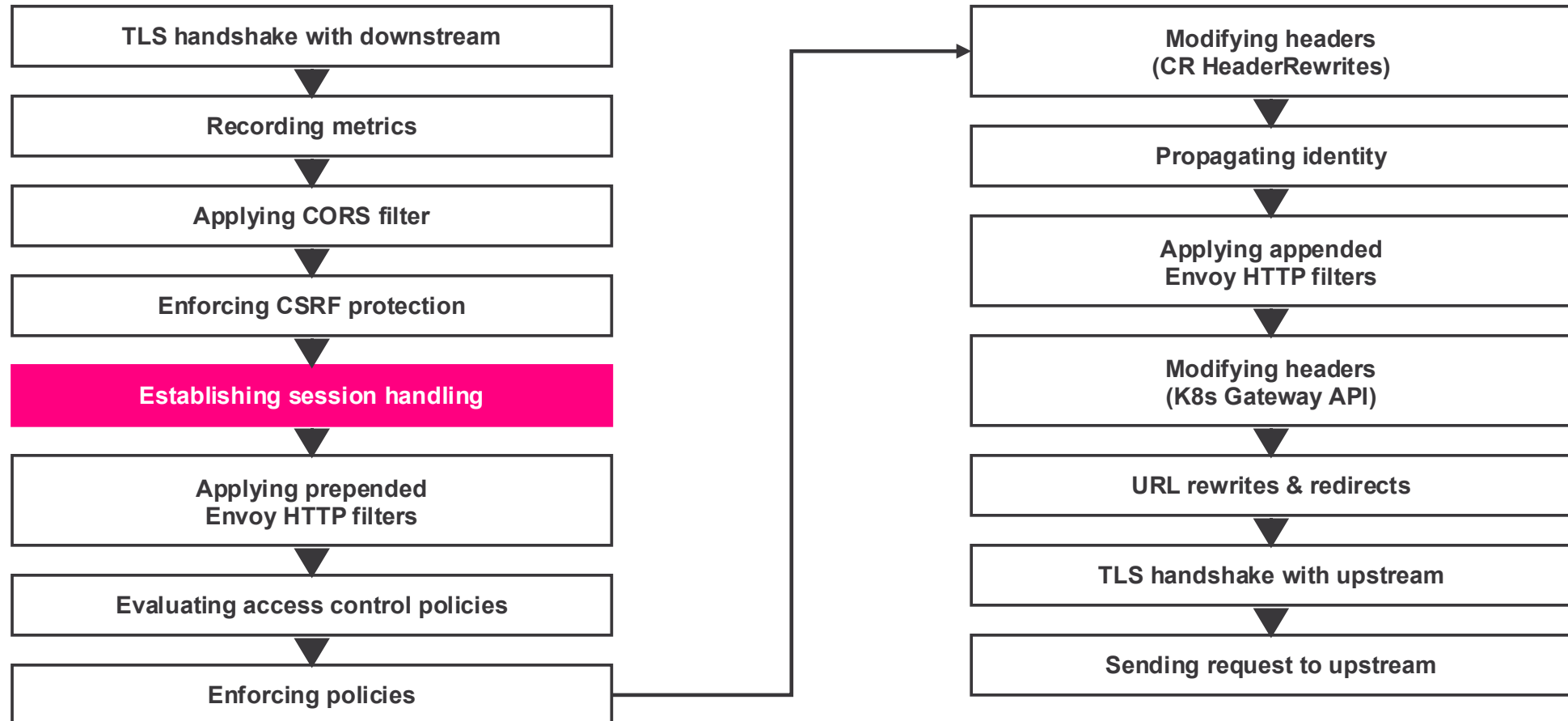


Overview – Session Handling Configuration

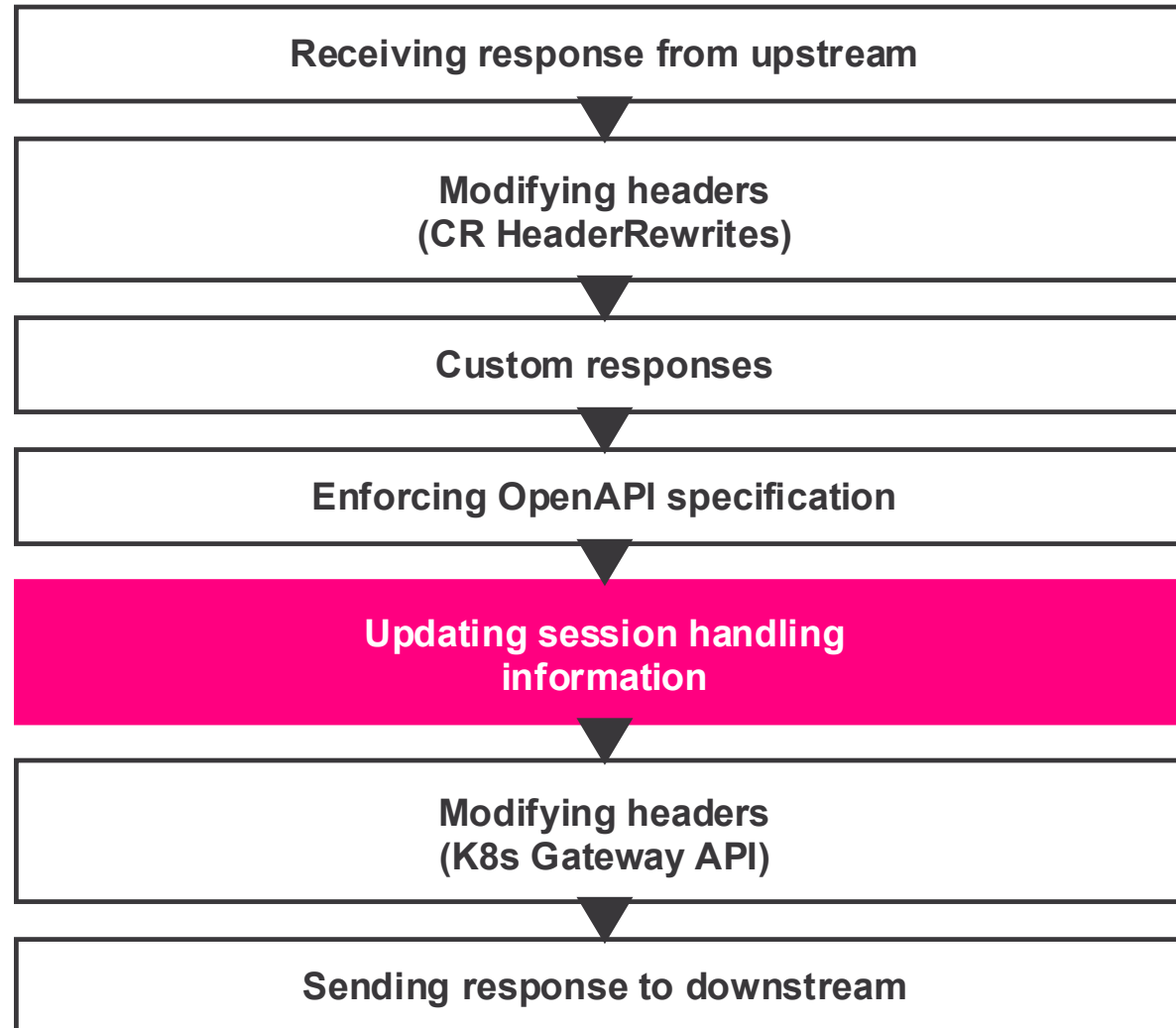


 crd  deployment  ns  service  secret  Airlock Microgateway  other

Processing Sequence – Request



Processing Sequence – Response



Feature Dependency

Session handling MUST be configured for **OIDC authentication**
and **access control functionality**

Session Handling Modes

spec.mode: Enforce

All requests are aggregated into sessions

- Requests without a session: new session is created
- Requests with existing session: session is reused + idle time reset

spec.mode: OnDemand (Default)

Reflects the «Enforce» behaviour for OIDC routes

For other routes: no session creation or usage

Session Handling Modes – Implications of «Enforce»



You need to set `spec.mode: Enforce` if session handling should be active for authentication **AND filtering** (e.g., DenyRules)

→ To do so, requires additional session storage

Session Limits

It is possible to specify limits for concurrent sessions per remote IP

```
...
  limits:
    sessionsPerIP:
      thresholds:
        # Specify the threshold above which an event is logged
        logging: 1000
        # Specify the threshold above which requests are blocked
        blocking: 1500
...
```

Session Limits – Exceptions

It is also possible to specify conditions under which the limit does not apply

```
...
limits:
  sessionsPerIP:
    thresholds:
      # Specify the threshold above which an event is logged
      logging: 1000
      # Specify the threshold above which requests are blocked
      blocking: 1500
    exceptions:
      # exceptions on remote IP CIDR ranges
      - remoteIP:
          cidrRanges:
            - 192.168.0.0/16
...

```

Overview

Prerequisite: Redis

Connect Microgateway to Redis

The Microgateway Session Agent

Estimate Session Storage Memory

Protect Traffic with mutual TLS



**Session
Handling**

Prerequisite: Redis or Valkey

Used by a «**SessionHandling**» CR for storing session data

- Required for **session persistence** and **token caching**
- Can be deployed within or outside Microgateway cluster
- Supported deployment modes
 - Standalone
 - Sentinel
 - Cluster
 - Cloud



Valkey



redis

Overview

Prerequisite: Redis

Connect Microgateway to Redis

The Microgateway Session Agent

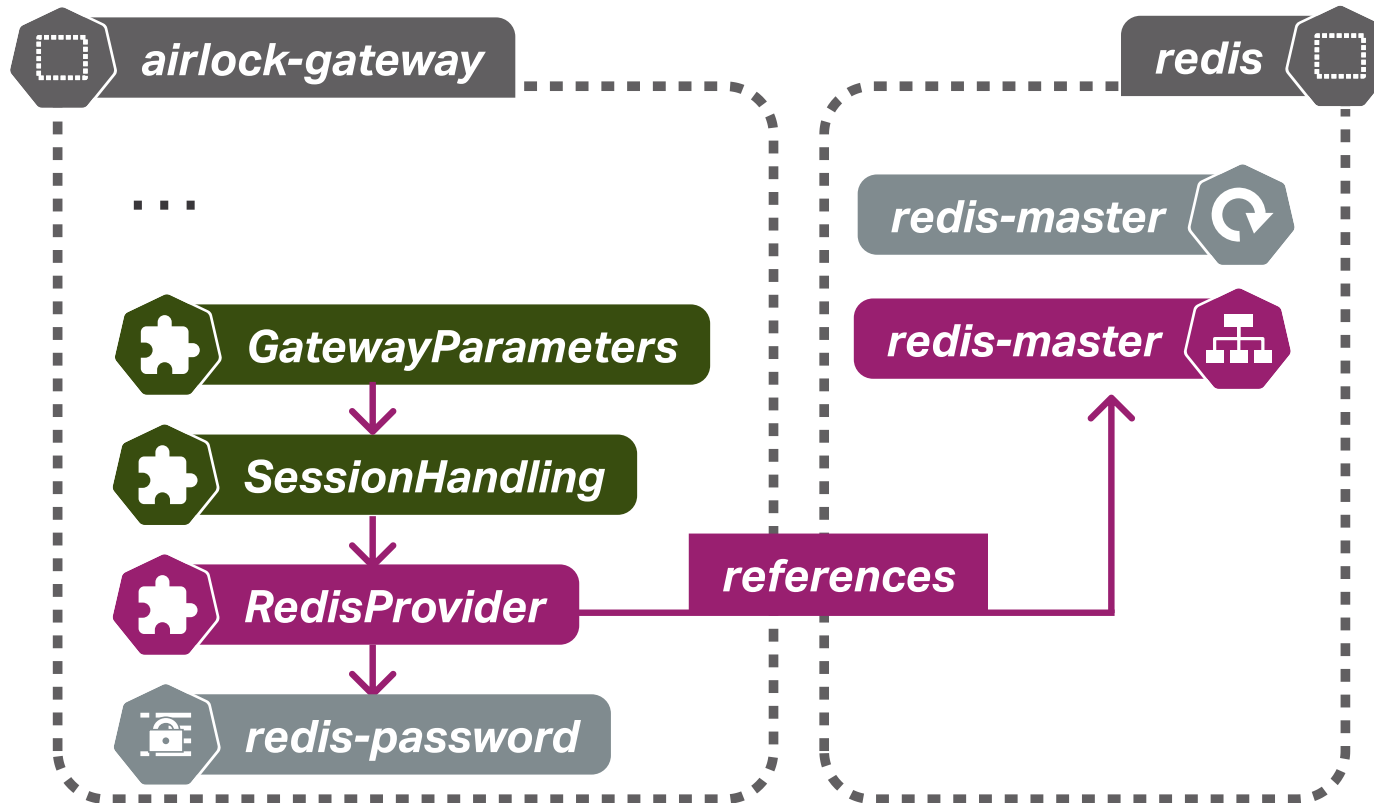
Estimate Session Storage Memory

Protect Traffic with mutual TLS



Session Handling

Connect Microgateway to Redis – CR «RedisProvider»



```
apiVersion: microgateway.airlock.com/v1alpha1
kind: RedisProvider
metadata:
  name: sessionhandling
  namespace: airlock-gateway
spec:
  mode:
    standalone:
      host: redis.example.local
      port: 6379
  auth:
    password:
      secretRef:
        name: redis-password
```

crd deployment ns service secret Airlock Microgateway other

Connect Microgateway to Redis – CR «RedisProvider»

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: RedisProvider
metadata:
  name: sessionhandling
  namespace: airlock-gateway
spec:
  mode:
    standalone:
      host: redis.example.local
      port: 6379
  auth:
    password:
      secretRef:
        name: redis-password
```

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: SessionHandling
metadata:
  name: sessionhandling
  namespace: airlock-gateway
spec:
  persistence:
    redisProviderRef:
      name: sessionhandling
```

```
apiVersion: v1
kind: Secret
metadata:
  name: redis-password
  namespace: airlock-gateway
...
```

Overview

Prerequisite: Redis

Connect Microgateway to Redis

The Microgateway Session Agent

Estimate Session Storage Memory

Protect Traffic with mutual TLS

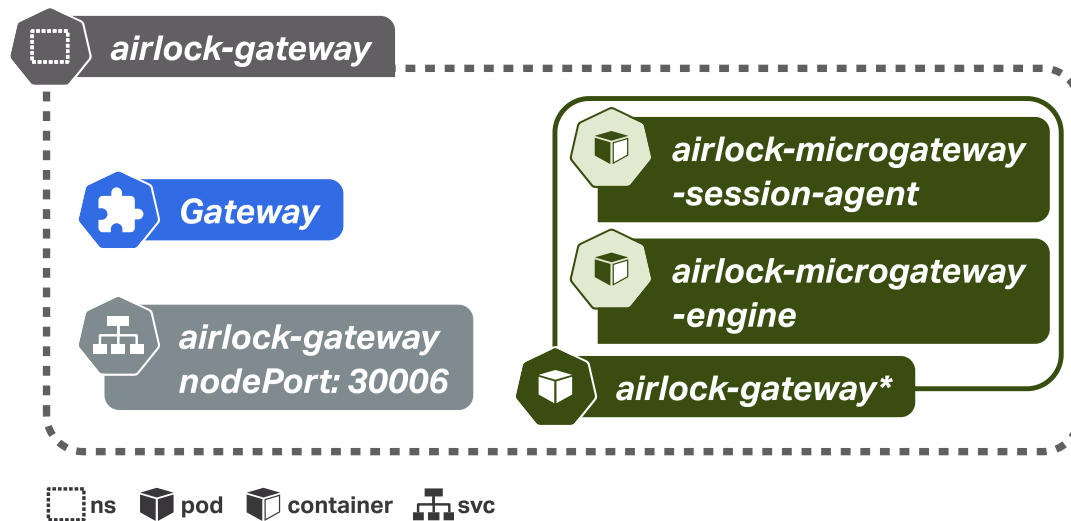


**Session
Handling**

The Microgateway Session Agent

Handles the Redis session store connection

- Deployed by the Airlock Microgateway **Operator**
- Deployed in **same Pod** as the Airlock Microgateway **Engine container**



Overview

Prerequisite: Redis

Connect Microgateway to Redis

The Microgateway Session Agent

Estimate Session Storage Memory

Protect Traffic with mutual TLS



**Session
Handling**

Session Storage Memory

When session handling is enabled, **each active session consumes memory** proportional to its stored data

OIDC-based sessions require **more memory** due to token storage

Session Storage Memory – Estimation

session size

≈ base data + optional OIDC tokens

Consider...

- ...session metadata and internal storage base data

- ...OIDC access, ID and refresh tokens (if applicable)

total memory

≈ average session size × number of concurrent sessions × safety margin

Overview

Prerequisite: Redis

Connect Microgateway to Redis

The Microgateway Session Agent

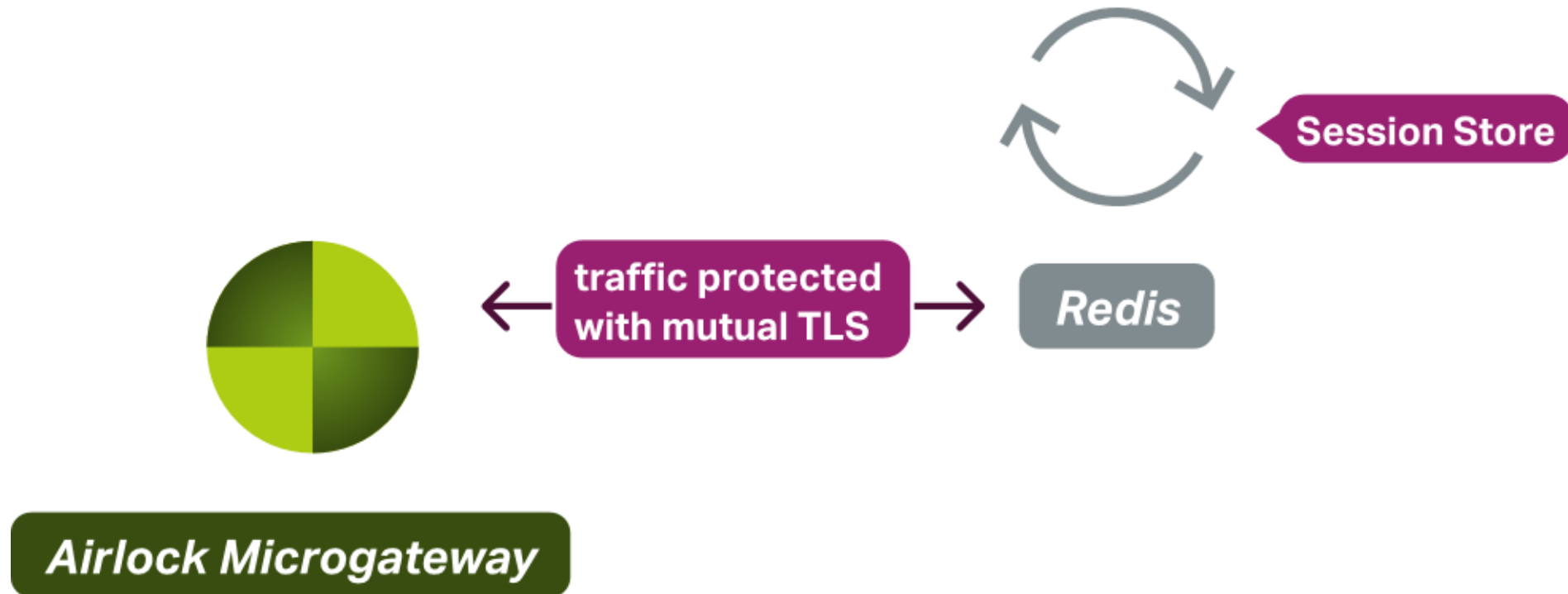
Estimate Session Storage Memory

Protect Traffic with mutual TLS

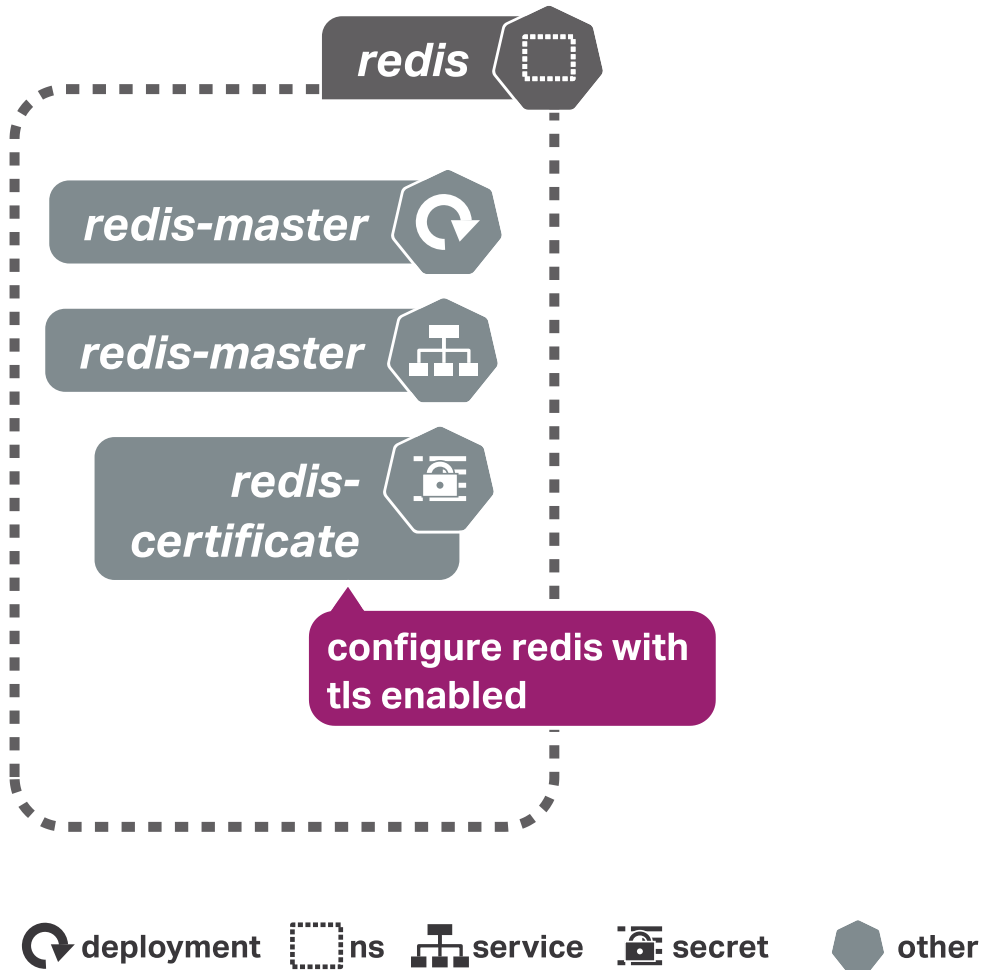


**Session
Handling**

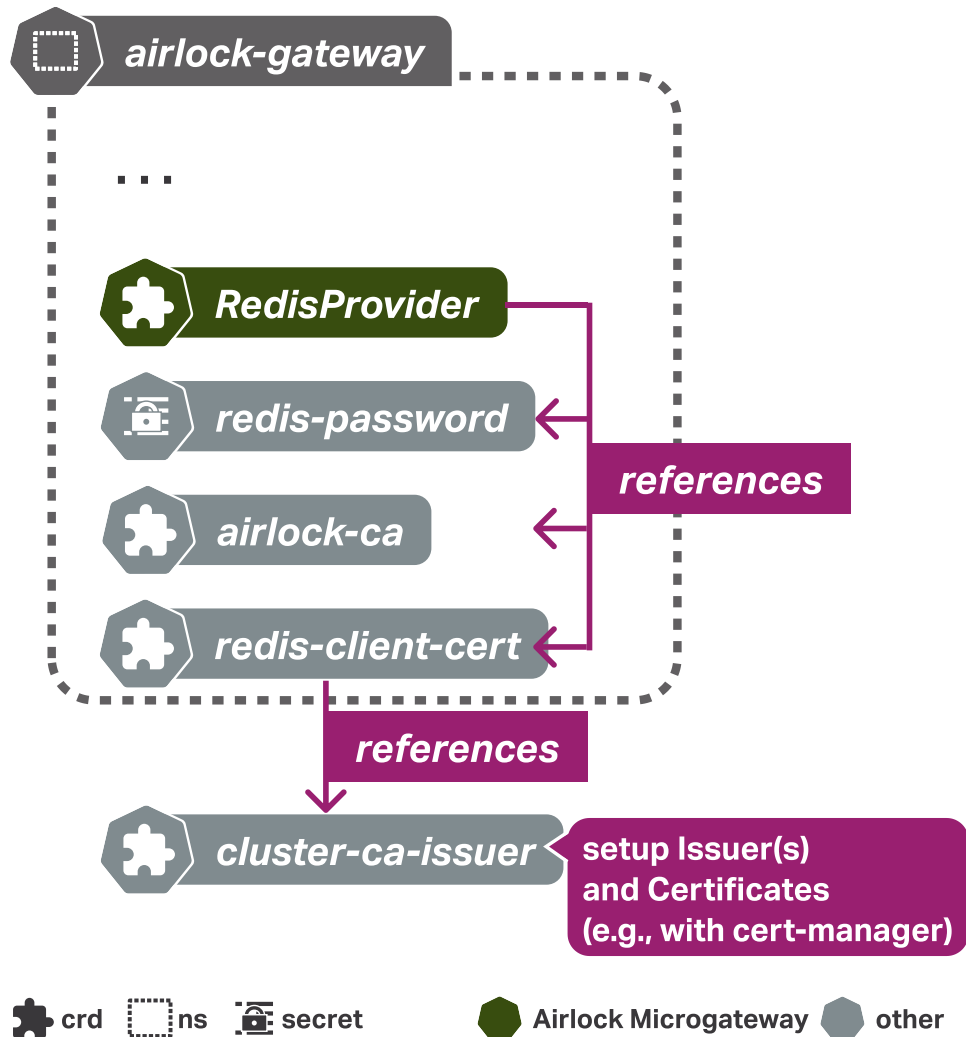
Protect Traffic with mutual TLS – Overview



Protect Traffic with mutual TLS – Enable Redis TLS



Protect Traffic with mutual TLS – Example Configuration



```
apiVersion: microgateway.airlock.com/v1alpha1
kind: RedisProvider
metadata:
  name: sessionhandling
  namespace: airlock-gateway
spec:
  mode:
    standalone:
      host: redis-master.redis.svc.cluster.local
      port: 6379
  tls:
    certificateVerification:
      custom:
        trustedCA:
          certificates:
            - secretRef:
                name: airlock-ca
    clientCertificate:
      secretRef:
        name: redis-client-cert
  auth:
    password:
      secretRef:
        name: redis-password
```

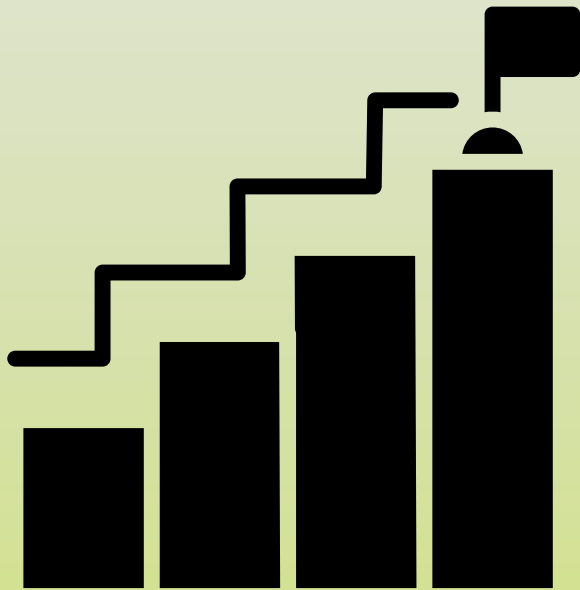
LAB 06

Session Handling



LAB 06

Session Handling

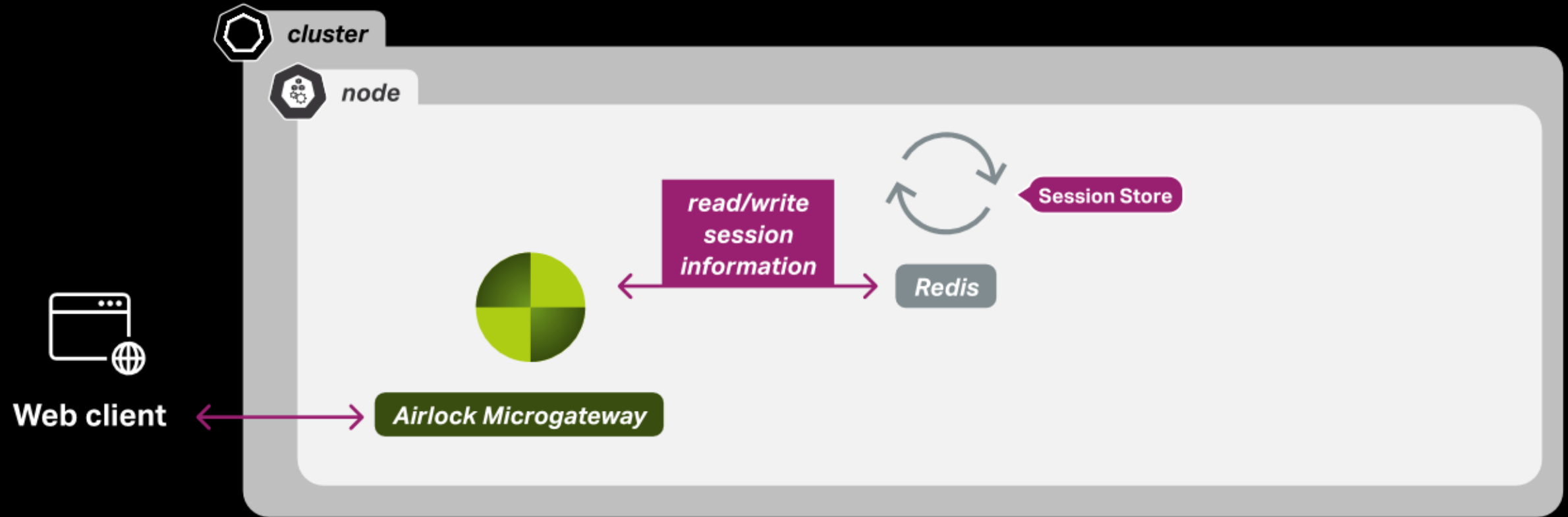


 30–40 min

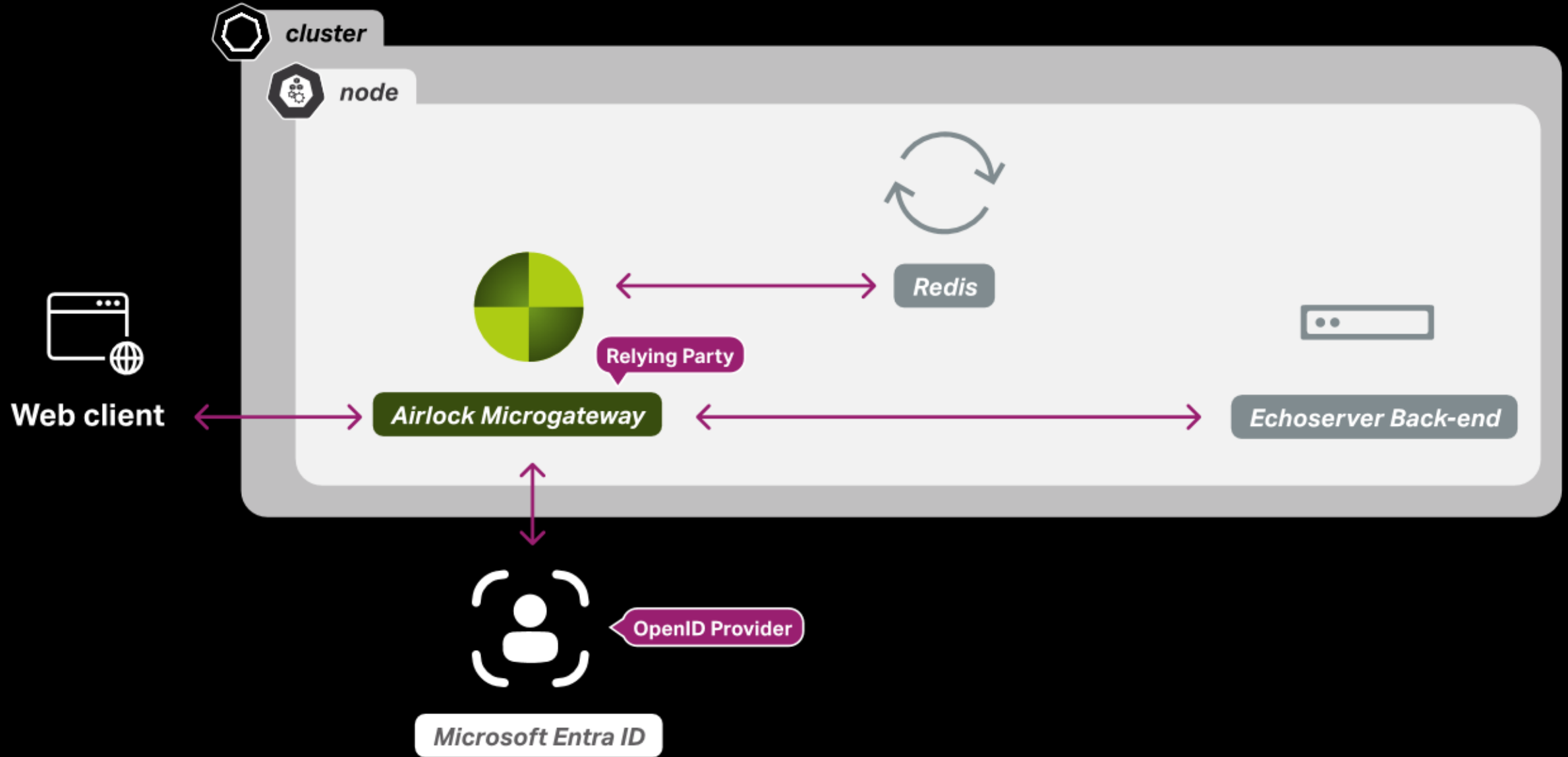
Steps

- Setup session handling and test your configuration
- Secure traffic between Airlock Microgateway and Redis with mutual TLS (optional)

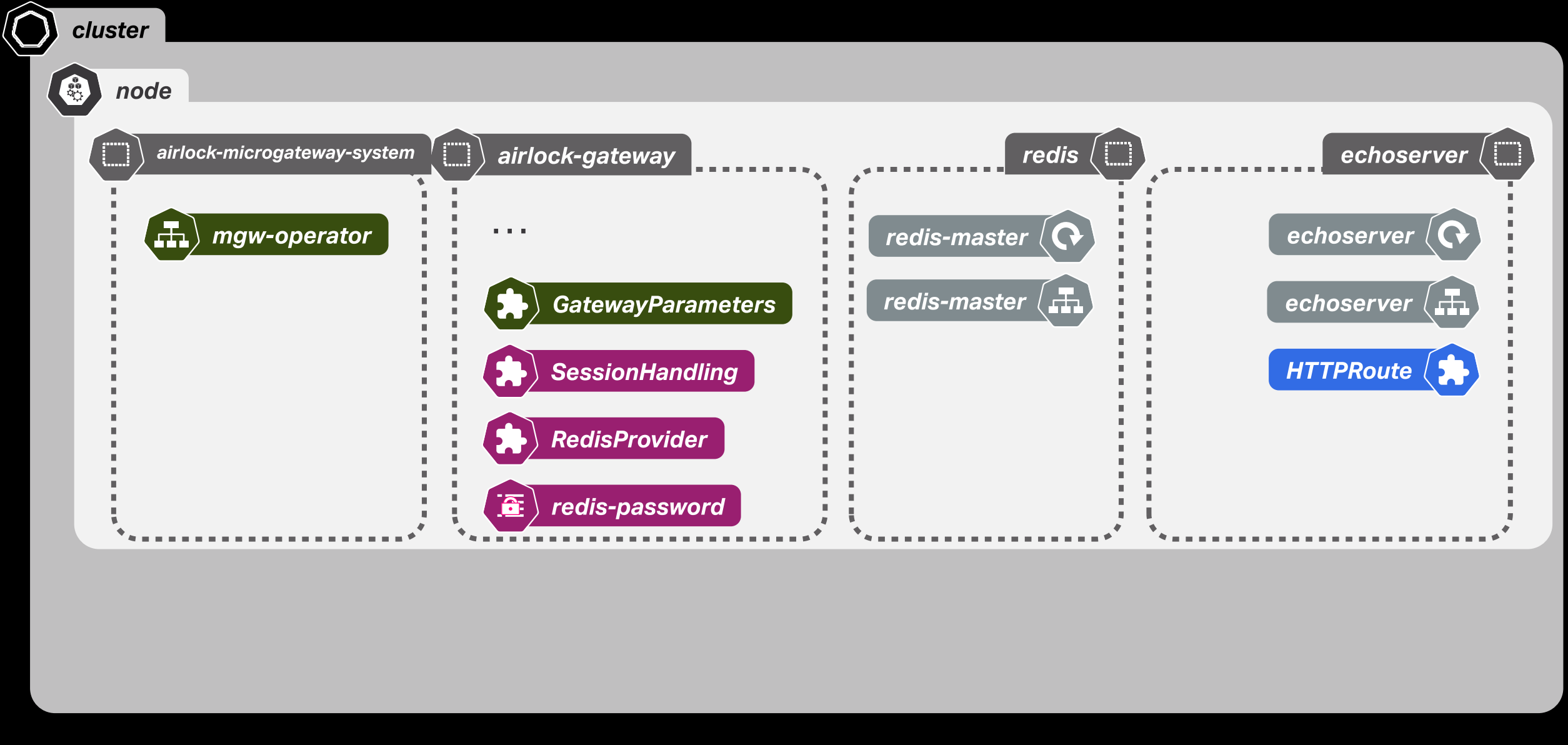
Focus of This Lab: Setup Session Handling as Foundation for an OIDC Configuration with Airlock Microgateway



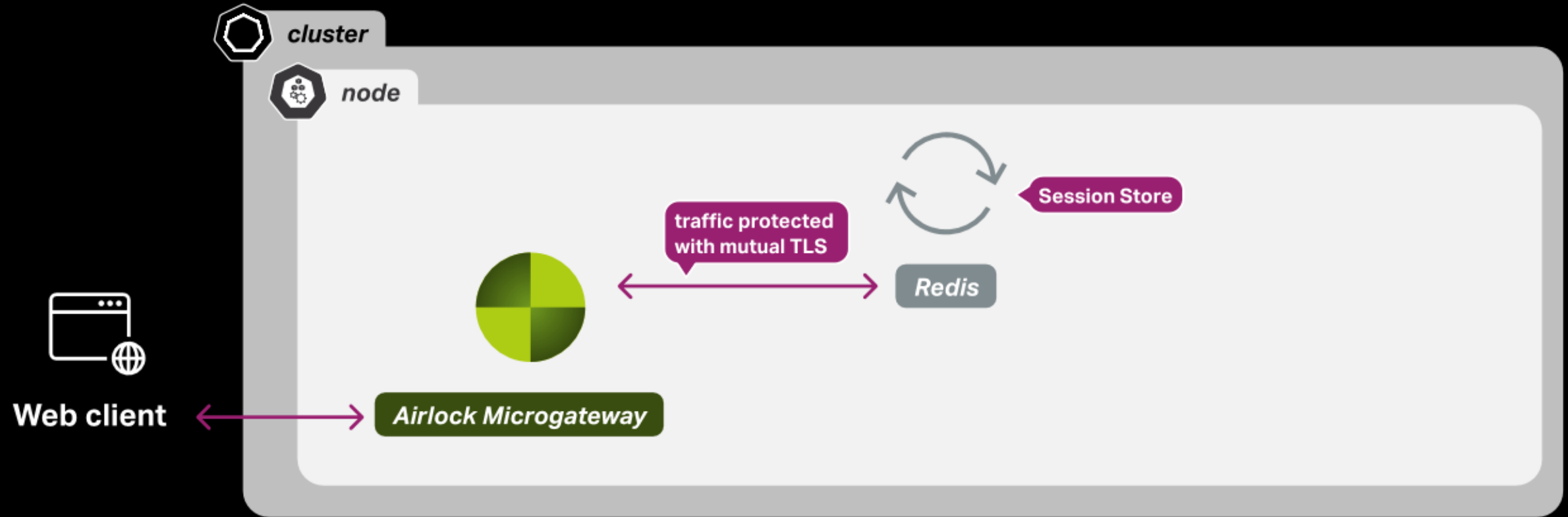
You Will be Able to Test Your Configuration with Microsoft Entra ID (OpenID Provider) and an Echoserver (Back-end)



In **Challenge 1**, You will Configure Airlock Microgateway Session Handling



In **Challenge 2**, You will Secure Traffic between Airlock Microgateway and Redis with mTLS



LAB 06

Session Handling



Any feedback?



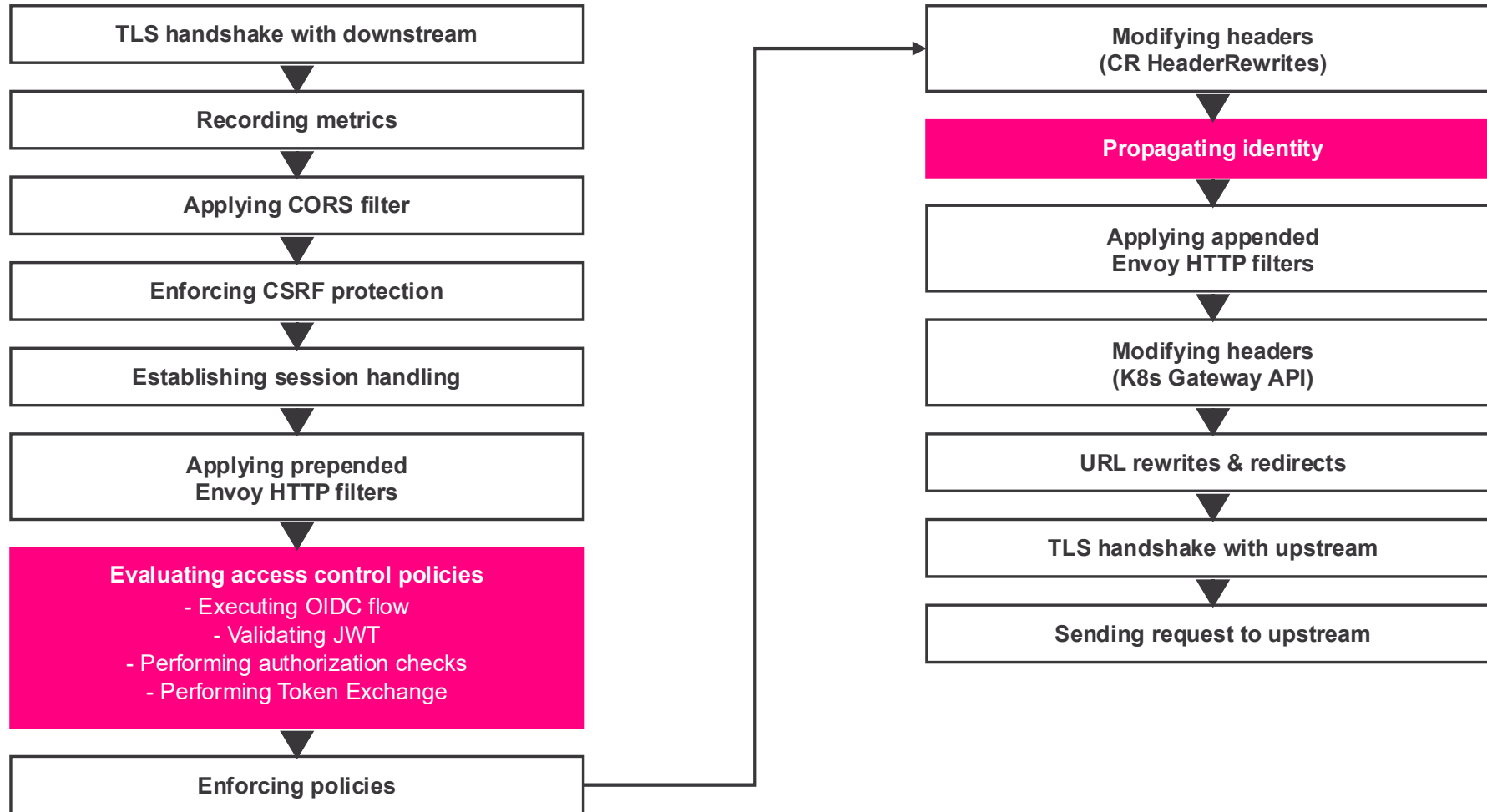
AuthN, AuthZ, and IdP

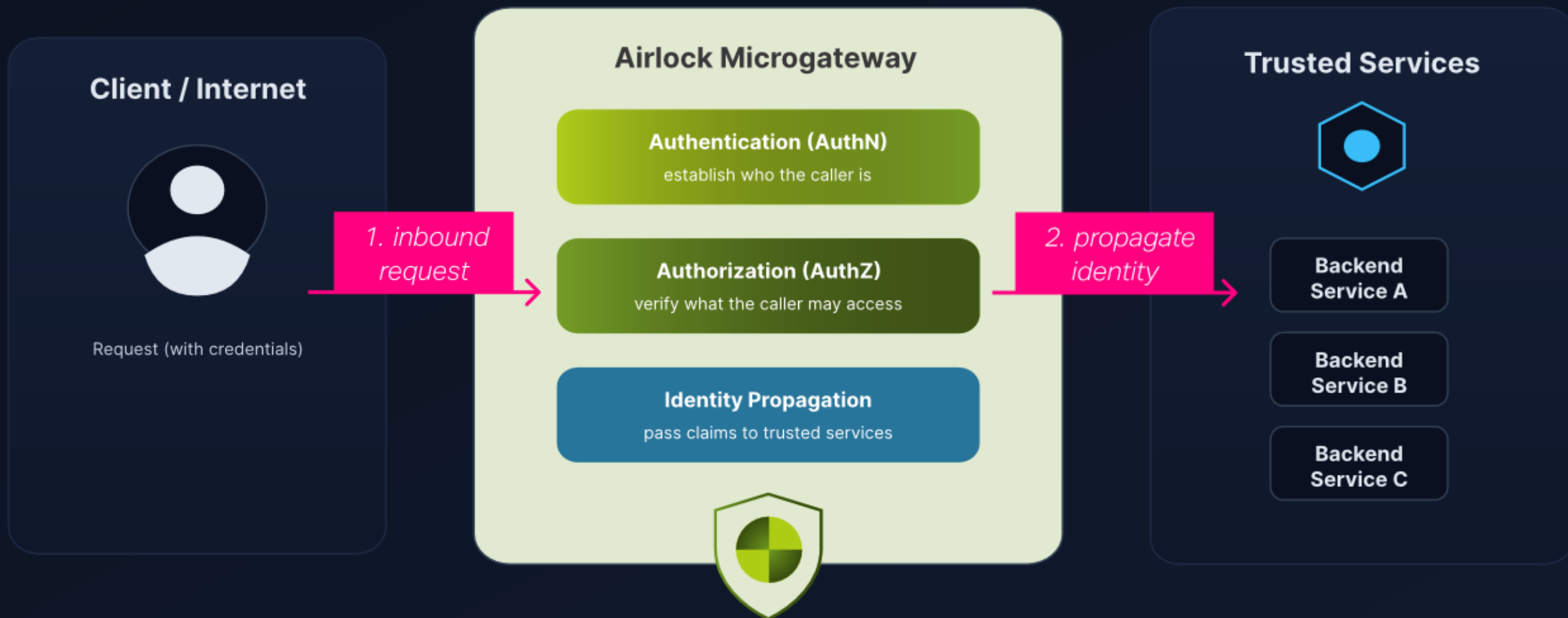
IdP = Identity Propagation



How can you configure **Authentication, Authorization**
and **Identity Propagation** with Airlock Microgateway?

Processing Sequence – Request





Forms of AuthN and AuthZ

Example: OIDC AuthN and AuthZ

Forms of Identity Propagation (IdP)

Example: OIDC IdP



AuthN, AuthZ and IdP

Forms of AuthN and AuthZ

Example: OIDC AuthN and AuthZ

Forms of Identity Propagation (IdP)

Example: OIDC IdP



AuthN, AuthZ and IdP

Supported Forms of AuthN and AuthZ

JWT

Client authentication via a JSON Web Token

OIDC

Authentication and authorization with an OpenID Provider (e.g., Airlock IAM)

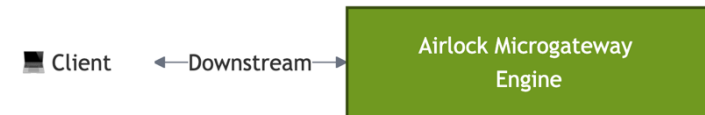
Supported Forms of AuthN and AuthZ

Client Certificate (mTLS)

Clients can be prompted during TLS handshake to authenticate with certificate

frontend [FrontendTLSConfig](#)

Frontend describes TLS config when client connects to Gateway.
Support: Core



Backend TLS

Microgateway can also use TLS to authenticate itself to backends

backend [GatewayBackendTLS](#)

Backend describes TLS configuration for gateway when connecting to backends.

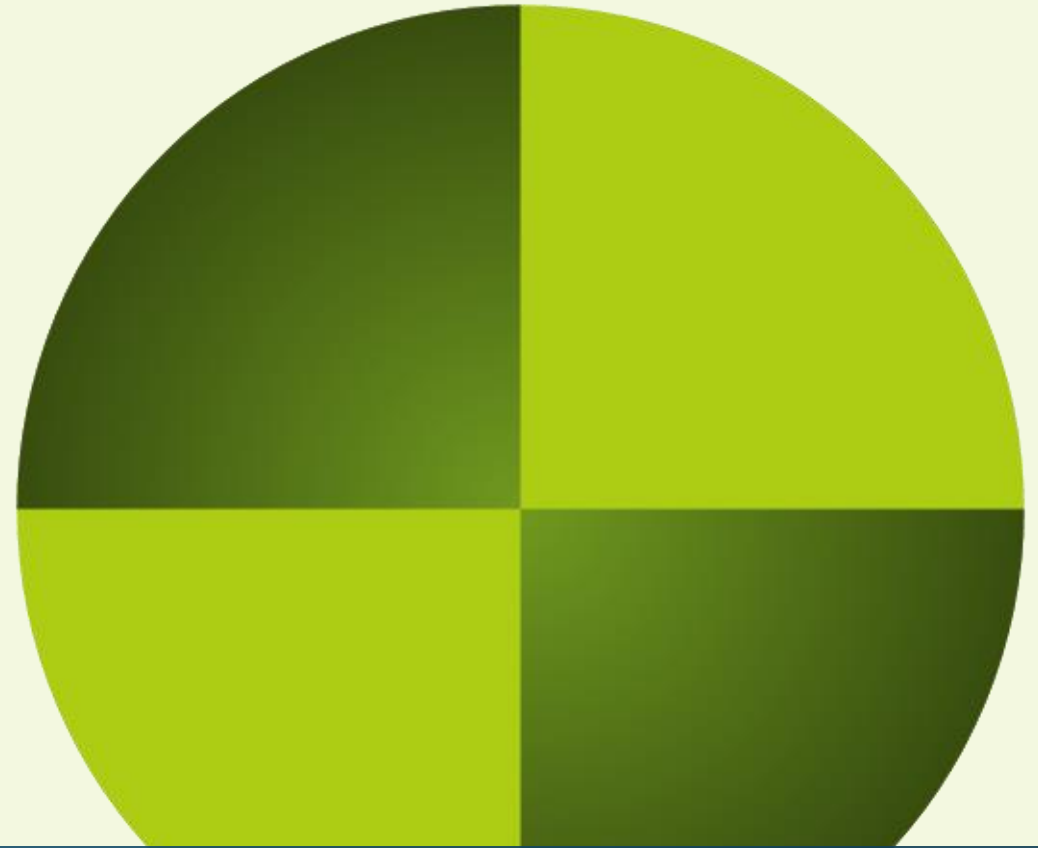


Forms of AuthN and AuthZ

Example: OIDC AuthN and AuthZ

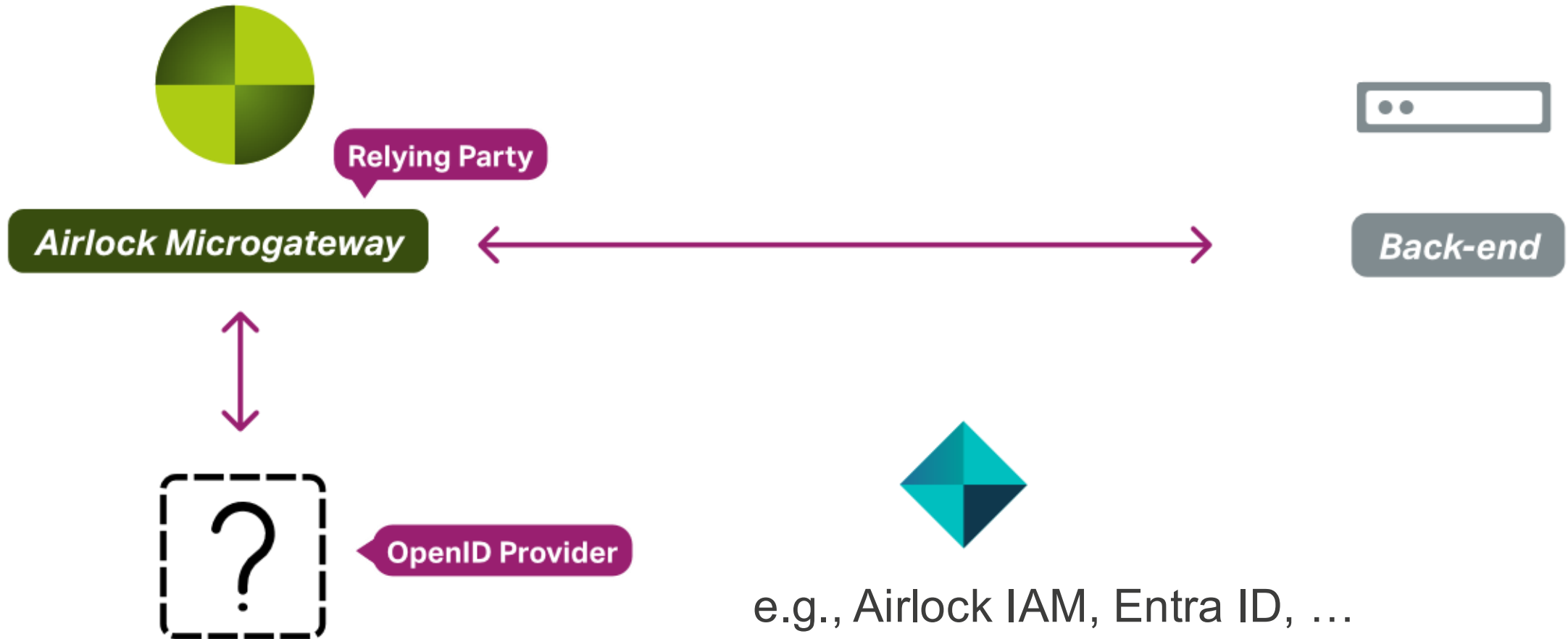
Forms of Identity Propagation (IdP)

Example: OIDC IdP



AuthN, AuthZ and IdP

OIDC – Overview



OIDC – Core Custom Resource Definitions

JWKS

To fetches signing keys from an OpenID Provider (OP)

OIDCProvider

To specify an OP with identifier and required endpoints

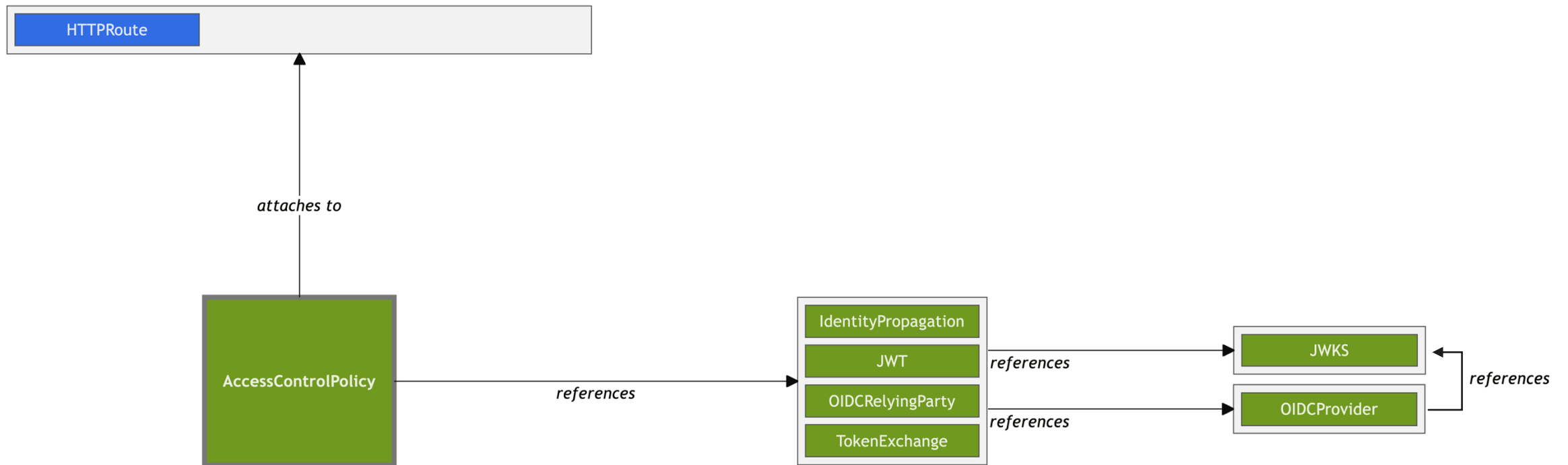
OIDCRelyingParty

To specify how the Airlock Microgateway Engine interacts with an OP

AccessControlPolicy

To enforce OIDC-based authorization

Custom Resources – Example «AccessControlPolicy»



OIDC – CR «JWKS», Entra ID Example

Provides config for JSON Web Key Set (JWKS) providers and endpoints

```
...
remote:
  uri: https://login.microsoftonline.com/<tenant-id>/discovery/v2.0/keys
  timeouts:
    connect: 5s
```

- To **verify OIDC token signatures**
- Configured via the **OP's JWKS endpoint URI**
- **Timeouts** ensure reliable retrieval and resilience

OIDC – CR «JWKS», Entra ID Example

Define JSON Web Key Set settings.
You need to specify the respective Entra ID endpoint.



Microsoft Entra ID

 crd  secret

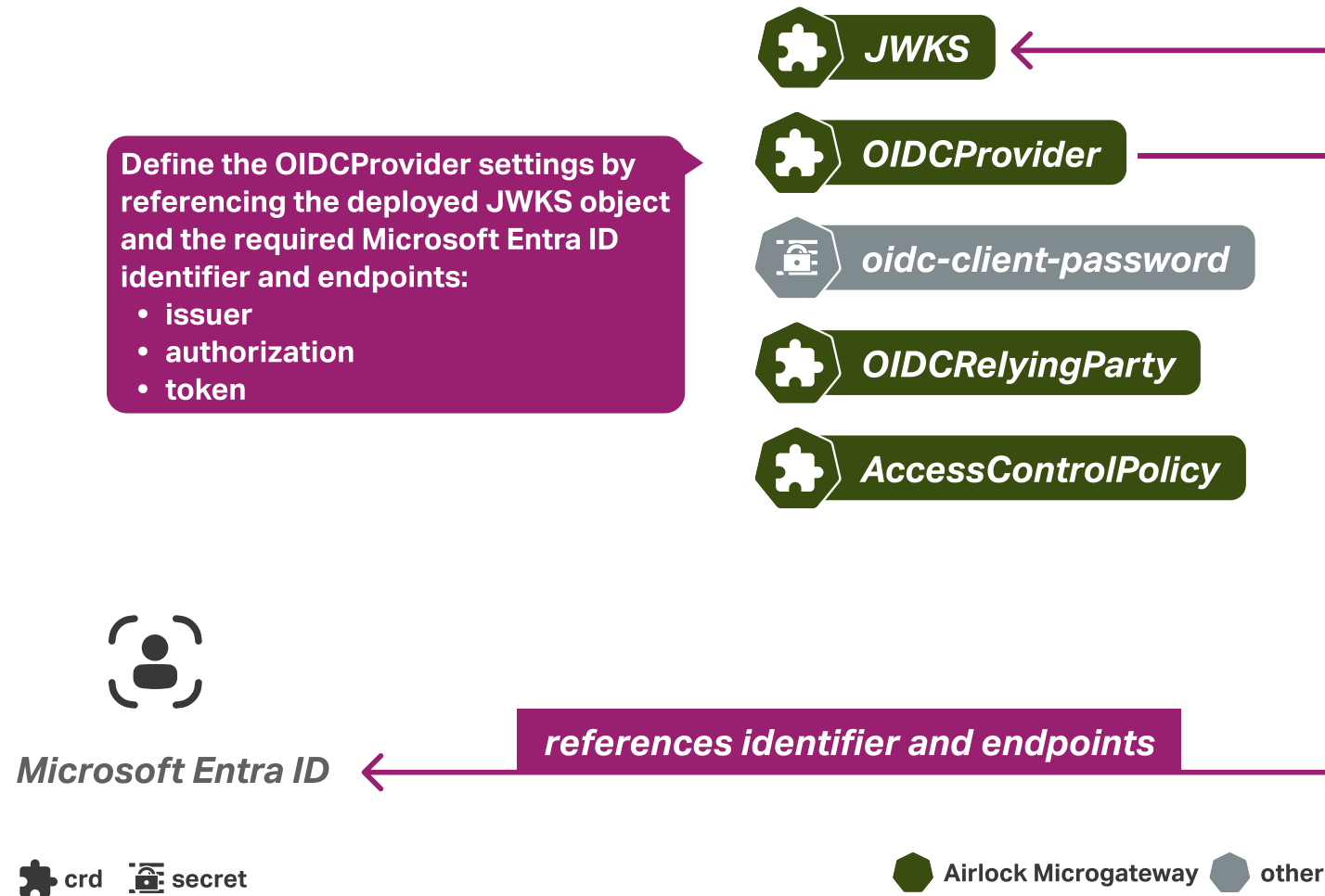
 Airlock Microgateway  other

OIDC – CR «OIDCProvider», Entra ID Example

Represents the **OpenID Provider (OP)** configuration

- Specifies **identifier** for issuer and **endpoints** for authentication and tokens
- References «**JWKS**» CR for signature verification

OIDC – CR «OIDCProvider», Entra ID Example

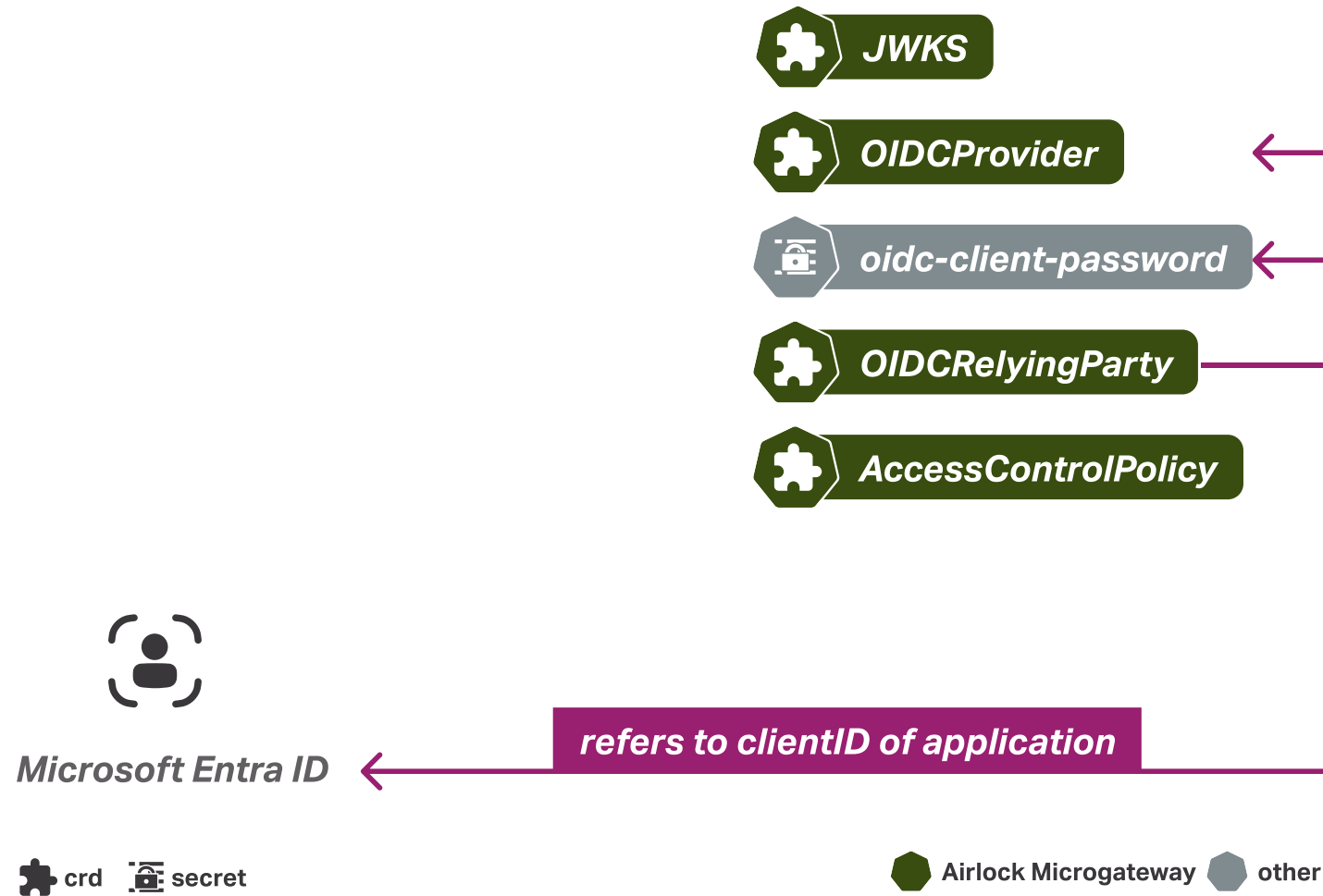


OIDC – CR «OIDCRelyingParty», Entra ID Example

Represents the **Relying Party (RP)** configuration

- Contains the registered **clientId** and a **reference to the clientSecret** (stored as K8s secret)
- References the **OIDCProvider** for token exchange

OIDC – CR «OIDCProvider», Entra ID Example

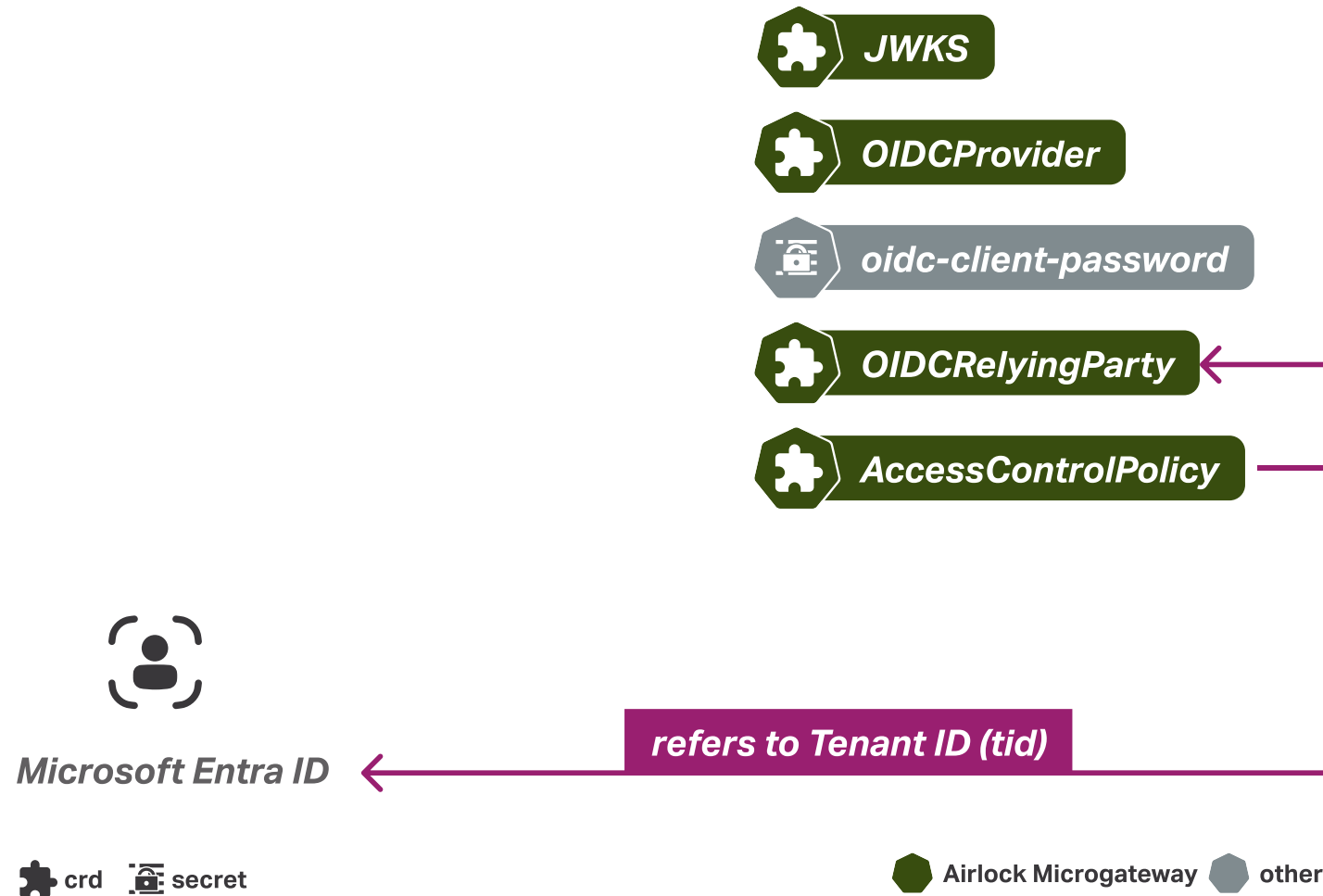


OIDC – CR «AccessControlPolicy», Entra ID Example

Required to **enforce OIDC-based authorization**

- References the **OIDCRelyingParty** configuration
- Possible to **check for conditions** (e.g., required claims) that need to be satisfied
- Example: claim «tid» (tenant id) must match exact value «1a400452-ft3...»

OIDC – CR «AccessControlPolicy», Entra ID Example



Forms of AuthN and AuthZ

Example: OIDC AuthN and AuthZ

Forms of Identity Propagation (IdP)

Example: OIDC IdP



AuthN, AuthZ and IdP

Supported Forms of IdP– Value Extraction

clientCertificate

extract value from the presented client certificate

jwt

extract value from a JWT

metadata

extract value from an Envoy dynamic filter metadata key

oidc

extract value from the result of an OpenID Connect flow

IdP = Identity Propagation

Supported Forms of IdP– Value Extraction

tokenExchange

use the token obtained via token exchange as value

→ Possible to specify a JWT claim to extract as value



Requires configuration of the CR **TokenExchange**

→ Allows to exchange token received from downstream to a new one for authorization or propagation to the upstream service

IdP = Identity Propagation

Supported Forms of IdP– Propagation

bearerToken

IdP via an **authorization header** containing a bearer token

header

IdP via a **request header**

IdP = Identity Propagation

Forms of AuthN and AuthZ

Example: OIDC AuthN and AuthZ

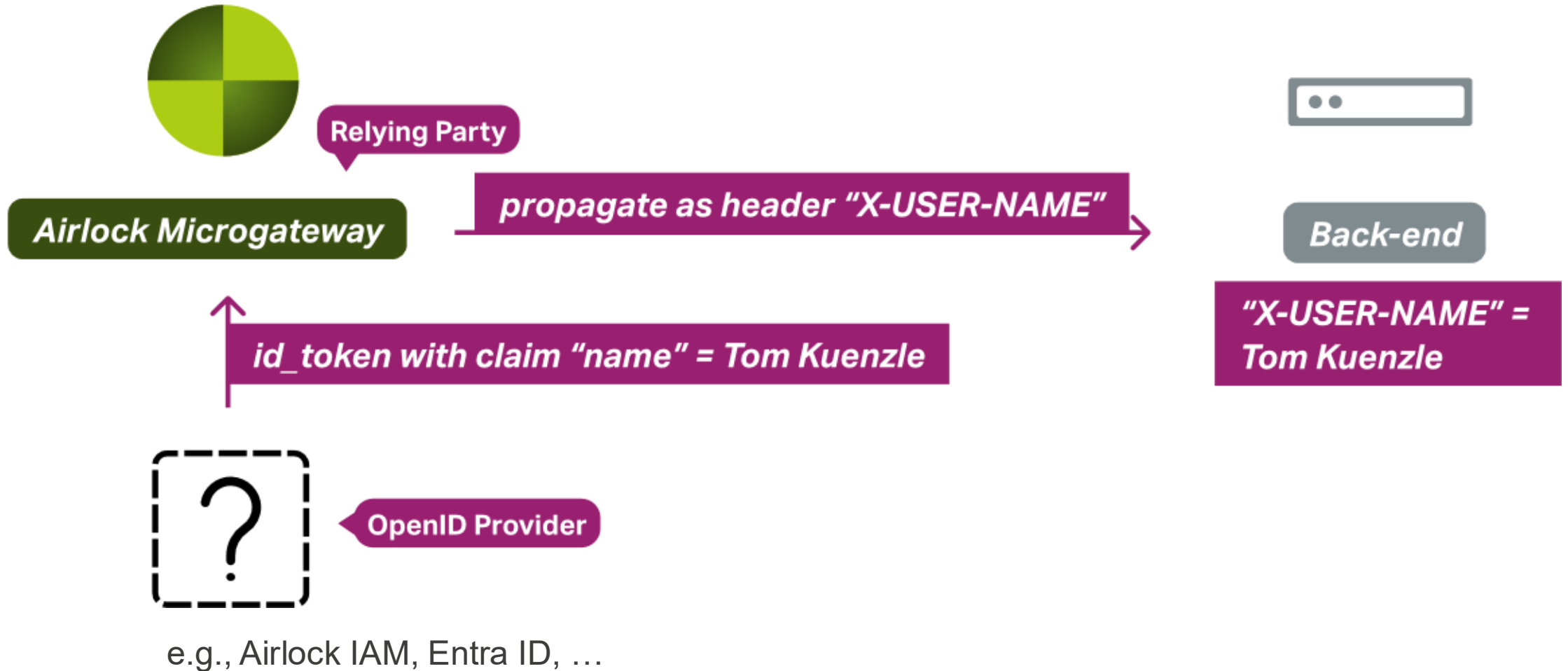
Forms of Identity Propagation (IdP)

Example: OIDC IdP



AuthN, AuthZ and IdP

OIDC IdP – Overview



IdP = Identity Propagation

OIDC IdP – Core Custom Resource Definitions

IdentityPropagation To define forms of value extraction and header-based identity propagation

AccessControlPolicy To enforce identity propagation

IdP = Identity Propagation

OIDC IdP – Examples CR «IdentityPropagation»

forward an oidc access_token as
authorization header

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: IdentityPropagation
metadata:
  name: idp-bearer-example
spec:
  bearerToken:
    source:
      oidc:
        accessToken: {}
```

forward a user's name claim as
X-USER-NAME header

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: IdentityPropagation
metadata:
  name: idp-header-example
spec:
  header:
    name: X-USER-NAME
    value:
      source:
        oidc:
          idToken:
            claim: "name"
```

IdP = Identity Propagation

OIDC IdP – Examples CR «AccessControlPolicy»

forward an oidc access_token as
authorization header

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: IdentityPropagation
metadata:
  name: idp-bearer-example
spec:
  bearerToken:
    source:
      oidc:
        accessToken: {}
```

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: AccessControlPolicy
metadata:
  name: access-control-policy-example
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: backend
  policies:
    - identityPropagation:
        actions:
          - identityPropagationRef:
              name: idp-bearer-example
            onFailure: Pass
    ...
```

IdP = Identity Propagation

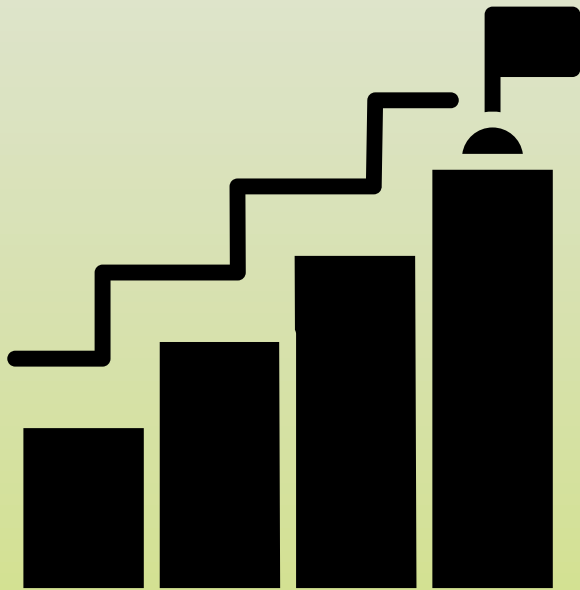
LAB 07

Authentication and Authorization with Entra ID



LAB 07

Authentication and Authorization with Entra ID

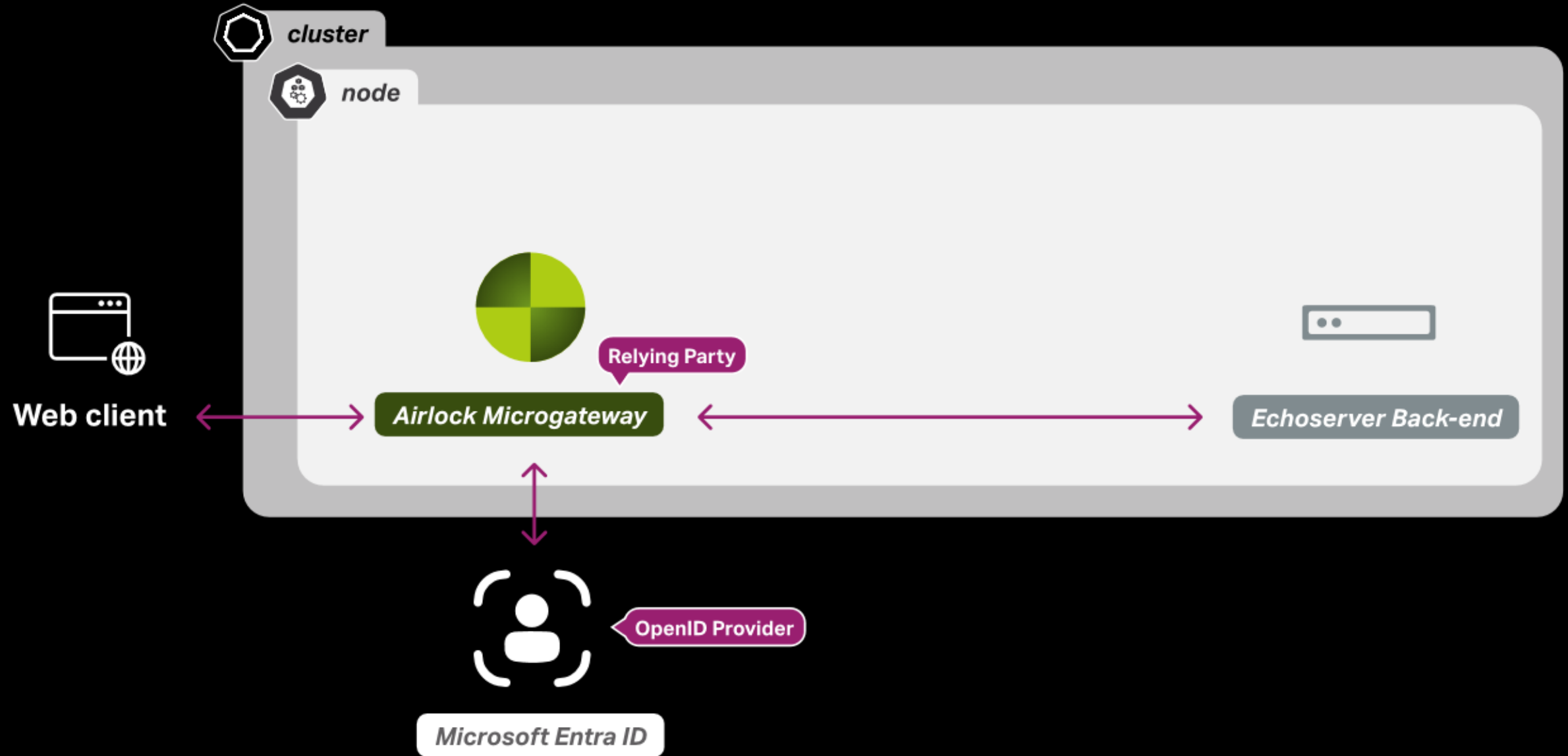


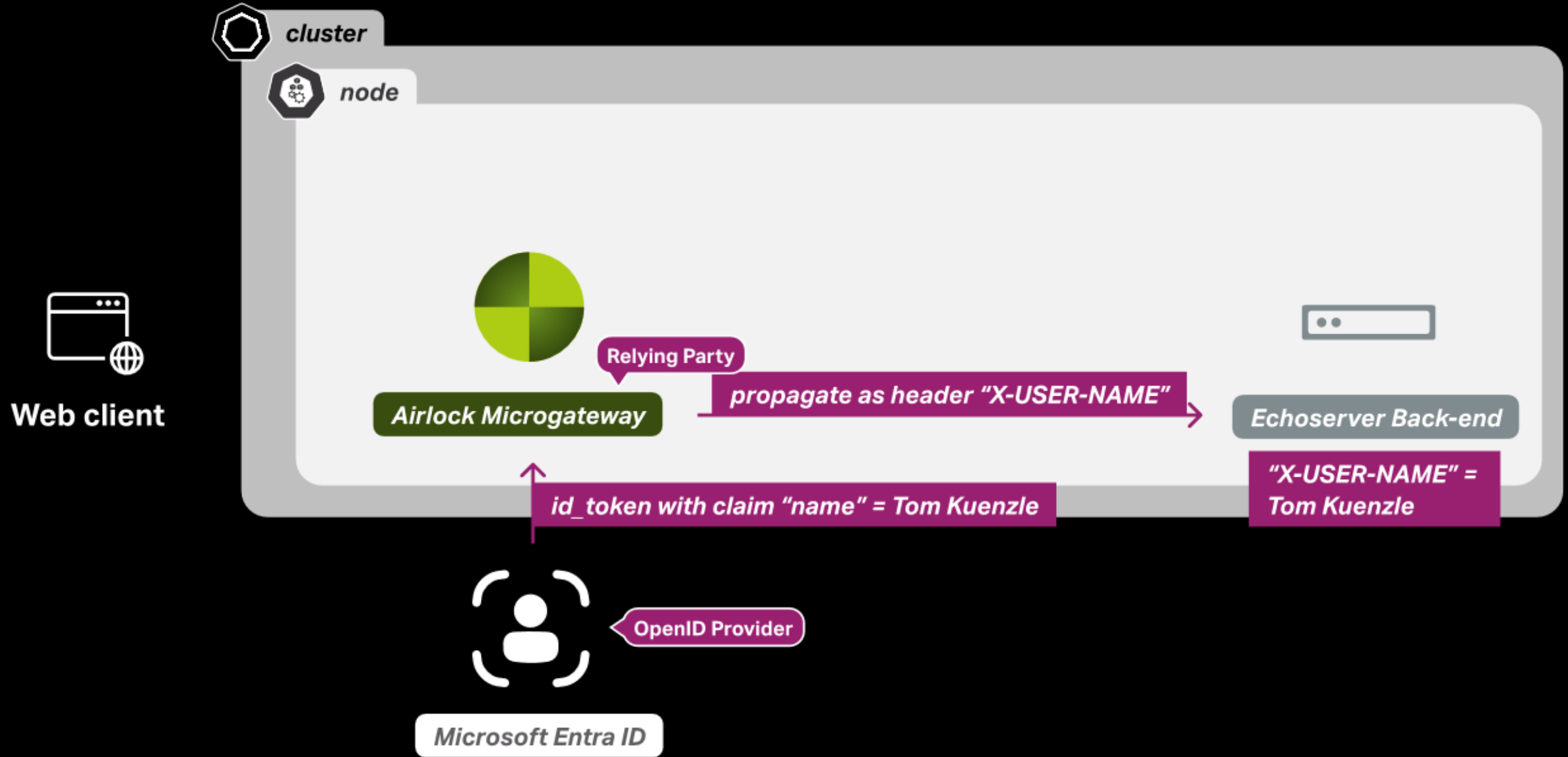
 30–40 min

Steps

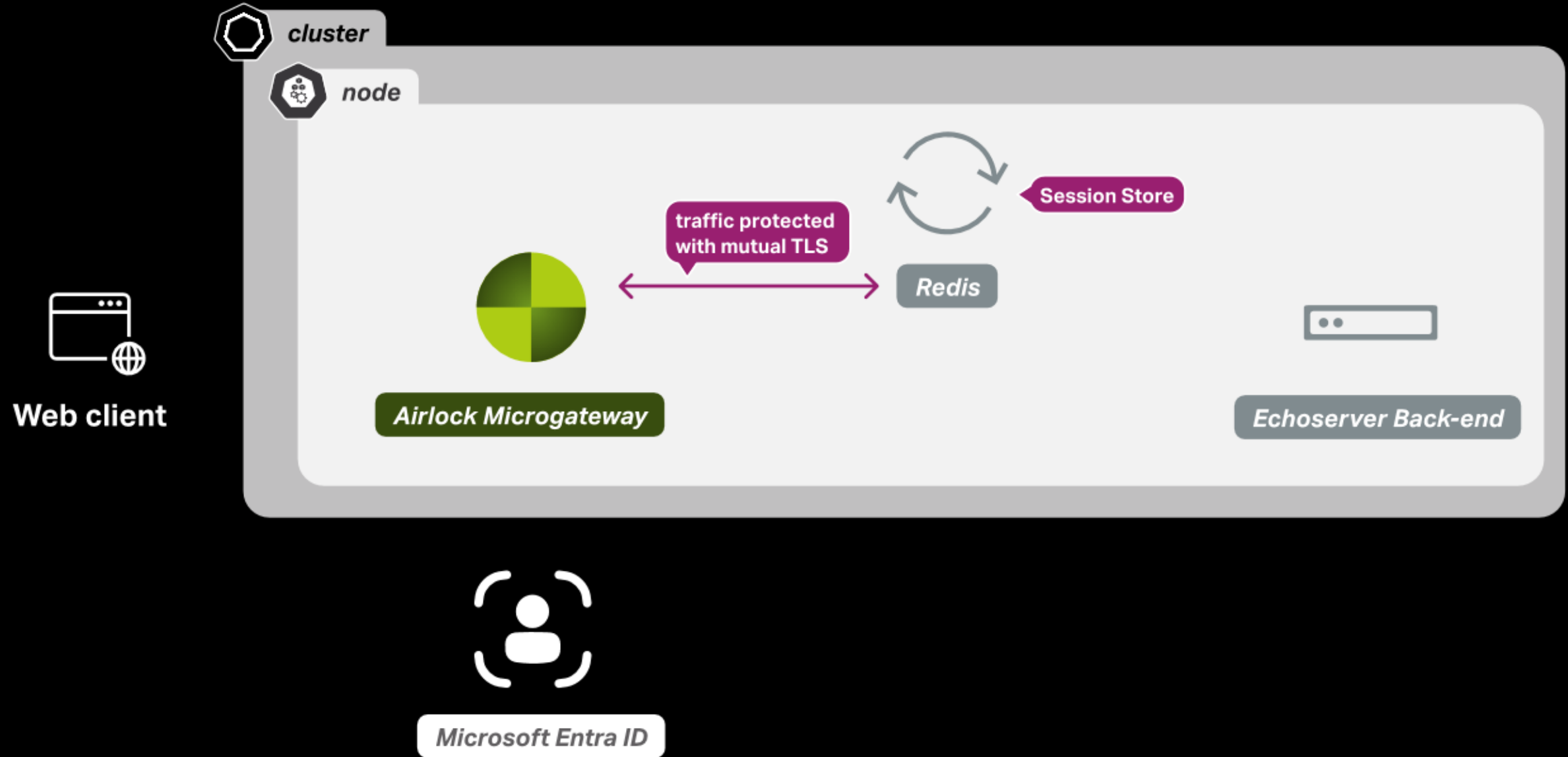
- Configure OIDC-based authentication with Entra ID
- Configure header-based Identity Propagation (optional)

Focus of This Lab: Setup an OIDC Configuration (Authentication Code Flow) with Airlock Microgateway...





For this Lab, an Echoserver Back-end, a Redis Session Store, Session Handling, and Entra ID are Pre-configured



LAB 07

Authentication and Authorization with Entra ID



Any feedback?

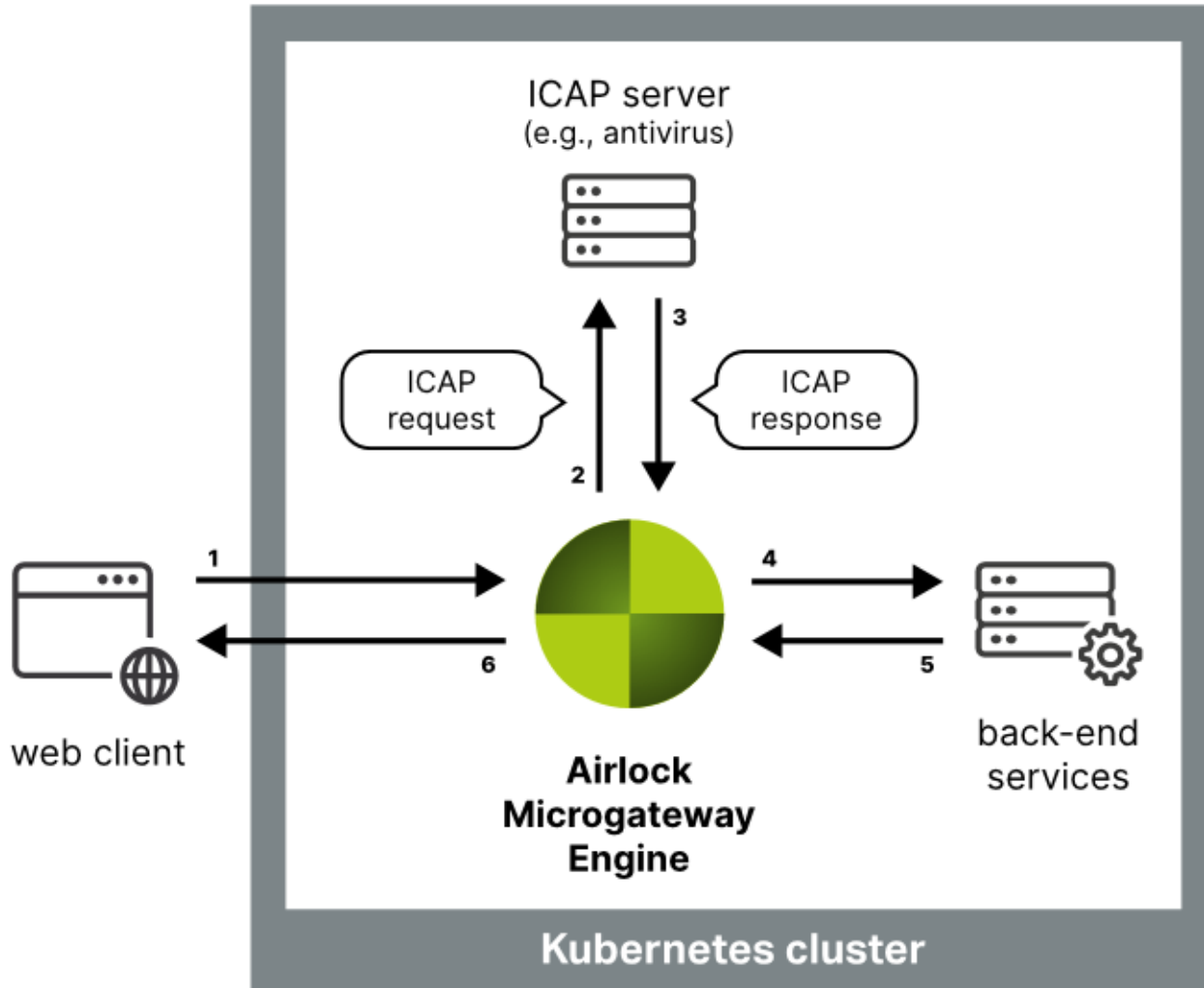


ICAP



Why and how does Airlock Microgateway
support **ICAP**?

ICAP – What is Supported by Airlock Microgateway?



Protocol to filter HTTP
requests with external
servers

ICAP – Why You Might Want to Use It

Benefits:

- increase security or add functionality
- standardize your server infrastructure by using an industry-proven protocol

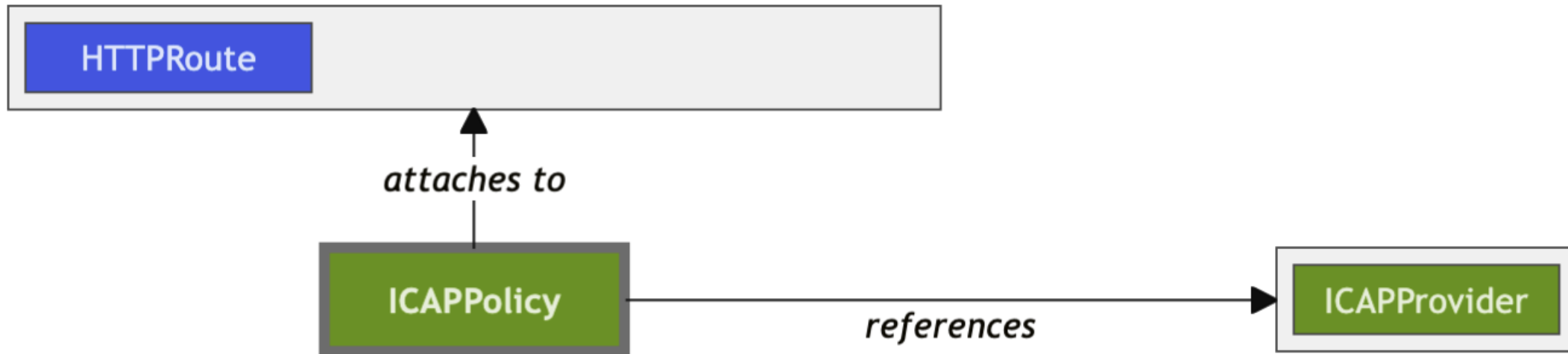
Usage:

malware detection, etc.



Only forward necessary requests to avoid unnecessary latency increase!

ICAP – Configuration



Limitation with Microgateway 5.1: Only **one** service can be connected

ICAP – Configuration

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: ICAPPolicy
metadata:
  name: juice-shop-icap-example
spec:
  targetRefs:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      name: http-r-juice-shop
  policies:
    - requestConditions:
        path:
          matcher:
            exact: /upload/file
      request:
        actions:
          - icapProviderRef:
              name: av-scan
            onError: Block
```

```
apiVersion: microgateway.airlock.com/v1alpha1
kind: ICAPProvider
metadata:
  name: av-scan
spec:
  endpoints:
    - uri: icaps://av-scanner.av.svc.cluster.local
      tls:
        protocol:
          minimum: TLSv1_3
  timeouts:
    connect: 2s
    maxDuration: 30s
```

Operations



How can you handle the **operations**
of Airlock Microgateway?

Monitoring License Validity
Monitoring Licensed Throughput
Versioning, Lifecycle and Upgrades
Support



Operations

Monitoring License Validity

Monitoring Licensed Throughput

Versioning, Lifecycle and Upgrades

Support



Operations

Monitoring License Validity – Prometheus Metrics

Operator Metrics

`microgateway_license_valid`

1 = valid license; 0 = no valid license

If `microgateway_license_valid` is 1

License is **valid**

→ Traffic is secured according to config of Airlock Microgateway Engine and kind and scope of license

Monitoring License Validity – Prometheus Metrics

Operator Metrics

`microgateway_license_valid`

1 = valid license; 0 = no valid license

`microgateway_license_expiry_timestamp_seconds`

timestamp when the active license expires

If `microgateway_license_expiry_timestamp_seconds` is reached

License is **expired**

→ `microgateway_license_valid` is set to 0

→ Pods created **before** the license expiry timestamp **run normally**

→ Pods created **after** the license expiry timestamp **block all traffic**

Monitoring License Validity – Prometheus Metrics

Operator Metrics

`microgateway_license_valid`

1 = valid license; 0 = no valid license

`microgateway_license_expiry_timestamp_seconds`

timestamp when the active license expires

If `microgateway_license_valid` is 0

AND `microgateway_license_expiry_timestamp_seconds` is **NOT** reached

License is **missing** or **invalid**

→ All traffic is blocked

Monitoring License Validity – Prometheus Metrics

Operator Metrics

`microgateway_license_valid`

1 = valid license; 0 = no valid license

`microgateway_license_expiry_timestamp_seconds`

timestamp when the active license expires

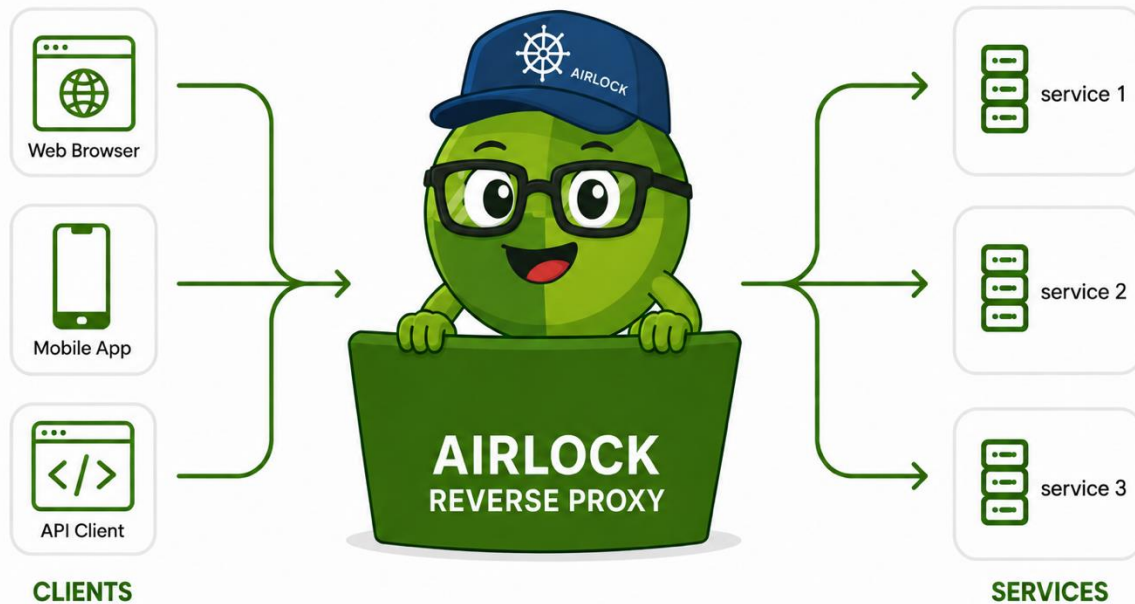
`microgateway_license_info`

label = ID of the license OR <MISSING>

`microgateway_license_is_premium`

1 = premium license; 0 = no premium license

Remember Content Security Defaults?



After Deployment

Content security defaults = **inactive**

Enable Content Security Defaults?

Only, if **module «filter»** is licensed

Monitoring License Validity – Insufficient License

Configuration

`GatewayParameters.spec.defaults.contentSecurity = secured`

If module **filter** is NOT licensed:

→ All traffic is blocked

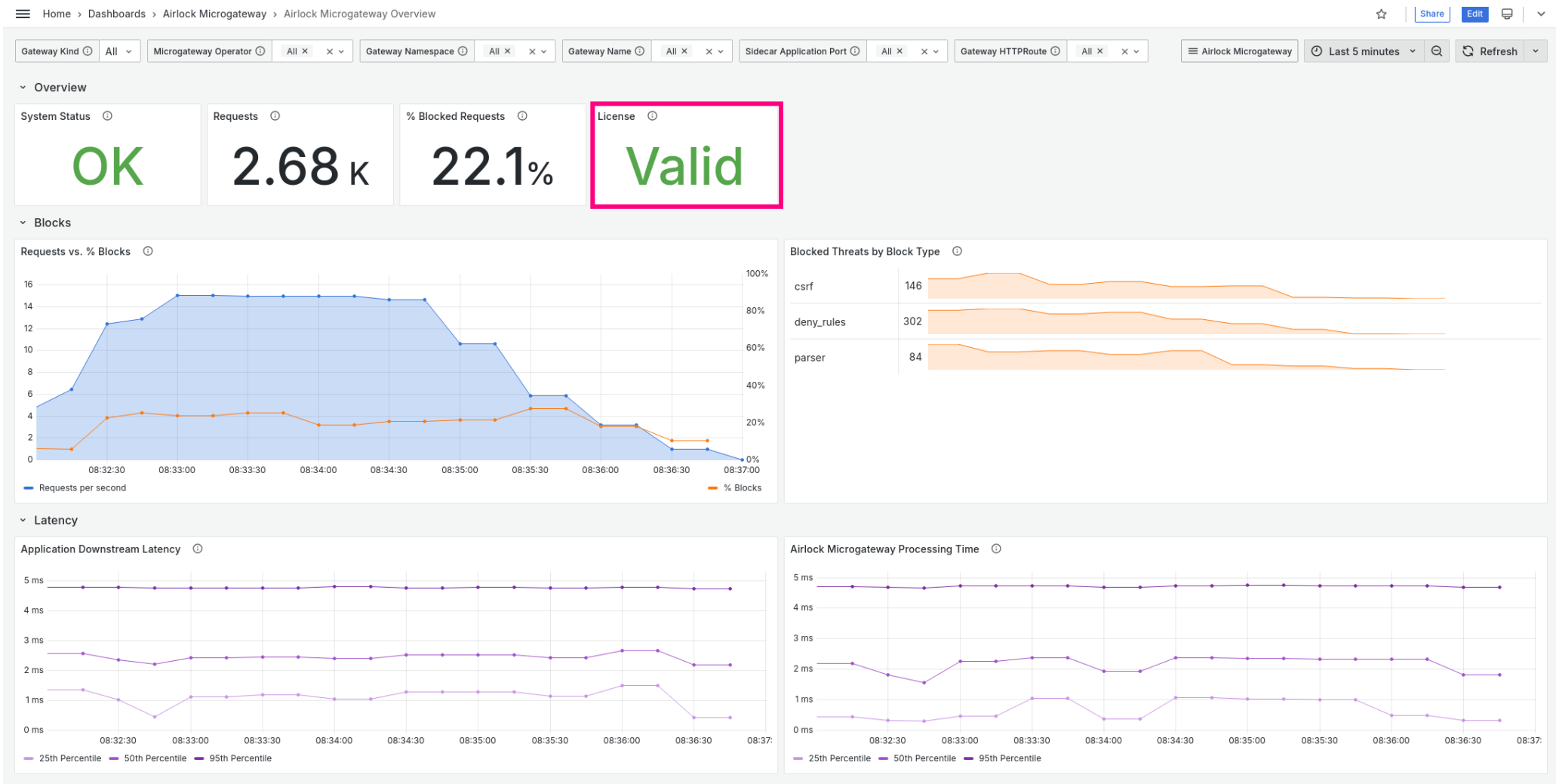
Monitoring License Validity – Insufficient License

Configuration

If a policy for an HTTPRoute is configured, but the required module NOT licensed

- Policy is rejected
- Traffic for the affected HTTPRoute is blocked

Airlock Microgateway Overview Dashboard



Monitoring License Validity

Monitoring Licensed Throughput

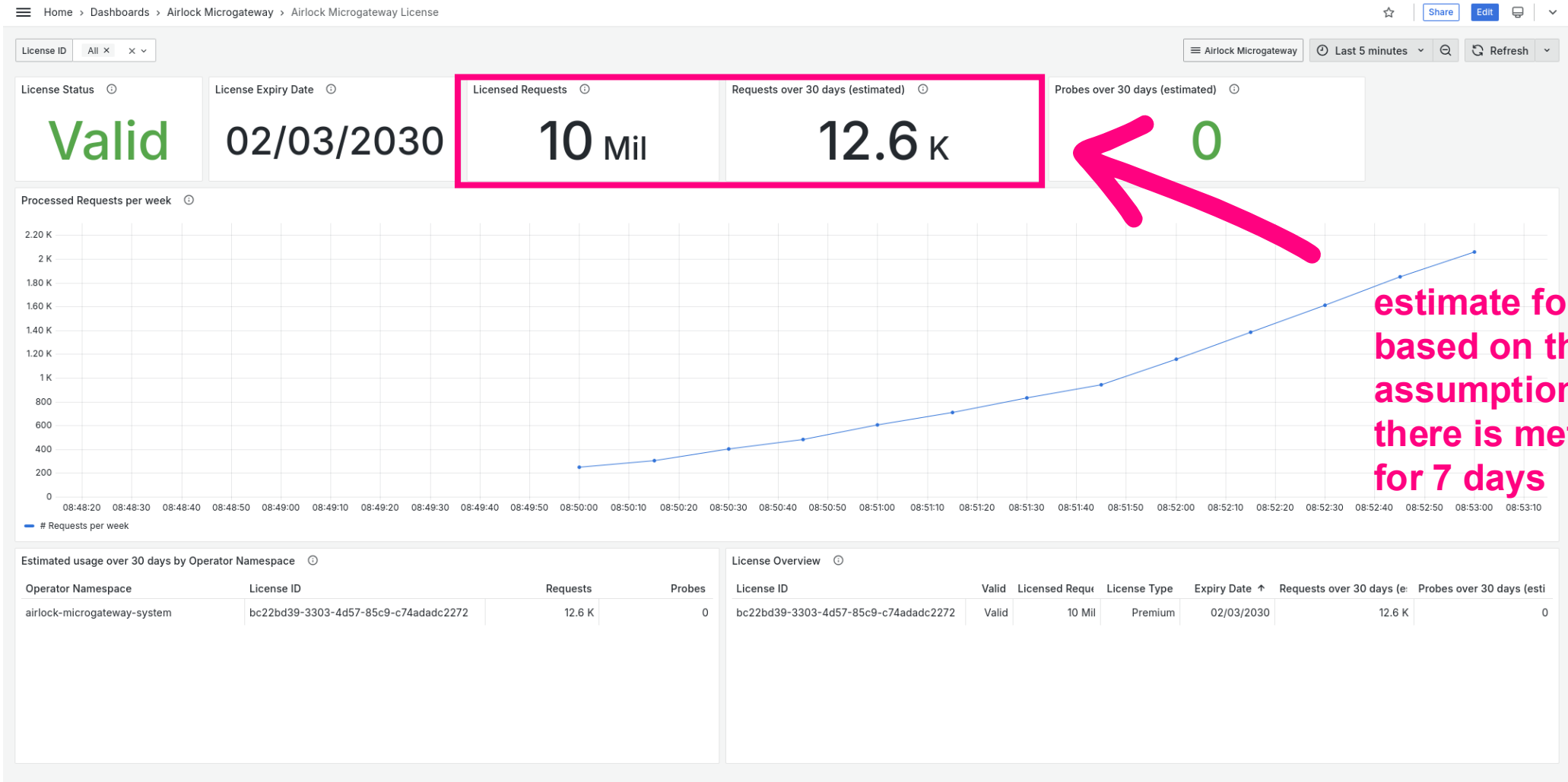
Versioning, Lifecycle and Upgrades

Support



Operations

Airlock Microgateway License Dashboard



Monitoring License Throughput

Default Prometheus storage retention usually < 30 days

Option 1

Set up Prometheus deployment with storage retention ≥ 30 days

Option 2

Monitor request counts by counting them via the access logs

Requires a storage of access logs for at least 30 days

Monitoring License Throughput – Prometheus Metrics

Operator Metrics

`microgateway_license_max_rq_count_per_month`

licensed request count per month

Engine Metrics

`microgateway_license_http_rq_total`

number of total requests

<https://docs.airlock.com/microgateway/5.1/index/1681413666718.html>

Monitoring License Validity
Monitoring Licensed Throughput
Versioning, Lifecycle and Upgrades
Support



Operations

Semantic Versioning Scheme

$x.y.z = \text{major}.\text{minor}.\text{patch}$

major = breaking changes

→ release a new x when changes break compatibility

Be Prepared to Upgrade!

Support lifecycle for Airlock Microgateway minor releases:

Minor version	Published	End-of-support
5.1	06/2026	03/2027
5.0	03/2026	12/2026
4.8	11/2025	08/2026
No longer supported: 4.7	08/2025	05/2026

i. e., **[ver]**
could be
5.1

[https://docs.airlock.com/microgateway/\[ver\]/index/1652373391626.html](https://docs.airlock.com/microgateway/[ver]/index/1652373391626.html)

How often are Upgrades Available?

Support lifecycle for Airlock Microgateway minor releases:

Minor version	Published
5.1	06/2026
5.0	03/2026
4.8	11/2025
No longer supported: 4.7	08/2025

New minor (or major)
version **every ~3 months**

`x.y.z` = major.minor.patch

Airlock Microgateway (MGW) Upgrades Include Envoy

Support lifecycle for Airlock Microgateway minor releases:

Minor version	Published
5.1	06/2026
5.0	03/2026
4.8	11/2025
No longer supported: 4.7	08/2025

Supported MGW versions
receive regular updates for
 **envoy** and other
dependencies

Be Prepared to Upgrade!

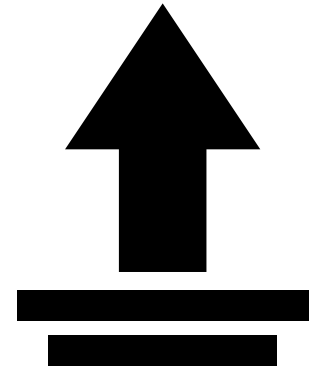
Support lifecycle for Airlock Microgateway minor releases:

Minor version	Published	End-of-support
5.1	06/2026	03/2027
5.0	03/2026	12/2026
4.8	11/2025	08/2026
No longer supported: 4.7	08/2025	05/2026

End of support
after ~9 months

How to Upgrade?

1. Upgrade the K8s Gateway API CRDs
2. Upgrade the Airlock Microgateway CRDs
3. Upgrade the Airlock Microgateway Operator



i. e., **[ver]** could be **5.1**

[https://docs.airlock.com/microgateway/\[ver\]/index/1726159368039.html#Upgrade](https://docs.airlock.com/microgateway/[ver]/index/1726159368039.html#Upgrade)

Monitoring License Validity
Monitoring Licensed Throughput
Versioning, Lifecycle and Upgrades
Support



Operations

Monitoring License Validity
Monitoring Licensed Throughput
Versioning, Lifecycle and Upgrades
Support



Operations

Support

1st level

usually on-site

*access to relevant systems and
Airlock documentation materials*

2nd / 3rd level

usually qualified partner

SLA incl. 7x24h coverage if contracted

*expertise for integrations of /
projects involving Airlock products*

4th level

by manufacturer Ergon Informatik AG

7x24h coverage if contracted

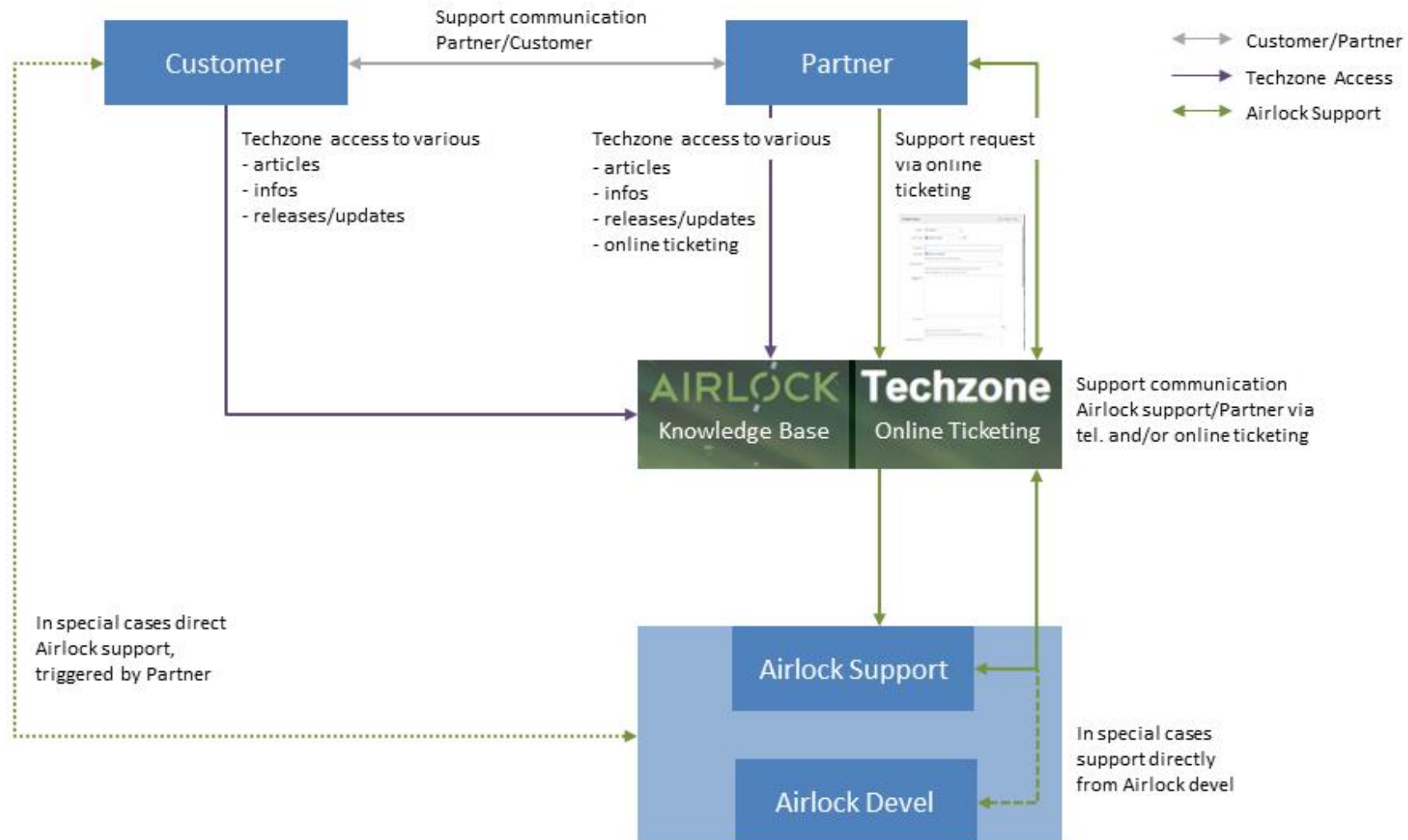
usually, 4h reaction time for critical issues

open support tickets at:

<https://techzone.ergon.ch/support>

in-depth Airlock expertise

Support



Support information: Detailed Description

- reproducible issue
- system information
- configuration files
- log files

Further Information

<https://www.airlock.com>

general product information

<https://docs.airlock.com>

product documentations

<https://techzone.ergon.ch>

product downloads

technical infos

link to Airlock online ticketing

→ Support Process: <https://techzone.ergon.ch/support-process>

Conclusion and Certification

Please Complete Your Feedback!



Airlock Microgateway Connaisseur Feedback

Hello!

We would appreciate it, if you could help us to improve the Airlock Microgateway training and product by filling our feedback questionnaire.

Please answer only half of the questionnaire at the begin of the training, and the other half at the end of the training (stopping-point indicated in questionnaire).

Your answers will help us to better address your needs.

[Click here to start →](#)

**← re-open the link received
via mail on day 1**

 **Thank you!**

 **Please stop here for now.**



Re-open the link you received via e-mail when prompted at the end of the last day of the training and answer the remain questions.

You are at the end of the last day of the training and re-opened the link?
Please, click **«Next»** to continue!

Back

**continue
with «Next»**



Next »

Recap

Questions and Feedback

Certification



**Conclusion and
Certification**

Certification Process



 Online test via Testportal

→ An access code will be sent to your mail

 60 min

 Open book

allowed: lab environment + resources provided in the training + product documentation

NOT allowed: use of AI agents like ChatGPT, Claude, etc. → if detected: certification = invalid

Certification Process



-  Online test via Testportal
 - An access code will be sent to your mail
-  60 min
-  Open book
-  Single and multiple choice questions
-  Answer 65% correctly to pass!

Certification Process



- Access the certification test and answer all questions
- If something is unclear, please ask now!

Certificate Delivery

- Certificates are sent out shortly after the certification **via email**
- Target: Your email address that was provided during the training booking

Creation of Support Cases

Preconditions

- Passed certification
- An account at the Ergon Techzone:

[*https://techzone.ergon.ch*](https://techzone.ergon.ch)



Afterward

- You will receive the permission to **create support cases**
- Usually, a few days after the certification



Thanks! Open Questions?

Airlock Microgateway Allrounder



Maximilian Dürr
Airlock Academy Teacher

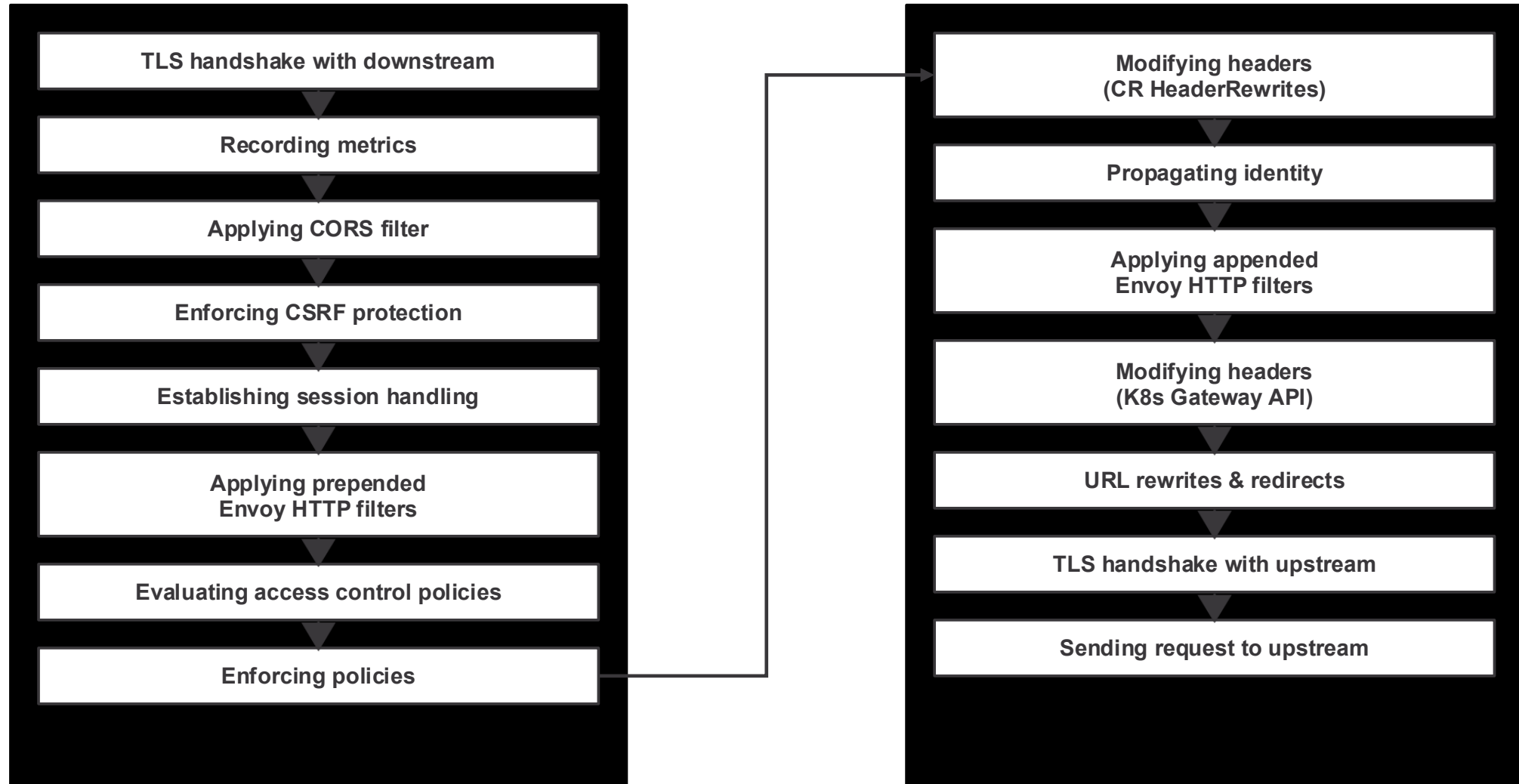


Nicolas Röschli
Airlock Academy Teacher

Attribution – Kubernetes Icons Set

- The figures in this slide set make use of the Kubernetes Icons Set <https://github.com/kubernetes/community/tree/master/icons>, licensed under CC-BY 4.0.
- The background color of the icons is partially changed in contrast to the original blue color.
- Also, the legend of the figures used in the slides includes versions of the icons that have a light/transparent background and a dark foreground.

Processing Sequence – Requests



Processing Sequence – Responses

